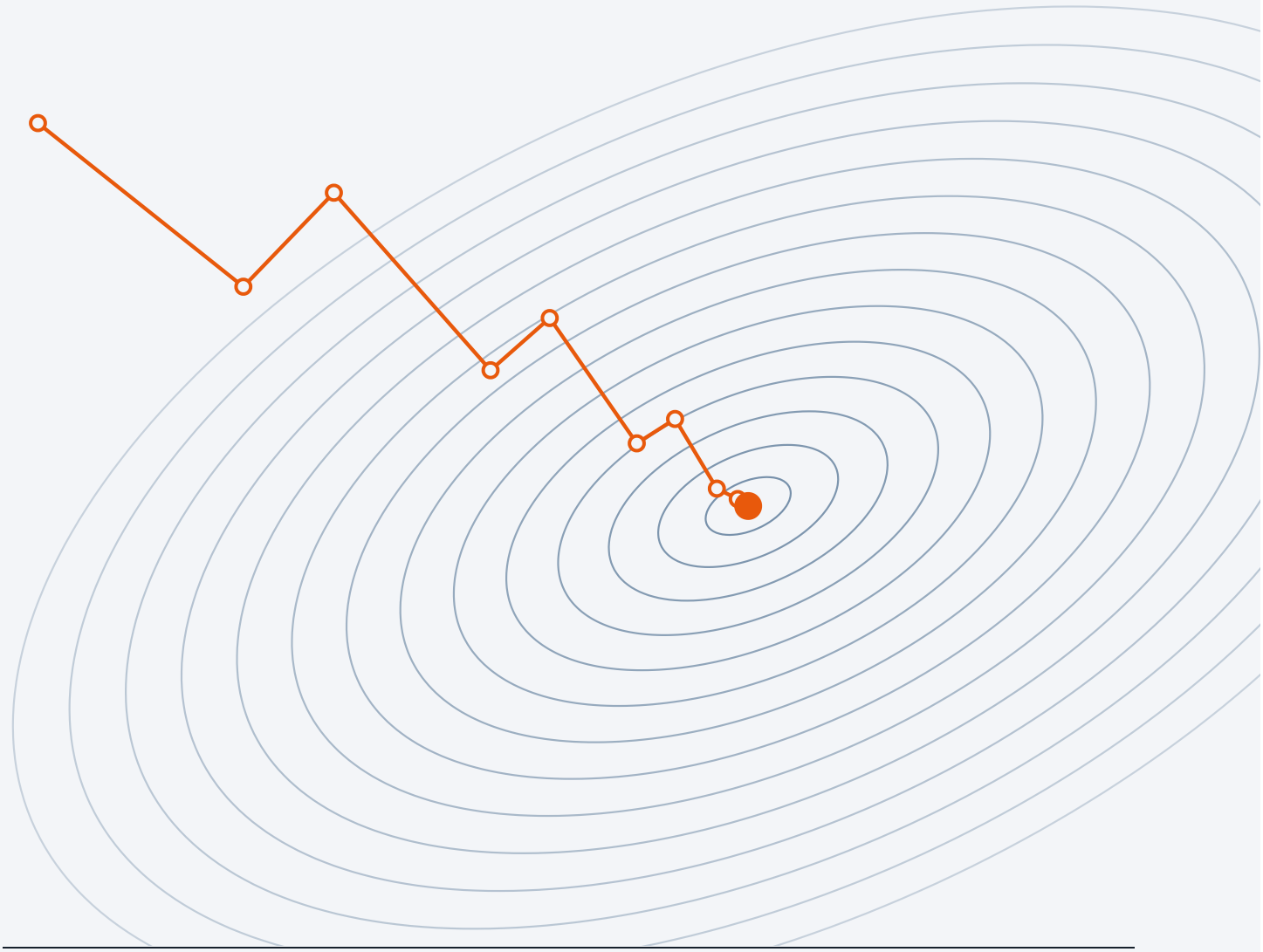
T.C.
PAMUKKALE ÜNİVERSİTESİ
İKTİSADİ VE İDARİ BİLİMLER FAKÜLTESİ
YÖNETİM BİLİŞİM SİSTEMLERİ BÖLÜMÜ

YBS 319 · KONU ANLATIMI

Yapay Zekâ ve Öğrenen Algoritmalara Giriş



Arş. Gör. Berker DAL

2026–2027 Güz yarıyılı

KISIM I

Yapay zekâya giriş

1 Yapay zekâyı tanımlamak

2 Bir yapay zekâ sisteminin anatomisi

3 Temel ayrımlar ve kavramlar

4 Yapay zekânın kısa tarihi

5 İşletmede yapay zekâ: değer, sınırlar ve güven

1

BÖLÜM

Yapay zekâyı tanımlamak

Bu bölümde

- "Zekâ" sözcüğünün neden tek bir tanıma sığmadığını açıklamak,
- Russell ve Norvig'in dört yaklaşımını ve her birinin hangi başarı ölçütünü kurduğunu ayırt etmek,
- Turing testinin neyi ölçüp neyi ölçmediğini söyleyebilmek,
- rasyonel ajan kavramını ve rasyonelliğin bağlama nasıl bağlı olduğunu bir örnek üzerinde göstermek,
- OECD'nin işlevsel yapay zekâ sistemi tanımını altı bileşenine ayırarak bir sisteme uygulamak.

MUĞLA ÜNİVERSİTESİ 2013 BİLİŞİM SİSTEMLERİ

Kaynak: YBS 319 ders notları

Bir hesap makinesi, bir e-posta spam filtresi, telefonunuzdaki navigasyon uygulaması, bir film platformunun öneri listesi, havalimanındaki yüz tanıma kapısı ve soru sorduğunuzda paragraf paragraf cevap yazan bir dil modeli. Bunlardan hangileri yapay zekâdır?

Derste bu soruyu sorduğumuzda gelen cevaplar genellikle aynı birkaç ölçüte dayanır: sistem insan gibi düşünüyorsa, öğreniyorsa, çok karmaşıksa, çok veri kullanıyorsa ya da kendiliğinden cevap üretiyorsa yapay zekâdır. Her ölçüt bazı örnekleri açıklar ama hiçbiri alanın tamamını kapsamaz. Satranç oynayan bir program hiçbir şey öğrenmeden, yalnızca hamleleri arayarak ve değerlendirerek dünya şampiyonunu yenebilir. Sabit kurallarla çalışan bir sistem son derece karmaşık ve otomatik olabilir, ama bu onu yapay zekâ yapmaz. Bir sistemi ürün adına, pazarlama diline ya da dış görünüşüne göre değil; neyi amaçladığına, hangi girdiyi aldığına, girdiden çıktıya nasıl geçtiğine ve çıktının nerede kullanıldığına bakarak değerlendirmemiz gerekir. Bu bölüm o değerlendirmenin dilini kuruyor.

1.1 Neden şimdi bu kadar konuşuluyor?

Yapay zekâ yeni bir araştırma alanı değildir; adı 1956'da konmuştur. Son yıllarda bu kadar görünür olmasının nedeni, tekniklerin laboratuvardan çıkıp gündelik ürünlere ve işletmelerin iş akışlarına girmesidir. Stanford Üniversitesi'nin *AI Index 2026* raporundaki kuruluş anketinde katılımcıların %88'i en az bir iş işlevinde yapay zekâ kullandığını, %70'i ise en az bir iş işlevinde üretken yapay zekâ kullandığını bildirmiştir.

Bu oranları kesin bir sayım gibi yorumlamamak gerekir; bunlar tanımı ve örneklemi belli olan bir anketin sonuçlarıdır. Yine de önemli bir eğilime işaret ederler. Aynı kurumda satış tahmini, müşteriye öneri, belge özetleme, sahtecilik tespiti ve çalışanlara karar desteği birlikte yürüyebilir ve bunların her biri farklı bir yöntemle çalışır. Kullanım yaygınlaştıkça doğruluk kadar yönetişim, veri güvenliği, kullanıcı davranışı ve sorumluluk soruları da büyür.

Oranları nasıl okumalı?
Bu yüzdeler bir dünya sayımı değil, belirli bir örneklemin öz bildirimidir. "Kullanıyoruz" demek, kullanımın ölçekli, güvenilir ya da değer üreten bir uygulama olduğunu göstermez.

Görünürlüğün arkasında birbirini besleyen üç gelişme vardır. Dijitalleşen süreçler çok daha fazla veri üretir hâle geldi. Hesaplama altyapısı büyük modellerin eğitilmesini ve düşük gecikmeyle kullanılmasını mümkün kıldı. Hazır kütüphaneler ve bulut servisleri de bu teknikleri kurumların iş akışlarına taşıdı. Üretken yapay zekâ ise kullanıcı arayüzünü doğal dile yaklaştırarak teknolojiyi uzman olmayanların da doğrudan kullanabileceği bir şeye dönüştürdü.

Bu kitapta güncel ürün adlarına mümkün olduğunca az yaslanacağız. Ürünler ve oranlar hızla değişir; amaç, veri, model, çıktı ve etki üzerine kurulan düşünme çerçevesi ise çok daha kalıcıdır.

1.2 Zekâ sözcüğünün güçlüğü

Yapay zekâyı tanımlamakta zorlanmamızın ilk nedeni, *zekâ* sözcüğünün kendisinin tek bir yeteneği anlatmamasıdır. İnsanlarda algılama, öğrenme, dil kullanma, planlama ve akıl yürütme birbirinden farklı yeteneklerdir. Bir kişi bazılarında çok güçlü, bazılarında zayıf olabilir. Makinelerde bu ayrışma çok daha keskindir: Satrançta dünya şampiyonunu yenen bir sistem, bir

bardağı masadan alıp lavaboya koymak gibi basit bir mutfak görevini yerine getiremeyebilir. Bir sistem görüntü sınıflandırmada üstün olup kendi hedefini belirleyemeyebilir; başka bir sistem uzun ve akıcı metinler üretip fiziksel çevreyi hiç algılayamayabilir.

Bu yüzden mühendislikte "makine gerçekten düşünüyor mu?" gibi doğrudan ölçülmesi zor bir sorudan daha somut sorulara geçeriz:

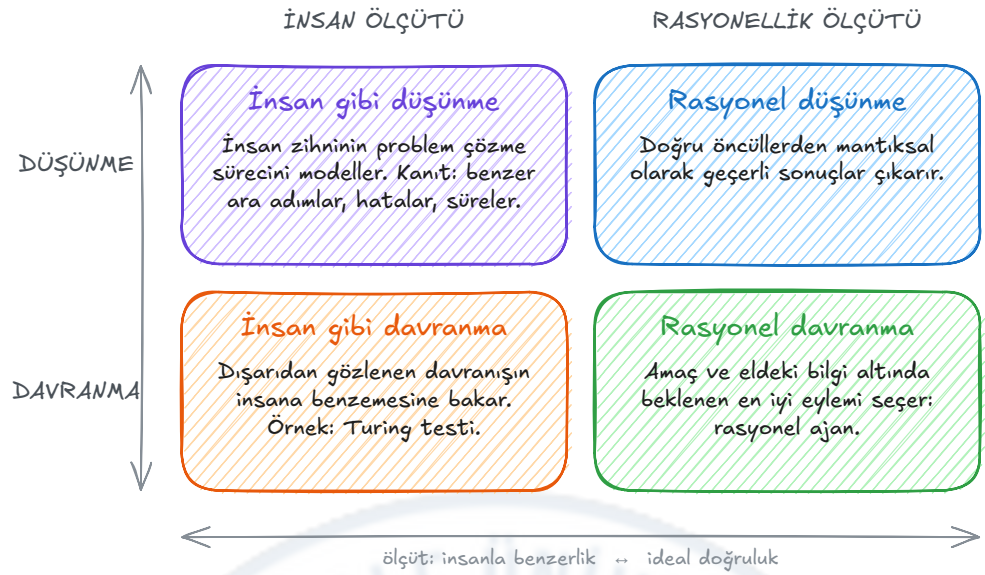
- Sistem hangi görevi yerine getiriyor?
- Başarıyı hangi ölçütle değerlendiriyoruz?
- Ortam ya da veri değiştiğinde performans korunuyor mu?
- Hata ne kadar maliyetli, geri döndürülebilir mi ve sorumluluğu kime ait?

Bu sorular soyut bir zekâ tartışmasını sınanabilir bir sistem problemine dönüştürür. Hesap makinesi aritmetikte insandan hem hızlı hem hatasızdır, ama onu genel anlamda zeki saymayız. Bir dil modeli akıcı metin üretir, fakat akıcılık doğruluk güvencesi vermez. Tek bir toplam zekâ puanı yerine görev, çevre ve başarı ölçütü tanımlarız.

> **Dikkat**
>
> İnsana benzemek tek başına bir mühendislik ölçütü değildir. İnsanlar yorgunluk, önyargı ve sınırlı bellek taşır; bir sistemin insan hatalarını taklit etmesi onu daha iyi bir yapay zekâ yapmaz. *İnsan bilişini açıklamak* bilimsel bir hedeftir, *belirli bir görevi güvenilir biçimde yerine getiren sistem tasarlamak* ise mühendislik hedefidir. Bu iki hedef kesişebilir ama aynı değildir.

1.3 Yapay zekâyı tanımlamanın dört yolu

Yapay zekâ ders kitaplarının en yaygın başvurusu olan *Artificial Intelligence: A Modern Approach* kitabında Stuart Russell ve Peter Norvig, alandaki tanımları iki eksen üzerinde düzenler. Birinci eksen başarı ölçütüyle ilgilidir: Sistemi gerçek insanlarla mı karşılaştırıyoruz, yoksa ideal bir doğruluk ölçütü olan *rasyonellikle* mi? İkinci eksen inceleme nesnesiyle ilgilidir: Sistemin iç düşünme sürecine mi bakıyoruz, yoksa dışarıdan gözlenen davranışına mı? İki eksen birleşince dört yaklaşım ortaya çıkar (Şekil 1.1).



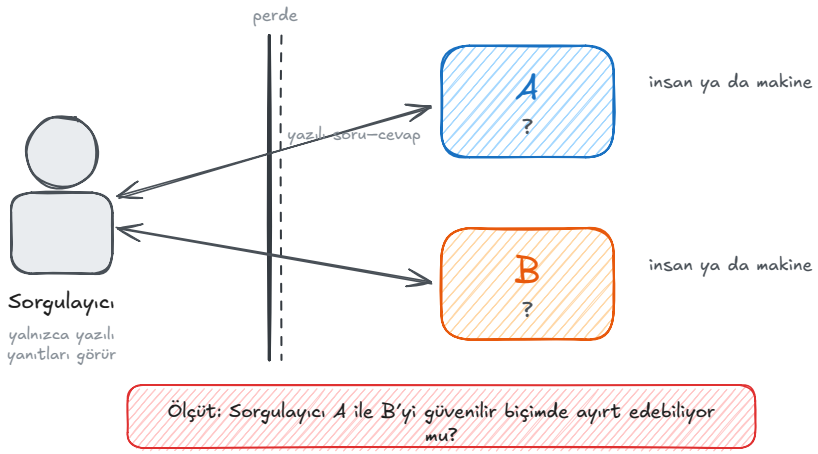
Şekil 1.1 Russell ve Norvig'in iki eksenli çerçevesi. Yatay eksen başarı ölçütünü, dikey eksen inceleme nesnesini belirler.

Her kutu farklı bir kanıt türü ister. İnsan gibi düşünme iddiası için doğru cevap yetmez; insanlarla benzer hata örüntüsü, çözüm sırası ya da tepki süresi gösterilmelidir. Rasyonel düşünme için biçimsel olarak geçerli çıkarım aranır. İnsan gibi davranma için sorgulayıcının gözleyebildiği performans önemlidir. Rasyonel davranma için ise eylemin beklenen sonucu, tanımlanmış bir performans ölçütüne göre değerlendirilir.

> **Dikkat**
>
> Dört kutuyu yapay zekâ tarihinin birbirini izleyen dört aşaması sanmak yaygın bir yanılgıdır. Bunlar aynı soruya ilişkin alternatif araştırma hedefleridir ve birbirini dışlamaz. Bir bilişsel mimari hem insan süreçlerini modelleyebilir hem de belirli bir görevde rasyonel davranabilir; ancak bu iki başarı ayrı deneylerle gösterilmelidir.

1.3.1 İnsan gibi davranma: Turing testi

Alan Turing 1950 tarihli "Computing Machinery and Intelligence" makalesinde, "Makineler düşünebilir mi?" sorusunun kolayca bitmeyen bir tanım tartışmasına dönüşeceğini görür. Bu yüzden düşünmenin özünü tanımlamaya çalışmak yerine gözlenebilir bir davranış deneyi önerir. *Taklit oyunu* adını verdiği bu düzenekte bir sorgulayıcı, karşısındakileri görmeden yalnızca yazılı bir kanal üzerinden bir insanla ve bir makineyle konuşur (Şekil 1.2). Ölçüt, makinenin gerçekten insan olması değil, sorgulayıcının yanıtların kaynağını güvenilir biçimde ayırt edememesidir.



Şekil 1.2 Taklit oyunu. Sorgulayıcı yalnızca yazılı yanıtları görür; değerlendirme dış davranış üzerinden yapılır.

Bu fikrin tarihsel önemi, ölçülmesi güç bir kavramı sınanabilir bir düzene çevirmesidir. Russell ve Norvig, ciddi bir Turing testini geçebilmek için bir makinenin dört yeteneğe ihtiyaç duyacağını söyler: sorgulayıcıyla iletişim kurmak için *doğal dil işleme*, bildiklerini ve konuşma sırasında öğrendiklerini saklamak için *bilgi temsili*, saklanan bilgiden yeni sonuçlar çıkarmak için *otomatik akıl yürütme* ve yeni koşullara uyum sağlamak için *makine öğrenmesi*. Fiziksel etkileşimi de içeren *toplam Turing testi* bunlara nesneleri algılamak için bilgisayarlı görüyü ve nesneleri hareket ettirmek için robotiği ekler. Bu liste, yapay zekâ alanının bugün hâlâ kullandığımız alt dallarının neredeyse tamamını içerir.

Testin sınırını açık tutmak gerekir. Turing testi bir bilinç testi değildir. Bir sistemin kısa bir konuşmada insana benzemesi, dünyayı insan gibi anladığını, iç mekanizmasının insan bilişine benzediğini ya da bütün bilişsel görevlerde insan düzeyinde olduğunu göstermez. Güncel sohbet sistemleri bu ayrımı her zamankinden önemli kılıyor: Akıcı bir dil, kullanıcıda güçlü bir zihin izlenimi yaratabilir. Bu kitapta gözlenen performansı bilinç, niyet ve genel zekâ iddialarından ayrı tutacağız. Turing'i yapay zekânın tek tanımı olarak değil, davranışsal ölçme fikrinin klasik başlangıç noktası olarak okuyacağız. Nitekim yapay zekâ araştırmalarının büyük bölümü insanı kandırmayı değil, zekice davranışın ilkelerini anlamayı hedefler.

Örnek 1.1 · Sohbet botu ve dört kutu

Bir bankanın sohbet botu, beş dakikalık yazışmalarda müşterilerin %70'ini bir insan temsilciyle konuştuğuna inandırıyor. Bu sonuç, *insan gibi davranma* ölçütü için bir kanıttır. Botun insan gibi düşündüğünü göstermez; bunun için iç süreçlerin insan bilişiyle karşılaştırılması gerekirdi. Mantıksal olarak geçerli çıkarımlar yaptığını da göstermez; insanı ikna etmek mantıksal geçerliliğin kanıtı değildir. Botun bilinçli olduğunu ise hiç göstermez.

1.3.2 İnsan gibi düşünme: bilişsel modelleme

Bu yaklaşımın sorusu, bir programın doğru sonuca ulaşıp ulaşmadığı değil, insan zihninin aynı sonuca *nasıl* ulaştığıdır. İnsan gibi düşünen bir program iddiasında bulunmak için önce insan düşüncesine ilişkin sınanabilir bir kurama ihtiyaç vardır. Russell ve Norvig bunun için üç bilgi

kaynağı sayar:

- **İçebakış:** Kişinin kendi düşünce akışını gözlemlemesi. Erişilebilir ama sınırlı ve özneldir.
- **Psikolojik deney:** İnsanların davranışını, hata örüntülerini ve tepki sürelerini ölçmek. Karşılaştırılabilir deneysel veri üretir.
- **Beyin görüntüleme:** Biliş sırasında etkinleşen sinirsel süreçleri gözlemlemek. Fiziksel süreç hakkında kanıt sağlar.

Kuram yeterince açık hâle geldiğinde bir bilgisayar programı olarak ifade edilebilir. Programın yalnızca son cevabı değil, insanlarla benzer ara adımları, süreleri ve hataları da üretmesi, iki mekanizmanın benzer olabileceğine dair kanıttır. Allen Newell ve Herbert Simon'un 1961'de yayımladıkları *General Problem Solver* (GPS) bu yaklaşımın klasik örneğidir. Araştırmacılar programın yalnızca doğru sonuca ulaşıp ulaşmadığına bakmadı; alt hedefleri hangi sırayla ele aldığını ve çözüm süresini, aynı problemleri çözen insan deneklerin sesli düşünme protokolleriyle karşılaştırdılar. Yapay zekâdaki bilgisayar modellerini psikolojideki deneysel yöntemlerle birleştiren bu çizgi bugün *bilişsel bilim* adıyla ayrı bir disiplin olarak sürer.

Hesap makinesi örneği yaklaşımın sınırını iyi gösterir. Hesap makinesi çoğu insandan hızlı ve hatasız işlem yapar; ama ara adımları, hata örüntüleri ve öğrenme süreci insanınkine benzemez. Dolayısıyla iyi görev performansı bilişsel benzerlik kanıtı değildir. Tersine, insanın hatalarını çok iyi taklit eden bir model de en iyi mühendislik çözümü olmayabilir. Performans iddiası ile bilişsel benzerlik iddiası ayrı deneyler gerektirir.

1.3.3 Rasyonel düşünme: düşüncenin yasaları

Üçüncü yaklaşım, düşünmenin biçimsel olarak doğru kurallara uymasını hedefler. Kökeni, doğru öncüllerden zorunlu olarak doğru bir sonuç çıkaran Aristoteles kıyaslarına kadar götürülür:

öncül 1	Bütün insanlar ölümlüdür.
öncül 2	Sokrates insandır.
sonuç	∴ Sokrates ölümlüdür.

On dokuzuncu yüzyılda mantıkçılar, dünyadaki nesneler ve aralarındaki ilişkiler hakkında daha genel cümleler kurmaya yarayan kesin gösterimler geliştirdi. Yapay zekâdaki *mantıkçı gelenek*, dünya bilgisini bu dillere çevirip çıkarım programlarıyla sonuç üretmeyi amaçladı. Hedef, akıl yürütmeyi denetlenebilir bir hesaplama sürecine dönüştürmekti.

Yaklaşımın iki temel sınırı vardır. Birincisi, gündelik bilgi çoğu zaman kesin ve eksiksiz değildir. Sağlık, siyaset ya da işletme kararlarının kuralları satranç kuralları kadar açık olmaz. Olasılık kuramı belirsizlik altında tutarlı akıl yürütmeyi destekler, fakat hangi bilginin ve hangi olasılıkların temsil edileceği yine çözülmesi gereken bir sorundur. İkincisi, mantıksal olarak çözülebilir bir problemde bile doğru sonuca ulaşmak hesaplama bakımından çok pahalı olabilir. Doğru düşünme kuralına sahip olmak, makul bir sürede eylem üretmeyi garanti etmez.

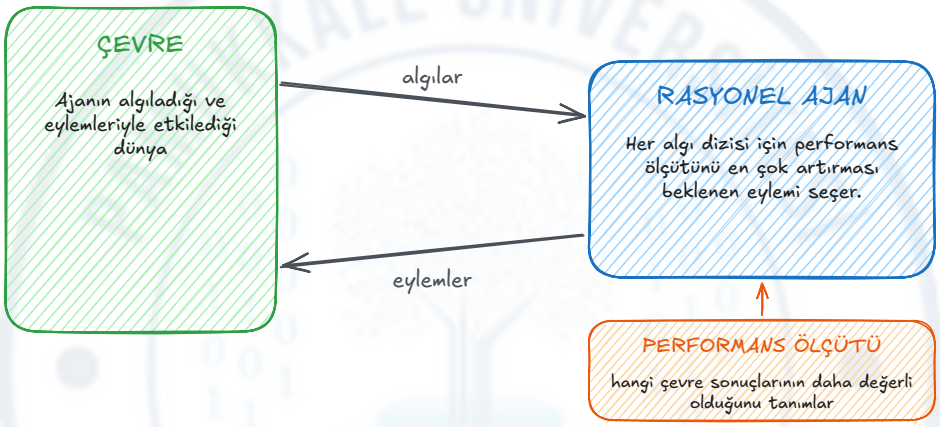
Burada *geçerlilik* ile dünyadaki *doğruluk* arasındaki ayrımı korumak gerekir. Geçerli bir çıkarım, öncüller doğruysa sonucun da doğru olacağını garanti eder; öncüllerin doğru olup olmadığını söylemez. Bu yüzden uygulamada bilgiyi edinmek, temsil etmek ve onunla çıkarım yapmak birlikte değerlendirilir.

Son olarak rasyonel düşünmek, rasyonel davranmanın tek yolu değildir. Sıcak bir yüzeye değen elimizi geri çekmemiz uzun bir mantıksal çıkarımın sonucu değildir, yine de son derece uygun bir eylemdir; bazen uzun düşünmek fırsatı kaçırmak demektir. Mantıksal çıkarım rasyonel bir ajanın araçlarından biridir, rasyonelliğin tamamı değildir.

Geçerli ama yanlış
"Bütün balıklar uçar; hamsi balıktır; öyleyse hamsi uçar." Çıkarım biçim olarak geçerlidir, ama ilk öncül yanlış olduğu için sonuç dünya hakkında doğru bilgi vermez. Mantık motoru öncüllerin doğruluğunu kendiliğinden denetlemez.

1.3.4 Rasyonel davranma: rasyonel ajan

Dördüncü yaklaşım yapay zekâyı *ajanlar* üzerinden tanımlar. Ajan, çevresini algılayan ve eylemleriyle bu çevreyi etkileyen her şeydir: bir robot, bir yazılım servisi ya da bir kredi değerlendirme sistemi olabilir. Ajan algılayıcılar veya veri girişleri üzerinden çevreyi gözler, iç temsili ve karar süreciyle seçenekleri değerlendirir, eyleyiciler veya kurumsal işlemler aracılığıyla çevre üzerinde bir eylem gerçekleştirir (Şekil 1.3).



Şekil 1.3 Rasyonel ajan döngüsü. Ajan algılardan eyleme giden bir karar döngüsü içinde çalışır; hangi sonuçların daha değerli olduğunu performans ölçütü tanımlar.

Tanım 1.1 · Rasyonel ajan

Rasyonel ajan, her olası algı dizisi için, o ana kadarki algılarının sağladığı kanıt ve sahip olduğu ön bilgi altında, **performans ölçütünü en çok artırması beklenen** eylemi seçen ajandır.

Tanımdaki *beklenen* sözcüğü kilit önemdedir. Belirsiz bir dünyada ajan eylemlerinin sonucunu kesin olarak bilemez; olası sonuçları ve bu sonuçların değerini dikkate alarak seçim yapar. Rasyonellik bu yüzden diğer üç yaklaşımdan daha geneldir. İnsan gibi davranmakla aynı değildir: İnsanlar sistematik hatalar yapar ve ajan, hiçbir insanın seçmeyeceği bir yöntemle daha iyi sonuç üretebilir. Rasyonel düşünmekle de aynı değildir: Doğru mantıksal çıkarım uygun eylemi bulmaya yardım edebilir, ama refleks, arama, öğrenilmiş bir politika ya da olasılıksal bir karar da rasyonel davranış üretebilir. Başarı iç mekanizmaya değil, belirli bir amaç ve çevre altında eylemin sonuçlarına bağlanır.

Russell ve Norvig bu yaklaşımın bugün alanın baskın standardı olduğunu söylerken bir uyarı da ekler: Amacı doğru tanımlamak sanıldığından zordur. Ajan kendisine verilen hedefi kusursuz biçimde en iyilerken insanın gerçek niyetine zarar verebilir. Satrançta kazanmak açık bir hedeftir;

gerçek dünyada ise güvenlik, hız, maliyet ve adalet birbiriyle çatışabilir. Yönetim bilişim sistemleri açısından performans ölçütünü kimin belirlediği ve bu ölçütten kimlerin etkilendiği, teknik tasarım kadar önemlidir. Bu konuya Bölüm 5'te *amaç uyumsuzluğu* başlığıyla döneceğiz.

1.4 Rasyonellik bağlama bağlıdır

Rasyonellik kusursuzluk ya da her şeyi bilmek demek değildir. Ajanın eylemi, karar anında sahip olduğu algı dizisi ve ön bilgi göz önüne alınarak değerlendirilir. Russell ve Norvig'in örneğinde bir kişi yolun iki yanına bakıp trafik olmadığını gördükten sonra karşıya geçmeye başlar, fakat o sırada uçan bir uçaktan kopan bir parça başına düşer. Sonuç kötüdür; ama eldeki kanıta göre karşıya geçme kararı rasyoneldir. Sonradan ortaya çıkan beklenmedik bir sonuç, kararın o anda irrasyonel olduğunu kanıtlamaz.

Bir eylemin rasyonel olup olmadığı dört şeye bağlıdır:

1. **Performans ölçütü:** Başarı hangi çevre sonuçlarıyla ölçülüyor?
2. **Ön bilgi:** Ajan çevre hakkında başlangıçta ne biliyor?
3. **Eylem kümesi:** Ajan hangi müdahaleleri gerçekten yapabiliyor?
4. **Algı dizisi:** Karar anına kadar hangi kanıtı gözledi?

Bu dört unsurdan biri değiştiğinde aynı davranışın değerlendirmesi de değişebilir. Bu yüzden rasyonelliği iki yakın kavramdan dikkatle ayırırız. *Kusursuzluk*, sonradan en iyi olduğu görülen sonucu seçmek demektir. *Her şeyi bilmek* ise eylemlerin gerçek sonuçlarını önceden kesin olarak bilmektir. Rasyonellik ikisini de gerektirmez; yalnızca eldeki kanıta göre beklenen performansı artıran seçimi ister.

Örnek 1.2 · Robot süpürge

Basit bir robot süpürgeyi düşünelim. Performans ölçütü yalnızca temizlenen kare sayısıysa, sürekli hareket etmek makul bir politikadır. Ölçüte enerji tüketimi ve aşınma için bir ceza eklendiğinde aynı politika zayıflar; gereksiz hareket artık irrasyoneldir. Robot kiri yalnızca bulunduğu karede algılıyorsa bilgisi sınırlıdır; bir harita sensörü eklendiğinde daha iyi bir plan yapabilir. Sağa ve sola gitmenin yanına "beklemek" eylemi eklendiğinde ise seçenek kümesi değişir. Görünüşte aynı olan bir davranışa, performans ölçütünü ve bilgi koşullarını belirtmeden "rasyonel" diyemeyiz.

Rasyonel bir ajan yalnızca sonuca yönelik eylemler seçmez; bilgi toplamak ve öğrenmek için de eylem seçebilir. Karşıya geçmeden önce yola bakmak, gelecekteki kararı iyileştiren bir *bilgi toplama* eylemidir. Ajanın deneyimden öğrenerek tasarımcının verdiği başlangıç bilgisinin eksiklerini düzeltebilmesine ise *özerklik* denir.

Uygulama 1.1 · Navigasyon ajanı

Bir navigasyon uygulaması, karar anındaki trafik verisine göre en kısa sürecek rotayı seçiyor. On dakika sonra rotada beklenmedik bir kaza oluyor ve varış gecikiyor. Seçim rasyonel miydi?

Evet. Rasyonellik, sonuca göre değil, karar anındaki bilgiyle beklenen performansa göre değerlendirilir. "Sonuç kötü olduğuna göre karar da yanlıştı" demek rasyonelliği kusursuzlukla; "ajan kazayı önceden bilmeliydi" demek ise her şeyi bilmekle karıştırmaktır. Öte yandan uygulamanın kazadan sonra rotayı güncellememesi irrasyonel olurdu, çünkü bu durumda algı dizisi değişmiştir.

1.5 İşlevsel bir tanım: OECD yapay zekâ sistemi

Dört yaklaşım, yapay zekâ araştırmasının neyi hedeflediğini anlamak için güçlü bir çerçeve sunar. Ancak bir kurumun "Bu sistem yapay zekâ mı, hangi kurallara tabi?" sorusuna cevap vermesi için daha işlevsel bir tanıma ihtiyaç vardır. Ekonomik Kalkınma ve İşbirliği Örgütü (OECD) 2024'te güncellediği tanımda bunu sağlar. Tanımın sadeleştirilmiş hâli şöyledir:

Tanım 1.2 · Yapay zekâ sistemi (OECD, 2024)

Açık ya da örtük amaçlar doğrultusunda, aldığı girdilerden fiziksel veya sanal çevreyi etkileyebilecek **tahmin, içerik, öneri ya da karar** gibi çıktıları nasıl üreteceğini **çıkarsayan** makine tabanlı bir sistemdir. Farklı yapay zekâ sistemleri, kullanıma alındıktan sonra farklı özerklik ve uyarlanabilirlik düzeylerinde çalışır.

Tanımı ezberlemek yerine altı bileşenine ayırmak daha yararlıdır (Şekil 1.4):



Şekil 1.4 OECD tanımının altı bileşeni. Özerklik, akışın tamamının ne ölçüde insan müdahalesi olmadan işlediğini anlatır.

Tablo 1.2 OECD tanımının bileşenleri ve bir kredi karar destek sistemine uygulanışı

Bileşen	Sorduğumuz soru	Kredi karar destek sisteminde
Amaç	Sistem neyi başarmaya çalışıyor?	Temerrüt riskini yönetmek
Girdi	Hangi veri, bilgi ya da sinyali alıyor?	Başvuru ve geçmiş ödeme bilgileri
Çıkarım	Girdiden çıktıya hangi yöntemle geçiyor?	Bir skor modeli ya da kural işlemi
Çıktı	Ne üretiyor?	Risk puanı veya "incelemeye al" önerisi
Çevre	Çıktı nereyi etkiliyor?	Kredi süreci ve başvuru sahibi
Özerklik	İnsan müdahalesinden ne ölçüde bağımsız?	İnsan onayının derecesi

Tanımın en önemli özelliği, makine öğrenmesini zorunlu bir koşul hâline getirmemesidir. Girdilerden çıkarım yaparak öneri ya da karar üreten bazı kural tabanlı sistemler de bu çerçevede yapay zekâ sayılabilir. Uyarlanabilirlik yani kullanım sırasında öğrenmeye devam etme bulunabilir, ama zorunlu değildir. Sınırlar her zaman keskin de değildir: Basit bir kural sistemi bağlamına göre sıradan bir otomasyon ya da bir yapay zekâ sistemi olarak sınıflandırılabilir. Bu kitapta bir sisteme yapıştırdığımız etiketten çok, o etiketin gerekçesine önem vereceğiz.

Dikkat

OECD tanımını dört yaklaşımın yerine geçen "tek doğru tanım" gibi okumayın. Bu tanım, politika ve yönetişim için bir sistemin kapsamını belirlemeye yarayan işlevsel bir çerçevedir. Dört yaklaşım ise araştırmanın hangi başarı ölçütünü hedeflediğini anlatır. İkisi farklı soruları cevaplar.

Uygulama 1.2 · Hangisi yapay zekâ sistemidir?

Aşağıdaki üç sistemi OECD tanımının bileşenleriyle karşılaştıralım.

- *Her gece 02.00'de veritabanının yedeğini alan betik.* Sabit bir zaman tetikleyicisiyle çalışır; girdiden bir tahmin, öneri ya da karar çıkarsamaz. Otomasyondur.
- *Uzmanların yazdığı kurallardan çıkarım yaparak bir kredi başvurusu için "incelemeye al" önerisi üreten sistem.* Girdiden çıkarım yaparak öneri üretir; makine öğrenmesi kullanmasa da yapay zekâ sistemi kapsamında değerlendirilebilir.
- *Geçmiş satış kayıtlarından gelecek ayın talebini tahmin eden model.* Hem yapay zekâ sistemidir hem de veriden öğrendiği için makine öğrenmesi kullanır.

Bu ayrımları Bölüm 3'te otomasyon, yapay zekâ ve makine öğrenmesi arasındaki ilişki üzerinden daha ayrıntılı ele alacağız. Bir sonraki bölümde ise OECD tanımındaki bileşenleri bir akışa dönüştürüp, bir yapay zekâ sisteminin model dosyasından ibaret olmadığını göreceğiz.

Bölüm özeti

- *Zekâ* tek bir yetenek değildir; bu yüzden yapay zekâyı soyut bir "düşünüyor mu?" sorusu yerine görev, başarı ölçütü, çevre ve hata maliyeti üzerinden değerlendiririz.
- Russell ve Norvig'in çerçevesi iki eksenden oluşur: ölçüt (insan ya da rasyonellik) ve inceleme nesnesi (düşünme ya da davranma). Ortaya çıkan dört yaklaşım tarihsel aşamalar değil, farklı başarı ölçütleridir.
- Turing testi insan gibi davranmayı ölçer. Bilinç, anlama ya da insan gibi düşünme kanıtı değildir.
- İnsan gibi düşünme iddiası, son cevabın değil ara adımların, hataların ve sürelerin insanlarla karşılaştırılmasını gerektirir (GPS örneği).
- Rasyonel düşünme biçimsel geçerliliği hedefler; geçerlilik, öncüllerin doğruluğunu garanti etmez.
- Rasyonel ajan, eldeki kanıt ve ön bilgi altında performans ölçütünü en çok artırması *beklenen* eylemi seçer. Rasyonellik ne kusursuzluktur ne de her şeyi bilmek.
- Rasyonellik performans ölçütüne, ön bilgiye, eylem kümesine ve algı dizisine bağlıdır.
- OECD (2024) tanımı amaç, girdi, çıkarım, çıktı, çevre ve özerklik bileşenlerinden oluşur ve makine öğrenmesini zorunlu tutmaz.

Kaynaklar

Russell, S. ve Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4. baskı). Pearson. Bölüm 1.1 ve 2.2.

Turing, A. M. (1950). Computing machinery and intelligence. *Mind*, 59(236), 433–460.

Newell, A. ve Simon, H. A. (1961). GPS, a program that simulates human thought.

OECD (2024). *Explanatory Memorandum on the Updated OECD Definition of an AI System*. OECD Artificial Intelligence Papers, No. 8.

Stanford HAI (2026). *AI Index Report 2026*, Ekonomi bölümü.

2

BÖLÜM

Bir yapay zekâ sisteminin anatomisi

Bu bölümde

- Bir yapay zekâ sistemini amaç, girdi, model ve çıkarım, çıktı, etki ve geribildirim halkalarına ayırmak,
- kurumsal amaç, karar amacı ve model hedefi arasındaki farkı ve amaç uyumsuzluğunu bir örnekle açıklamak,
- verinin neden "dünyanın tarafsız kopyası" olmadığını ve veri sızıntısının ne olduğunu söyleyebilmek,
- tahmin, sınıf etiketi, sıralama, içerik ve eylem çıktılarını birbirinden ayırmak,
- anlamlı insan gözetiminin koşullarını ve geribildirim döngüsünün sistemi nasıl değiştirdiğini tartışmak,
- algoritma, model, yapay zekâ sistemi ve uygulama kavramlarını birbirine karıştırmadan kullanmak.

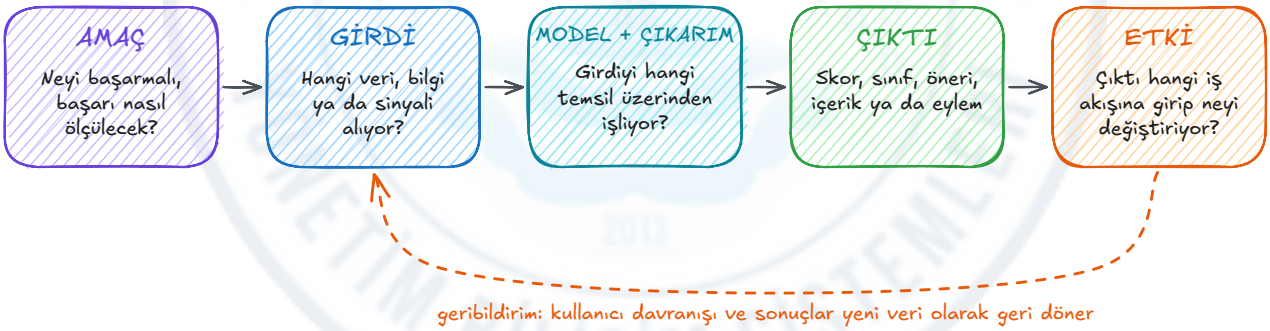
Kaynak: YBS 319 ders notları

Bir önceki bölümde OECD tanımını altı bileşene ayırmıştık. Bu bölümde aynı fikri bir adım ileri götürüp bir yapay zekâ sisteminin içini açacağız. Amacımız tek tek algoritmaları öğrenmek değil; hangi algoritma kullanılırsa kullanılsın her sistemde sorulması gereken soruları sabitlemek. Dönem boyunca karşımıza çıkacak her örneği, kredi skorundan Titanic verisinde yolcuların hayatta kalıp kalmadığını tahmin eden modele kadar, bu bölümde kuracağımız şemayla okuyacağız.

Derste bu şemayı bir otomobil benzetmesiyle açmıştık. Model sistemin motoruna benzer. Motor olmadan araç gitmez, ama motor tek başına otomobil de değildir. Yakıt hattı, direksiyon, fren, gösterge paneli ve sürücü olmadan motorun ürettiği güç bir yere ulaşmaz. Kurumsal bir yapay zekâ sisteminde de veri hattı, arayüz, kullanıcı rolleri, iş kuralları, güvenlik, izleme ve bakım modelin etrafında yer alır. Laboratuvarda çok iyi sonuç veren bir model, veri zamanında gelmediği ya da kullanıcı çıktıyla ne yapacağını bilmediği için sahada hiçbir şey değiştirmeyebilir. Yönetim bilişim sistemleri açısından ilginç olan da tam bu aradaki boşluktur.

2.1 Amaçtan geribildirime: bir sistemin akışı

Bir yapay zekâ sistemini incelerken beş halkalı bir zincir izleriz (Şekil 2.1). Önce **amaç** gelir: Sistem neyi başarmaya çalışıyor ve başarıyı nasıl ölçeceğiz? Sonra **girdi**: Sistem çevreden hangi veriyi, bilgiyi ya da sinyali alıyor? Üçüncü halkada **model ve çıkarım** vardır: Girdi, bir temsil üzerinden işlenir. Dördüncü halka **çıktıdır**: Sistem bir skor, sınıf, öneri, içerik, karar ya da eylem üretir. Beşinci halka **etki ve geribildirimdir**: Çıktı bir iş akışına girer, insanların ya da başka sistemlerin davranışını değiştirir ve bu değişiklik yeni veri olarak geri döner.



Şekil 2.1 Bir yapay zekâ sisteminin akışı. Çıktı süreci bitirmez; etki yeni veri üretir ve bir sonraki modelin girdisine dönüşebilir.

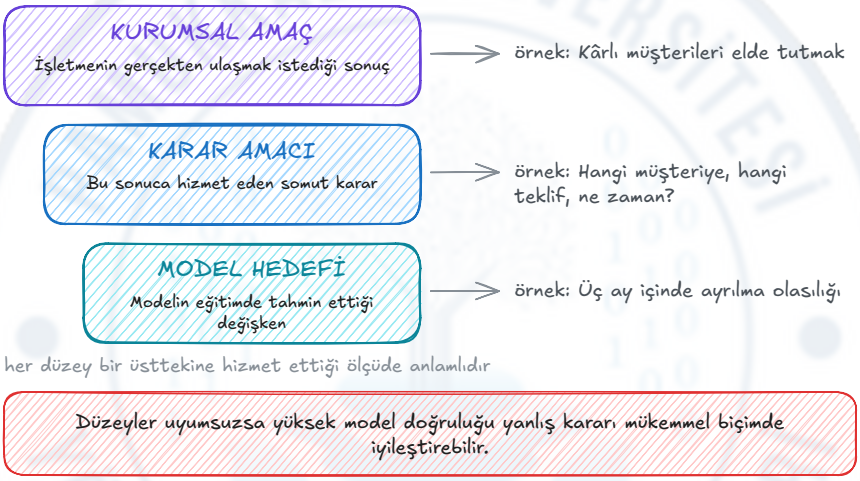
Son halkadaki geri dönüş oku şemanın en çok gözden kaçan parçasıdır. Bir öneri sistemi kullanıcıya hangi içeriği gösterdiğine karar verir; kullanıcı da büyük ölçüde gösterilen içerik arasından seçim yapar. Bu tıklamalar bir sonraki eğitim verisini oluşturur. Yani sistemin bugünkü çıktısı yarınki girdisini biçimlendirir. Bu yüzden yapay zekâ sistemini yalnızca bir model dosyası olarak düşünemeyiz. Bundan sonraki her örnekte aynı zinciri kuracağız: amaç, girdi, model, çıkarım, çıktı, etki ve geribildirim.

2.2 Amaç ve bağlam

Bir yapay zekâ projesinin ilk sorusu “hangi modeli kullanalım?” değildir. Önce neyi iyileştirmek istediğimiz açık olmalıdır: hız mı, maliyet mi, kalite mi, risk mi, yoksa hizmet düzeyi mi? Bu soru yalnızca teknik bir hedef seçimi de değildir. Kimin sorununu çözdüğümüzü, başarıyı kimin açısından ölçtüğümüzü, hangi maliyetleri hesabın dışında bıraktığımızı ve olumsuz sonuçların kimleri etkileyeceğini de belirler.

2.2.1 Üç düzeyde amaç

Pratikte “amaç” sözcüğü üç farklı düzeyi birlikte taşır ve bunları ayırmak gerekir (Şekil 2.2). **Kurumsal amaç**, işletmenin gerçekten ulaşmak istediği sonuçtur; örneğin kârlı müşterileri elde tutmak. **Karar amacı**, bu sonuca hizmet eden somut karardır; hangi müşteriye hangi teklifin ne zaman yapılacağı gibi. **Model hedefi** ise modelin eğitim sırasında tahmin etmeye çalıştığı değişkendir; örneğin müşterinin önümüzdeki üç ay içinde ayrılma olasılığı.



Şekil 2.2 Amacın üç düzeyi. Model hedefi karar amacına, karar amacı da kurumsal amaca hizmet ettiği ölçüde anlamlıdır.

Bu üç düzey uyumlu değilse yüksek model doğruluğu yanlış kararı mükemmel biçimde iyileştirebilir. Ayrılma olasılığını çok iyi tahmin eden bir model, en riskli müşterilere indirim gönderilmesine yol açabilir. Oysa bu müşterilerin bir kısmı ne yapılırsa yapılsın ayrılacaktır, bir kısmı da zaten kalacaktır. Kurumsal amaç açısından değerli olan, teklif sayesinde davranışı değişecek müşteriyi bulmaktır. Bu konuya Bölüm 5’teki müşteri kaybı örneğinde ayrıntılı olarak döneceğiz.

2.2.2 Amaç uyumsuzluğu

Ölçtüğümüz gösterge, gerçekte ulaşmak istediğimiz amacı her zaman temsil etmez. Bir çağrı merkezinde temsilcilerin başarısını yalnızca ortalama görüşme süresiyle ölçtüğümüzü düşünelim. Kısa görüşme genellikle verimli hizmetin işaretidir. Fakat süre tek hedef hâline geldiğinde temsilciler müşterinin sorunu çözülmeden görüşmeyi kapatmaya yönelebilir. Gösterge iyileşir, hizmet kalitesi düşer. Ölçülen hedef ile kurumun gerçek niyeti arasındaki bu farka **amaç uyumsuzluğu** diyoruz.

Aynı örüntü pek çok yerde görülür. Hastanede yalnızca yatak devir hızını artırmayı hedeflemek bakım kalitesini hesabın dışında bırakabilir. Bir içerik platformunda yalnızca izlenme süresini en üst düzeye çıkarmak, kullanıcının memnuniyetini değil bağımlılığını ödüllendirebilir. Burada modelin "hata" yaptığı söylenemez; model kendisine verilen hedefi gayet iyi optimize etmektedir. Sorun hedefin kendisindedir. Bu yüzden model metriği ile iş sonucu ayrı ayrı izlenir.

Goodhart yasası
İngiliz iktisatçı Charles Goodhart'ın adıyla anılan sezgi: Bir gösterge hedef hâline geldiğinde, ölçmesi beklenen şeyi temsil etme gücünü yitirmeye başlar.

> **Örnek 2.1 · Çağrı süresini kısaltan asistan**
>
> Bir banka, müşteri temsilcilerine görüşme sırasında hazır yanıt öneren bir yapay zekâ asistanı kuruyor. Başarı ölçütü olarak ortalama görüşme süresi seçiliyor. İlk ay süre %18 kısalıyor. Ancak aynı dönemde aynı müşterinin bir hafta içinde yeniden araması da artıyor; çünkü öneriler sorunu kapatmaya değil, görüşmeyi kapatmaya göre seçiliyor. Doğru tasarımda süre ile birlikte ilk aramada çözüm oranı ve tekrar arama oranı da izlenmelidir.

2.2.3 Bağlam

Amaç kadar bağlam da önemlidir. Bağlam, modelin çalıştığı nüfus, zaman, mevzuat, ekonomik koşul ve iş sürecidir. Aynı model farklı bir şehirde, farklı bir müşteri grubunda ya da farklı bir mevzuat altında aynı sonucu vermez. Ekonomik koşullar ve kullanıcı davranışı zamanla değişir. Pandemi döneminde alışveriş alışkanlıkları birkaç hafta içinde değiştiğinde, önceki yılların verisiyle eğitilmiş talep tahmin modellerinin pek çoğu bir anda işe yaramaz hâle gelmişti.

Bu nedenle performans yalnızca modelin bir özelliği değildir; model ile kullanım ortamı arasındaki ilişkinin bir özelliğidir. Teknik testte iyi sonuç almak, gerçek ortamda kalıcı güvenilirlik garantisi vermez.

2.3 Girdi ve veri

Girdi yalnızca sayısal bir tablo değildir. Metin, görüntü, ses, sensör ölçümü, kullanıcı isteği, uzman bilgisi ve iş kuralları da girdi olabilir. Bir sigorta hasar sisteminde kaza fotoğrafı, eksperin notu, poliçe koşulları ve geçmiş hasar kayıtları aynı karara birlikte girer.

2.3.1 Veri dünyanın kopyası değildir

Veriyi dünyanın tarafsız bir kopyası olarak düşünmek yaygın bir hatadır. Veri, bir ölçme, kayıt ve seçme sürecinin ürünüdür. Hangi olayın kayda girdiği, hangi değişkenin nasıl ölçüldüğü ve hangi grubun görünmez kaldığı, modelin öğrenebileceği ilişkiyi baştan sınırlar.

Kredi örneği bunu açıkça gösterir. Bir banka geçmişte belirli bir müşteri grubuna çok az kredi vermişse, o gruptaki kişilerin krediyi geri ödeyip ödemeyeceğini hiçbir zaman gözlememiştir. Eğitim verisinde yalnızca kredi verilmiş kişilerin ödeme davranışı vardır. Model, gözlenmeyen grup hakkında güvenilir bir şey öğrenemez; daha kötüsü, "bu gruba az kredi verilmiş" bilgisini "bu grup risklidir" diye okuyabilir. Geçmiş kararlar bir eşitsizlik taşıyorsa model bu örüntüyü başarı ölçütü gibi öğrenir.

Etiketler de karar içerir. Bir e-postanın spam olup olmadığını, bir işlemin sahte olup olmadığını ya da bir başvurunun "iyi aday" olup olmadığını birileri belirlemiştir. Eksik ya da yanlış etiketler modelin öğrendiği ilişkiyi doğrudan bozar.

2.3.2 Verinin farklı rolleri

Makine öğrenmesinde aynı veri kaynağından gelen kayıtlar farklı roller üstlenir. **Eğitim verisi**, modelin ilişkileri öğrendiği örneklerdir. **Doğrulama ve test verisi**, modelin daha önce görmediği örneklerde nasıl çalıştığını ölçmek için ayrılır. **Kullanım anındaki yeni girdi** ise eğitilmiş modelden çıktı almak için verilen kayıttır. Bu ayrımı Bölüm 6 ve 7'de ayrıntılı işleyeceğiz; burada şimdilik bir tehlikeye dikkat çekelim.

> **Dikkat**
>
> **Veri sızıntısı**, eğitim sırasında modelin gerçek kullanımda erişemeyeceği bir bilgiye erişmesidir. Örneğin bir hastanın taburcu olup olmayacağını tahmin eden modelde "taburcu evrakı düzenlenme tarihi" değişkeninin bulunması. Bu değişken sonuç oluştuktan sonra bilinir. Model testte olağanüstü başarılı görünür, gerçek kullanımda ise işe yaramaz.

2.3.3 Veri bir kurumsal varlıktır

Veri yalnızca teknik bir hammadde değil, ekonomik ve hukuki bir varlıktır. Kimin sahibi olduğu, kimlerin erişebileceği, ne kadar süre saklanacağı, hangi hukuki dayanakla işlendiği ve nasıl korunacağı sistem tasarımının başında düşünülmelidir. Türkiye'de kişisel verilerin işlenmesi 6698 sayılı Kişisel Verilerin Korunması Kanunu'na tabidir; teknik olarak elde edilebilen her verinin bir modelde kullanılabileceği anlamına gelmez.

2.4 Model, çıkarım ve çıktı

Model, girdiler ile çıktılar arasındaki ilişkiyi temsil eden yapıdır. Bu temsil bir uzmanın yazdığı kurallar dizisi, doğrusal bir denklem, bir karar ağacı ya da milyonlarca parametreli bir sinir ağı olabilir. **Algoritma** ise modeli kurmak, aramak ya da kullanmak için izlenen adımlar dizisidir.

> **Tanım 2.1 · Çıkarım**
>
> Çıkarım, mevcut bir girdinin kurulu bir model üzerinden geçirilerek yeni bir çıktı elde edilmesidir. Çoğu üretim sisteminde her istek geldiğinde model yeniden eğitilmez; eğitilmiş model yeni girdiye yanıt verir. Üretken modellerde çıkarım, yeni içerik oluşturma sürecidir.

Bu üç kavramı tek bir örnek üzerinde ayıralım. Karar ağacı *algoritması*, geçmiş müşteri kayıtlarından bir ağaç kurar. Kurulan ağacın düğümleri ve eşikleri, örneğin "son üç ayda iki kez geç ödeme yapmışsa sağa, yapmamışsa sola", *modeldir*. Yeni gelen bir müşterinin bilgilerini ağaçta yukarıdan aşağı ilerletip bir risk sınıfına ulaşmak ise *çıkarımdır*. Kural tabanlı bir sistemde de durum benzerdir: Bilgi mühendisi kuralları yazar, çıkarım motoru bu kuralları mevcut olgulara uygular.

2.4.1 Çıktının türü

Çıktının türünü doğru adlandırmak, sistemin riskini anlamanın ilk adımıdır. Beş temel tür sayabiliriz:

- **Tahmin:** Bilinmeyen ya da gelecekteki bir değer. “Bu müşterinin ayrılma olasılığı %18.”
- **Sınıf etiketi:** Örneğin tanımlı sınıflardan birine atanması. “Bu işlem şüpheli.”
- **Sıralama veya öneri:** Alternatiflerin bir öncelik sırasına konması. “Bu ürün önce gösterilsin.”
- **İçerik:** Yeni üretilmiş metin, özet, görsel ya da kod. “İşte müşteriye yanıt taslağı.”
- **Karar veya eylem:** Sürece doğrudan müdahale. “Kartın erişimi engellendi”, “rota değiştirildi.”

Aynı model farklı arayüz ve iş kurallarıyla bambaşka bir etki yaratabilir. %82’lik bir risk olasılığı bir tahmindir. “İncelemeye gönder” ifadesi bir öneridir. “Başvuruyu reddet” ise sürece doğrudan müdahaledir. Model hiç değişmeden yalnızca bir eşik ve bir iş kuralı eklenerek tahmin karara dönüştürülebilir. Bu yüzden bir modelin teknik özelliklerinden sistemin gerçek etkisini çıkaramayız. Çıktının kim tarafından, hangi yetkiyle ve geri döndürülebilir biçimde kullanılıp kullanılmadığını ayrıca incelememiz gerekir.

2.5 İnsan, iş akışı ve geribildirim

2.5.1 Anlamlı insan gözetimi

“İnsan döngüde” ifadesi çoğu zaman ekranda bir onay düğmesi bulunmasıyla eşitlenir. Oysa bir çalışana yüzlerce kararı birkaç dakika içinde onaylatmak, görünüşte insan kontrolü olsa da gerçekte yüzeysel otomasyondur. Anlamlı gözetimi üç soruyla sınayabiliriz:

1. İnsan, sonucu değerlendirmek için gereken **bilgiye** sahip mi?
2. Karar için yeterli **zamanı** var mı?
3. Sistemin sonucunu değiştirme **yetkisi** bulunuyor mu?

Bu koşullardan biri eksikse insanın yalnızca onay düğmesine basması anlamlı kontrol sayılmaz. Buna bir de etkilenen kişinin itiraz edebileceği bir kanal eklenmelidir.

İnsanlar sistemle iki ters yönde hata yapabilir. **Otomasyon yanlılığında** kullanıcı, sistemin önerisini kendi bilgisiyle çelişse bile kabul eder; navigasyonun gösterdiği yolu, yolun kapalı olduğunu gördüğü hâlde izleyen sürücü gibi. **Otomasyon reddinde** ise kullanıcı sisteme güvenmediği için yararlı uyarıları da görmezden gelir. İyi tasarım, kullanıcıya sistemin ne zaman güvenilir olduğunu ve ne zaman olmadığını gösteren bilgiyi verir.

İki ters eğilim
Otomasyon yanlılığı: kullanıcının sistem önerisini gereğinden fazla doğru kabul etmesi. Otomasyon reddi: yararlı çıktının sürekli göz ardı edilmesi. İkisi de gözetimi işlevsiz kılar.

2.5.2 Geribildirim döngüsü

Sistemin kararı, gelecekte hangi verinin oluşacağını da etkiler. Öneri sistemi yalnızca popüler içeriği öne çıkarırsa kullanıcılar daha çok o içeriği görür, daha çok ona tıklar ve gelecekteki veri de aynı yönde biçimlenir. Az bilinen içerik hiçbir zaman şans bulamaz.

Daha ağır bir örnek kamu güvenliğinden verilebilir. Polis devriyeleri bir modelin “riskli” dediği bölgeye yönlendirilirse, o bölgede daha çok olay kayda geçer; çünkü orada daha çok gözlemci vardır. Bu kayıtlar bir sonraki modelde bölgeyi daha da riskli gösterir. Kendi kendini doğrulayan

bu döngü, verideki ilişkinin gerçek suç oranından değil devriye dağılımından kaynaklanmasına yol açar. Kredi ve işe alım kararlarında da benzer bir mekanizma işler: Kime kredi verdiğimiz, kimin geri ödeme davranışını gözlemleyebileceğimizi belirler.

Bu nedenle kullanım başladıktan sonra yalnızca doğruluk değil; veri dağılımı, kullanıcı tepkisi, farklı gruplar üzerindeki etki, itirazlar ve iş değeri birlikte izlenmelidir. Veri ve çevre değiştiğinde yeniden eğitim gerekebilir. Bazı durumlarda sistemi devreden çıkarmak en doğru yönetim kararıdır. Sistemi durdurma ve güvenli biçimde eski sürece geri dönme planı da tasarımın parçasıdır. Yapay zekâ sistemi tek seferlik bir proje değil, yaşam döngüsü boyunca yönetilen bir kurumsal varlıktır.

2.6 Üç sistem, aynı anatomi

Şimdi kurduğumuz şemayı birbirinden çok farklı üç sisteme uygulayalım (Tablo 2.1).

Tablo 2.1 Üç sistemin aynı halkalarla çözümlenmesi.

Sistem	Girdi	Model ve çıkarım	Çıktı ve kullanım
Film ve müzik önerisi	İzleme geçmişi, içerik özellikleri	Öğrenilmiş sıralama modeli	Gösterilecek içerik listesi
Kredi risk desteği	Başvuru bilgileri, geçmiş ödemeler	Skor kartı ya da öğrenilmiş model	Risk skoru ve inceleme önerisi
Üretken metin aracı	Kullanıcı isteği ve bağlam	Büyük dil modeliyle olasılıksal metin üretimi	İnsan incelemesine giden taslak

Öneri sisteminde çıktı yalnızca bir liste değildir; kullanıcının neyi göreceğini ve dolayısıyla gelecekteki davranış verisini de belirler. Ayrıca sıralamanın üzerine yaş sınırı ya da içerik güvenliği gibi iş kuralları eklenebilir. Kredi sisteminde aynı skorun otomatik bir karar mı yoksa uzmana yardımcı bir bilgi mi olduğu tamamen sistem tasarımına bağlıdır; skor, mevzuat ve yetki kurallarıyla birlikte kullanılır. Üretken metin aracında büyük dil modeli taslağı yazar, ama kaynak doğrulama ve insan onayı sistem düzeyinde eklenir. Taslağı doğrudan müşteriye göndermek ile bir çalışanın incelemesine sunmak aynı riski taşımaz.

Bu tabloyu okurken her satır için aynı dört soruyu sorabiliriz: Girdiyi kim üretti? Model neyi temsil ediyor? Çıktı hangi karara giriyor? Hata kimi etkiliyor?

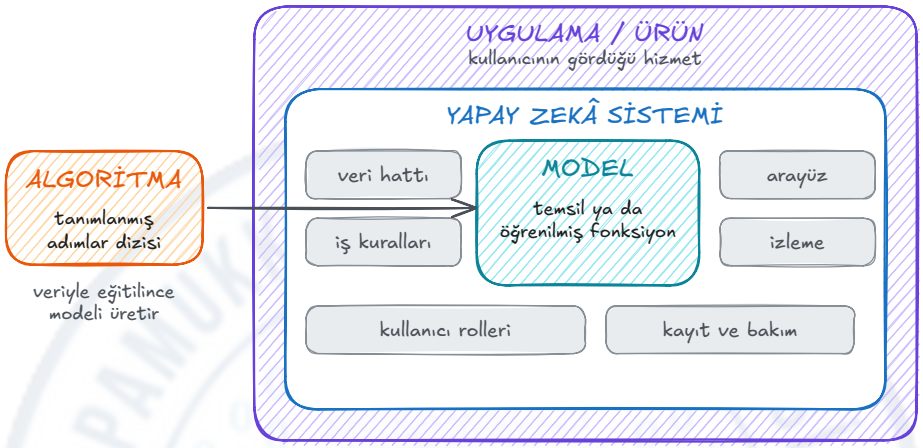
> **Dikkat**
>
> Ürünün adından yöntemini çıkarmaya çalışmayın. Tek bir ürün birden çok model, kural ve insan adımını birleştirebilir. “Akıllı” sözcüğü taşıyan bir uygulama yalnızca sabit kurallarla çalışıyor olabilir; sade görünen bir ekranın arkasında da birkaç farklı model bulunabilir.

Derste her örnek için alternatif bir mimari de sormuştuk. Kredi sistemi bütünüyle kurallarla kurulabilir mi? Yaş, gelir belgesi ya da kara liste kontrolü gibi uygunluk koşulları kolayca kurala dönüşür; fakat karmaşık geri ödeme örüntülerini elle yazmak çok daha zordur. İçerik önerisi

yalnızca “en çok izlenenleri göster” kuralıyla çalışabilir mi? Çalışabilir, ama kişiselleştirme sınırlı kalır. Bu karşılaştırma, teknik seçimin problemin yapısına göre değiştiğini gösterir ve bir sonraki bölümde ele alacağımız açık kurallar ile veriden öğrenme ayrımına köprü kurar.

2.7 Algoritma, model, sistem ve uygulama

Bu dört kavram gündelik dilde birbirinin yerine kullanılır, ama ayrı tutulmaları dönem boyunca pek çok karışıklığı önler (Şekil 2.3).



Şekil 2.3 Dört kavramın ilişkisi. Algoritma veriyle eğitilince bir model üretir; model, sistemin yalnızca bir parçasıdır; uygulama ise sistemi kullanıcıya sunar.

Algoritma, bir problemi çözmek ya da bir hesaplama yapmak için tanımlanmış adımlar dizisidir. Doğrusal regresyonda en küçük kareler yöntemi, karar ağacında her düğümde en iyi bölmeyi arayan prosedür birer algoritmadır.

Model, belirli bir görevde kullanılan temsil ya da öğrenilmiş fonksiyondur. Model insan tarafından kurulmuş kurallar içerebilir ya da veriden öğrenilmiş olabilir. Aynı eğitim algoritması farklı veri kümelerinde farklı modeller üretir: Aynı regresyon algoritması İstanbul’daki konut verisiyle başka, Denizli’deki konut verisiyle başka katsayılar bulur.

> **Tanım 2.2 · Yapay zekâ sistemi**
>
> Model, veri hattı, yazılım, iş kuralları, kullanıcı arayüzü, izleme, bakım ve kullanıcı rollerinin bir bütün olarak çalıştığı yapıdır. Veri alma, doğrulama, çıkarım servisi, kayıt tutma ve müdahale mekanizmaları sistemin parçasıdır.

Uygulama ya da ürün ise kullanıcının gördüğü ve belirli bir ihtiyacı karşılayan hizmettir. Aynı ürün arka planda birden fazla model ve sistem bileşeni kullanabilir.

Derste kullandığımız mutfak benzetmesiyle: Tarif algoritmaya, o tarifle hazırlanmış sos modele, mutfaktaki bütün araç gereç ve çalışma düzeni sisteme, müşteriye masada sunulan deneyim ise uygulamaya benzer. İyi bir sos tek başına iyi bir restoran demek değildir.

Güncel bir örnekle aynı ayrımı yapabiliriz. *Transformer* bir yöntem ailesi, yani bir mimaridir. Bu mimariyle büyük bir metin derlemi üzerinde eğitilmiş ağırlıklar bir modeldir. Bu modelin güvenlik filtreleri, kullanım kayıtları, arayüz ve hız sınırlamalarıyla birlikte çalışan yapısı bir sistemdir. Son kullanıcının telefonundan eriştiği sohbet hizmeti ise uygulamadır. İnternetten bir model dosyası indirmek, çalışan ve yönetilebilir bir yapay zekâ sistemi kurmak anlamına gelmez. Kurumsal değer, bu parçaların birlikte tasarlanmasıyla ortaya çıkar.

Uygulama 2.1 · İade sahteciliği sistemini anatomisine ayırmak

Bir e-ticaret şirketi, kötüye kullanılan iade taleplerini yakalamak için bir sistem kuruyor. Bu sistemi bölümdeki halkalara ayırın.

Amaç. Kurumsal amaç iade kaynaklı zararı azaltmak, ama dürüst müşteriyi kaybetmemek. Karar amacı, hangi iade talebinin elle incelemeye gönderileceği. Model hedefi, talebin sonradan kötüye kullanım olarak teyit edilme olasılığı.

Girdi. Sipariş geçmişi, iade sıklığı, ürün kategorisi, kargo durumu, iade gerekçesi metni. Geçmişte hiç incelenmemiş iadelerin gerçekte kötüye kullanım olup olmadığı bilinmiyor; etiketler yalnızca incelenmiş taleplerden geliyor.

Model ve çıkarım. Geçmiş teyitli vakalarla eğitilmiş bir sınıflandırma modeli, her yeni talep için bir skor üretiyor.

Çıktı. Skor kendi başına karar değil; belirli bir eşiğin üzerindeki talepler inceleme kuyruğuna öneri olarak düşüyor.

Etki ve geribildirim. Yalnızca yüksek skorlu talepler incelendiği için gelecekteki etiketler de yalnızca bu taleplerden gelecek. Düşük skorlu ama gerçekte sorunlu talepler hiç görülmeyebilir. Bunu dengelemek için düşük skorlu taleplerden de rastgele küçük bir örneklem düzenli olarak incelenmeli. Ayrıca inceleme ekibinin günlük kapasitesi eşiğin nerede durması gerektiğini doğrudan belirliyor.

Bölüm özeti

- Bir yapay zekâ sistemi amaç, girdi, model ve çıkarım, çıktı, etki ve geribildirim halkalarından oluşur. Çıktı süreci bitirmez; etki yeni veri üretir.
- Amaç üç düzeyde düşünülür: kurumsal amaç, karar amacı ve model hedefi. Bunlar uyumlu değilse yüksek doğruluk yanlış kararı iyileştirebilir. Ölçülen gösterge ile gerçek amaç arasındaki farka amaç uyumsuzluğu denir.
- Performans, model ile kullanım bağlamı arasındaki ilişkinin bir özelliğidir; nüfus, zaman, mevzuat ve süreç değiştiğinde değişebilir.
- Veri bir ölçme, kayıt ve seçme sürecinin ürünüdür. Görünmeyen gruplar, yanlış etiketler ve geçmiş eşitsizlikler modele taşınır. Sonuç oluştuktan sonra bilinebilen bir bilginin eğitimde kullanılmasına veri sızıntısı denir.
- Algoritma adımları, model temsili, çıkarım ise yeni girdiden çıktı üretilmesini anlatır.
- Çıktı tahmin, sınıf etiketi, sıralama, içerik ya da eylem olabilir. Aynı model farklı eşik ve iş kuralıyla farklı etkiye dönüşür.
- Anlamlı insan gözetimi bilgi, zaman, yetki ve itiraz kanalı gerektirir. Otomasyon yanlılığı ve otomasyon reddi gözetimi işlevsiz kılar.
- Algoritma, model, yapay zekâ sistemi ve uygulama farklı katmanlardır; bir model dosyası çalışan bir sistem değildir.

Kaynaklar

OECD (2024). *Explanatory Memorandum on the Updated OECD Definition of an AI System*. OECD Artificial Intelligence Papers, No. 8.

Russell, S. ve Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4. baskı). Pearson. Bölüm 2.

Goodhart, C. A. E. (1984). Problems of monetary management: The U.K. experience. *Monetary Theory and Practice* içinde (s. 91–121). Macmillan.

6698 sayılı Kişisel Verilerin Korunması Kanunu. Resmî Gazete, 7 Nisan 2016, Sayı 29677.

3

BÖLÜM

Temel ayrımlar ve kavramlar

Bu bölümde

- Otomasyon, yapay zekâ ve makine öğrenmesinin neden birbirinin alt kümesi olmadığını örneklerle göstermek,
- açık kural yaklaşımı ile veriden öğrenme yaklaşımını bilgi kaynağı, güncelleme, açıklama ve risk açısından karşılaştırmak,
- eğitim, çıkarım ve genelleme kavramlarını ayırmak; aşırı uyum ile yetersiz uyumu tanımlamak,
- yapay zekâ yeteneklerini problem türlerine göre adlandırmak, bilgi çıkarımını akıl yürütmeden ayırmak,
- tahmin, öneri ve karar arasındaki farkın riski nasıl değiştirdiğini açıklamak,
- yapay zekâ, makine öğrenmesi, derin öğrenme ve üretken yapay zekâyı doğru eksenlere yerleştirmek,
- dar, genel ve süper zekâ kavramlarını yöntem ekseninden ayırmak.

Kaynak: YBS 319 ders notları

Bu bölüme derste yaptığımız küçük bir yoklamayla başlayalım. Yapay zekâ, makine öğrenmesi, derin öğrenme ve üretken yapay zekâ arasındaki ilişkiyi kâğıda çizmeye çalışın. Hangisini hangisinin içine koyarsınız? Sembolik yapay zekâyı nereye yerleştirirsiniz? Otomasyonu bu resme nasıl eklersiniz?

Çoğumuz iç içe geçmiş halkalar çizeriz: en dışta yapay zekâ, içinde makine öğrenmesi, onun içinde derin öğrenme, en içte de üretken yapay zekâ. Bu çizimin bir kısmı doğrudur, bir kısmı yanıltıcıdır. Yanıltıcı olan kısım, birbirinden farklı soruların aynı eksene sıkıştırılmasından kaynaklanır. Otomasyon bir işin *nasıl yürütüldüğüyle*, makine öğrenmesi modelin *nasıl oluşturulduğuyla*, derin öğrenme belirli bir *yöntem ailesiyle*, üretken yapay zekâ ise sistemin *ne tür çıktı ürettiğiyle* ilgilidir. Bu bölümde bu soruları tek tek ayıracağız. Sonunda aynı çizime döndüğünüzde hangi halkaların gerçekten iç içe olduğunu, hangilerinin ayrı bir eksende durduğunu görebileceksiniz.

Bölüm boyunca dört soruyu ayrı ayrı soracağız:

- Sistem insan müdahalesini azaltıyor mu? Bu soru **otomasyonla** ilgilidir.
- Model örnek veriden mi kuruluyor? Bu soru **makine öğrenmesiyle** ilgilidir.
- Çok katmanlı sinir ağları kullanılıyor mu? Bu soru **derin öğrenmeyle** ilgilidir.
- Sistem yeni içerik mi üretiyor? Bu soru **üretken yapay zekâyla** ilgilidir.

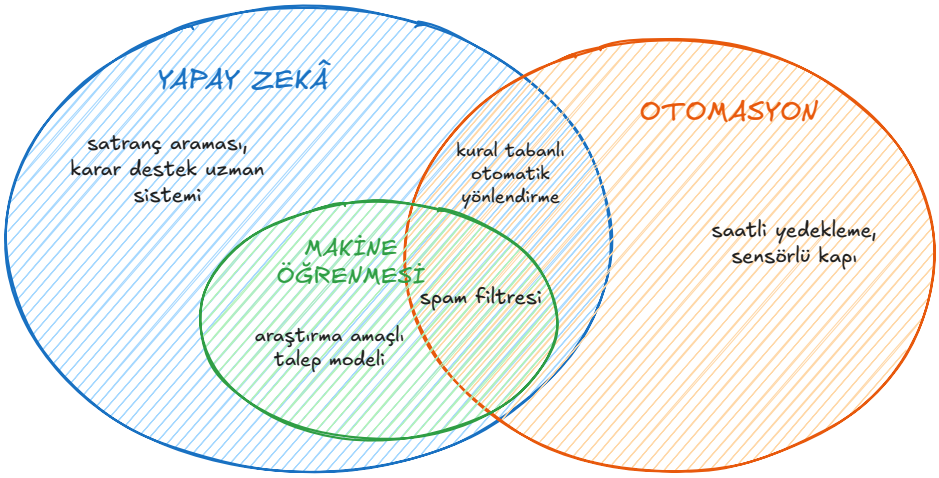
Aynı sistem bu soruların birkaçına birden “evet” diyebilir. Ama birine verilen cevap ötekinin cevabını belirlemez.

3.1 Otomasyon, yapay zekâ ve makine öğrenmesi

Otomasyon, bir işi insan müdahalesini azaltarak kurallar, yazılım ya da makinelerle yürütmektir. Her gece saat 02.00’de sunucu yedeği alan bir görev, belirli bir tutarın üzerindeki faturayı yöneticiye yönlendiren sabit bir kural ya da hareket sensörüyle açılan bir kapı otomatik çalışır. Bu sistemler son derece karmaşık olabilir ama yapay zekâ olmak zorunda değildir.

Yapay zekâ, algılama, çıkarım, arama, planlama, dil, öğrenme, tahmin ya da karar gibi görevleri yerine getiren sistemleri kapsayan daha geniş bir alandır. Bir uzman sistem ya da satranç arama programı veriden hiçbir şey öğrenmeden de yapay zekâ olabilir.

Makine öğrenmesi, her davranışın açıkça yazılması yerine verideki örüntülerden model oluşturan yöntemlerin genel adıdır. Sahtecilik tespiti, talep tahmini ve görüntü sınıflandırma tipik örneklerdir. Makine öğrenmesi yapay zekâ içinde önemli bir yöntem ailesidir.



Şekil 3.1 Üç kavramın ilişkisi. Otomasyon ile yapay zekâ örtüşür ama biri ötekini kapsamaz. Makine öğrenmesi yapay zekânın içinde bir yöntem ailesidir ve otomasyonla kesişebilir. Kesişimdeki spam filtresi hem öğrenilmiş bir modeldir hem de e-postayı otomatik olarak taşır.

Şekil 3.1'de görüldüğü gibi otomasyon ile yapay zekâ birbirini kapsayan halkalar değildir; makine öğrenmesi de otomasyonun içinde değil, onunla kesişen bir alandır. Her otomasyon yapay zekâ içermez: Saatli yedekleme buna örnektir. Her yapay zekâ otomatik karar vermez: Bir doktora olası tanıları sıralayan karar destek sistemi son sözü insana bırakır. Makine öğrenmesi de her zaman üretim otomasyonuna bağlanmaz: Bir analist yalnızca araştırma amacıyla bir model eğitip sonuçlarını bir rapora yazabilir.

Gerçek ürünlerde ise bu kavramlar sıkça örtüşür. Öğrenilmiş bir spam modeli gelen e-postayı otomatik olarak gereksiz klasörüne taşıdığında hem makine öğrenmesi, hem bir yapay zekâ görevi, hem de otomasyon vardır. Önemli olan bu bileşenleri ayrı ayrı gösterebilmektir. Bir sistemi incelerken sorulacak soru şudur: Çıktı süreci ne ölçüde otomatik olarak değiştiriyor?

3.2 Açık kurallar ve veriden öğrenme

Bir sisteme bilgiyi iki temel yoldan aktarabiliriz.

Açık kural yaklaşımında bilgi bir uzman ya da geliştirici tarafından kurallara dönüştürülür. "Sepet tutarı 500 TL'yi aşarsa kargo ücretsizdir" ya da "aynı kart son bir saatte üç farklı ülkeden kullanılmışsa işlemi durdur" gibi. Kural değiştiğinde sistem doğrudan güncellenir: Kampanya bitince eşik değiştirilir, ertesi gün yeni kural geçerlidir. Kararın hangi kural zincirinden çıktığını izlemek çoğu zaman kolaydır.

Veriden öğrenme yaklaşımında ise ilişkiyi tek tek yazmak yerine örneklerden model oluşturulur. Geçmişteki binlerce sahte ve gerçek işlemi gören bir model, hiçbir uzmanın açıkça tarif edemeyeceği örüntüleri yakalayabilir. Davranış değişikliği kural düzenlemesiyle değil, yeni veri ve yeniden eğitimle gerçekleşir.

Tablo 3.1 Açık kural yaklaşımı ile veriden öğrenme yaklaşımının karşılaştırılması.

	Açık kural yaklaşımı	Veriden öğrenme
Bilginin kaynağı	Uzman, mevzuat, süreç bilgisi	Örnekler ve örüntüler
Davranış nasıl değişir	Kural düzenlenir	Yeni veriyle yeniden eğitilir
Açıklama	Kural zinciri gösterilebilir	Modele göre zorlaşabilir
Uygun olduğu durum	Açık politika, mevzuat, az sayıda istisna	Elle yazılamayacak karmaşık örüntüler
Temel risk	Eksik, çelişkili ya da eskimiş kurallar	Yanlış ya da değişmiş veri, zayıf genelleme

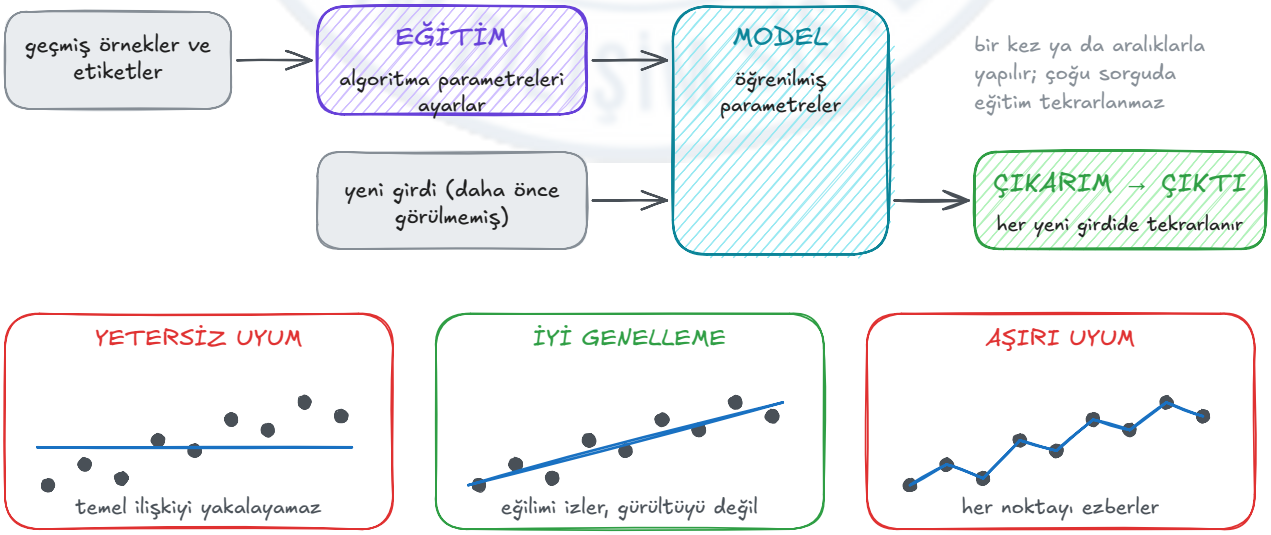
Bu iki yaklaşımı mutlak rakipler olarak görmemek gerekir. Bir kredi sisteminde yasal uygunluk koşulları (yaş sınırı, gelir belgesi, yasal takip kaydı) açık kurallarla uygulanırken geri ödeme riski öğrenilmiş bir modelle tahmin edilebilir. Kurumsal sistemlerin çoğunda kurallar ile öğrenilmiş modeller birlikte çalışır.

Dikkat

Kuralların okunabilir olması onların doğru ve tarafsız olduğunu göstermez. Bir uzmanın önyargısı ya da eskimiş bir politika açık bir kural olarak kodlanabilir. Tersine, öğrenilmiş bir modelin karmaşık olması onun bütünüyle açıklanamaz olduğu anlamına gelmez; tek bir kararı açıklayan yerel yöntemler, değişken önem analizleri ve benzer örneklerle karşılaştırma gibi araçlar vardır. Asıl soru hangi bilginin elde bulunduğu, değişimin nasıl gerçekleştiği ve hatanın nasıl denetleneceğidir.

3.3 Eğitim, çıkarım ve genelleme

Veriden öğrenen sistemlerde üç ayrı kavram sıkça birbirine karıştırılır (Şekil 3.2).



Şekil 3.2 Eğitim, çıkarım ve genelleme. Eğitim bir kez ya da aralıklarla yapılır; çıkarım her yeni girdide tekrarlanır. Genelleme ise modelin görmediği örneklerdeki başarısıdır.

Eğitim sırasında algoritma, örnek veriler ve bir hedef ölçüt kullanarak modelin parametrelerini ayarlar. Geçmiş e-postalar ve bunların spam etiketleriyle bir sınıflandırma modeli eğitmek buna örnektir.

Çıkarım sırasında eğitilmiş modele yeni bir girdi verilir ve model bir çıktı üretir. Gelen yeni e-postanın spam olma olasılığını hesaplamak çıkarımdır. Çoğu üretim sisteminde her sorgu geldiğinde eğitim yeniden yapılmaz.

Genelleme, modelin daha önce görmediği ama aynı problemden gelen örneklerde işe yarayan ilişkiyi kullanabilmesidir. Bu, modelin eğitim verisini ezberlemesinden çok farklıdır.

> **Tanım 3.1 · Aşırı uyum ve yetersiz uyum**
>
> **Aşırı uyumda** (*overfitting*) model eğitim verisindeki ayrıntıları ve gürültüyü de ezberler; eğitim başarısı yüksek, yeni örneklerdeki başarısı düşüktür. **Yetersiz uyumda** (*underfitting*) model eğitim verisindeki temel ilişkiyi bile yakalayamaz; hem eğitimde hem yeni örneklerde zayıftır.

Sınava hazırlanan iki öğrenci düşünelim. Biri geçmiş yılların sorularını cevaplarıyla birlikte ezberlemiştir; aynı sorular gelirse tam puan alır, soru biraz değişirse ne yapacağını bilemez. Bu aşırı uyumdur. Öteki konuya hiç çalışmamıştır ve her soruya aynı şıkkı işaretler. Bu da yetersiz uyumdur. İstediğimiz, konunun mantığını kavramış ve yeni bir soruyu da çözebilen öğrencidir.

Zaman da genellemeyi etkiler. Bir model eğitildiği dönemde iyi genelleyebilir ama veri dağılımı değiştiğinde performansı düşebilir. Bu nedenle model yalnızca eğitim başarısıyla değerlendirilmez. Ayrılmış test verisi, çapraz doğrulama ve kullanım sırasındaki izleme farklı aşamaların araçlarıdır; Bölüm 7'de bunları ayrıntılı göreceğiz. Ders boyunca tekrar tekrar soracağımız temel soru şudur: Model geçmişi mi tekrar ediyor, yoksa yeni durumda işe yarayan bir ilişki mi öğreniyor?

3.4 Yapay zekâ yetenekleri ve problem türleri

Yapay zekâ tek bir görev yapmaz. Bir projeye model adıyla değil, çözülmesi gereken görevin türüyle başlamak daha sağlıklıdır. Sık karşılaşılan problem türlerini şöyle sıralayabiliriz:

Algılama
: Girdide ne var? Bir üretim bandındaki kamera görüntüsünde hatalı ürünü bulmak.

Sınıflandırma
: Örnek hangi sınıfa ait? Bir kart işleminin sahte mi normal mi olduğuna karar vermek.

Tahmin
: Bilinmeyen ya da gelecekteki değer ne olacak? Gelecek ayın talebini öngörmek.

Sıralama ve öneri
: Hangisi önce gösterilmeli? Bir müşteriye ürün önermek.

Bilgi çıkarımı
: Metinde hangi varlıklar ve ilişkiler var? Bir sözleşmeden tarafları, tarihleri ve yükümlülükleri çekmek.

Akıl yürütme
: Eldeki bilgi ve kurallardan hangi sonuç çıkar? Bir başvurunun şirket politikasına göre incelemeye gidip gitmeyeceğini belirlemek.

Planlama
: Hangi eylem dizisi hedefe götürür? Bir fabrikanın haftalık üretim çizelgesini oluşturmak.

İçerik üretme
: Yeni bir örnek nasıl oluşturulur? Bir müşteriye yanıt taslağı ya da bir kod parçası yazmak.

Gerçek bir iş problemi çoğu zaman bu yeteneklerin birkaçını birlikte gerektirir. Bir müşteri hizmetleri sistemi gelen talebi sınıflandırabilir, ilgili prosedürü arayabilir, metinden sipariş numarasını çıkarabilir ve bir yanıt taslağı üretebilir. Otonom bir araç algılamayla çevredeki nesneleri tanır, tahminle diğer araçların hareketini öngörür, planlamayla rota ve manevra seçer, kontrolle de direksiyon ve fren gibi fiziksel eylemleri uygular. Yapay zekâ bu anlamda tek bir model türünden çok, yeteneklerin bir bileşimidir. Her yetenek de farklı bir ölçütle değerlendirilir: Sınıflandırmada hata oranı, öneride sıralama kalitesi, üretimde doğruluk ve uygunluk önem kazanır.

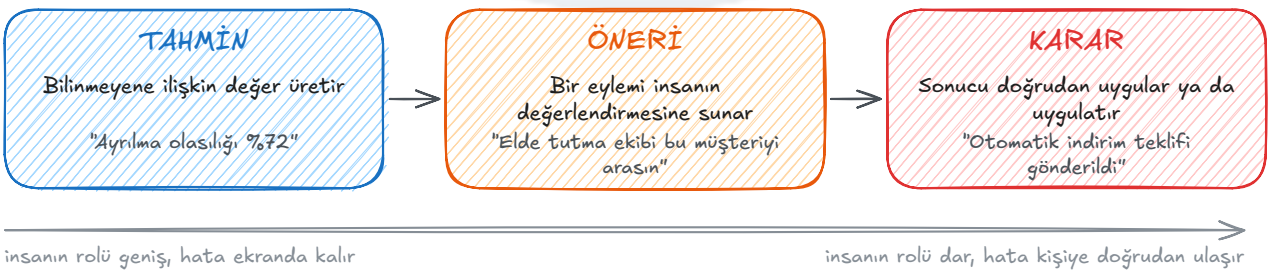
Örnek 3.1 · Bilgi çıkarımı mı, akıl yürütme mi?

Bir müşteri e-postasında adı, sipariş numarasını, ürün adını ve “kargo 12 gündür gelmedi” ifadesini bulmak **bilgi çıkarımıdır**; metinde zaten bulunan bilgiyi yapılandırır. Aynı e-postadaki teslimat süresini, şirketin “10 iş gününü aşan gecikmede kargo ücreti iade edilir” kuralını ve müşterinin üyelik statüsünü birlikte değerlendirip “müşterinin kargo ücreti iadesi hakkı var” sonucuna ulaşmak ise **akıl yürütmedir**; metinde açıkça yazmayan yeni bir sonuç üretir.

İçerik üretiminin de tek başına doğruluk garantisi taşımadığını not edelim. Üretken bir sistem yeni bir metin oluşturabilir; bu metnin doğru olup olmadığı ayrı bir sorudur.

3.5 Tahmin, öneri ve karar

Bu üç kavram teknik olarak aynı modelden çıkabilir, ama sonuçları çok farklıdır (Şekil 3.3).



Şekil 3.3 Tahmin, öneri ve karar. Model aynı kalsa bile çıktı sağa doğru ilerledikçe insanın rolü daralır ve hatanın etkisi büyür.

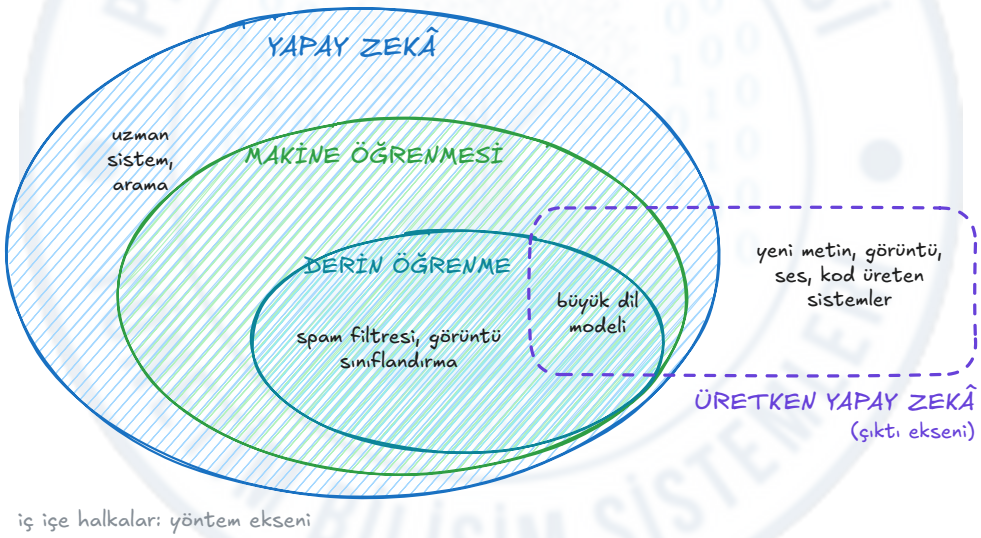
Tahmin, bilinmeyen bir değer ya da olay hakkında bir çıktı üretir: "Bu müşterinin ayrılma olasılığı %72." **Öneri**, bu tahmine ve başka ölçütlere dayanarak bir eylemi insanın değerlendirmesine sunar: "Bu müşteri elde tutma ekibince aranmalı." **Karar** ise sonucu doğrudan uygular ya da başka bir sisteme uygulatır: "Müşteriye otomatik indirim teklifi gönderildi."

Bu aşamalar arasında iş kuralları ve insan yargısı bulunabilir. Aynı %72'lik ayrılma olasılığı, yüksek değerli bir kurumsal müşteri için "hesap yöneticisi hemen arasın" önerisine, düşük değerli bir müşteri için "standart e-posta kampanyasına ekle" önerisine dönüşebilir.

Hatanın maliyeti bu zincir boyunca büyür. Yanlış bir tahmin ekranda bilgi olarak kaldığı sürece bir analist onu fark edip düzeltebilir. Aynı tahmin otomatik bir kampanyaya, fiyata ya da kredi reddine bağlandığında, çok sayıda kişiyi doğrudan ve hızla etkiler. Bu yüzden bir projeyi değerlendirirken iki soruyu ayrı ayrı sorarız: Model ne tahmin ediyor? Bu tahmin kimin eline gidiyor ve hangi eylemi tetikliyor?

3.6 Yapay zekâ, makine öğrenmesi, derin öğrenme ve üretken yapay zekâ

Şimdi bölümün başındaki çizime dönebiliriz (Şekil 3.4).



Şekil 3.4 İç içe halkalar ve ayrı eksen. Yapay zekâ, makine öğrenmesi ve derin öğrenme yöntem açısından iç içedir. Üretken yapay zekâ ise çıktı türünü anlatır ve bu halkaları kesen ayrı bir eksende durur.

Yöntem açısından iç içe halkalar çizmek doğrudur. Yapay zekâ en geniş alandır. **Makine öğrenmesi**, veriden model oluşturan yöntemler ailesi olarak yapay zekânın içinde yer alır. Makine öğrenmesinin kendi içinde farklı öğrenme düzenleri vardır: Doğru cevabın bilindiği örneklerden öğrenen *denetimli öğrenme*, etiketsiz veride yapı arayan *denetimsiz öğrenme* ve deneme yanılmayla ödül toplamayı öğrenen *pekiştirmeli öğrenme*. Bunları Bölüm 6'da daha ayrıntılı ele alacağız.

Derin öğrenme, çok katmanlı yapay sinir ağlarını kullanan bir makine öğrenmesi yaklaşımıdır. Gücü, görüntü, ses ve dil gibi yüksek boyutlu verilerde farklı soyutlama düzeylerinde temsilleri kendiliğinden öğrenebilmesinden gelir. Bir görüntüdeki kenarları, sonra şekilleri, sonra nesneleri katman katman öğrenir. Eskiden bu özelliklerin elle tasarlanması gerekiyordu.

Üretken yapay zekâ ise yeni metin, görüntü, ses, video ya da kod üreten sistemleri anlatır. Burada dikkatli olmak gerekir. Günümüzdeki üretken sistemlerin çoğu derin öğrenmeye dayanır, ama üretken yapay zekâ esas olarak bir *çıktı davranışını* tanımlar. Derin öğrenme ise bunun *nasıl* yapıldığıyla ilgili bir yöntem ailesidir. Bu iki kavram farklı eksenlerdedir:

- Bir spam filtresi derin öğrenme kullanabilir ama üretken değildir; çünkü bir sınıf etiketi üretir.
- Bir görüntü sınıflandırma ağı derin öğrenmedir ama üretken değildir.
- Bir müşteriye yanıt taslağı yazan sistem üretkendir.
- Sembolik bir planlayıcı yeni bir eylem planı üretebilir; ancak güncel kullanımda genellikle "üretken yapay zekâ" etiketiyle anılmaz.

Bu yüzden üretken yapay zekâyı her durumda dördüncü iç halka gibi çizmek kavramsal olarak tam doğru değildir. Ayrıca metin üreten bir sistemin bütün kararları derin bir sinir ağıyla vermesi de gerekmez; istemin işlenmesi, ilgili belgelerin aranması, kural filtreleri ve insan kontrolü aynı sisteme eklenebilir. Sistem mimarisini bileşenlerine ayırırken bu ayrım önem kazanır.

Akılda tutulacak cümle
Derin öğrenme bir şeyin nasıl yapıldığıyla, üretken yapay zekâ ne tür çıktı üretildiğiyle ilgilidir.

3.7 Sembolik yaklaşım ve yetenek kapsamı

3.7.1 Sembolik yapay zekâ

Sembolik yapay zekâ, kavramları, nesneleri, ilişkileri ve kuralları açık temsillerle, yani sistemin işleyebileceği sembollerle ifade eder. Mantıksal çıkarım, durum uzayında arama, planlama, uzman sistemler ve bilgi grafikleri bu geleneğin parçalarıdır. Bilgi, öğrenilmiş ağırlıklara dağılmak yerine adlandırılmış yapılarda durur. Bu nedenle bir sonucun hangi bilgi ve kurala dayandığı çoğu zaman izlenebilir.

Sembolik yaklaşım açık politika ve uzman bilgisinin belirleyici olduğu alanlarda etkilidir. Ancak gerçek dünyanın bütün bilgisini kurallara dönüştürmek zordur. Belirsiz ve çok yüksek boyutlu ortamlarda, örneğin bir fotoğraftaki yüzü tanımak için, bilgi tabanı kurmanın maliyeti hızla büyür. Ayrıca bir temsilin açık olması, insanın her ayrıntıyı kolayca anlayacağı anlamına da gelmez; on binlerce kurallı bir bilgi tabanı da karmaşıklaşır.

3.7.2 Dar, genel ve süper zekâ

Yöntem sorusundan tamamen ayrı bir soru da sistemin **yetenek kapsamıdır**:

Dar yapay zekâ
: Belirli bir görev ya da görev kümesinde çalışan sistemler. Bugün kullandığımız bütün sistemler bu gruptadır.

Genel yapay zekâ
: İnsanın farklı alanlar arasında bilgi aktarabilmesine benzeyen geniş ve esnek bir yetenek hedefi.

Süper zekâ
: Bilişsel alanların çoğunda insanı aşan varsayımsal sistemler.

Bu kavramları güncel ürün etiketleri gibi kullanmamak gerekir. Genel yapay zekâ için üzerinde uzlaşılmış tek bir test ya da eşik yoktur. Genel yapay zekâ da belirli bir yöntem ailesinin adı değil, bir yetenek iddiasıdır.

Dikkat

Yöntem ile yetenek kapsamı farklı eksenlerdir. Sembolik bir sistem dar bir görevde ya da geniş bir bilgi alanında çalışabilir; öğrenilmiş bir model de dar ya da çok amaçlı olabilir. Bu yüzden "sembolik eşittir dar, öğrenen eşittir genel" gibi bir sonuç çıkarılamaz.

Uygulama 3.1 · Sistemleri eksenlere yerleştirmek

Aşağıdaki sistemlerin her biri için dört soruyu cevaplayın: Otomasyon var mı? Makine öğrenmesi var mı? Derin öğrenme var mı? Üretken mi?

1. **Fatura onay akışı:** 10.000 TL üzerindeki faturalar otomatik olarak müdüre yönlendiriliyor. Otomasyon var; makine öğrenmesi, derin öğrenme ve üretkenlik yok. Bu bir yapay zekâ sistemi bile değildir.
2. **Kredi uygunluk uzman sistemi:** Mevzuat kurallarından bir başvurunun uygun olup olmadığını çıkarıyor, sonucu uzmana gösteriyor. Sembolik yapay zekâ; öğrenme yok; karar insanda olduğu için otomasyon sınırlı.
3. **Kamera ile hatalı ürün tespiti:** Evrişimli bir sinir ağı hatalı ürünü buluyor, bant bu ürünü otomatik olarak ayırıyor. Otomasyon, makine öğrenmesi ve derin öğrenme var; üretken değil.
4. **Müşteri hizmetleri yanıt taslağı:** Büyük dil modeli taslak yazıyor, temsilci düzenleyip gönderiyor. Makine öğrenmesi, derin öğrenme ve üretkenlik var; son eylem insanda.

Bölüm özeti

- Otomasyon işin nasıl yürütüldüğünü, makine öğrenmesi modelin nasıl kurulduğunu, derin öğrenme bir yöntem ailesini, üretken yapay zekâ ise çıktı türünü anlatır. Aynı sistem bu soruların birkaçına birden "evet" diyebilir.
- Her otomasyon yapay zekâ değildir; her yapay zekâ otomatik karar vermez; her yapay zekâ veriden öğrenmez.
- Açık kural yaklaşımında bilgi uzmandan gelir ve kural değiştirilerek güncellenir; veriden öğrenmede bilgi örneklerden gelir ve yeniden eğitimle güncellenir. Kurumsal sistemlerin çoğu ikisini birlikte kullanır. Okunabilir kural tarafsız kural demek değildir.
- Eğitim parametreleri ayarlar, çıkarım yeni girdiye çıktı üretir, genelleme ise görülmemiş örneklerdeki başarıdır. Aşırı uyumda model ezberler; yetersiz uyumda temel ilişkiyi bile yakalayamaz.
- Yapay zekâ yetenekleri algılama, sınıflandırma, tahmin, sıralama ve öneri, bilgi çıkarımı, akıl yürütme, planlama ve içerik üretmedir. Bilgi çıkarımı metinde olanı yapılandırır; akıl yürütme yeni bir sonuç üretir.
- Tahmin, öneri ve karar aynı modelden çıkabilir; karara doğru gidildikçe insanın rolü daralır ve hatanın etkisi büyür.
- Yapay zekâ ⊃ makine öğrenmesi ⊃ derin öğrenme ilişkisi yöntem eksenindedir. Üretken yapay zekâ bu halkaları kesen ayrı bir çıktı eksenidir.
- Dar, genel ve süper zekâ yetenek kapsamını anlatır; sembolik ya da öğrenen olmak ise yöntemle ilgilidir.

Kaynaklar

Russell, S. ve Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4. baskı). Pearson. Bölüm 1 ve 19.

Goodfellow, I., Bengio, Y. ve Courville, A. (2016). *Deep Learning*. MIT Press. Bölüm 1.

LeCun, Y., Bengio, Y. ve Hinton, G. (2015). Deep learning. *Nature*, 521, 436–444.

4

BÖLÜM

Yapay zekânın kısa tarihi

Bu bölümde

- Yapay zekânın beslendiği disiplinleri ve her birinin katkısını tanımak,
- 1950'den 2020'lere uzanan temel kırılmaları yöntem, altyapı ve sınır açısından okumak,
- sembolik ve öğrenen geleneklerin birbirini izleyen dönemler değil, baştan beri yan yana gelişen yaklaşımlar olduğunu göstermek,
- yapay zekâ kışlarının neden tek bir nedene indirgenemeyeceğini açıklamak,
- Deep Blue, AlexNet, AlphaGo ve Transformer örneklerinde başarının hangi bileşenlerden geldiğini ayırt etmek,
- güncel yükselişin beş nedenini ve tarihten çıkan yöntemsel ve kurumsal dersleri sıralamak.

Kaynak: YBS 319 ders notları

Bu bölümün amacı yılları ve araştırmacı adlarını ezberlemek değil. Derste de söylediğimiz gibi sınavda isim eşleştirmesinden çok, bir kırılmanın hangi fikri ya da imkânı değiştirdiğini açıklayabilmeniz önemlidir. Her dönüm noktasında üç şeye bakacağız:

- **Baskın yöntem:** O dönemde problemler nasıl çözülmeye çalışılıyordu?
- **Altyapı:** Bu yöntemi mümkün kılan veri, hesaplama ve kurumsal koşullar nelerdi?
- **Sınır:** Yöntem hangi noktada tıkandı ve bu tıkanma neyi görünür kıldı?

Yapay zekâ düz bir çizgide ilerlemedi. Bazı dönemlerde büyük vaatler ortaya çıktı, ardından teknik sınırlar ve uygulama maliyetleri görünür hâle geldi, ilgi ve finansman azaldı. Daha sonra yeni veri kaynakları, hesaplama gücü, yöntemler ve değerlendirme altyapısı bir araya gelerek yeni bir yükseliş oluşturdu. Bu döngüyü bilmek, bugün üretken yapay zekâ hakkında duyduğumuz iddiaları daha dengeli değerlendirmemize yardım eder.

4.1 Disipliner temeller

Yapay zekâ tek bir bilim dalından doğmadı. Russell ve Norvig alanın temellerini sekiz düşünce geleneği üzerinden anlatır:

Felsefe
: Doğru çıkarım nedir? Zihin ile beden nasıl ilişkilidir? Bilgi nereden gelir ve nasıl eyleme dönüşür?

Matematik
: Biçimsel mantık, hesaplanabilirlik, olasılık ve optimizasyon araçları.

Ekonomi
: Tercih, fayda, oyun kuramı ve belirsizlik altında karar verme.

Sinirbilim
: Beynin bilgiyi nasıl işlediği.

Psikoloji
: İnsan ve hayvan davranışının, algının ve öğrenmenin deneysel modelleri.

Bilgisayar mühendisliği
: Bu modellerin üzerinde çalışacağı hesaplama makineleri ve sistem altyapısı.

Kontrol kuramı
: Geribildirim, kararlılık ve zaman içinde amaçlı davranış.

Dilbilim
: Dilin yapısı ve anlam ile düşünce arasındaki ilişki.

Bu disiplinler yapay zekâya hizmet etmek için var olmuş alanlar değildir; her birinin kendi geniş soruları vardır. Yapay zekâ bu alanlardan fikir ve teknik almış, karşılığında onlara yeni deney ve model araçları sunmuştur. Bu kökenler, alanın neden yalnızca programlamaya indirgenemeyeceğini de açıklar.

YBS açısından bu listeyi tek bir karar destek sistemi üzerinde düşünebiliriz. Matematik tahmin ve optimizasyonu, ekonomi fayda ve teşvikleri, psikoloji kullanıcının sistem önerisine nasıl tepki verdiğini, dilbilim metin girdisini, bilgisayar mühendisliği ölçek ve altyapıyı, kontrol kuramı

geribildirim döngüsünü, felsefe ise bilgi ve sorumluluk sorularını taşır.

4.2 1950–1956: sorunun kurulması

Alan Turing'in 1950 tarihli makalesini Bölüm 1'de taklit oyunu üzerinden tanımıştık. Makalenin önemi yalnızca Turing testinde değildir. Turing aynı metinde öğrenen makineler fikrini, olası itirazları, dil ve hesaplamanın sınırlarını da tartışır. "Makineler düşünebilir mi?" sorusunu gözlenebilir davranışa çevirerek ölçme problemini dönüştürmüştür.

1955'te John McCarthy, Marvin Minsky, Nathaniel Rochester ve Claude Shannon, ertesi yaz Dartmouth Koleji'nde yapılacak bir çalışma için bir araştırma önerisi yazdılar. *Artificial intelligence* ifadesi bu kurucu belgede kullanıldı. Önerinin temel varsayımı, öğrenmenin ve zekânın diğer özelliklerinin ilke olarak bir makinenin benzetebileceği kadar kesin tarif edilebileceğiydi.

> **Dikkat**
>
> Dartmouth'u yapay zekânın bir günde doğduğu an gibi anlatmak doğru olmaz. McCulloch ve Pitts'in 1943'teki yapay nöron modeli, Hebb'in 1949'daki öğrenme kuralı, Turing'in hesaplama ve zekâ üzerine çalışmaları, Wiener'in sibernetiği ve erken oyun programları zaten vardı. Dartmouth'un önemi, dağınık çalışmaları ortak bir ad ve araştırma gündemi altında toplamasıdır. Toplantı beklenen ani atılımı üretmedi; ama kurduğu gündem kalıcı oldu.

4.3 Semboller, arama ve ilk öğrenen ağlar

1950'lerin ikinci yarısında iki farklı çizgi aynı anda gelişiyordu.

Birinci çizgi **sembolik problem çözmedir**. Allen Newell ve Herbert Simon'ın *Logic Theorist* programı (1956), matematiksel teoremleri semboller ve sezgisel arama kullanarak kanıtladı. Burada problem, olası durumlar ve bu durumlar arasında geçişi sağlayan işlemlerden oluşan bir *uzay* olarak görülür; sistem başlangıç durumundan hedefe giden yolu arar. Newell ve Simon'ın sonraki programı *General Problem Solver* (GPS) ise yalnızca doğru sonuca ulaşmayı değil, insanların alt hedefleri hangi sırayla ele aldığını taklit etmeyi de amaçlıyordu; bu yüzden Bölüm 1'deki "insan gibi düşünme" yaklaşımının erken bir örneğidir. Newell ve Simon daha sonra *fiziksel sembol sistemi hipotezini* formüle ettiler: Sembol işleyen bir sistem, genel zekâlı davranış için gerekli ve yeterli araçlara sahiptir. Bu gelenek planlama, oyun oynama ve bilgi temsili çalışmalarını uzun süre biçimlendirdi.

İkinci çizgi **örneklerden öğrenmedir**. Frank Rosenblatt'ın 1958'de tanıttığı *perceptron*, örnekleri gördükçe bağlantı ağırlıklarını ayarlayan erken bir öğrenme modeliydi. Arthur Samuel'in dama programı da kendi kendine oynayarak oyun kalitesini geliştiriyordu; "makine öğrenmesi" terimi de Samuel'le birlikte yaygınlaştı. Perceptron sinir ağları ve veriden öğrenme geleneği için tarihsel bir dönüm noktasıdır. Ancak ilk dönem beklentileri modelin gerçek yeteneklerinin çok önüne geçti; sınırlılıklar görünür olunca ilgi hızla geriledi.

Bu iki çizgiyi bir "sembolik dönem bitti, öğrenme dönemi başladı" anlatısıyla okumamak gerekir. İki gelenek başlangıçtan itibaren birlikte vardı. Bugünkü sistemlerin çoğu da arama, kural ve öğrenilmiş modelleri birlikte kullanır.

Kahramanlar savaşı değil
Sembolik temsil ile öğrenen ağlar yapay zekânın baştan beri yan yana duran iki geleneğidir. Donanım, veri ve teori koşulları hangisinin hangi dönemde görünür olacağını belirledi.

4.4 Ölçek sınırı, uzman sistemler ve kışlar

Erken sistemler küçük oyunlarda, teoremlerde ve basitleştirilmiş "mikrodünyalarda" etkileyici sonuçlar verdi. Bu başarılar geniş bir yetenek beklentisi doğurdu. Gerçek problemlere geçildiğinde ise iki darboğaz ortaya çıktı (Şekil 4.1).



Şekil 4.1 Erken başarılardan bakım darboğazına. Her aşama bir sonraki sorunun görünür olmasına yol açtı.

Birinci darboğaz **ölçekti**. Küçük bir problemde işe yarayan arama yöntemi, problem büyüdükçe *kombinatoryal patlama* yaşıyordu: Değerlendirilmesi gereken olasılık sayısı üstel biçimde artıyordu. Makine çevirisi, gündelik dilin ve dünya bilgisinin yüzeysel kelime eşlemeden çok daha karmaşık olduğunu gösterdi. ABD'de 1966 tarihli ALPAC raporu makine çevirisindeki ilerlemeyi yetersiz bulunca bu alandaki finansman daraldı. Birleşik Krallık'ta 1973 tarihli Lighthill raporu, yöntemlerin gerçek ölçekte kombinatoryal patlamaya takıldığını vurguladı. Minsky ve Papert'in 1969'da yayımlanan *Perceptrons* kitabı da tek katmanlı perceptronun çözemediği problem sınıflarını gösterdi.

> **Dikkat**
> Minsky ve Papert'in analizi *tek katmanlı* yapının sınırlarını gösteriyordu. Bunu "sinir ağları işe yaramaz" biçiminde genellemek yanlıştır; nitekim çok katmanlı ağlar bu sınırların bir kısmını aşar.

İkinci darboğaz **bilgiydi**. Genel amaçlı problem çözücüler, gerçek bir alanda yön verecek ayrıntılı bilgi olmadan çok geniş arama yapıyordu. 1970'lerde bu dersten yola çıkan **uzman sistemler** yükseldi. DENDRAL kimyasal analizde alan uzmanlığını aramayı daraltan kurallara dönüştürdü. MYCIN bakteriyel enfeksiyonların değerlendirilmesinde ve antibiyotik önerisinde yüzlerce kural ve kesinlik faktörleri kullandı. Edward Feigenbaum'un öne çıkardığı **bilgi mühendisliği** yaklaşımının temel sezgisi şuydu: Gerçek problemlerde alan bilgisi, genel amaçlı akıl yürütmeden çoğu zaman daha belirleyicidir.

1980'lerde uzman sistemler ticari bir dalgaya dönüştü. Fakat bu kez **bakım darboğazı** ortaya çıktı. Uzmanlardan bilgi edinmek pahalı ve yavaştı. Kural sayısı arttıkça çelişkileri ve istisnaları yönetmek zorlaştı. Sistemler dar bağlamlarının dışına çıktığında kırılganlaşıyordu. Ticari beklentiler gerçek kapasiteyi aşınca 1980'lerin sonunda ikinci bir gerileme yaşandı.

Bu gerileme dönemleri **yapay zekâ kışı** olarak anılır. Kışı tek bir başarısız deneye ya da tek bir tarihe bağlamak yanlıştır. Aşırı vaatler, sınırlı hesaplama, veri azlığı, ölçeklenemeyen yöntemler, bakım maliyeti ve finansman kararları farklı ülkelerde farklı zamanlarda birlikte etkili oldu. Yöntemler de bütünüyle kaybolmadı; kural tabanlı çıkarım ve optimizasyon farklı adlarla kurumlarda yaşamaya devam etti.

4.5 Öğrenmenin yeniden yükselişi

1986'da Rumelhart, Hinton ve Williams'ın *Nature*'da yayımlanan makalesi, çok katmanlı ağlarda hatanın çıktıdan geriye doğru yayılarak ağırlıkların sistematik biçimde ayarlanabileceğini etkili bir şekilde gösterdi. **Geri yayılım** (*backpropagation*) adı verilen bu fikrin kökleri daha eskidir, ama bu çalışma sinir ağları araştırmasını yeniden canlandıran tarihsel bir simge oldu. Gizli katmanların, görev için yararlı *iç temsiller* öğrenebileceği fikri güç kazandı.

1990'lardaki yükseliş yalnızca sinir ağlarından oluşmadı. Karar ağaçları, destek vektör makineleri, Bayesçi modeller ve olasılıksal yöntemler yaygınlaştı. Judea Pearl'ün olasılıksal ağlar üzerine çalışmaları belirsizlik altında akıl yürütmeye sağlam bir zemin kazandırdı. Belki daha da önemlisi, ortak bir değerlendirme kültürü oluştu: Yöntemler ortak veri kümelerinde, eğitim ve test verisi ayrılarak, ortak ölçütlerle karşılaştırılmaya başlandı. Belirsizlik, hata ve genelleme sayısal olarak ölçülür hâle geldi.

Buradaki ders, tek bir yöntemin bütün alanı devraldığı değildir. Öğrenmenin yeniden yükselişi veri, hesaplama, istatistik ve değerlendirme yöntemlerinin birlikte olgunlaşmasıyla gerçekleşti. Bugün de bir problemi çözerken yalnızca en yeni sinir ağına bakmayız; veri miktarı, açıklama gereksinimi, maliyet ve görev türü hangi yöntemin uygun olduğunu belirler. Bu kitapta öğrenen modellere doğrusal regresyonla başlamamızın nedeni de budur.

4.6 1997: Deep Blue

1997'de IBM'in Deep Blue sistemi, dünya satranç şampiyonu Garry Kasparov'u standart turnuva koşullarında oynanan altı oyunluk maçta yendi. Bu, kamuoyu gözünde makine zekâsının güçlü bir simgesi oldu. Ancak sistemin nasıl çalıştığına baktığımızda insan beynini taklit eden genel bir zihin görmüyoruz. Başarı dört bileşenin birleşimiydi:

- **Güçlü arama:** Çok sayıda olası hamle dizisini derinlemesine değerlendirdi.
- **Değerlendirme işlevi:** Bir tahta konumunun göreli değerini sayısal olarak hesapladı.
- **Satranç bilgisi:** Büyükustalardan gelen alan bilgisini aramaya dahil etti.
- **Özel donanım:** Saniyede yaklaşık 200 milyon konumu değerlendirebildi.

Deep Blue dar yapay zekâ kavramını çok iyi açıklar. Sistem satrançta insanı aşabiliyordu, ama satrancın dışındaki en basit gündelik soruna bile bilgi aktaramıyordu. Bölüm 1'deki dört yaklaşımla düşünürsek, Deep Blue insan gibi oynamak ya da insanın satranç düşüncesini taklit

etmek zorunda değildi; başarı ölçütü oyunu kazanmaktı. Bu yüzden *rasyonel davranma* yaklaşımına daha yakındır.

Aynı dış başarı, farklı iç süreç
Dar bir görevde insanı aşmak, insan gibi düşünmeyi ya da genel zekâyı gerektirmez. Etkileyici bir gösteri, yetenek kapsamı hakkında tek başına yeterli kanıt değildir.

4.7 2000'ler ve 2012: ölçek etkisi

2000'lerde altyapı köklü biçimde değişti. İnternet, dijital işlemler, sensörler ve kurumsal sistemler çok büyük veri kaynakları üretti. Grafik işlemciler (GPU), dağıtık hesaplama ve bulut hizmetleri büyük modellerin eğitimini erişilebilir kıldı. Açık kaynak kütüphaneler araştırma sonuçlarının uygulamaya geçişini hızlandırdı.

Fei-Fei Li ve ekibinin oluşturduğu **ImageNet**, milyonlarca etiketli görüntüden oluşan bir veri kümesi ve yıllık bir karşılaştırma yarışmasıydı. 2012'de Krizhevsky, Sutskever ve Hinton'ın **AlexNet** adlı derin evrişimli ağı bu yarışmada ilk beş tahmin hatasını yaklaşık %15'e indirdi; ikinci sıradaki yöntemin hatası %26 civarındaydı. Bu fark, alandaki pek çok araştırmacının yönünü değiştirdi.

Bu başarıyı yalnızca model mimarisine bağlamak eksik olur. Büyük ve etiketli bir veri kümesi, GPU üzerinde eğitim, derin evrişimli ağlar, aşırı uyumu azaltan teknikler ve ortak bir değerlendirme altyapısı birlikte etkiliydi. Ortak veri ve ölçütler sayesinde araştırmacılar sonuçlarını doğrudan karşılaştırabiliyordu. Yöntem, veri ve donanım birbirini güçlendirdiğinde bir performans sıçraması oluşabileceğini gösteren bu olaya **ölçek etkisi** diyoruz. Yine de belirli bir yarışmadaki başarı bütün görme problemlerini çözmez; veri dağılımı ve görev tanımı sonucu sınırlar.

4.8 2015–2016: derin öğrenme ve AlphaGo

2015'te LeCun, Bengio ve Hinton, derin öğrenmeyi farklı temsil katmanlarını ortak bir çerçevede ele alan bir derleme makalesiyle *Nature*'da özetlediler. Görüntü, konuşma ve dil görevlerindeki başarılar aynı yöntem ailesiyle ilişkilendirildi. Bu, derin öğrenmenin bütün makine öğrenmesinin yerini aldığı anlamına gelmez. Ağaç tabanlı yöntemler, doğrusal modeller ve sembolik yöntemler pek çok problemde hâlâ en uygun seçenektir; özellikle tablo biçimindeki kurumsal verilerde.

2016'da DeepMind'ın **AlphaGo** sistemi, Go ustası Lee Sedol'ü beş oyunluk maçta 4–1 yendi. Go, satrançtan çok daha geniş bir olasılık uzayına sahip olduğu için kaba kuvvet aramaya uygun değildir. AlphaGo'yu "saf bir sinir ağı" gibi anlatmak da eksik olur. Sistem, hangi hamlelerin umut verici olduğunu söyleyen bir *politika ağını* ve bir konumun ne kadar iyi olduğunu tahmin eden bir *değer ağını*, klasik bir arama yöntemi olan *Monte Carlo ağaç aramasıyla* birleştirdi. Ağlar önce uzman oyunlarından, sonra kendine karşı oynayarak öğrendi.

AlphaGo'nun dersi, farklı yaklaşım ailelerinin hibrit biçimde birleşebileceğidir. Öğrenilmiş değerlendirme, klasik aramanın hangi dallara odaklanacağını belirledi. Kurumsal sistemlerde de öğrenilmiş tahminler, kurallar, optimizasyon ve insan kararı birlikte kullanılabilir. Ve yine: Dar bir görevde olağanüstü başarı, genel dünya bilgisi ya da başka alanlara otomatik aktarım anlamına gelmez.

4.9 2017 ve sonrası: Transformer ve temel modeller

2017'de Vaswani ve arkadaşları, *Attention Is All You Need* başlıklı makalede **Transformer** mimarisini tanıttılar. Bu mimari, bir dizideki öğelerin birbirleriyle ilişkisini *öz dikkat* (*self-attention*) ağırlıkları üzerinden kurar. Dizisel veriyi kelime kelime sırayla işleme zorunluluğunu azalttığı için eğitim çok daha fazla paralelleştirilebildi. Güçlü sonuçlar ve ölçeklenebilirlik, sonraki büyük dil modellerine zemin hazırladı.

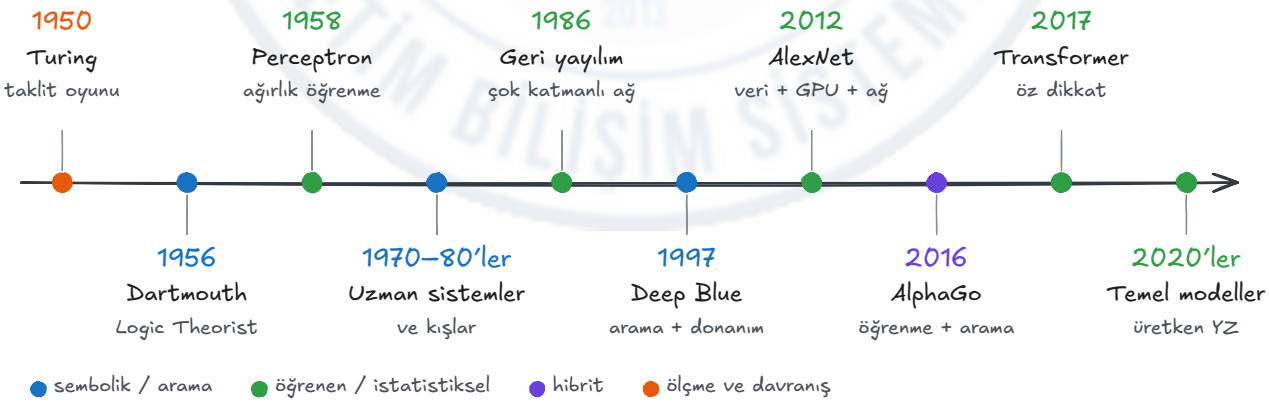
2020'lerde büyük ve çeşitli veri üzerinde geniş amaçlarla eğitilen, sonra istem, ince ayar ya da araç kullanımıyla farklı görevlere uyarlanabilen **temel modeller** yaygınlaştı. Metin, görüntü, ses ve kod üretimi doğrudan son kullanıcıya ulaştı.

> **Dikkat**
>
> Transformer bir mimaridir; büyük dil modeli daha geniş bir model türüdür; sohbet uygulaması ise kullanıcıya sunulan bir üründür. Bölüm 2'deki algoritma, model, sistem ve uygulama ayrımı burada da geçerlidir.

Bu yaygınlaşma yeni riskleri de görünür kıldı. Üretken bir modelin çıktısı akıcı olabilir, ama akıcılık doğrulanmış bilgi anlamına gelmez; model eğitim verisindeki örüntülerden olası bir devam üretir. Bu yüzden sistem düzeyinde bilgi erişimi, kaynak gösterme, kural denetimi ve insan incelemesi gerekebilir. Telif, mahremiyet, güvenlik, enerji maliyeti ve iş gücü üzerindeki etkiler de teknik performans kadar önemli hâle geldi.

4.10 Zaman çizelgesini okumak

Şekil 4.2'deki zaman çizelgesini tek tek tarihler olarak değil, problem kurma biçiminin nasıl değiştiğini gösteren bir dizi olarak okuyalım.



Şekil 4.2 Kısa zaman çizelgesi. Her nokta bir yöntemi, bir altyapıyı ya da bir sınırı görünür kılan kırılmadır.

Turing ölçme sorusunu davranışa çevirdi. Dartmouth alana ad verdi. Logic Theorist semboller ve aramayı, perceptron örneklerden ağırlık öğrenmeyi öne çıkardı. Uzman sistemler alan bilgisinin gücünü, kışlar ise aşırı beklentinin maliyetini gösterdi. Geri yayılım çok katmanlı öğrenmeyi

yeniden canlandırdı. Deep Blue arama, alan bilgisi ve özel donanımın birleşimini, AlexNet veri, GPU ve derin ağ birleşiminin ölçek etkisini kanıtladı. AlphaGo öğrenme ile aramayı birleştirdi. Transformer ve temel modeller geniş görev uyarlamasını ve içerik üretimini yaygınlaştırdı.

Çizelgede tekrarlayan bir desen var: Büyük vaatler oluşur, gerçek sınırlar görülür, ardından yeni yöntem, veri ve hesaplama birleşimi yeni bir uygulama dalgası yaratır. Ama bu çizelgeyi doğrusal bir ilerleme resmi gibi de okumamak gerekir. Sembolik arama yükselirken öğrenen ağlar da vardı; uzman sistemler güçlüyken istatistiksel yöntemler gelişiyordu; derin öğrenmenin yükselişi mantığı, aramayı ve olasılığı geçersiz kılmadı.

Derste vurguladığımız isimleri, temsil ettikleri fikirle birlikte şöyle özetleyebiliriz:

Alan Turing
Ölçülebilir davranış sorusu ve öğrenen makine gündemi.

John McCarthy
Alanın adlandırılması, Dartmouth, mantıksal bilgi temsili.

Newell ve Simon
Semboller, arama, bilişsel modelleme, fiziksel sembol sistemi.

Frank Rosenblatt
Perceptron ve örneklerden ağırlık öğrenme.

Edward Feigenbaum
Uzman sistemler ve bilgi mühendisliği.

Judea Pearl
Olasılıksal ve nedensel akıl yürütme.

Rumelhart, Hinton ve Williams
Geri yayılım ve iç temsil öğrenme.

Fei-Fei Li
ImageNet ve ortak karşılaştırma altyapısı.

Hinton, LeCun ve Bengio
Derin öğrenmenin uzun dönemli gelişimi.

Silver ve Hassabis
AlphaGo; öğrenme ile aramanın birleşimi.

Vaswani ve ekibi
Transformer ve öz dikkat.

4.11 Güncel yükselişin beş nedeni

Bugünkü yükselişi tek bir nedenle açıklayamayız. Birbirini besleyen beş unsur vardır:

1. **Dijital veri.** İşlemler, sensörler, metin ve görüntü arşivleri, kullanıcı etkileşimleri çok büyük veri kaynakları oluşturdu.
2. **Hesaplama.** GPU'lar, dağıtık sistemler ve büyük veri merkezleri karmaşık modelleri eğitmeyi ve hızlı çıkarım yapmayı mümkün kıldı.
3. **Yöntemler.** Temsil öğrenme, dikkat mekanizmaları ve optimizasyon teknikleri zaman içinde olgunlaştı.
4. **Yazılım ve bulut.** Açık kütüphaneler, programlama arayüzleri (API) ve hazır modeller küçük ekiplerin bile güçlü yeteneklere erişmesini sağladı.
5. **Kurumsal talep.** Tahmin, kişiselleştirme, karar desteği ve içerik üretimi için ekonomik kullanım alanları oluştu.

Bu unsurlar birbirinden bağımsız değildir. Veri olmadan öğrenen bir yöntem eğitilemez. Hesaplama olmadan büyük deneyler yapılamaz. Yöntemsel gelişme olmadan yalnızca daha çok veri ve donanım yeterli olmaz. Bulut altyapısı teknik kapasiteyi daha çok kuruma taşır, kurumsal talep de yatırım ve geribildirim üretir.

Her unsurun bir de maliyeti vardır. Veri toplama mahremiyet ve temsil sorunları yaratabilir. Yoğun hesaplama enerji tüketimi ve pazarın birkaç büyük şirkette yoğunlaşması sonucunu doğurabilir. Hazır servisler bağımlılık ve veri egemenliği riski taşır. Talep baskısı da sistemlerin

aceleyle kullanıma alınmasına yol açabilir. Yükselişi yalnızca “daha iyi bir algoritma bulundu” cümlesiyle değil, ekonomik ve kurumsal bir ekosistemin değişimi olarak okumak gerekir.

4.12 Tarihten çıkan dersler

Yöntemsel dersler. Yapay zekâ tek bir yöntem üzerinden ilerlemedi; genel arama, alan bilgisi, olasılık, öğrenme ve optimizasyon farklı dönemlerde eksik parçaları tamamladı. Sembolik akıl yürütme, istatistiksel öğrenme, arama ve optimizasyon bugün de birlikte var olabilir. Dar bir gösteride insanı aşmak genel zekâ kanıtı değildir. Ölçülebilen teknik yetenek ile gerçek kullanım değerini birbirine karıştırmamak gerekir.

Kurumsal dersler. Aşırı vaat ile gerçek kapasite arasındaki açıklık güveni ve yatırımı sarsar. Bir model başarılı olsa bile bakım, veri kalitesi, kullanıcı kabulü ve süreç değişimi başarısız olabilir. Bilgi edinme, veri hazırlama, donanım ve bakım maliyetleri çoğu zaman algoritmanın dışında kalır ve hesaba katılmaz. Büyük sıçramalar genellikle tek bir algoritmadan değil; yöntem, veri, donanım ve değerlendirme altyapısının birleşiminden doğar.

Bu tarih bizi karamsar yapmamalı; ama güncel iddialara ihtiyatlı yaklaşmayı öğretmelidir. Bir ürün etkileyici bir demo sunduğunda şu soruları sorarız: Hangi görevde, hangi veriyle, hangi ölçüt altında ve hangi maliyetle çalışıyor? Demo başarısı, üretim ortamında güvenilirlik demek değildir.

Bölüm özeti

- Yapay zekâ felsefe, matematik, ekonomi, sinirbilim, psikoloji, bilgisayar mühendisliği, kontrol kuramı ve dilbilimden beslenen disiplinler arası bir alandır.
- Turing (1950) zekâ sorusunu gözlenebilir davranışa çevirdi; Dartmouth önerisi (1955–56) alana adını verdi ve ortak bir gündem kurdu, ama alanı tek başına icat etmedi.
- Sembolik problem çözme (Logic Theorist, GPS) ile örneklerden öğrenme (perceptron, Samuel) aynı dönemde başladı; iki gelenek baştan beri yan yanadır.
- Kombinatoryal patlama ve bilgi eksikliği erken yöntemlerin sınırlarını gösterdi. Uzman sistemler (DENDRAL, MYCIN) alan bilgisinin gücünü kanıtladı ama bakım darboğazına takıldı.
- Yapay zekâ kışları aşırı vaat, hesaplama ve veri sınırı, ölçeklenemeyen yöntem, bakım maliyeti ve finansman kararlarının birlikte etkisidir.
- Geri yayılım (1986) çok katmanlı ağları yeniden canlandırdı; 1990'larda istatistiksel öğrenme ve ortak değerlendirme kültürü yerleşti.
- Deep Blue (1997) arama, alan bilgisi ve özel donanımı; AlexNet (2012) büyük veri, GPU ve derin ağı; AlphaGo (2016) öğrenme ile aramayı birleştirdi. Hiçbiri tek bir sihirli algoritmanın zaferi değildir.
- Transformer (2017) bir mimaridir; temel modeller onun üzerine kuruldu. Akıcı çıktı doğrulanmış bilgi demek değildir.
- Güncel yükselişin beş nedeni dijital veri, hesaplama, yöntemler, yazılım ve bulut ile kurumsal taleptir.

Kaynaklar

Russell, S. ve Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4. baskı). Pearson. Bölüm 1.2–1.3.

McCarthy, J., Minsky, M. L., Rochester, N. ve Shannon, C. E. (1955). *A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence*.

Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386–408.

Buchanan, B. G. ve Shortliffe, E. H. (1984). *Rule-Based Expert Systems: The MYCIN Experiments of the Stanford Heuristic Programming Project*. Addison-Wesley.

Rumelhart, D. E., Hinton, G. E. ve Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.

Krizhevsky, A., Sutskever, I. ve Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25.

LeCun, Y., Bengio, Y. ve Hinton, G. (2015). Deep learning. *Nature*, 521, 436–444.

Silver, D. ve diğerleri (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, 529, 484–489.

Vaswani, A. ve diğerleri (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.

5

BÖLÜM

İşletmede yapay zekâ: değer, sınırlar ve güven

Bu bölümde

- Yapay zekâyı veri, model, iş süreci, kullanıcı ve yönetişimden oluşan sosyoteknik bir sistem olarak tanımlamak,
- iş hedefinden sonuç ölçümüne uzanan altı halkalı zinciri bir örneğe uygulamak,
- bir işletme probleminin yapay zekâya uygun olup olmadığını sekiz soruyla sınamak,
- teknik performans ile işletme değerini ayırmak; doğruluk ölçütünün neden yanıltabileceğini sayısal bir örnekle göstermek,
- beş otomasyon düzeyini ve her birinin tipik riskini tanımak,
- yapay zekâ hakkındaki yaygın yanlış genellemeleri, temel sınırlılık türlerini ve NIST güvenilirlik özelliklerini sıralamak,
- bir sistemi dokuz soruluk bir çerçeveyle değerlendirmek.

Kaynak: YBS 319 ders notları

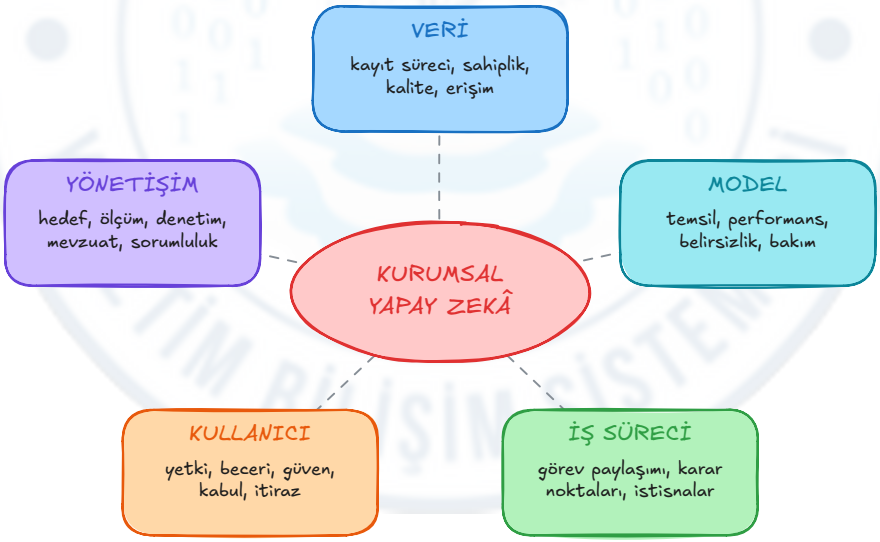
Şimdiye kadar bir yapay zekâ sisteminin ne olduğunu, hangi parçalardan oluştuğunu ve nereden geldiğini konuştuk. Bu bölümde konuyu yönetim bilişim sistemleri açısından ele alıyoruz. Yapay zekâ modeli teknik bir bileşendir. Kurumsal değer ise bu modelin veri, iş süreci, kullanıcı, karar ve yönetişimle birlikte çalışmasından doğar.

Bir YBS öğrencisinin temel sorusu yalnızca "hangi algoritma daha doğru?" değildir. Hangi iş kararını iyileştirmeye çalışıyoruz? Veri nereden geliyor ve kimin sorumluluğunda? Çıktı hangi kullanıcıya, ne zaman ve hangi arayüzle ulaşacak? İnsan sonucu nasıl değerlendirecek? Hata kimi etkileyecek? Performans ve iş değeri nasıl izlenecek? Bu sorular teknik sistem ile örgütsel sistem arasında köprü kurar.

Basit bir örnek: Bir talep tahmin modeli çok başarılı olabilir. Fakat satın alma birimi siparişlerini hâlâ geçen yılın rakamlarına bakarak veriyorsa stok kararı değişmez ve model hiçbir değer üretmez. YBS bakışı bu yüzden iki dili aynı anda kullanır. Teknik dil bize modelin girdisini, temsilini ve performansını anlatır. Yönetim dili karar sahibini, iş sürecini, maliyeti, sorumluluğu ve etkilenen kişileri anlatır. Bir yapay zekâ projesi ancak bu iki anlatım aynı sistem üzerinde birleştiğinde kurumsal bir çözüme dönüşür.

5.1 Yapay zekâ sosyoteknik bir sistemdir

Sosyoteknik sistem yaklaşımı, teknolojinin sonuçlarını onu kullanan insanlardan ve kurumdan ayrı düşünmez. Kurumsal bir yapay zekâ sisteminde beş bileşen birbirine bağlıdır (Şekil 5.1).



Şekil 5.1 Sosyoteknik sistem olarak kurumsal yapay zekâ. Bileşenlerden biri zayıfsa teknik olarak güçlü bir model değer üretmeyebilir ya da kurumsal risk yaratabilir.

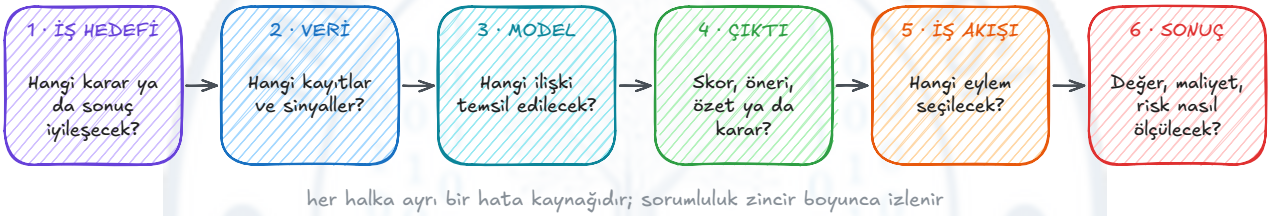
Veri tarafında kayıt süreçleri, sahiplik, kalite ve erişim önemlidir. **Model** tarafında temsil, performans, belirsizlik ve bakım vardır. **İş süreci**, görevin kimler arasında nasıl paylaşıldığını, karar noktalarını ve istisnaların nasıl yönetildiğini belirler. **Kullanıcının** yetkisi, becerisi, güveni, sistemi kabul edip etmemesi ve itiraz imkânı gerçek kullanım sonucunu değiştirir. **Yönetişim** ise hedefleri, ölçümü, denetimi, mevzuata uyumu ve sorumluluğu kurar.

Bu bileşenlerden herhangi biri değiştiğinde sistemin davranışı da değişir. Bir bankanın sahtecilik modelini düşünelim. Model hiç değişmese bile inceleme ekibinin kapasitesi azalırsa kuyruk uzar ve yüksek riskli işlemler zamanında ele alınamaz. Uyarı eşiği düşürülürse daha çok alarm üretilir; inceleme ekibi bunlara yetişemez ve daha çok dürüst müşterinin kartı gereksiz yere bloke edilir. Teknik doğruluk sabit kaldığı hâlde kurumsal sonuç değişmiştir. Bu yüzden sistem performansı yalnızca model metriğiyle değil; süreç süresi, kullanıcı davranışı, hata maliyeti ve etkilenen gruplar üzerinden de ölçülür.

Bir çağrı merkezi örneğinde de benzer durum vardır. Sistem çok iyi yanıtlar önerebilir. Ama önerinin hangi kaynağa dayandığı görünmüyorsa, temsilci öneriyi kontrol edemiyorsa ya da temsilcinin performansı yalnızca görüşme süresine göre ölçülüyorsa sistem yanlış davranışı teşvik eder. Sorun bu durumda modelde değil, sistemin diğer bileşenlerindedir. YBS'nin katkısı, teknik bileşenleri kurumsal süreçle birlikte tasarlamaktır.

5.2 Veriden karara uzanan zincir

Bölüm 2'deki akışı işletme açısından altı halkalı bir zincire dönüştürebiliriz (Şekil 5.2).



Şekil 5.2 Veriden karara uzanan zincir. Her halka ayrı bir hata kaynağıdır ve her halkada ölçülebilir bir soru sorulmalıdır.

1. **İş hedefi:** Hangi karar ya da sonuç iyileşecek?
2. **Veri:** Bu kararla ilişkili hangi kayıtlar ve sinyaller kullanılacak?
3. **Model:** Hangi ilişki ya da örüntü temsil edilecek?
4. **Çıktı:** Sistem skor, öneri, özet, içerik ya da karar mı üretecek?
5. **İş akışı:** İnsan ya da başka bir sistem bu çıktıyla hangi eylemi seçecek?
6. **Sonuç:** Değer, maliyet, risk ve etki nasıl ölçülecek?

Zincirin her halkası ayrı bir hata kaynağıdır. İş hedefi belirsizse doğru problem çözülmez. Veri eksikse model dünyayı yanlış temsil eder. Model ilişkileri hatalı öğrenebilir. Çıktı yanlış yorumlanabilir. İş akışı zamanında eylem üretmeyebilir. Sonuç yanlış ölçülürse sistemin yararlı olup olmadığı hiç anlaşılamaz. Veri vardır ama hedef belirsizdir; proje dağılır. Model iyidir ama çıktı geç gelir; karar değişmez. Çıktı kullanılır ama sonuç izlenmez; gerçek katkıyı bilemeyiz.

Zincir boyunca sorumluluk da izlenmelidir: Hedefi kim koydu, veriyi kim üretti, eşiği kim seçti, son kararı kim verdi ve olumsuz bir sonuçta düzeltmeyi kim yapacak?

Yapay zekânın değeri veriyi sihirli biçimde akıllı hâle getirmesinde değil, belirli bir iş bağlamında daha iyi bir karar ya da eylem üretmesindedir.

5.3 İşletme işlevlerinde yapay zekâ

Yapay zekâ işletmenin hemen her işlevinde kullanılabilir. Ancak iyi bir problem tanımı “yapay zekâ kullanalım” diye başlamaz; karar kalitesini sınırlayan bilgi ya da süreç eksikliğini bularak başlar. Bu yüzden her işlevde önce karar sorusunu, sonra yapay zekâ görevini ararız (Tablo 5.1).

Nedensellik uyarısı
Teklif yapılan müşterilerin çoğunun kalması, teklifin onları tuttuğunu tek başına göstermez. Belki ekip zaten kalma ihtimali yüksek müşterileri seçmiştir. Karşılaştırma grubu olmadan etki ölçülemez.

Tablo 5.1 İşletme işlevlerinde karar soruları ve ilgili yapay zekâ görevleri.

İşlev	Tipik uygulama	Karar sorusu
Pazarlama	Segmentasyon, öneri, terk tahmini	Hangi müşteriye hangi teklif?
Finans	Kredi riski, sahtecilik, nakit akışı	Hangi işlem önce incelenmeli?
İnsan kaynakları	Aday eşleştirme, iş gücü analitiği	Hangi beceri açığı geliştirilmeli?
Operasyon	Talep tahmini, çizelgeleme, kestirimci bakım	Hangi makine ne zaman bakıma alınmalı?
Tedarik zinciri	Stok, rota, teslimat süresi tahmini	Nerede ne kadar stok tutulmalı?
Müşteri hizmetleri	Talep sınıflandırma, arama, özet, yanıt taslağı	Talep hangi ekibe gitmeli?
Bilgi yönetimi	Kurumsal arama, bilgi çıkarımı	Çalışan hangi güncel prosedüre ulaşmalı?
Üst yönetim	Senaryo analizi, karar desteği	Alternatiflerin maliyeti ve riski ne?

Aynı tabloda hata maliyetlerinin çok farklı olduğuna dikkat edin. Bir film önerisindeki hata ile bir kredi ya da işe alım kararındaki hata aynı düzeyde yönetişim gerektirmez. İnsan kaynakları uygulamaları özellikle ayrımcılık ve açıklanabilirlik açısından yüksek dikkat ister. Yöntem seçimini işlevin adına değil, kararın yapısına ve etki düzeyine bağlamak gerekir.

Ayrıca her işlevde yapay zekâ gerekmeyebilir. Bir SQL sorgusu, iyi tasarlanmış bir gösterge paneli, bir süreç standardı ya da bir karar tablosu aynı ihtiyacı karşılayabilir. Gereksiz yere kurulmuş karmaşık bir model, kalıcı bir bakım maliyeti yaratır.

5.4 Yapay zekâya uygun problem seçimi

Her işletme problemi bir yapay zekâ problemi değildir. Bir problemi seçmeden önce şu sekiz soruyu sorarız:

1. Karar ya da görev açık biçimde tanımlanabiliyor mu?
2. Başarılı sonuç ölçülebiliyor mu?
3. Görev yeterince sık tekrarlanıyor mu, ya da tekil olsa bile yüksek değer taşıyor mu?
4. İlgili, güvenilir ve hukuka uygun veri mevcut mu?
5. Tahmin ya da öneri iş akışı tarafından zamanında kullanılabilir mi?

6. Yanlış pozitif ve yanlış negatif hataların maliyeti nedir?
7. Daha basit bir kural ya da raporlama çözümü yeterli olur mu?
8. İzleme, düzeltme, itiraz ve sorumluluk mekanizması kurulabilir mi?

Her soru bir başarısızlık biçimine karşılık gelir. Karar açık tanımlanamıyorsa eğitim etiketi ve başarı ölçütü de belirsiz kalır. Görev nadiren tekrarlanıyorsa model geliştirme maliyeti geri dönmeyebilir. Veri yoksa makine öğrenmesi yerine kural ya da süreç yeniden tasarımı düşünülmelidir. Hata maliyeti yüksekse otomasyon düzeyi düşürülebilir.

Altıncı soru ayrıca açıklanmayı hak ediyor. Yanlış pozitif ile yanlış negatif aynı etkiyi yaratmaz. Sahtecilik sisteminde normal bir işlemi durdurmak (yanlış pozitif) müşteri deneyimini bozar; sahte bir işlemi kaçırmak (yanlış negatif) doğrudan finansal zarar yaratır. Hangisinin daha pahalı olduğu, modelin nasıl ayarlanacağını belirler.

Dikkat

Bu sorulardan birkaçına "hayır" cevabı çıkması, daha büyük bir modele ihtiyaç olduğunu değil, önce problemin, verinin ya da iş sürecinin yeniden tasarlanması gerektiğini gösterir. Bir yapay zekâ projesine başlamama kararı da iyi bir yönetim kararı olabilir. Başarısız projelerin çoğu, modelin zayıflığından önce problem ve süreç tanımının zayıflığından kaynaklanır.

5.5 Model performansı ve işletme değeri

Teknik performans ile işletme değeri aynı şey değildir.

Teknik performans, modelin test verisindeki davranışını anlatır: doğruluk, hata, duyarlılık, kesinlik, sıralama kalitesi gibi ölçütler. Test verisindeki başarı, modelin *belirli koşullar altında* ne kadar iyi çalıştığını gösterir. Kullanım ortamı değiştikçe bu performans bozulabilir.

İşletme değeri ise gelir artışı, maliyet azalması, süre kısalması, hizmet kalitesi ya da risk azalması gibi sonuçlardaki değişimdir. İki katman arasında eşik, süreç, kullanıcı ve eylem bulunur. Çıktının zamanında doğru kişiye ulaşması, kullanıcının onu benimsemesi ve sürecin buna göre tasarlanması teknik skoru değere dönüştürür.

Örnek 5.1 · %99 doğruluk neden yanıltır?

Bir bankada her 1.000 kart işleminden 10'u sahtedir. Hiçbir şey öğrenmeyen ve her işleme "normal" diyen bir model düşünelim. Bu model 1.000 işlemin 990'ını doğru sınıflandırır:

$$\text{Doğruluk} = \frac{990}{1000} = \%99$$

Ama tek bir sahte işlemi bile yakalamaz. İşletme açısından değeri sıfırdır. Sınıflardan biri çok nadir olduğunda tek bir doğruluk oranı yanıltıcıdır; sahte işlemlerin ne kadarının yakalandığına (duyarlılık) ve "sahte" denen işlemlerin ne kadarının gerçekten sahte olduğuna (kesinlik) ayrıca bakmak gerekir. Bu konuya Bölüm 14'te sınıf dengesizliği başlığıyla döneceğiz.

Teknik ölçütler kendi aralarında da gerilim içindedir. Sahtecilik modelinin duyarlılığını artırmak, genellikle yanlış alarm sayısını da artırır. İnceleme kapasitesi sınırlıysa ek alarmlar gerçek zararı azaltmak yerine kuyruğu tıkar. Benzer şekilde talep tahmini hatası düşmüş olabilir; ama sipariş süreci değişmiyorsa fazla stok ya da ürün yokluğu azalmaz.

Bu yüzden değer iddiası her zaman bir karşılaştırma gerektirir: Sistem olmasaydı ne olurdu? Çevrimdışı model metriği, kontrollü bir pilot uygulama, süreç metrikleri ve gerçek iş sonucu birlikte izlenir. Uygun bir karşılaştırma grubu ya da önce-sonra tasarımı olmadan katkı yanlış yorumlanabilir.

5.6 Karar desteğinden tam otomasyona

Bir yapay zekâ çıktısı farklı otomasyon düzeylerinde kullanılabilir. Her düzeyin kendine özgü bir riski vardır (Şekil 5.3).



Şekil 5.3 Beş otomasyon düzeyi ve tipik riskleri. Sağa doğru gidildikçe insanın rolü daralır; ama bu bir olgunluk sıralaması değildir.

Bilgilendirme
: Sistem bilgi sunar, insan yorumlar. Örnek: talep tahmini panosu. Risk: yanlış güven ya da yanlış yorum.

Öneri
: Sistem alternatif önerir, insan seçer. Örnek: bakım önceliği önerisi. Risk: otomasyon yanlılığı; kullanıcının öneriyi sorgulamadan kabul etmesi.

Onaylı otomasyon
: Sistem hazırlar, insan onaylar. Örnek: taslak müşteri yanıtı. Risk: yüzeysel onay; kullanıcının yeterli zaman ya da bilgisi yoksa onay biçimsel kalır.

Koşullu otomasyon
: Düşük riskli durumlar otomatik işlenir, istisnalar insana gider. Örnek: açıkça spam olan e-postaları ayırmak. Risk: eşiğin ve istisna tanımının hatalı olması.

Tam otomasyon
: Sistem kararı doğrudan uygular. Örnek: çok düşük riskli, kolay geri alınabilir görevler. Risk: hatanın ölçeklenmesi ve sorumluluğun belirsizleşmesi.

Dikkat

Otomasyon düzeylerini bir olgunluk sıralaması gibi okumayın; daha fazla otomasyon her zaman daha iyi değildir. Otomasyon düzeyini teknik yetenek değil; kararın geri döndürülebilirliği, etkilenen kişinin hakları, hata maliyeti, zaman baskısı ve itiraz imkânı birlikte belirlemelidir. İnsan onayı eklemek de riski kendiliğinden çözmez: Onaylayan kişi yüzlerce kararı inceleyemiyorsa kontrol biçimseldir. Her düzey için durdurma, itiraz ve kayıt mekanizması tasarlanmalıdır.

5.7 Örnek olay: müşteri kaybı

Bölümün kavramlarını tek bir örnekte birleştirelim.

Uygulama 5.1 · Telekom şirketinde müşteri kaybı

Bir telekom şirketi abonelerinin ayrılmasını azaltmak istiyor. Yönetim "bir müşteri kaybı modeli yapalım" diyor. Bu cümle henüz yeterli bir problem tanımı değildir. Zinciri halka halka kuralım.

İş kararı. Hangi müşteriye, hangi teklifle, ne zaman ve hangi bütçeyle ulaşılacak? Model ancak bu karara hizmet ettiği ölçüde anlamlıdır.

Hedef değişken. "Ayrılma" nasıl tanımlanacak? Numara taşıma, sözleşmeyi yenilememe ve uzun süre hattı kullanmama farklı etiketlerdir ve farklı modeller üretir.

Girdi. Sözleşme süresi, kullanım miktarındaki değişim, ödeme gecikmeleri, şikâyet kayıtları, yaşanan hizmet kesintileri.

Eylem. En riskli görünen herkese indirim vermek hem bütçeyi bozar hem de adalet sorusu doğurur. Daha önemlisi, yüksek ayrılma olasılığı olan müşteri tekliften en çok etkilenecek müşteri olmayabilir. Zaten kalacak birine indirim vermek gelir kaybıdır; ne yapılırsa yapılsın ayrılacak birine teklif vermek de etkisizdir. Asıl hedef, teklif sayesinde *davranışı değişebilecek* müşteriyi bulmaktır.

Ölçüm. Sonucu değerlendirirken yalnızca aranan müşterilerin kaçının kaldığına bakamayız. Teklif yapılmayan benzer müşterilerden oluşan bir kontrol grubuyla karşılaştırma gerekir. Ölçülecek şey kampanyayı kabul etme oranı değil, gerçekten elde kalma ve bunun maliyetidir.

Görüldüğü gibi yapay zekâ projesi tahminden çok daha geniştir: veri mühendisliği, kampanya tasarımı, deney düzeni, bütçe, kullanıcı arayüzü, müşteriyle etik iletişim ve performans izleme aynı sistemin parçalarıdır.

5.8 Güven, sınırlar ve insan sorumluluğu

Bir sistem yüksek performans gösterse bile hangi koşullarda güvenilmez olabilir? Birkaç kısa durum düşünelim. Bir model %95 doğruluk veriyor, ama hatalarının büyük bölümü belirli bir müşteri grubunda toplanıyor. Bir dil modeli akıcı yanıtlar üretiyor, ama verdiği bilginin kaynağını gösteremiyor. Bir tıbbi görüntü sistemi ortalamada başarılı, ancak hastaneye yeni alınan cihazdan gelen görüntülerde bozuluyor. Bu örneklerin hiçbirinde tek bir performans sayısı güvenilirliği açıklamaz.

Doğruluk, belirli bir veri kümesindeki performansı anlatır. **Güvenilirlik** ise sistemin farklı koşullarda emniyetli, izlenebilir ve öngörülebilir biçimde çalışmasını da kapsar. Bir model test verisinde yüksek doğruluk gösterip yine de güvenilir bir sistem oluşturmayabilir: Kullanım verisi değişebilir, kötü niyetli kişiler girdileri manipüle edebilir, kullanıcı çıktıyı yanlış yorumlayabilir ya da sistem hata anında güvenli biçimde durmayabilir.

5.8.1 Yapay zekâ ne değildir

Derste yaygın yanlış genellemeleri sekiz maddede toplamıştık:

- **Sihir değildir.** Girdisi, hedefi, yöntemi ve hata koşulları incelenebilir ve ölçülebilir.
- **Her karmaşık yazılım değildir.** Büyük bir kod tabanı ya da zor bir algoritma tek başına yapay zekâ etiketi üretmez.
- **Her otomasyon değildir.** Sabit bir iş akışı öğrenme ya da anlamlı bir çıkarım içermeyebilir.
- **İnsan beyninin kopyası değildir.** İşlevsel başarı biyolojik benzerlik gerektirmez.
- **Bilinç kanıtı değildir.** Bir sistemin birinci tekil şahısla konuşması iç deneyime sahip olduğunu göstermez.
- **Her zaman doğru değildir.** Tahmin ve üretim belirsizlik ve hata taşır.
- **Tarafsız değildir.** Veri, hedef ve kullanım biçimi insan ve kurum tercihlerini yansıtır.
- **Her probleme uygun değildir.** Basit bir kural, iyi bir rapor ya da süreç düzeltmesi daha ucuz, daha açıklanabilir ve daha güvenli olabilir.

Bu listenin amacı teknolojiye ne kör bir hayranlıkla ne de gereksiz bir korkuyla yaklaşmamaktır. İyi mühendislik, teknolojinin sınırını doğru tanımlamakla başlar.

5.8.2 Temel sınırlılık türleri

Sınırlılıkları sekiz başlıkta toplayabiliriz. İlk dördü daha çok teknik düzeydedir:

Veri bağımlılığı
: Model, eğitim verisinde bulunmayan koşulları kendiliğinden bilemez.

Dağılım değişimi
: Kullanım ortamı, eğitim dönemindeki koşullardan farklılaşabilir.

Belirsizlik ve hata
: Skor ya da olasılık kesin gerçek değil, bir tahmindir.

Açıklama güçlüğü
: Karmaşık bir model, karar gerekçesini insan dilinde veremeyebilir.

Son dördü ise sosyoteknik düzeydedir:

Yanlılık ve adalet
: Hata oranları farklı gruplar için farklı olabilir; aynı hata oranı da farklı grupları farklı etkileyebilir.

Gizlilik ve güvenlik
: Veri sızıntısı, kötü niyetli girdilerle yanıltma ve kötüye kullanım riski vardır.

Amaç uyumsuzluğu
: Ölçülen hedef iyileşirken gerçek amaç bozulabilir (Bölüm 2).

İnsan etkenleri
: Otomasyon yanlılığı, aşırı güven ya da kullanıcı reddi sonuçları değiştirir.

Bu listeyi her sisteme aynı ağırlıkla uygulanan bir kontrol listesi gibi kullanmamak gerekir. Bir riskin önemi bağlama bağlıdır. Müzik önerisindeki kötü bir tahmin küçük bir rahatsızlık yaratır; yanlış bir sağlık önceliği ya da haksız bir kredi reddi çok daha ağır sonuç doğurur. Her sınırlılık için üç şey sorulur: Nasıl önlenecek, nasıl fark edilecek ve fark edildiğinde nasıl müdahale edilecek?

5.8.3 NIST güvenilir yapay zekâ çerçevesi

ABD Ulusal Standartlar ve Teknoloji Enstitüsü'nün (NIST) 2023'te yayımladığı Yapay Zekâ Risk Yönetimi Çerçevesi, güvenilir yapay zekâyı tek bir doğruluk sayısına indirgemez. Çerçeve yedi özellik tanımlar (Şekil 5.4).



Şekil 5.4 NIST çerçevesindeki güvenilir yapay zekâ özellikleri. Geçerlilik ve güvenilirlik diğerlerinin zeminidir; hesap verebilirlik ve şeffaflık hepsiyle ilişkilidir.

Geçerli ve güvenilir
: Tanımlı koşullarda tutarlı ve doğru çalışır.

Emniyetli
: İnsanlara, kuruma ve çevreye zararı azaltır.

Güvenli ve dayanıklı
: Saldırı, hata ve bozulmaya karşı dirençlidir.

Hesap verebilir ve şeffaf
: Roller, kayıtlar ve karar süreçleri izlenebilir.

Açıklanabilir ve yorumlanabilir
: Sonucun anlamı ve sınırları kullanıcıya aktarılabilir.

Mahremiyeti güçlendiren
: Kişisel veriyi koruyan tasarım ve kullanım uygular.

Adil ve zararlı yanlılığı yöneten
: Farklı gruplar üzerindeki etkileri izler ve yönetir.

Bu özellikler arasında gerilim olabilir. Daha ayrıntılı kayıt tutmak şeffaflığı artırır ama mahremiyet riskini de büyütebilir. Daha basit ve açıklanabilir bir model biraz performans kaybına yol açabilir. Veri minimizasyonu ile izleme ihtiyacı çatışabilir. Risk yönetimi bütün özellikleri aynı anda en üst düzeye çıkarma iddiası değil, bağlama göre dengeli kararlar almak, bu kararları belgelemek ve yaşam döngüsü boyunca yönetmektir.

5.8.4 Sorumluluk modelde kalmaz

İnsan sorumluluğu, modelin karar verip vermediğinden ibaret değildir. Amacı kim belirledi, veriyi kim seçti, eşiği kim onayladı, kullanıcı neyi değiştirebilir ve hata olduğunda kim müdahale eder? Sorumluluk bir model dosyasına devredilemez. Amaçları, veriyi, yetkileri ve kullanım koşullarını belirleyen insanlar ve kurumlar hesap vermeye devam eder.

Dört işlev
NIST çerçevesi risk yönetimini dört işlevle anlatır: yönetişimi kur (*Govern*), bağlamı haritala (*Map*), riski ölç (*Measure*) ve müdahaleyi yönet (*Manage*).

5.9 Bir sistemi değerlendirmek için dokuz soru

Kısım I boyunca kurduğumuz kavramları dokuz soruluk bir değerlendirme çerçevesinde toplayabiliriz. Bu soruları dönem boyunca karşılaşacağımız her sistem için kullanacağız:

1. Sistem hangi somut amacı izliyor ve bu amaç kime ait?
2. Girdi verisi nasıl üretildi; kimi temsil ediyor, kimi dışarıda bırakıyor?
3. Model hangi ortamda eğitildi, nerede kullanılacak?
4. Çıktı bir tahmin mi, öneri mi, karar mı, yoksa içerik mi?
5. Başarı hangi ölçütle ve hangi karşılaştırmayla değerlendirildi?
6. Yanlış bir sonuçtan kim, nasıl etkileniyor?
7. Kullanıcı sonucu anlayabiliyor, düzeltebiliyor ve itiraz edebiliyor mu?
8. Performans, güvenlik ve etkiler kullanım sonrasında izlenecek mi?
9. Sistem kapatıldığında ya da sağlayıcı değiştiğinde süreç çalışmaya devam edebilir mi?

İlk üç soru amacı, veriyi ve bağlamı; dördüncü soru yetki düzeyini; beşinci ve altıncı sorular teknik kanıtı ve hata dağılımını; yedinci soru sürecin denetlenebilirliğini; sekizinci soru zaman içindeki değişimi; dokuzuncu soru ise iş sürekliliğini sınar. Bu soruların amacı her projeyi durdurmak değil, belirsizliği görünür kılarak daha iyi bir tasarım yapmaktır.

Bölüm özeti

- Kurumsal yapay zekâ veri, model, iş süreci, kullanıcı ve yönetişimden oluşan sosyoteknik bir sistemdir. Model sabit kalsa bile kapasite, eşik ya da ölçüt değiştiğinde kurumsal sonuç değişir.
- Veriden karara zincir altı halkadan oluşur: iş hedefi, veri, model, çıktı, iş akışı ve sonuç. Her halka ayrı bir hata kaynağıdır; sorumluluk zincir boyunca izlenmelidir.
- İyi bir problem tanımı "yapay zekâ kullanalım" diye değil, karar kalitesini sınırlayan eksikliği bularak başlar. Bazen bir sorgu, rapor ya da kural yeterlidir.
- Uygun problem seçimi sekiz soruyla sınanır. Birkaç "hayır", daha büyük model değil yeniden tasarım ihtiyacını gösterir.
- Teknik performans ile işletme değeri farklıdır. Nadir sınıflarda doğruluk yanıltıcıdır; değer iddiası bir karşılaştırma grubu gerektirir.
- Beş otomasyon düzeyi bilgilendirme, öneri, onaylı otomasyon, koşullu otomasyon ve tam otomasyondur. Bu bir olgunluk sıralaması değildir; düzeyi geri döndürülebilirlik, haklar ve hata maliyeti belirler.
- Doğruluk belirli bir veri kümesindeki performanstır; güvenilirlik farklı koşullarda emniyetli, izlenebilir ve öngörülebilir çalışmayı da kapsar.
- NIST çerçevesi yedi güvenilirlik özelliği tanımlar; bunlar arasında gerilim olabilir ve bağlama göre dengelenir.
- Sorumluluk modele devredilemez; amaçları, veriyi, yetkileri ve kullanım koşullarını belirleyen insanlarda ve kurumlarda kalır.

Kaynaklar

NIST (2023). *Artificial Intelligence Risk Management Framework* (AI RMF 1.0). NIST AI 100-1. https://doi.org/10.6028/NIST.AI.100-1

Russell, S. ve Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4. baskı). Pearson. Bölüm 27.

KISIM II

Makine öğrenmesine giriş

6 Makine öğrenmesi problemini kurmak

7 Güvenilir bir öğrenme çalışması

6

BÖLÜM

Makine öğrenmesi problemini kurmak

Bu bölümde

- Mitchell'in görev, performans ve deneyim tanımıyla bir sistemin öğrenip öğrenmediğini sınamak,
- bir problemin makine öğrenmesine uygun olup olmadığını dört koşulla değerlendirmek,
- gözlem, özellik ve hedef kavramlarını bir veri tablosu üzerinde göstermek; hedef tanımının öğrenmeyi nasıl yönlendirdiğini açıklamak,
- bir iş sorusunu tahmin hedefi, karar anı ve kullanım biçimiyle bir öğrenme görevine çevirmek,
- denetimli, denetimsiz, yarı denetimli ve pekiştirmeli öğrenmeyi öğrenme sinyaline göre ayırmak; sınıflandırma ile regresyonu hedef türüne göre ayırmak,
- toplu ve çevrim içi öğrenme ile örnek temelli ve model temelli yaklaşımın ayrı sorulara cevap verdiğini göstermek.

Kaynak: YBS 319 ders notları

Kitabın ilk kısmında yapay zekânın geniş çerçevesini kurduk ve kural tabanlı yaklaşım ile veriden öğrenmeyi karşılaştırdık. Bu bölümde öğrenen sistemlere odaklanıyoruz. Temel sorumuz şu: Geçmiş örneklerden kurduğumuz bir model, daha önce görmediği yeni bir durumda nasıl işe yarar?

Bu bölüm bir algoritma kataloğu değildir. Bir makine öğrenmesi problemini kurabilmek için neyin tahmin edileceğini, hangi verinin gerçekten elde bulunduğunu, öğrenme sinyalinin nereden geldiğini ve başarının nasıl sınanacağını konuşacağız. Bölüm boyunca bir perakende zincirinin stok ve talep problemini örnek olarak tekrar tekrar kullanacağız. Böylece farklı kavramlar tanım düzeyinde kalmayacak, aynı problemin farklı yüzleri olarak görünecek.

6.1 Bir sistemin öğrendiğini nasıl anlarız?

Derse üç sistemle başlamıştık:

1. Her gün aynı eşik kuralını uygulayan bir stok alarmı: “Stok 20 birimin altına düşerse uyarı ver.”
2. Geçmiş satışlardan gelecek haftanın talebini tahmin eden bir model.
3. Satış tahminine göre otomatik sipariş açan bir iş akışı.

Birinci sistem deneyimle davranışını değiştirmez; yıl boyunca aynı eşiği uygular. İkinci sistem geçmiş satış örneklerinden bir ilişki kurar ve bunu yeni haftaya uygular. Makine öğrenmesi için güçlü bir adaydır. Üçüncü sistem ise iki bileşenden oluşur: öğrenen bir tahmin modeli ve onun çıktısını işleme dönüştüren bir otomasyon.

Buradan çıkan ders şudur: Öğrenmenin izini, sistemin davranışının veriye dayanarak değişmesinde ararız. “Akıllı” ürün adlarından çok, “Model hangi deneyimden ne öğrendi?” sorusu yararlıdır. Bir şeyi daha not edelim: Veri eklendi diye sistem kendiliğinden her an öğrenmez. Çoğu uygulamada eğitim ayrı, kullanım ayrı bir aşamadır.

6.1.1 Mitchell’in tanımı

Tom Mitchell’in 1997’deki ders kitabında verdiği tanım, bu sezgiyi kesin bir biçime sokar:

> **Tanım 6.1 · Makine öğrenmesi (Mitchell, 1997)**
>
> Bir bilgisayar programı, bir **T** görev sınıfı ve bir **P** performans ölçüsüne göre, **E** deneyiminden öğreniyor denir; eğer T’deki görevlerde P ile ölçülen performansı E deneyimiyle artıyorsa.



Şekil 6.1 Mitchell tanımının üç parçası. Üçü birlikte söylenmeden "model öğreniyor" cümlesi eksik kalır.

Bu tanımın üç parçasını (Şekil 6.1) her zaman birlikte söylemeye çalışacağız. Spam filtresinde **görev** e-postaları spam ya da normal olarak sınıflandırmaktır; **performans** doğru sınıflandırılan e-postaların oranıdır; **deneyim** geçmişte kullanıcıların işaretlediği e-posta örnekleridir. Perakende örneğimizde görev, mağaza ve ürün bazında gelecek haftanın talebini tahmin etmektir. Deneyim, geçmiş satışlar, takvim, fiyat ve kampanya kayıtlarıdır. Başarıyı ise modelin eğitimde gördüğü haftalardan *farklı* haftalardaki tahmin hatasıyla ölçeriz. Yalnızca geçmişi iyi açıklayan bir denklem yeterli değildir.

Buradaki "ilişki öğrenmek" ifadesi, doğada değişmez bir yasa keşfettiğimiz anlamına da gelmez. Örnekler hangi dönemden ve hangi mağazalardan geldiyse model çoğunlukla o koşullarda işe yarayan düzenlilikleri yakalar. Fiyat değişimi, yeni bir rakip, bir kayıt hatası ya da müşteri davranışındaki bir değişim bu düzenliliği bozabilir. Bu yüzden bir modelin kapsamını açıkça söylemek gerekir.

6.2 Neden makine öğrenmesi?

Makine öğrenmesini tercih etmemiz için genellikle üç gerekçeden en az biri bulunur.

Kuralları yazmak zordur. Géron, spam filtresi örneğiyle her istenmeyen e-posta kalıbını ayrı bir kural olarak yazmanın zahmetini anlatır. Spam gönderenler sözcüklerini ve yöntemlerini değiştirdikçe kural listesi uzar ve bakımı zorlaşır. Öğrenen bir sistem ise yararlı örüntüleri örneklerden günceller. Binlerce ürün ve mağazanın satışı için tek tek eşik yazmak da çoğu zaman uygulanabilir değildir.

Koşullar değişir. Talep mevsime, kampanyaya ve bölgeye göre farklılaşır; müşteri davranışı zamanla değişir. Modeli yeniden eğitmek değişime uyumu kolaylaştırabilir. Fakat bu uyum kendiliğinden ve risksiz değildir: Yeni örneklerin güvenilir, karşılaştırılabilir ve zamanında gelmesi gerekir.

Veride keşfedilecek bir yapı vardır. Etiket olmadığında bile verideki benzerlikler ve olağandışı kayıtlar insan analizine yardımcı olabilir.

> **Dikkat**
>
> "Veri varsa makine öğrenmesi kullanırız" sonucunu çıkarmayın. Bir görev açık bir kuralla güvenilir ve ucuz biçimde çözülüyorsa, öğrenen bir model yalnızca ek karmaşıklık yaratır. Gerekçe verinin varlığı değil, görevin niteliği ve modelin yeni örneklerde değer üretme beklentisidir.

6.3 Bir problem öğrenmeye uygun mu?

Bölüm 5'te bir işletme probleminin yapay zekâya uygun olup olmadığını sekiz soruyla sınamıştık. Makine öğrenmesi için bu filtreyi dört koşula daraltabiliriz. Buradaki fark, eğitim örneklerinin ve karar anındaki girdilerin *gerçekten* elde bulunup bulunmadığına odaklanmamızdır.

1. **Tekrarlanan ve ölçülebilir bir görev bulunmalı.** Talep tahmini tekrar eden bir görevdir: Her hafta yeni örnek gelir ve geçmiş tahminlerin hatası ölçülebilir. Görevin kapsamı, tahmin ufku ve ölçüm birimi de açık olmalıdır: Günlük mü haftalık mı? Adet mi ciro mu?
2. **Örnekler kullanım koşullarını yeterince temsil etmeli.** Eğitim verisi yalnızca büyükşehir mağazalarından oluşuyorsa kırsal bir mağazaya genelleme güvenli değildir. Hiç kaydı olmayan yeni bir mağaza açılışında aynı güveni bekleyemeyiz.
3. **Gerekli girdiler karar anında elde bulunmalı.** Bir bilgiyi karar verildikten sonra öğreniyorsak model onu gerçek kullanımda kullanamaz. Geçmiş tabloda bulunması yeterli değildir.
4. **Hata maliyeti ve insan denetimi baştan belirlenmeli.** Fazla stok ile stokun tükenmesi farklı maliyetler doğurur. Bu fark, hangi performans ölçüsüne ve ne düzeyde insan onayına ihtiyaç olduğunu belirler.

6.4 Veri kümesinin temel dili

Bir veri tablosuna baktığımızda ilk sorumuz şudur: **Bir satır neyi temsil ediyor?** Satır bazen bir müşteri, bazen bir sipariş, bazen bir mağaza-gün ya da mağaza-ürün-hafta olabilir. Satırın birimi değiştiğinde öğrenme görevi de değişir.

Gözlem (örnek)
: Bir satırın temsil ettiği birim. Perakende örneğinde mağaza, ürün ve hafta birleşimi. Aynı ürünün farklı mağazalardaki ya da farklı haftalardaki kayıtları ayrı gözlemlerdir.

Özellik (girdi)
: O birim için **tahmin anında bilinen** açıklayıcı bilgi: fiyat, kampanya planı, mevsim, geçmiş satışlar.

Hedef (etiket)
: Öğrenilmesi istenen çıktı. Talep tahmini için sonraki haftada satılan adet; stok tükenmesini tahmin edeceksek evet ya da hayır.

Tablo 6.1, ders için oluşturduğumuz küçük bir örnektir; gerçek bir işletmenin verisi değildir.

Tablo 6.1 Her satır bir mağaza-ürün-hafta birimidir. İlk beş sütun hafta başında bilinen girdiler, son sütun geçmiş örnekler için öğrenme hedefidir.

Mağaza	Ürün	Hafta	Fiyat	Kampanya	Sonraki hafta satış
A	Su	36	18 TL	Evet	142
A	Su	37	18 TL	Hayır	119
B	Süt	36	31 TL	Evet	76

Model, geçmiş satırlardan girdiler ile son sütun arasındaki ilişkiyi öğrenir. Yeni bir mağaza-ürün-hafta satırında ise son sütun henüz bilinmez; model onu tahmin eder.

Bu tabloda iki ayrıntı dikkat çekiyor. Birincisi, "hafta" sütunu yalnızca 36 ve 37 sayılarından ibaret değildir. Arkasında tatil, hava durumu, kampanya ve dönem etkileri durur. İkincisi, gerçekleşmiş satış her zaman gerçek talebin mükemmel bir ölçüsü değildir. Bunu biraz açalım.

6.4.1 Özellik ve hedef farklı sorulara cevap verir

Etiket yalnızca sınıf adı değildir
"Etiket" sözcüğü bir kategori adını çağrıştırsa da sayısal bir hedef de denetimli öğrenmede etikettir. 142 adet satış da bir etikettir.

Özellik ile hedef arasındaki ayrım yalnızca tablonun sağı ile solu arasındaki ayrım değildir. Her biri farklı sorular sorar.

Bir **özellik** için şunları sorarız: Karar anında bu bilgiyi biliyor muyuz? İlişkiyi açıklamaya yardım ediyor mu? Kayıt güvenilir ve tutarlı mı? Örneğin gelecek haftanın sonundaki stok sayımı geçmiş veri dosyasında görünebilir; ama hafta başında sipariş kararı verirken bilinmez. Böyle bir sütunu modele verirsek laboratuvardaki performans yapay biçimde yükselir.

Derste sorduğumuz küçük bir soruyu burada tekrar edelim: Fiyat gelecek hafta değişecekse hangi fiyatı girdi yaparız? Cevap, karar anında bilinen *planlanmış* fiyattır. Gerçekleşmiş ve sonradan değişmiş fiyatı kullanmak sakıncalıdır.

Bir **hedef** için ise şunları sorarız: Neyi tahmin etmek istiyoruz? Sonuç nasıl ölçülüyor? Hedef iş amacını gerçekten temsil ediyor mu?

6.4.2 Hedef tanımı öğrenmeyi yönlendirir

Bir ürün 80 adet satıldıysa talebin 80 olduğunu hemen söyleyemeyiz. Raf saat 15.00'te boşalmış olabilir; günün kalan saatlerindeki talep hiç gözlenmemiştir. Bu durumda satış, gerçek talebin altında kalır. Bir mağazada satış düşük çünkü talep düşük; başka bir mağazada satış düşük çünkü stok erken bitmiş. Amacımız gerçek talebi karşılamaksa yalnızca satış adedini hedef yapmak eksik bir temsil olur. Sipariş iptalleri, iadeler ve kayıt gecikmeleri de hangi dönemde ve hangi işletme kuralıyla kaydedildiklerine göre sayıyı değiştirir.

Makine öğrenmesi, mevcut kayıtların tanımını öğrenir. Etiket yanlışsa, eksikse ya da iş amacının yalnızca bir vekiliyse, model yüksek test başarısıyla bile yanlış soruya iyi cevap verebilir. Bu yüzden hedef tanımı ve verinin üretim süreci teknik ekibin tek başına vereceği bir karar değildir; operasyon ekibiyle birlikte belirlenir ve belgelenir. Veri sütununun adı ile kararın amacı her zaman aynı şey değildir.

Örnek 6.1

Mağaza çalışanları stok sayımını hafta sonunda topluca sisteme giriyorsa, tabloda "günlük stok" diye görünen sütun gerçekte günlük bir ölçüm değildir. Pazartesi ile cuma arasındaki değerler ya boştur ya da cuma gününün sayısı geriye doğru kopyalanmıştır. Bu sütunla eğitilen bir model, gün içi stok hareketini öğrendiğini sanır ama aslında bir kayıt alışkanlığını öğrenir.

6.5 İş sorusundan öğrenme görevine

İş sorusu ile model hedefini ayrı ayrı kurmak gerekir. "Stok sorununu azaltalım" geniş bir amaçtır. Bunun içinden somut bir öğrenme görevi seçeriz (Şekil 6.2).



Şekil 6.2 İş sorusundan öğrenme görevine. Karar anı, hangi bilgilerin özellik olarak kullanılabileceğini sınırlar.

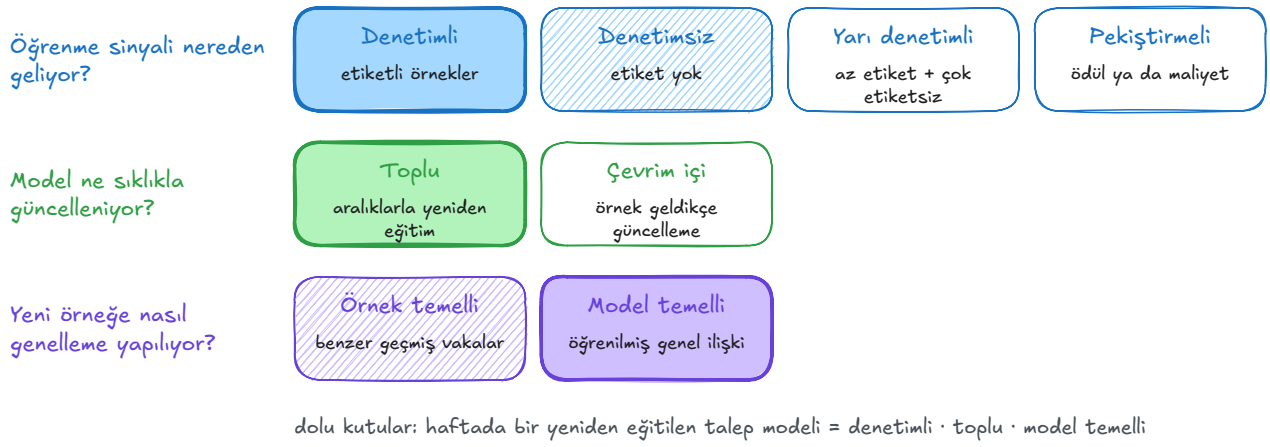
İş sorumuz "Hangi ürün ne zaman tükenebilir?" olsun. Bunu "Gelecek yedi gün içinde stok tükenir mi?" biçiminde bir tahmin hedefine çevirirz. Satır birimimiz mağaza-ürün-pazartesi olur. Karar anı her pazartesi sipariş verilmeden öncesidir. Kullanılacak özellikleri de bu ana göre sınırlarız: Pazartesi sabahı bilinmeyen hiçbir bilgi girdi olamaz.

Tahminin nasıl kullanılacağını da açık bırakmayız. Model riskli ürünleri bir liste hâlinde sıralayabilir; ama sipariş miktarını belirlemek başka bir karardır. Planlamacı tedarik süresini, tedarikçinin güvenilirliğini ve bütçeyi ayrıca değerlendirir. Hedef bir sınıflandırma etiketi olsa bile süreç yalnızca modelin sonucundan ibaret değildir.

Aynı veri kaynağıyla başka bir görev de kurabiliriz: "Gelecek hafta kaç adet satılır?" Bu kez hedef sayısal olur. Görev değiştiği için hedef tanımı, girdilerin zamanlaması ve başarı ölçütü yeniden kurulur.

6.6 Makine öğrenmesinin türleri

Makine öğrenmesi türleri çoğu zaman tek bir liste hâlinde ezberlenir: denetimli, denetimsiz, yarı denetimli, pekiştirmeli, çevrim içi, örnek temelli... Bu liste yanıltıcıdır, çünkü terimlerin her biri farklı bir soruya cevap verir (Şekil 6.3).



Şekil 6.3 Üç ayrı soru, üç ayrı eksen. Bir sistem her eksende ayrı bir yerde durur.

- **Öğrenme sinyali nereden geliyor?** Denetimli, denetimsiz, yarı denetimli ve pekiştirmeli öğrenme bu soruya cevap verir.
- **Model ne sıklıkla güncelleniyor?** Toplu ve çevrim içi öğrenme bu soruya cevap verir.
- **Yeni bir örneğe nasıl genelleme yapılıyor?** Örnek temelli ve model temelli yaklaşım bu soruya cevap verir.

Örneğin denetimli bir talep modeli ayda bir toplu olarak eğitilebilir; uygun bir altyapıyla yeni kayıtlar geldikçe de güncellenebilir. Yani "denetimli" ile "çevrim içi" birbirinin rakibi değildir. Aynı sistem hem denetimli, hem toplu, hem model temelli olabilir.

6.6.1 Denetimli öğrenme

Denetimli öğrenmede eğitim örnekleri için beklenen çıktı bilinir. Perakende örneğinde geçmiş mağaza-ürün-hafta kayıtları ve bu haftalardan sonra gerçekleşen satışlar elimizdedir. Model girdiler ile hedef arasında bir ilişki kurar. Yeni hafta geldiğinde aynı tür girdileri görür; hedef henüz oluşmadığı için onu tahmin eder.

"Denetim" sözcüğü, bir insanın modelin her adımını gözetlediği anlamına gelmez. Öğrenme sırasında kullanılan hedef değerin varlığını anlatır. Bu hedeflerin güvenilirliği ayrıca incelenmelidir: Stok tükenmesi nedeniyle satış hedefi sistematik olarak düşük görünüyorsa model de bu örüntüyü öğrenir. Veri ne kadar büyük olursa olsun yanlış hedef sorunu kendiliğinden çözülmez.

Denetimli öğrenmenin iki temel görevi, **hedefin türüne** göre ayrılır:

> **Tanım 6.2 · Sınıflandırma ve regresyon**
>
> **Sınıflandırmada** hedef bir kategoridir: "Stok tükenir / tükenmez", "düşük, orta, yüksek risk". Model bir sınıf ya da sınıfa ait bir olasılık üretir. **Regresyonda** hedef sayısal bir miktardır: "Gelecek hafta kaç adet satılır?" Model sayısal bir tahmin üretir.

İkisi de denetimli öğrenmedir; aralarındaki fark yalnızca hedefin türüdür. Derste kısa bir alıştırma yapmıştık: "İade olur mu?" sınıflandırmadır; "Kaç adet iade olur?" regresyondur.

Dikkat

Model ailesinin adından görev türünü çıkarmayın. “Lojistik regresyon” adını taşıyan yöntem, adına rağmen sınıflandırma için kullanılır.

Regresyonda tahmin 93,4 adet çıkabilir. Bunu kaç adet sipariş vereceğimize çevirmek, yuvarlama ve tedarik kısıtlarına bağlı ayrı bir karardır. Tahmin çıktısı ile sipariş eylemini aynı kavram yapmayız.

6.6.2 Bir skor kararın kendisi değildir

Bir sınıflandırıcı bazen doğrudan bir sınıf, bazen sınıfa ilişkin bir skor ya da olasılık üretir. “%70 stok tükenme olasılığı” tek başına kaç adet sipariş verileceğini söylemez. Karar için tedarik süresi, eldeki güvenlik stoku, ürünün bozulma maliyeti ve yanlış alarmın maliyeti gerekir.

Bölüm 3’te tahmin ile karar arasındaki farkı görmüştük. Burada bu farkın değerlendirmeye etkisini görüyoruz. Skoru bir karara çevirmek için bir **eşik** seçeriz: “Olasılık 0,6’nın üzerindeyse incelemeye al.” Eşik düşük seçilirse daha çok yanlış alarm, yüksek seçilirse daha çok kaçırılan olay ortaya çıkar. Stok tükenmesini kaçırmanın maliyeti yüksekse yalnızca genel doğruluğa bakmak yetmez; kaçırılan olayları ayrıca izlemek gerekir. Fazla sipariş pahalıysa yanlış alarmlar önem kazanır.

Örnek 6.2

Bozulabilir bir ürün olan çilek ile raf ömrü uzun bir deterjan düşünelim. İkisi için de model %70 tükenme riski veriyor. Deterjanda fazladan sipariş vermenin maliyeti düşüktür; ürün depoda bekler. Çilekte fazla sipariş birkaç gün içinde çöpe gider. Aynı skor, iki üründe farklı eşiklere ve farklı sipariş kararlarına yol açar.

6.6.3 Denetimsiz öğrenme

Denetimsiz öğrenmede örneklerin girdileri vardır, ama her örnek için bir “doğru cevap” sütunu yoktur. Mağazaların haftalık satış profillerini ve ürün çeşitlerini bildiğimizi, ama mağazaları önceden tanımlanmış türlere ayıran güvenilir bir etiket bulunmadığını düşünelim. Verideki benzerliklere göre yapılar bulabiliriz. Denetimsiz öğrenmenin üç temel kullanım amacı vardır:

Gruplama

Benzer davranış gösteren mağazaları birlikte incelemek. Sonuçta “şehir merkezi mağazası” gibi anlamlı bir ad kendiliğinden doğmaz; ekip profilleri yorumlayıp adlandırır.

Boyut azaltma ve temsil

Çok sayıdaki değişkeni daha az boyutlu bir temsille özetlemek. Görselleştirme ya da sonraki analizler için yararlıdır, ama her ayrıntıyı korumaz.

Aykırılık arama

Alışılmadık kayıtları işaretlemek. Bir mağazanın çok yüksek iade oranı bir suistimal, bir kampanya etkisi ya da bir veri hatası olabilir. İşaret, incelemenin başlangıcıdır.

Etiketsiz veride sonuçları değerlendirmek daha zordur. Bu yüzden denetimsiz öğrenme çıktılarını dört soruyla sorgularız:

1. **"Benzer" ne demek?** Mağazalar ciroya göre mi, ürün sepetine göre mi, büyüme hızına göre mi benziyor? Benzerlik ölçüsü seçildiğinde "yakınlık" anlamı da seçilmiş olur.
2. **Bulunan grup doğal bir sınıf mı?** Verideki grup sınırları her zaman doğal ve sabit değildir; kullanılan temsil ve amaçla birlikte oluşur.
3. **Aykırı kayıt hata mı?** Ani bir satış artışı veri giriş hatası, başarılı bir kampanya ya da talepte gerçek bir değişim olabilir. Sistem bu olasılıkları tek başına ayırt etmez.
4. **Sonuç işe yarıyor mu?** Mağaza grupları farklı stok planları tasarlamaya yardım ediyor mu? Gruplar yeni dönemlerde de benzer mi? Alan uzmanı gözlemleriyle uyum görüyor mu?

> **Dikkat**
>
> Denetimsiz öğrenme, etiketsiz veriden "gizli doğru etiketleri" çıkarmaz. Bulunan gruplar matematiksel olarak tutarlı olabilir ama bir iş kararına yaramayabilir. Amaç belirsizse bulunan örüntü ilginç ama yararsız kalır.

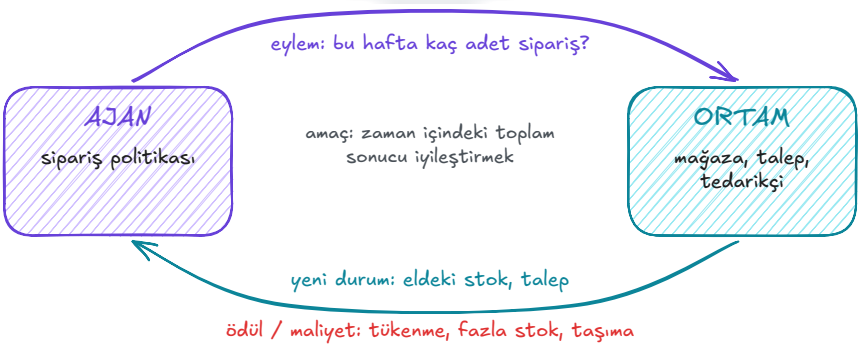
6.6.4 Yarı denetimli öğrenme

Yarı denetimli öğrenmede iki veri kaynağı birlikte kullanılır: hedefi bilinen az sayıda örnek ve hedefi bilinmeyen çok sayıda örnek. Örneğin uzmanlar ürün iadelerinin yalnızca küçük bir kısmının iade nedenini ayrıntılı olarak kodlamış olabilir; geri kalan iade açıklamaları etiketsizdir. Yöntem, sınırlı etiketle birlikte etiketsiz verinin yapısından yararlanmaya çalışır.

Bütün verinin neden etiketlenemediğini sorarsak cevaplar tanıdıktır: uzman zamanı, maliyet, mahremiyet ve belirsiz vakalar. Ancak etiketsiz veri bedava doğru cevap sağlamaz. Etiketli örnekler yanlı ya da hatalıysa bu hata etiketsiz örneklere yayılarak büyüyebilir. Etiketsiz örnekler başka bir dönemden ya da başka bir müşteri grubundan geliyorsa öğrenilen ilişki yanlış aktarılır.

6.6.5 Pekiştirmeli öğrenme

Pekiştirmeli öğrenmede örnek başına verilmiş tek bir doğru cevap yoktur. Bir **ajan** ortamın durumunu gözler, bir eylem seçer, sonra bir ödül ya da maliyet görür. Amaç, zaman içindeki toplam sonucu iyileştiren davranışı bulmaktır (Şekil 6.4).



Şekil 6.4 Pekiştirmeli öğrenme döngüsü. Doğru cevap yerine eylemin sonucu olarak gelen ödül ya da maliyet vardır.

Stok örneğinde ajan belirli koşullarda sipariş miktarını seçebilir; sonra stok tükenmesi, fazla stok ve taşıma maliyetleriyle karşılaşır. Burada kararlar ardışıktır: Bugün çok sipariş vermek yarınki seçenekleri ve maliyetleri değiştirir. Bölüm 1'deki rasyonel ajan fikrinin öğrenmeyle birleştiği yer burasıdır.

Bu örnek gerçek bir mağazada deneme yapma önerisi değildir. Kötü eylemler müşteriye ve bütçeye zarar verir. Gerçek kullanımdan önce güvenli bir simülasyon, kısıtlar ve insan denetimi gerekir. Ayrıca geribildirim alan her sistem pekiştirmeli öğrenme değildir; ödül üzerinden bir davranış politikası öğrenen bir yapı bulunmalıdır.

6.6.6 Toplu ve çevrim içi öğrenme

Bu ayrım öğrenme sinyalinden bağımsızdır; modelin ne sıklıkla ve nasıl güncellendiğiyle ilgilidir.

Toplu öğrenmede örnekler biriktirilir, model belirli aralıklarla yeniden eğitilir ve yeni sürüm test edildikten sonra kullanıma alınır. Perakendede haftalık ya da aylık güncelleme buna örnektir. **Çevrim içi öğrenmede** ise yeni örnekler geldikçe model küçük adımlarla güncellenir.

Çevrim içi güncelleme değişime daha hızlı tepki verir; ama bozuk, kötü niyetli ya da alışılmadık veri de modeli aynı hızla etkiler. Toplu yöntem değişime daha yavaş uyum sağlar, buna karşılık sürüm onayı ve eski sürüme geri dönüş daha kolay yönetilir. Seçim yalnızca bir hız meselesi değildir; veri güvenilirliği, izleme kapasitesi ve iş riskiyle ilgilidir.

> **Dikkat**
>
> Çevrim içi öğrenme, gerçek zamanlı tahmin yapmakla aynı şey değildir. Toplu olarak ayda bir eğitilmiş bir model de saniyeler içinde tahmin üretebilir. "Her gün tahmin yapıyor" ifadesi, modelin her gün yeniden öğrendiğini kanıtlamaz.

6.6.7 Örnek temelli ve model temelli yaklaşım

Son eksen, yeni bir örneğe nasıl cevap verildiğiyle ilgilidir. **Örnek temelli yaklaşımda** sistem yeni vakayı geçmişteki benzer vakalarla karşılaştırır: "Bu mağaza haftası, geçmişteki hangi haftalara benziyor ve o haftalarda ne olmuştu?" Benzerliğin hangi özelliklere göre tanımlandığı sonucu doğrudan belirler. **Model temelli yaklaşımda** ise geçmiş örneklerden daha genel bir ilişki öğrenilir ve yeni girdiye bu ilişki uygulanır. Kitabın son kısmında göreceğimiz doğrusal regresyon model temelli bir yaklaşımdır.

İki yaklaşımın da kendine özgü bir genelleme sorunu vardır. Örnek temelli sistem, yanlış bir benzerlik ölçüsüyle alakasız geçmiş haftaları yakın sanabilir. Model temelli sistem gereğinden karmaşık bir ilişki kurup geçmişe aşırı uyabilir. Bu ayrımın denetimli ve denetimsiz ayrımından bağımsız olduğuna dikkat edin.

Uygulama 6.1 · İade tahminini çerçevelemek

Bir e-ticaret şirketi iade oranını azaltmak istiyor. Bu iş sorusunu bir öğrenme görevine çevirelim.

Satır birimi. Sipariş satırı: bir müşterinin bir siparişindeki bir ürün.

Karar anı. Sipariş onaylandıktan hemen sonra, kargoya verilmeden önce.

Hedef. "Bu ürün 30 gün içinde iade edilecek mi?" Hedef kategoriktir; görev sınıflandırmadır. "Bu ay kaç adet iade gelecek?" sorusu ise mağaza-ay düzeyinde bir regresyon görevi olurdu.

Uygun özellikler. Ürün kategorisi, beden bilgisi, müşterinin geçmiş iade oranı, ürün sayfasındaki beden tablosunun güncel olup olmadığı. Hepsi karar anında bilinir.

Uygun olmayan özellikler. Kargo teslim süresi ve müşterinin ürünü değerlendirme puanı. Bunlar karar anından sonra oluşur.

Hedefin sınırı. Müşterinin "iade etmek isteyip vazgeçtiği" durumlar kayda girmez; ölçtüğümüz şey gerçekleşmiş iadedir.

Türler. Geçmiş siparişlerin iade bilgisi elimizde olduğundan denetimli öğrenme; model haftada bir yeniden eğitilirse toplu öğrenme.

Bölüm özeti

- Mitchell'e göre bir program, T görevlerinde P ile ölçülen performansı E deneyimiyle artıyorsa öğrenir. Görev, performans ve deneyim birlikte söylenmelidir.
- Makine öğrenmesi, kuralların yazılması zor olduğunda, koşullar değiştiğinde ya da veride keşfedilecek bir yapı olduğunda tercih edilir. Verinin varlığı tek başına gerekçe değildir.
- Bir problem öğrenmeye uygunsa görev tekrarlanır ve ölçülebilir, örnekler kullanım koşullarını temsil eder, girdiler karar anında bilinir, hata maliyeti ve insan denetimi baştan belirlenmiştir.
- Gözlem bir satırın birimidir; özellik tahmin anında bilinen bilgidir; hedef öğrenilmesi istenen çıktıdır. Satış, gerçek talebin yalnızca bir vekili olabilir; hedefin nasıl üretildiği belgelenmelidir.
- Bir iş sorusu tahmin hedefi, satır birimi, karar anı ve kullanım biçimiyle öğrenme görevine çevrilir. Karar anı kullanılabilecek özellikleri sınırlar.
- Türler üç ayrı soruya cevap verir: öğrenme sinyali (denetimli, denetimsiz, yarı denetimli, pekiştirmeli), güncelleme biçimi (toplu, çevrim içi) ve genelleme biçimi (örnek temelli, model temelli).
- Sınıflandırmada hedef kategorik, regresyonda sayısaldır. Bir skor kararın kendisi değildir; eşik hata maliyetlerine göre seçilir.
- Denetimsiz öğrenmede bulunan gruplar ve aykırılıklar kendiliğinden iş etiketi değildir; insan yorumu ve doğrulama gerekir.

Kaynaklar

Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill. Bölüm 1.

Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3. baskı). O'Reilly. Bölüm 1–2.

James, G., Witten, D., Hastie, T. ve Tibshirani, R. (2021). *An Introduction to Statistical Learning* (2. baskı). Springer. Bölüm 2.1.

Müller, A. C. ve Guido, S. (2016). *Introduction to Machine Learning with Python*. O'Reilly. Bölüm 1–3.

Russell, S. ve Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4. baskı). Pearson. Bölüm 19 ve 23.

7

BÖLÜM

Güvenilir bir öğrenme çalışması

Bu bölümde

- Bir makine öğrenmesi çalışmasının dört adımlı akışını algoritma seçimini merkeze almadan kurmak,
- eğitim, doğrulama ve test kümelerinin görevlerini ayırmak; zaman yapısı olan verilerde bölmenin nasıl yapılacağını açıklamak,
- genellemeyi "hangi görülmemiş örnek?" sorusuyla birlikte tanımlamak,
- aşırı uyum ve yetersiz uyumu eğitim ve test başarısı üzerinden teşhis etmek,
- veri sızıntısının açık ve örtük biçimlerini tanımak,
- temsil edici veri, veri kalitesi ve basit başlangıç düzeyi kavramlarını değerlendirmede kullanmak,
- kullanımdaki bir modelin neden eskidiğini ve nasıl izleneceğini açıklamak.

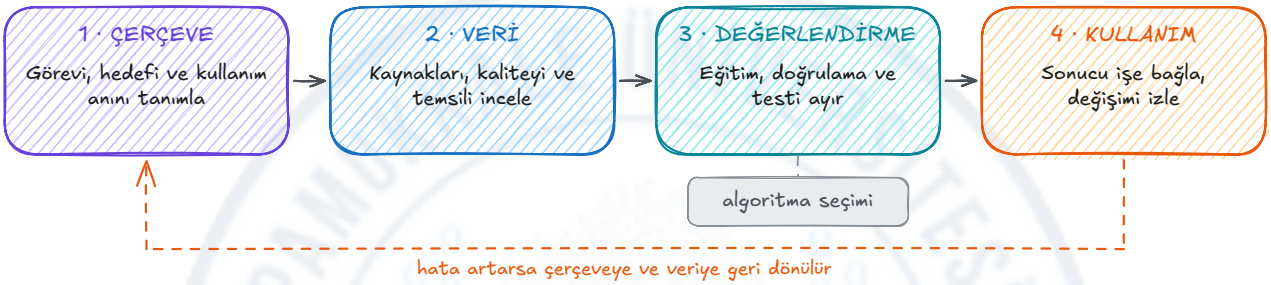
Kaynak: YBS 319 ders notları

Önceki bölümde bir makine öğrenmesi problemini nasıl kuracağımızı konuştuk. Bu bölümde modelin nasıl eğitildiğinden çok, elde edilen sonucun ne kadar güvenilir olduğunu soracağız. Bir model eğitim tablosundaki satırları çok iyi açıklayabilir. Bizim asıl ilgilendiğimiz ise henüz görmediğimiz haftalarda işe yarayıp yaramadığıdır.

Bölüm boyunca iki soruyu bir alışkanlık hâline getirmeye çalışacağız. Her veri sütunu için: **Bunu tahmin anında biliyor muyuz?** Her başarı sayısı için: **Bunu hangi veride ölçtük?**

7.1 Bir makine öğrenmesi çalışmasının akışı

Bir makine öğrenmesi çalışmasını dört adımda düşünebiliriz (Şekil 7.1). Bu akış algoritma seçimini kasıtlı olarak merkeze koymaz.



Şekil 7.1 Bir makine öğrenmesi çalışmasının dört adımı. Algoritma seçimi akışın küçük bir parçasıdır; değerlendirme en baştan tasarlanır.

1. Çerçeve. Görevi, hedefi ve kullanım anını tanımlarız. Tahmin birimi nedir? Hedef ne zaman oluşur? Çıktı hangi karara girecek?

2. Veri. Kaynakları, kaliteyi ve temsil sorununu inceleriz. Eksik değerler, tekrarlanan kayıtlar, dönem değişiklikleri ve mağazalar arasındaki farklı kayıt uygulamaları model ailesinden önce gelir.

3. Değerlendirme. Eğitim, doğrulama ve test verisini ayırırız. Değerlendirmeyi en baştan tasarlarız: Modeli eğitirken ve seçerken kullandığımız örnekler ile son performansı raporlayacağımız örnekler ayrı olmalıdır.

4. Kullanım. Sonucu işe bağlarız ve değişimi izleriz. Sonuçların nasıl denetlendiğini ve yeni dönemlerde hatanın değişip değişmediğini takip ederiz. Bir model dosyasının üretilmiş olması, iş hedefinin gerçekleştiğini kanıtlamaz.

Perakende örneğimize uygularsak her adımda somut bir soru sorabiliriz. Çerçeve: “Pazartesi sabahı hangi ürünün tükeneceğini mi tahmin ediyoruz, yoksa kaç adet satılacağını mı?” Veri: “İade kayıtları her mağazada aynı biçimde mi düşülüyor?” Değerlendirme: “Testi hangi haftalarda yapacağız?” Kullanım: “Planlamacı listeyi ne zaman görecek ve hata arttığında kim fark edecek?”

7.2 Eğitim, doğrulama ve test

Elimizdeki veriyi üç kümeye ayırırız ve her kümenin görevi farklıdır.

Tablo 7.1 Üç veri kümesinin görevleri.

Küme	Ne için kullanılır?	Temel kural
Eğitim	İlişkiyi öğrenmek	Model bu örnekleri görür
Doğrulama	Seçenekleri ve ayarları karşılaştırmak	Model geliştirme sırasında kullanılır
Test	Son kez yeni veri başarısını ölçmek	Bütün seçimler bitene kadar saklı tutulur

Eğitim verisi, modelin ilişkiyi kurduğu örneklerdir. **Doğrulama verisi**, aday modelleri ya da ayarları karşılaştırmak için kullanılır: "Bu model mi daha iyi, şu model mi?" **Test verisi** ise bütün seçimler bittikten sonra yeni veri başarısını bir kez ölçmek içindir.

Test kümesini neden baştan ayırırız? Çünkü kendi seçimlerimizden bağımsız bir son sınama istiyoruz. Teste defalarca bakıp tasarımı ona göre değiştirirsek test verisi de dolaylı olarak eğitim sürecine karışır. Sınavdan önce soruları görüp çalışmaya benzer: Aldığımız not artık gerçek bilgimizi ölçmez.

7.2.1 Zaman yapısı olan veride bölme

Satırları rastgele karıştırıp bölmek her zaman doğru değildir. Perakende örneğinde aynı mağaza ve ürünün ardışık haftaları birbirine çok benzer. Satırları rastgele dağıtırsak 37. hafta eğitimde, 36. ve 38. haftalar testte olabilir. Model, testteki haftanın komşularını görmüş olur; test kolaylaşır.

Neden doğrulama ayrı?
Doğrulama kümesinde en iyi görünen modeli seçtiğimizde, o kümede biraz şanslı olan modeli de seçmiş olabiliriz. Test kümesi bu seçimden etkilenmemiş tarafsız bir ölçüm sağlar.

Gerçekte yapacağımız iş, *gelecek* haftayı tahmin etmektir. Bu yüzden eski dönemlerle eğitip daha sonraki dönemde test etmek kullanım koşulunu daha iyi taklit eder (Şekil 7.2).



Şekil 7.2 Rastgele bölme ve zamana göre bölme. Zaman yapısı olan veride test, eğitimden sonraki dönemden alınır.

Ürünlerin piyasaya çıkış tarihleri, kampanya düzeni ve mevsim de bölme stratejisini etkiler. Tek bir standart bölme oranını (örneğin %80 eğitim, %20 test) ezberlemekten çok, kullanım koşulunu taklit eden bir bölme seçmek gerekir.

7.3 Genelleme

Bölüm 3'te genellemeyi kısaca tanımlamıştık: Modelin değeri, görmediği örneklerdeki başarısıdır. Eğitim başarısı yalnızca bir başlangıç bilgisidir. Bir model geçmiş satış tablosundaki her satırı ezberleyebilir; ama bu yetenek yeni haftanın talebini tahmin etmeye yetmez. İşletme açısından değer, yeni dönemde karar kalitesine yardımcı olacak kadar doğru ve güvenilir tahmin üretmesidir.

"Görülmemiş örnek" ifadesi de bağlama bağlıdır. Görülmemiş örnek şunlardan biri olabilir:

- gelecek hafta, aynı mağazadaki aynı ürün,
- mevcut mağazalarda satılmaya başlanan tamamen yeni bir ürün,
- yeni bir şehirde açılan bir mağaza.

Bunlar aynı zorluk düzeyinde değildir. Test tasarımını hangi kullanım senaryosunu değerlendirmek istediğimize göre seçeriz. Eğitim örneklerinden rastgele çekilmiş bir test kümesi, yeni bir bölgeye açılmanın riskini ölçmez. Bir performans iddiasıyla karşılaştığımızda şu soruları sorarız: Hangi dönemde, hangi mağazalarda, hangi ürünlerde ölçüldü?

7.4 Aşırı uyum ve yetersiz uyum

Aşırı uyumda model, eğitim verisindeki tesadüfi ayrıntıları da yakalar. Örneğin belirli bir haftada yaşanmış tek seferlik bir kampanyayı kalıcı bir talep kuralı gibi öğrenebilir. Eğitim hatası düşüktür, ama yeni haftalarda hata artar.

Yetersiz uyumda model temel ilişkiyi bile temsil edemez. Örneğin mevsimselliği hiç yakalayamayan çok basit bir model hem eğitimde hem yeni veride zayıf kalır.

Eğitim ve test başarısını yan yana koyarak kaba bir teşhis yapabiliriz:

Tablo 7.2 Eğitim ve test başarısından kaba teşhis.

Eğitim başarısı	Test başarısı	Olası durum
Yüksek	Düşük	Aşırı uyum olasılığı
Düşük	Düşük	Yetersiz uyum ya da veri ve kurulum sorunu
Yüksek	Yüksek (ve yakın)	İyi genelleme, ama sızıntı ihtimali kontrol edilmeli

Tabloyu kesin bir teşhis aracı gibi kullanmayın; tek bir sayıdan kesin teşhis konmaz. Son satırdaki uyarıya özellikle dikkat edin: Olağanüstü yüksek bir test başarısı, çoğu zaman bir sorunun işaretidir.

Dikkat

Bu kavramları “karmaşık model kötüdür” sloganına çevirmeyin. Uygun karmaşıklık; veri miktarına, gürültüye ve göreve bağlıdır. Çok basit bir yaklaşım mevsimselliği kaçırabilir; aşırı ayrıntılı bir yaklaşım tek seferlik kampanya gürültüsünü ezberleyebilir. Hedef, yeni örneklerde tutarlı başarıdır. Karmaşıklığı ayarlamanın teknik yollarını dersin ilerleyen konularında göreceğiz.

7.5 Veri sızıntısı

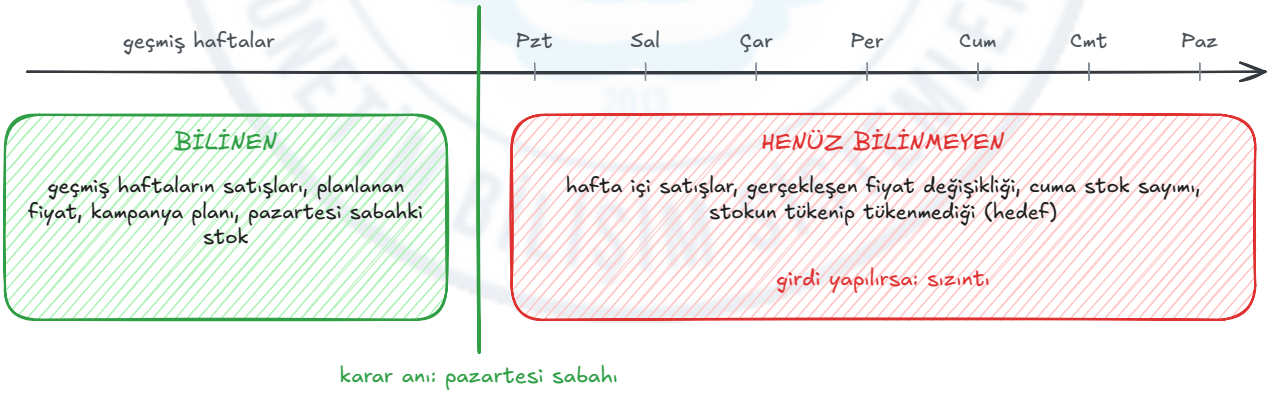
Bölüm 2’de veri sızıntısını kısaca tanımlamıştık. Burada konuyu ayrıntılı ele alıyoruz, çünkü sızıntı makine öğrenmesi çalışmalarındaki en sinsi hatalardan biridir.

Tanım 7.1 · Veri sızıntısı

Modelin, gerçek kullanımda erişemeyeceği bir bilgiyi eğitim ya da değerlendirme sırasında görmesidir. Sızıntı test sonucunu olduğundan iyi gösterir; model kullanıma alındığında beklenen başarı gerçekleşmez.

7.5.1 Açık sızıntı: gelecekten gelen sütun

Perakende örneğinde pazartesi sabahı, gelecek yedi gün içinde stokun tükenip tükenmeyeceğini tahmin ediyoruz. Veri tablosunda o haftanın cuma günkü stok sayımı da var. Bu sütunu girdi yaparsak test başarısı çok yüksek görünür; cuma günü stok sıfırsa ürünün tükendiğini model kolayca “tahmin eder”. Ama pazartesi sabahı bu değer henüz bilinmez (Şekil 7.3).



Şekil 7.3 Karar anı ve bilgi sınırı. Karar anından sonra oluşan her bilgi, tabloda bulunsa bile girdi olamaz.

7.5.2 Örtük sızıntı

Sızıntı yalnızca açıkça gelecekten gelen bir sütunla oluşmaz. Daha zor fark edilen iki biçimi vardır.

Ön işlemenin bütün veriden öğrenilmesi. Eksik değerleri doldurmak için ortalamayı, ölçeklendirme için en büyük ve en küçük değeri ya da hangi özelliklerin seçileceğini bütün veri üzerinden hesaplarsak, test verisinin bilgisi dolaylı olarak eğitime taşınır. Kural basittir: Ön

işleme ve seçim adımları yalnızca eğitim verisinden öğrenilir, sonra aynen test verisine uygulanır. Bu konuya Kısım III'te, özellikle Bölüm 9 ve Bölüm 15'te ayrıntılı döneceğiz.

Benzer kopyaların bölünmesi. Aynı olayın birbirine çok benzeyen kopyaları hem eğitime hem teste dağılırsa, test bağımsız bir sınama olmaktan çıkar. Aynı müşterinin aynı gün verdiği birkaç neredeyse aynı sipariş buna örnektir.

Her özellik için kendimize sorduğumuz küçük kontrol sorusu sızıntının en etkili ilacıdır: **Bu değeri, kararı verdiğimiz anda gerçekten biliyor muyuz?**

7.6 Temsil edici veri

Eğitim verisi, modelin kullanılacağı alanı temsil etmelidir. Yalnızca büyük mağazaların kayıtlarıyla eğitilmiş bir model küçük bir ilçedeki yeni mağazaya güvenle uygulanamaz. Yalnızca sakin haftaları içeren veri, bayram dönemindeki talebi yansıtmaz. Daha çok satır eklemek de, satırlar hep aynı dar gruptan geliyorsa, sorunu çözmez.

> **Örnek 7.1**
>
> Soğuk içecek talebini yalnızca kış aylarının kayıtlarıyla öğrenen bir model düşünelim. Model kışın düşük ve dengeli satışı çok iyi tahmin eder. Haziran geldiğinde ise talepteki sıçramayı hiç görmemiş olduğu için ciddi biçimde düşük tahmin yapar. Eğitim verisi kullanım dönemini temsil etmemektedir.

Temsil sorunu performans raporuna da yansır. Genel ortalama hata düşükken belirli ürün türlerinde, yeni mağazalarda ya da tatil haftalarında hata yüksek olabilir. Ortalama başarı, alt gruplardaki başarısızlığı gizleyebilir. Bu yüzden kullanım kitlesini açıkça tanımlar, ilgili dönemlerde ve alt gruplarda ayrı ayrı değerlendirme yaparız. Bunu yalnızca bir adalet tartışması olarak görmeyin; işletme performansı ve hizmet sürekliliği açısından da önemlidir.

7.7 Veri kalitesi ve özellik seçimi

Veri kalitesi makine öğrenmesinin yan işi değildir. Bu konuda dört ilkeyi akılda tutalım:

1. **Eksik, yanlış ya da tekrarlanan kayıt ilişkiyi bozar.** Yanlış fiyat, eksik kampanya bilgisi ya da iki kez girilmiş sipariş, modelin öğrendiği ilişkiyi doğrudan çarpıtır.
2. **Özellik karar anında erişilebilir ve anlamlı olmalıdır.** Bilgi tahminle gerçekten ilgili mi ve karar anında güvenilir biçimde bulunuyor mu?
3. **Çok kayıt, sistematik ölçüm hatasını tek başına düzeltmez.** Bir mağaza iadeleri satıştan düşüyor, öteki düşmüyorsa aynı başlıktaki sayılar karşılaştırılabilir değildir. Bu sorunu ne veri miktarı ne de gelişmiş bir algoritma çözer.
4. **Veri üretim süreci de model kadar izlenmelidir.** Kayıt yazılımı, ürün kodları ya da kampanya tanımı değiştiğinde girdilerin anlamı da değişir.

Çok sayıda ilgisiz özellik gürültüyü ve sızıntı riskini artırabilir. Tersine, önemli bir kampanya bilgisi eksikse model temel bir farkı hiç göremez. Veri hazırlamanın amacı yalnızca boş hücreleri doldurmak değil, ölçümlerin iş anlamını korumaktır. Kısım III bu işin tamamen kendisine ayrılmıştır.

7.8 Basit başlangıç düzeyi

Bir modelin hata oranı tek başına iyi ya da kötü değildir. Değerlendirmeye önce basit bir **başlangıç düzeyi** (*baseline*) kurarak başlarız. Perakende örneğinde en basit tahmin "gelecek haftanın satışı, geçen haftanın satışı kadar olacak" demektir. Mevsimsel ürünlerde "geçen yılın aynı haftası kadar" demek başka bir basit kıyaslamadır.

> **Tanım 7.2 · Başlangıç düzeyi**
>
> Herhangi bir öğrenme gerektirmeyen, kurulumu kolay bir kıyaslama tahminidir. Aday model, aynı test döneminde ve aynı hata ölçüsüyle bu düzeyi geçmeden "başarılı" sayılmaz.

Başlangıç düzeyi, karmaşık yöntemlerin neden gerekli olduğunu somutlaştırır. Aday model birkaç puan daha iyi görünüyorsa, bu farkın veri toplama, bakım ve açıklama maliyetini karşılayıp karşılamadığını tartışırız. Başlangıç düzeyi çok güçlü de çıkabilir. Bu bir başarısızlık değildir; problemin mevcut basit yöntemle zaten iyi yönetildiğini gösteren değerli bir bilgidir.

Karşılaştırmanın adil olması da gerekir. Yalnızca eğitildiği veride iyi sonuç veren bir modeli, başlangıç düzeyinin başka bir dönemde ölçülmüş hatasıyla karşılaştırmak anlamsızdır. İkisi aynı test verisinde, aynı ölçüyle sınanmalıdır.

7.9 Kullanımda model neden eskir?

Testte iyi sonuç veren bir model sonsuza kadar aynı performansı sürdürmez. Eskimenin iki kaynağı vardır.

Dünya değişir. Yeni bir fiyat politikası, yeni bir rakip, tedarik sorunu, enflasyon, mağaza açılışları ya da tüketici alışkanlıklarındaki değişim, modelin öğrendiği ilişkiyi geçersiz kılabilir.

Kayıt değişir. Yeni bir kasa (POS) yazılımı yeni ürün kodları üretir; kampanya kaydının tanımı farklılaşır; bazı alanlar boş gelmeye başlar. Dünya aynı kalsa bile modele gelen girdilerin anlamı değişmiş olur.

Kullanımda yalnızca genel hata değil, kritik ürün ve mağaza gruplarındaki hata da izlenmelidir. Hata arttığında refleks olarak modeli yeniden eğitmek yeterli olmayabilir. Önce hedefin ve kayıtların değişip değişmediği incelenir. Gerekirse veri toplama, problem tanımı, model ve iş akışı birlikte güncellenir. Bölüm 2'deki "sistemi izleyin" öğüdünün somut karşılığı budur: veri dağılımını, etiketleri ve tahmin hatasını izlemek.

Uygulama 7.1 · Bir performans iddiasını sorgulamak

Bir yazılım firması, stok tükenmesi tahmin modelinin "%97 doğrulukla" çalıştığını söylüyor. Bölümdeki kavramlarla hangi soruları sorarsınız?

Hangi veride ölçüldü? Eğitim verisinde mi, ayrılmış bir test kümesinde mi? Test kümesi model seçiminde de kullanıldı mı?

Nasıl bölündü? Satırlar rastgele mi bölündü, yoksa eğitim eski haftalardan, test sonraki haftalardan mı alındı?

Sızıntı var mı? Girdiler arasında hafta içinde ya da hafta sonunda oluşan bir bilgi, örneğin stok sayımı ya da teslimat durumu, bulunuyor mu? Ön işleme adımları bütün veri üzerinden mi hesaplandı?

Başlangıç düzeyi ne? Ürünlerin yalnızca %3'ü tükeniyorsa her ürüne "tükenmez" diyen bir kural da %97 doğruluk verir (Bölüm 5'teki örneği hatırlayın). Model bu kuralı geçiyor mu?

Kimin için geçerli? Hangi mağazalarda, hangi ürünlerde ve hangi dönemde ölçüldü? Yeni mağazalar ve bayram haftaları dahil mi?

Bölüm özeti

- Bir makine öğrenmesi çalışması dört adımdan oluşur: çerçeve, veri, değerlendirme ve kullanım. Algoritma seçimi bu akışın küçük bir parçasıdır.
- Eğitim kümesi ilişkiyi öğrenmek, doğrulama kümesi seçenekleri karşılaştırmak, test kümesi son ölçümü yapmak içindir. Test, bütün seçimler bitene kadar saklı tutulur.
- Zaman yapısı olan veride rastgele bölme testi kolaylaştırır; eski dönemle eğitip sonraki dönemde test etmek kullanım koşulunu daha iyi taklit eder.
- Genelleme, görülmemiş örneklerdeki başarıdır; ama "görülmemiş" olanın ne olduğu (yeni hafta, yeni ürün, yeni mağaza) açıkça tanımlanmalıdır.
- Eğitimde iyi ve testte kötü sonuç aşırı uyuma, ikisinde de kötü sonuç yetersiz uyuma ya da veri sorununa işaret eder. Olağanüstü iyi bir test sonucu sızıntı açısından kontrol edilmelidir.
- Veri sızıntısı, kullanım anında bilinmeyecek bilginin eğitimde ya da değerlendirmede kullanılmasıdır. Gelecekten gelen sütunlar, bütün veriden öğrenilen ön işleme ve bölünen benzer kopyalar sızıntı kaynağıdır.
- Eğitim verisi kullanım alanını temsil etmelidir; ortalama başarı alt gruplardaki başarısızlığı gizleyebilir.
- Model, aynı test verisinde ve aynı ölçüyle basit bir başlangıç düzeyiyle karşılaştırılmadan başarılı sayılmaz.
- Dünya ya da kayıt sistemi değiştiğinde model eskir; genel ve alt grup hatası, veri dağılımı ve etiketler izlenmelidir.

Kaynaklar

Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3. baskı). O'Reilly. Bölüm 1–2.

James, G., Witten, D., Hastie, T. ve Tibshirani, R. (2021). *An Introduction to Statistical Learning* (2. baskı). Springer. Bölüm 2.2.

Müller, A. C. ve Guido, S. (2016). *Introduction to Machine Learning with Python*. O'Reilly. Bölüm 1–3 ve 5.

Russell, S. ve Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4. baskı). Pearson. Bölüm 19.9.

KISIM III

Veriyi tanımak ve hazırlamak

8 Keşifsel veri analizi: Titanic

9 Veri ön işlemeye giriş: kalite, tekrarlar ve veri ayrımı

10 Eksik veri

11 Hatalı ve aykırı değerler

12 Özellik mühendisliği ve kategorik kodlama

13 Dağılım dönüşümleri ve ölçeklendirme

14 Öznitelik seçimi, boyut indirgeme ve sınıf dengesizliği

15 Tekrarlanabilir bir ön işleme hattı

8

BÖLÜM

Keşifsel veri analizi: Titanic

Bu bölümde

- Jupyter Notebook'ta kod ve metin hücrelerini kullanmak; `pandas` ile bir CSV dosyasını okumak,
- bir veri tablosunun boyutunu, veri tiplerini, tekrarlarını ve eksik değerlerini incelemek,
- veri tipi ile ölçüm türünü ayırmak; bir veri sözlüğü okumak,
- sayım, yüzde, ortalama, medyan, histogram ve kutu grafiğinin hangi sorulara cevap verdiğini ayırt etmek,
- grup içi oranları doğru payda ile hesaplamak; iki değişkeni birlikte inceleyerek karıştırıcı değişkenleri fark etmek,
- eksiklik örüntülerini, uç değerleri ve korelasyonu dikkatle yorumlamak,
- keşifsel bulguları nedensellik iddiasına dönüştürmeden raporlamak.

Kaynak: YBS 319 Titanic keşifsel veri analizi defteri

Önceki iki bölümde bir makine öğrenmesi problemini kurmayı ve sonucun güvenilirliğini sınamayı konuştuk. Her iki bölümde de aynı uyarı tekrar tekrar karşımıza çıktı: Model ailesinden önce veriyi tanımak gerekir. Bu bölümde ilk kez gerçek bir veri kümesini açıyoruz.

Keşifsel veri analizi (KVA; İngilizcede *exploratory data analysis*, EDA), bir veri kümesini model kurmadan önce tanımak için yapılan incelemedir. Amacımız bir tahmin modeli kurmak değil; veriyi okumak, yapısını ve sorunlarını görmek, uygun özetler ve grafikler üretmek ve bulguları dikkatle yorumlamaktır. Bu bölümdeki bulgular, Kısım III'ün geri kalanında yapacağımız her ön işleme kararının gerekçesi olacak.

Bölüm boyunca kod ile kavramı yan yana ilerletiyoruz. Her kod parçasının altında, gerçek veriyle çalıştırıldığında elde edilen çıktıyı bulacaksınız. Kodları derste kullandığımız defterle aynı sırada çalıştırırsanız aynı sonuçları alırsınız.

8.1 Titanic veri kümesi

1912'de batan RMS Titanic'in yolcularına ait bu veri kümesi, makine öğrenmesi eğitiminde en sık kullanılan örneklerden biridir. Bizim kullandığımız dosyada her satır bir yolcuyu, `Survived` sütunu ise kayıtlı sonucu temsil eder. Dosya 891 yolcu içerir; gemideki yolcuların tamamı değildir.

Bu tarihî ve gözlemsel bir veri kümesidir. Bulgular, dosyadaki yolcularla ilgilidir. Bütün yolculara genelleme yapmak ya da neden-sonuç çıkarımında bulunmak için tek başına yeterli değildir. Bu cümleyi bölüm boyunca birkaç kez hatırlayacağız.

8.1.1 Veri sözlüğü

Bir veri kümesiyle çalışmaya başlamadan önce her sütunun ne anlattığını bilmemiz gerekir. **Veri sözlüğü** her sütunun anlamını, ölçüm türünü ve dikkat edilmesi gereken yanlarını açıklar.

Tablo 8.1 Titanic veri kümesinin veri sözlüğü.

Sütun	Anlamı	Ölçüm türü ve dikkat
`PassengerId`	Yolcu kimliği	Sayı görünür ama ölçülen bir nicelik değildir
`Survived`	0 = hayatta kalmadı, 1 = hayatta kaldı	İkili hedef değişken
`Pclass`	Bilet sınıfı: 1, 2, 3	Sayısal kodlu sıralı kategori; 1 ile 3 arasındaki fark bir ölçü birimi değildir
`Name`	İsim	Metin; mahremiyet açısından dikkat gerektirir
`Sex`	Kayıttaki cinsiyet etiketi	Kategori
`Age`	Yaş	Sayısal; bazı kayıtlarda eksik
`SibSp`	Birlikte seyahat eden kardeş ya da eş sayısı	Tam sayı sayım
`Parch`	Birlikte seyahat eden ebeveyn ya da çocuk sayısı	Tam sayı sayım
`Ticket`	Bilet kodu	Metin; aynı bilet birden çok yolcuda görülebilir
`Fare`	Kayıttaki ücret değeri	Sayısal; dosya para birimini belirtmez
`Cabin`	Kabin kodu	Metin; çok sayıda eksik var
`Embarked`	Biniş limanı kodu: C, Q, S	Kategori; kodlar bir büyüklük sırası ifade etmez

Bir satırın birimi yolcudur. Bu yüzden bir bilet kodunun tekrar etmesi, tek başına mükerrer bir yolcu kaydı anlamına gelmez. Aile üyeleri aynı biletle seyahat etmiş olabilir.

8.2 Jupyter ve Python araçları

Derste kodları **Jupyter Notebook** üzerinde çalıştırıyoruz. Bir defter iki tür hücreden oluşur: Açıklama yazdığımız **metin hücreleri** ve Python kodunu çalıştırdığımız **kod hücreleri**. Bir hücreyi çalıştırmak için hücreye tıklayıp `Shift + Enter` tuşlarına basarız. Hücreler yukarıdan aşağıya sırayla çalıştırılmalıdır, çünkü sonraki hücreler önceki hücrelerde tanımlanan değişkenleri kullanır. Kod hücresinde `#` işaretinden sonra yazılanlar açıklamadır; Python bunları çalıştırmaz.

İlk iş olarak kullanacağımız araçları çağırırız:

```
from pathlib import Path          # dosya yolunu işletim sistemine uygun kurar
import pandas as pd               # tablo verileriyle çalışmak için
import matplotlib.pyplot as plt   # grafik çizmek için
from IPython.display import display  # tabloları Jupyter'da biçimli gösterir
```

```
plt.rcParams["font.family"] = "DejaVu Sans"   # Türkçe karakterler için yazı tipi
plt.rcParams["figure.figsize"] = (8, 4.5)     # varsayılan grafik boyutu
print("Araçlar hazır.")
```

çıktı

```
Araçlar hazır.
```

`import`, başka bir paketin işlevlerini kullanıma açar. `as pd` ve `as plt` yaygın kısaltmalardır; başka kaynaklarda da bunları göreceksiniz. `from ... import ...` yalnızca adı yazılan öğeyi getirir. `=` bir isme değer atar. Nokta (`.`) ise bir nesnenin özelliğine ya da işlevine erişir: `plt.rcParams` gibi.

8.3 Dosyayı okumak ve ilk satırlara bakmak

Neden pandas?
`pandas`, Python'da tablo biçimindeki veriyle çalışmanın standart kütüphanesidir. Excel'de satır ve sütunlarla yaptığımız işlerin çoğunu kodla, tekrarlanabilir biçimde yapmamızı sağlar.

```
veri_yolu = Path("Titanic-Dataset.csv")
if not veri_yolu.exists():
    raise FileNotFoundError("CSV dosyasını defterle aynı klasöre koyun.")

df = pd.read_csv(veri_yolu)   # CSV'yi DataFrame adlı tabloya okur
df[["PassengerId", "Survived", "Pclass", "Sex", "Age", "Fare", "Cabin",
"Embarked"]].head()
```

çıktı

	PassengerId	Survived	Pclass	Sex	Age	Fare	Cabin	Embarked
0	1	0	3	male	22.0	7.2500	NaN	S
1	2	1	1	female	38.0	71.2833	C85	C
2	3	1	3	female	26.0	7.9250	NaN	S
3	4	1	1	female	35.0	53.1000	C123	S
4	5	0	3	male	35.0	8.0500	NaN	S

`pd.read_csv(...)` CSV dosyasını **DataFrame** adı verilen satır-sütun yapısına çevirir. `df`, bu yapı için yaygın olarak kullanılan bir addır; farklı bir ad da verebilirsiniz. `.head()` varsayılan olarak ilk beş satırı döndürür. Sayfaya sığması için burada sütunların bir kısmını seçtik; derste bütün sütunlara bakmıştık.

İlk satırlar, sütunların görünümünü, metin ile sayı karışımını ve boş hücreleri fark etmemizi sağlar. `PassengerId` bir kayıt numarası, `Survived` ise 0/1 sonucu olarak görünüyor. İlk satırdaki boş `Cabin` hücresi `NaN` olarak gösteriliyor. Bu, sıfır ya da "kabin yok" bilgisi değildir; **kayıtlı değer eksik** demektir.

> **Dikkat**
>
> İlk beş satırdan genel bir sonuç çıkaramayız. Bu beş yolcunun üçü hayatta kalmış diye genel sağkalım oranının %60 olduğunu söyleyemeyiz. İlk satırlar veri kümesinin tamamını temsil etmez.

8.3.1 DataFrame'i elle kurmak

DataFrame'in mantığını görmek için küçük bir tabloyu kendimiz de kurabiliriz. Python'da süslü parantezle tanımlanan **sözlük** (*dictionary*), anahtar ve değer çiftlerinden oluşur. Her anahtar bir sütun adı, her değer o sütunun değerlerini taşıyan bir liste olur:

```
veri = {
    "İsim":  ["Ahmet", "Ayşe", "Mehmet"],
    "Yaş":   [25, 30, 22],
    "Şehir": ["Ankara", "İstanbul", "İzmir"],
}
print(veri["Yaş"][0])          # Yaş listesinin ilk elemanı
df_ornek = pd.DataFrame(veri)
df_ornek
```

çıktı

```
25
     İsim  Yaş     Şehir
0   Ahmet   25    Ankara
1    Ayşe   30  İstanbul
2  Mehmet   22     İzmir
```

`veri["Yaş"][0]` ifadesi önce "Yaş" anahtarının listesini, sonra bu listenin ilk elemanını seçer. Python'da sıralama sıfırdan başlar.

8.4 Tablo boyutu ve veri tipleri

```
print("Satır ve sütun sayısı:", df.shape)
print("Sütun adları:", df.columns.tolist())
```

çıktı

```
Satır ve sütun sayısı: (891, 12)
Sütun adları: ['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked']
```

`df.shape` iki öğeli bir demettir: (satır sayısı, sütun sayısı). Tablomuz **891 satır ve 12 sütun** içeriyor. Analizin paydasını belirlemek için kaç kayıt olduğunu bilmemiz gerekir.

```
df.dtypes
```

çıktı

```
PassengerId      int64
Survived         int64
Pclass           int64
Name               str
Sex                str
Age              float64
SibSp            int64
Parch            int64
Ticket             str
Fare             float64
Cabin              str
```

```
Embarked           str
dtype: object
```

`df.dtypes` her sütunun pandas veri tipini gösterir. `Age` ve `Fare` ondalıklı sayı (`float64`), `Survived`, `Pclass`, `SibSp`, `Parch` ve `PassengerId` tam sayı (`int64`) olarak okunmuş. Metin sütunları pandas sürümüne göre `object` ya da `str` olarak görünür.

> **Tanım 8.1 · Veri tipi ve ölçüm türü**
>
> **Veri tipi**, bir değerin bilgisayarda nasıl saklandığını belirtir: tam sayı, ondalıklı sayı, metin. **Ölçüm türü** ise değişkenin neyi ifade ettiğini anlatır: sürekli bir ölçüm mü, bir sayım mı, sıralı ya da sırasız bir kategori mi? İkisi aynı şey değildir.

`Pclass` tam sayı olarak saklanır, ama analitik olarak bir kategoridir. 1, 2 ve 3 bilet sınıflarını belirtir; üçüncü sınıf, birinci sınıfın "üç katı" değildir. `Survived` da sayı olarak saklanır ama iki sınıflı bir hedeftir. Yalnızca veri tipine bakarak bir sütunun istatistiksel anlamını belirleyemeyiz.

8.5 Kayıt bütünlüğü ve eksik değerler

```
df.info()
```

çıktı

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    str
 4   Sex          891 non-null    str
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    str
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    str
 11  Embarked     889 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 83.7 KB
```

`df.info()` her sütunun dolu kayıt sayısını ve veri tipini birlikte özetler. Toplam 891 satır varken `Age` sütununda 714, `Cabin` sütununda 204, `Embarked` sütununda 889 dolu değer olduğunu görüyoruz. Demek ki bu üç sütunda eksik değerler var.

Şimdi iki farklı tekrar türünü kontrol edelim:

```
print("Tamamen aynı satır sayısı:", df.duplicated().sum())
print("Tekrarlanan PassengerId sayısı:", df["PassengerId"].duplicated().sum())
```

çıktı
```
Tamamen aynı satır sayısı: 0
Tekrarlanan PassengerId sayısı: 0
```

`df.duplicated()` bütün alanları aynı olan satırları `True` olarak işaretler; `.sum()` bu işaretleri sayar. Python'da `True` değeri 1, `False` değeri 0 gibi toplanır. Tam satır tekrarı ile kimlik tekrarı farklı kalite sorunlarıdır. Bu veride ikisi de sıfır.

Eksik değerleri sayı ve yüzde olarak bir tabloda toplayalım:

```
eksik_sayilari = df.isna().sum()
eksik_yuzdeleri = (eksik_sayilari / len(df) * 100).round(1)
eksik_ozeti = pd.DataFrame({"Eksik sayı": eksik_sayilari,
                            "Eksik yüzde": eksik_yuzdeleri})
eksik_ozeti = eksik_ozeti.sort_values("Eksik sayı", ascending=False)
eksik_ozeti.head(4)
```

çıktı
```
             Eksik sayı  Eksik yüzde
Cabin               687         77.1
Age                 177         19.9
Embarked              2          0.2
PassengerId           0          0.0
```

`df.isna()` boş hücreleri işaretler, `.sum()` her sütun için sayar. Bölme ve 100 ile çarpma, oranı yüzdeye çevirir. `pd.DataFrame({...})` bir sözlükten yeni bir özet tablo kurar; `sort_values` en çok eksik olan sütunu üste taşır.

Eksikler üç sütunda yoğunlaşıyor: `Cabin` 687/891 = %77,1; `Age` 177/891 = %19,9; `Embarked` 2/891 = %0,2. Bunun iki önemli sonucu var. Yaş üzerinden yapacağımız her hesap yalnızca yaşı bilinen 714 kişiyi kapsayacak. `Cabin` sütunundaki çok yüksek eksiklik ise bu sütunla yapılacak çıkarımları ciddi biçimde sınırlıyor. Bu aşamada eksik değerlere dokunmuyoruz; eksikliği silmek ya da doldurmak analizin kendisini değiştirebilir. Önce örüntüyü inceleyeceğiz.

8.6 Sayısal değişkenlerin dağılımı

```
df[["Age", "Fare"]].describe().round(2)
```

çıktı
```
            Age     Fare
count    714.00   891.00
mean      29.70    32.20
std       14.53    49.69
min        0.42     0.00
25%       20.12     7.91
50%       28.00    14.45
75%       38.00    31.00
max       80.00   512.33
```

`df[["Age", "Fare"]]` ifadesindeki iç içe köşeli parantezlere dikkat edin: İçteki liste sütun adlarıdır, dıştaki parantez DataFrame'den seçim yapar. `.describe()` dolu kayıt sayısını (`count`), ortalamayı (`mean`), standart sapmayı (`std`), en küçük ve en büyük değeri ve çeyrekleri hesaplar. `.round(2)`

görüntüyü iki ondalığa yuvarlar; ham veriyi değiştirmez.

Bu tablodan okuyabileceklerimiz:

- `Age` için 714 gözlem var; ortalama 29,70, medyan (`50%`) 28, aralık 0,42–80. 0,42 yaş bir bebeğe aittir; ondalıklı yaşlar hata değildir.
- `Fare` için 891 gözlem var; ortalama 32,20, medyan 14,45, en büyük değer 512,33.
- Ücret ortalamasının medyanın iki katından fazla olması, sağa uzanan bir dağılıma işaret eder.

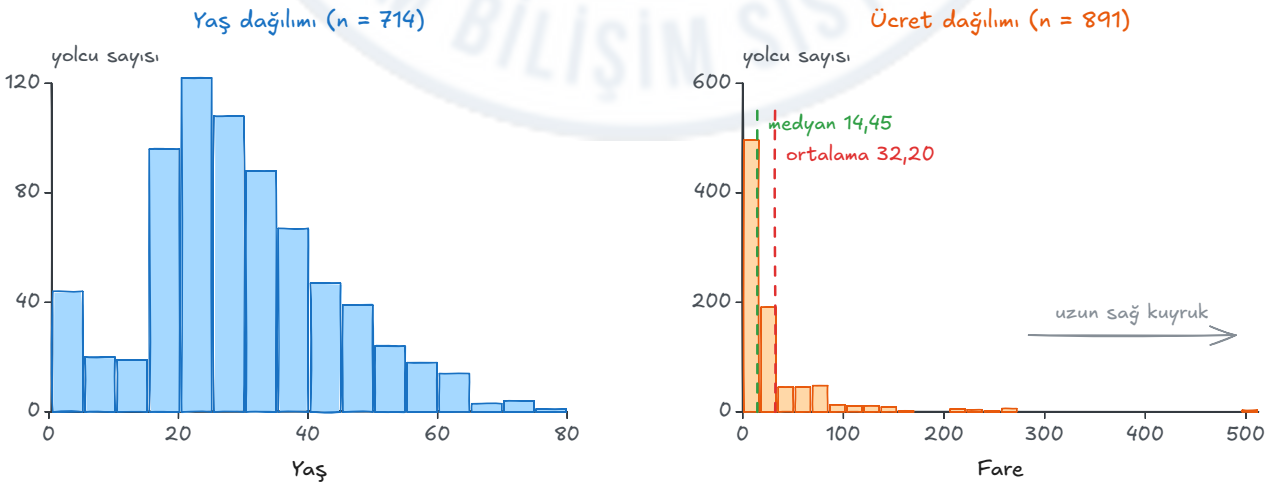
> **Tanım 8.2 · Ortalama, medyan ve çeyrekler**
>
> **Ortalama**, değerlerin toplamının gözlem sayısına bölümüdür ve uç değerlere duyarlıdır. **Medyan**, değerler sıralandığında ortada kalan değerdir; uç değerlerden çok daha az etkilenir. **Birinci çeyrek** (Q1) değerlerin %25'inin, **üçüncü çeyrek** (Q3) değerlerin %75'inin altında kaldığı noktadır.

Ortalama ile medyan arasındaki fark bize dağılımın şekli hakkında bir ipucu verir. Çok zengin birkaç kişinin bulunduğu bir mahallede ortalama gelir yüksek çıkar, ama mahalledeki tipik insanın gelirini medyan daha iyi anlatır. Ücret sütunu da böyledir.

8.6.1 Dağılımları grafikle görmek

Bir **histogram**, sayısal değerleri eşit genişlikte aralıklara (*bins*) ayırıp her aralığa düşen gözlem sayısını çubukla gösterir. Ortalama ve medyanın gizlediği şekli görünür kılar.

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))   # yan yana iki grafik alanı
axes[0].hist(df["Age"].dropna(), bins=16)          # eksik yaşları çıkarır
axes[0].set(title="Yaş dağılımı", xlabel="Yaş", ylabel="Yolcu sayısı")
axes[1].hist(df["Fare"], bins=30)
axes[1].set(title="Ücret değeri dağılımı", xlabel="Fare", ylabel="Yolcu sayısı")
plt.tight_layout()
plt.show()
```



Şekil 8.1 Yaş ve ücret dağılımları. Yaş histogramı yalnızca yaşı bilinen 714 yolcuya dayanır. Ücretlerin çoğu alt aralıklarda toplanır; az sayıdaki büyük değer sağa doğru uzun bir kuyruk oluşturur.

`plt.subplots(1, 2, ...)` bir satırda yan yana iki grafik alanı açar. `axes[0]` birinci, `axes[1]` ikinci alandır. `.dropna()` eksik yaşları grafikten çıkarır; histogram eksik değer çizemez.

Yaş dağılımında genç ve orta yaşlarda daha çok gözlem var. Ücret grafiğinde ise değerlerin büyük çoğunluğu en soldaki birkaç aralıkta toplanıyor ve sağa doğru seyrek ama uzun bir kuyruk uzanıyor (Şekil 8.1). Buna **sağa çarpık** dağılım diyoruz. Böyle bir dağılımda yalnızca ortalamayı raporlamak tipik yolcuyu olduğundan yüksek ücretli gösterir.

8.7 Kategorik değişkenleri saymak

Aralık sayısı
Histogramdaki aralık sayısını (`bins`) değiştirmek grafiğin görünümünü değiştirir, ama altta yatan 891 ücreti değiştirmez. Birkaç farklı değer deneyip dağılımın şeklinin değişmediğinden emin olmak iyi bir alışkanlıktır.

```
print(df["Sex"].value_counts(dropna=False))
print(df["Pclass"].value_counts(dropna=False).sort_index())
print(df["Embarked"].value_counts(dropna=False))
```

çıktı

```
Sex
male      577
female    314
Name: count, dtype: int64
Pclass
1    216
2    184
3    491
Name: count, dtype: int64
Embarked
S      644
C      168
Q       77
NaN      2
Name: count, dtype: int64
```

`.value_counts(dropna=False)` her farklı değeri sayar; `dropna=False` eksikleri de ayrı bir satırda tutar. `.sort_index()` sınıf kodlarını 1, 2, 3 sırasına getirir.

Sonuçlar: 577 erkek, 314 kadın; 216 birinci, 184 ikinci, 491 üçüncü sınıf yolcu; 644 S, 168 C, 77 Q ve 2 eksik biniş limanı. Eksik `Embarked` değerlerini ayrı saymak önemlidir. Sessizce yok sayılırlarsa toplam 891'den 889'a iner ve bunu fark etmeyiz.

Grupların büyüklükleri birbirine eşit değil. Üçüncü sınıfta birinci sınıfın iki katından fazla yolcu var. Bu yüzden ileride yalnızca "hayatta kalan sayısını" karşılaştırmak yanıltıcı olur; her grup içindeki oranı hesaplamamız gerekir.

8.8 Hedef değişken: kaç kişi hayatta kaldı?

```
sonuc_sayilari = df["Survived"].value_counts().sort_index()
sonuc_yuzdeleri = (df["Survived"].value_counts(normalize=True)
                   .sort_index() * 100).round(1)
pd.concat([sonuc_sayilari.rename("Yolcu sayısı"),
           sonuc_yuzdeleri.rename("Yüzde")], axis=1)
```

çıktı

```
          Yolcu sayısı  Yüzde
Survived
0                  549   61.6
1                  342   38.4
```

`value_counts()` frekansı, `value_counts(normalize=True)` ise toplam içindeki payı verir. `pd.concat(..., axis=1)` iki seriyi sütun olarak yan yana getirir.

`Survived`, bir model kurarsak tahmin etmeye çalışacağımız **hedef** sütundur (Bölüm 6). Bir modele geçmeden önce hedefin dağılımını görmek gerekir: 549 kişi (%61,6) hayatta kalmamış, 342 kişi (%38,4) hayatta kalmıştır. İki sınıf eşit büyüklükte değildir. Bu dağılım ileride iki işe yarayacak: Sınıf dengesini değerlendirmek (Bölüm 14) ve bir başlangıç düzeyi kurmak (Bölüm 7). Her yolcuya "hayatta kalmadı" diyen hiçbir şey öğrenmemiş bir tahmin, bu dosyada %61,6 doğruluk verir.

8.9 Gruplara göre hayatta kalma

8.9.1 Cinsiyet

Şimdi bir değişkenin hedefle ilişkisine bakalım. Kullanacağımız araç `groupby`'dır.

```
cinsiyet_ozeti = df.groupby("Sex")["Survived"].agg(["count", "sum", "mean"])
cinsiyet_ozeti = cinsiyet_ozeti.rename(
    columns={"count": "Yolcu sayısı", "sum": "Hayatta kalan", "mean": "Oran"})
cinsiyet_ozeti["Oran (%)"] = (cinsiyet_ozeti["Oran"] * 100).round(1)
cinsiyet_ozeti[["Yolcu sayısı", "Hayatta kalan", "Oran (%)"]]
```

çıktı

```
        Yolcu sayısı  Hayatta kalan  Oran (%)
Sex
female           314            233      74.2
male             577            109      18.9
```

`groupby("Sex")` satırları cinsiyet etiketine göre gruplar. `["Survived"]` her gruptaki hedef sütunu seçer. `.agg(["count", "sum", "mean"])` tek adımda üç özet üretir. Hedef 0/1 olduğu için burada küçük bir hile işe yarar: `sum` hayatta kalanların **sayısını**, `mean` ise hayatta kalma **oranını** verir. Birlerin ortalaması, birlerin payıdır.

Kadın etiketli 314 yolcunun 233'ü (%74,2), erkek etiketli 577 yolcunun 109'u (%18,9) hayatta kalmıştır. Her grubun paydası kendi yolcu sayısıdır. Aradaki fark yaklaşık **55,3 yüzde puandır** (74,2 − 18,9).

> **Dikkat**
>
> Yüzde puan ile yüzde değişimi aynı kavram değildir. %18,9'dan %74,2'ye geçiş 55,3 yüzde puanlık bir farktır; yüzde değişim olarak ise yaklaşık %293'lük bir artışa karşılık gelir. Raporlarken hangisini kullandığınızı açıkça yazın.

8.9.2 Bilet sınıfı ve biniş limanı

Aynı `groupby` örüntüsünü başka sütunlara da uygulayabiliriz. Bir kod fikrinin farklı araştırma sorularında tekrar kullanılabildiğini görmek, programlama öğrenmenin önemli bir parçasıdır.

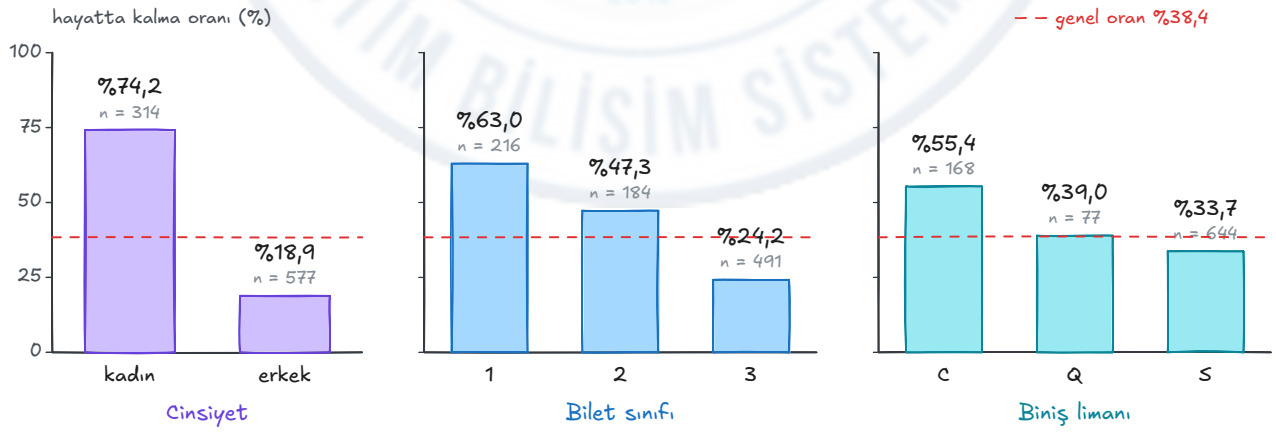
```
for sutun in ["Pclass", "Embarked"]:
    ozet = df.groupby(sutun)["Survived"].agg(["count", "sum", "mean"])
    ozet["Oran (%)"] = (ozet["mean"] * 100).round(1)
    print(ozet[["count", "sum", "Oran (%)"]], "\n")
```

çıktı

```
        count  sum  Oran (%)
Pclass
1         216  136      63.0
2         184   87      47.3
3         491  119      24.2

          count  sum  Oran (%)
Embarked
C           168   93      55.4
Q            77   30      39.0
S           644  217      33.7
```

Burada `for` döngüsüyle aynı işlemi iki sütun için tekrarladık. Sınıfa göre: birinci sınıfta 136/216 = %63,0; ikinci sınıfta 87/184 = %47,3; üçüncü sınıfta 119/491 = %24,2. Limana göre: C'de 93/168 = %55,4; Q'da 30/77 = %39,0; S'de 217/644 = %33,7.



Şekil 8.2 Gruplara göre hayatta kalma oranları. Her çubuk kendi grubunun içindeki oranı gösterir; çubukların üstündeki sayılar grup büyüklüğüdür. Kesikli çizgi genel oranı (%38,4) gösterir.

Şekil 8.2'de üç değişkenin hepsinde gruplar arasında belirgin farklar görülüyor. Liman tablosundaki üç grubun toplamı 889'dur. `groupby` eksik etiketli iki satırı varsayılan olarak dışarıda bırakır; bunu raporda ayrıca belirtmek gerekir. Q grubunun yalnızca 77 kişi olması da küçük gruplardaki oranların daha oynak olabileceğini hatırlatır.

Bu farklar dosyada güçlü ilişkiler olduğunu gösterir. Ama `Pclass` kodunun kendisinin yaşam şansını belirlediğini ya da C limanından binmenin hayat kurtardığını ispatlamaz. Bilet sınıfı ücretle, kabinin gemideki konumuyla ve başka koşullarla ilişkilidir. Limanların yolcu bileşimi de birbirinden farklı olabilir. Bunu birazdan iki değişkeni birlikte inceleyerek göreceğiz.

8.10 Yaş ile sonuç arasındaki ilişki

```
yas_ozeti = df.groupby("Survived")["Age"].agg(["count", "mean", "median"])
yas_ozeti.round(1)
```

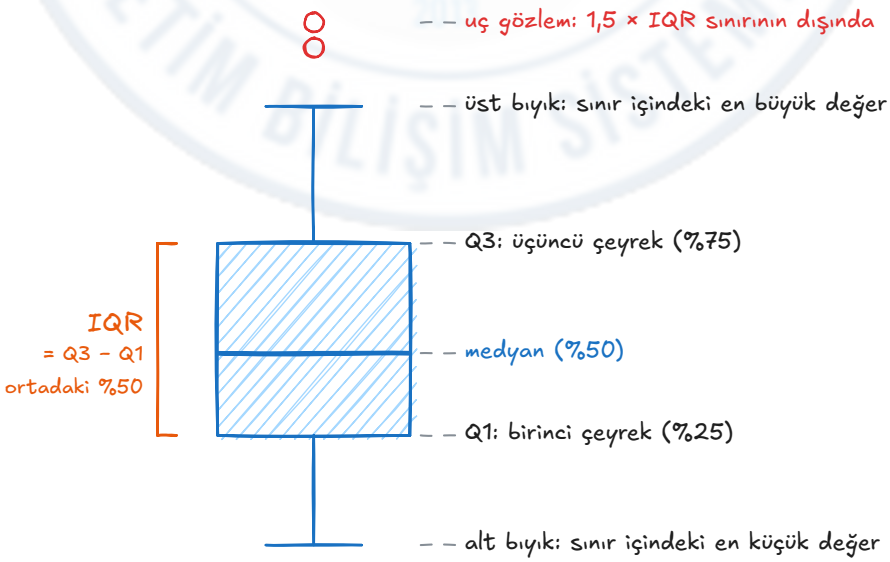
çıktı

```
          count  mean  median
Survived
0           424  30.6    28.0
1           290  28.3    28.0
```

Yaşı bilinenlerden hayatta kalmayan 424 kişinin ortalama yaşı 30,6, hayatta kalan 290 kişininki 28,3. İki grubun medyanı aynı: 28. Ortalama farkı küçük ve iki dağılım büyük ölçüde örtüşüyor. Bu hesaba yaşı eksik 177 kişi girmedi. Eksik yaşların hangi gruplarda yoğunlaştığı sonucu etkileyebilir.

8.10.1 Kutu grafiğini okumak

İki grubun dağılımlarını karşılaştırmak için **kutu grafiği** (*boxplot*) çok kullanışlıdır. Kutu grafiği beş sayıyı tek bir şekilde özetler (Şekil 8.3).



Şekil 8.3 Kutu grafiğinin anatomisi. Kutu ortadaki %50'yi, kutunun içindeki çizgi medyanı gösterir. Bıyıklar 1,5 × IQR sınırına kadar uzanan en uç gözlemlerde biter; bu sınırın ötesindeki değerler ayrı noktalar olarak çizilir.

Kutunun alt kenarı Q1'i, üst kenarı Q3'ü gösterir. Aradaki uzaklığa **çeyrekler açıklığı** (*interquartile range*, IQR) denir: IQR = Q3 − Q1. Kutunun içindeki çizgi medyandır. Bıyıklar, kutudan en fazla 1,5 × IQR uzaklığa kadar uzanır. Bu sınırların dışında kalan gözlemler tek tek nokta olarak çizilir. Bu noktalar "hatalı kayıt" anlamına gelmez; yalnızca dağılımın geri kalanından uzak oldukları için ayrıca dikkat çekilen gözlemlerdir.

```
df.boxplot(column="Age", by="Survived", grid=False)
plt.suptitle("")
plt.title("Sonuca göre yaş dağılımı")
plt.xlabel("Survived: 0 veya 1")
plt.ylabel("Yaş")
plt.show()
```

`boxplot(column="Age", by="Survived")` her sonuç grubu için ayrı bir kutu çizer. İki kutunun medyanları aynı hizadadır ve kutular neredeyse tamamen örtüşür. "Genç olmak tek başına hayatta kalmayı sağlar" sonucu buradan çıkmaz.

8.10.2 Yaşı aralıklara ayırmak

Yaş ile sonuç arasındaki ilişki doğrusal olmayabilir: Çocuklar ve yaşlılar farklı davranışlar gösterebilir. Bunu görmek için yaşı aralıklara bölelim:

```
yas_sinirlari = [0, 12, 18, 35, 60, float("inf")]
yas_etiketleri = ["0-12", "13-18", "19-35", "36-60", "61+"]
df["YasGrubu"] = pd.cut(df["Age"], bins=yas_sinirlari, labels=yas_etiketleri)

yas_grubu = df.groupby("YasGrubu", observed=True)["Survived"].agg(
    ["count", "sum", "mean"])
yas_grubu["Oran (%)"] = (yas_grubu["mean"] * 100).round(1)
yas_grubu[["count", "sum", "Oran (%)"]]
```

çıktı

YasGrubu	count	sum	Oran (%)
0-12	69	40	58.0
13-18	70	30	42.9
19-35	358	137	38.3
36-60	195	78	40.0
61+	22	5	22.7

`pd.cut` sürekli bir değişkeni aralıklara böler. `bins` sınır listesidir; `labels` her aralığın adıdır. Aralıklar varsayılan olarak sağ ucu dahil edecek biçimde kurulur: 12 yaşındaki bir çocuk "0–12" grubuna girer. `float("inf")` üst sınırın açık kalmasını sağlar. Yeni `YasGrubu` sütunu yalnızca defterdeki `df` nesnesine eklenir; CSV dosyası değişmez.

0–12 yaş grubunda oran %58,0 ile en yüksek; 61+ grubunda %22,7 ile en düşük. Ama 61+ grubunda yalnızca 22 kişi var; bu oranı ihtiyatla okumalıyız. Grup toplamı 714'tür; 177 eksik yaş hiçbir gruba girmedi. Aralık sınırlarını değiştirirsek tablonun ayrıntısı da değişebilir. Sınırların seçimi bir analiz kararıdır.

8.11 Ücret ve bilet sınıfı birlikte

```
df.groupby("Survived")["Fare"].agg(["count", "mean", "median"]).round(2)
```

çıktı

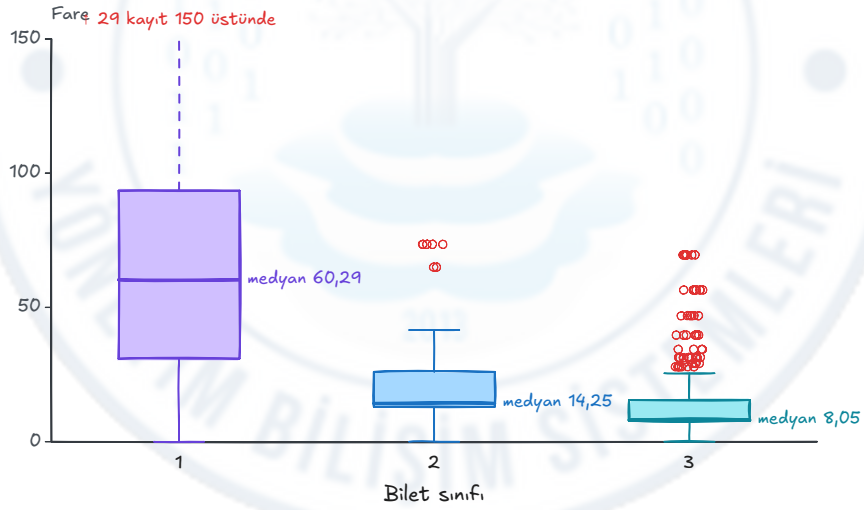
```
          count   mean  median
Survived
0           549  22.12    10.5
1           342  48.40    26.0
```

Hayatta kalmayanlarda medyan ücret 10,50, hayatta kalanlarda 26,00. Ortalamalar sırasıyla 22,12 ve 48,40. Her iki grupta da ortalamanın medyandan çok yüksek olması sağa çarpıklığa uygundur. İlk bakışta "yüksek ücret ödeyenler daha çok hayatta kalmış" diyebiliriz. Ama ücret, bilet sınıfıyla doğrudan bağlantılıdır:

```
df.groupby("Pclass")["Fare"].agg(["count", "mean", "median"]).round(2)
```

çıktı

```
        count   mean  median
Pclass
1         216  84.15   60.29
2         184  20.66   14.25
3         491  13.68    8.05
```



Şekil 8.4 Bilet sınıfına göre ücret dağılımı. Grafik 0–150 aralığına yakınlaştırılmıştır; daha yüksek ücretler veride durur ama burada görünmez.

Medyan ücret birinci sınıfta 60,29, ikinci sınıfta 14,25, üçüncü sınıfta 8,05 (Şekil 8.4). Birinci sınıfta ücretler genel olarak yüksek; aynı sınıfta hayatta kalma oranı da yüksekti. Dolayısıyla ücret ile sonuç arasındaki ham farkın bir bölümü, sınıf bileşiminden kaynaklanıyor olabilir. Bu veri, ikisinin ayrı etkilerini belirlemek için tek başına yeterli değildir.

8.12 Türetilmiş bir değişken: birlikte seyahat

Mevcut sütunlardan yeni bir değişken de üretebiliriz:

Görüntü ve veri
Kutu grafiğinde `plt.ylim(0, 150)` ile görünümü daraltmak yalnızca pencereyi küçültür; hesaplanan istatistikleri ya da veriyi değiştirmez. Görünmeyen büyük değerleri metinde ayrıca belirtmek gerekir.

```
df["AileBuyuklugu"] = df["SibSp"] + df["Parch"] + 1
df["TekYolcu"] = df["AileBuyuklugu"].eq(1)

tek_yolcu = df.groupby("TekYolcu")["Survived"].agg(["count", "sum", "mean"])
tek_yolcu["Oran (%)"] = (tek_yolcu["mean"] * 100).round(1)
tek_yolcu[["count", "sum", "Oran (%)"]]
```

çıktı

```
          count  sum  Oran (%)
TekYolcu
False       354  179      50.6
True        537  163      30.4
```

`SibSp + Parch + 1` iki sayım sütununu toplar ve yolcunun kendisini ekler. `.eq(1)` her kaydın değerinin 1'e eşit olup olmadığına bakar ve `True` ya da `False` üretir. Bu, Bölüm 12'de ayrıntılı göreceğimiz **özellik mühendisliğinin** ilk örneğidir.

Bu tanıma göre tek seyahat eden 537 yolcunun %30,4'ü, yanında kayıtlı yakını bulunan 354 yolcunun %50,6'sı hayatta kalmıştır. `True` satırı "tek", `False` satırı "tek değil" anlamındadır. Ama bu fark tek seyahat etmenin doğrudan etkisini kanıtlamaz; iki grubun yaş, cinsiyet ve sınıf bileşimi farklı olabilir. Ayrıca bu değişken yalnızca `SibSp` ve `Parch` alanlarında kayıtlı yakınları kapsar; gerçek aile ilişkilerinin tamamını ölçmez.

8.13 İki değişkeni birlikte incelemek

Tek bir değişkenle görülen ilişkiyi başka bir değişkenin içinde tekrar incelemek, **karıştırıcı değişkenleri** fark etmenin ilk adımıdır. Cinsiyet ve sınıfı birlikte ele alalım:

```
iki_grup = (df.groupby(["Sex", "Pclass"])["Survived"]
              .agg(["count", "mean"]).reset_index())
iki_grup["Oran (%)"] = (iki_grup["mean"] * 100).round(1)
iki_grup.pivot(index="Sex", columns="Pclass", values="Oran (%)")
```

çıktı

```
Pclass     1     2     3
Sex
female  96.8  92.1  50.0
male    36.9  15.7  13.5
```

`groupby(["Sex", "Pclass"])` iki anahtarla altı grup kurar. `.reset_index()` grup etiketlerini yeniden normal sütunlara çevirir. `pivot(...)` uzun tabloyu bir **çapraz tabloya** dönüştürür: satırda cinsiyet, sütunda sınıf.

Tablo 8.2 Cinsiyet ve sınıfa göre hayatta kalma oranları (%) ve hücre büyüklükleri.

	1. sınıf	2. sınıf	3. sınıf
Kadın	96,8 (91/94)	92,1 (70/76)	50,0 (72/144)
Erkek	36,9 (45/122)	15,7 (17/108)	13,5 (47/347)

Tablo 8.2'de iki şey birlikte görülüyor. Her cinsiyetin içinde sınıf farkı var: Kadınlarda birinci sınıftan üçüncü sınıfa oran %96,8'den %50,0'ye iniyor. Her sınıfın içinde de cinsiyet farkı var. Yani ne cinsiyet ne de sınıf tek başına bütün hikâyeyi anlatıyor. Çapraz tablodaki yüzdeler yeni bir hesaplama değil, ilk tablodaki oranların yeniden düzenlenmiş hâlidir.

> **Tanım 8.3 · Karıştırıcı değişken**
>
> İki değişken arasındaki gözlenen ilişkiyi kısmen ya da tamamen açıklayabilen üçüncü bir değişkendir. Ücret ile hayatta kalma arasındaki ilişkide bilet sınıfı böyle bir değişken olabilir: Sınıf hem ücreti hem de hayatta kalma koşullarını etkiler.

Bu ayrıştırma bile nedensellik ya da bütün olası etkenler için bir düzeltme anlamına gelmez. Ama tek değişkenli bir tablodan çıkaracağımız aceleci sonuçlara karşı güçlü bir kontroldür.

8.14 Eksik değerler rastgele mi?

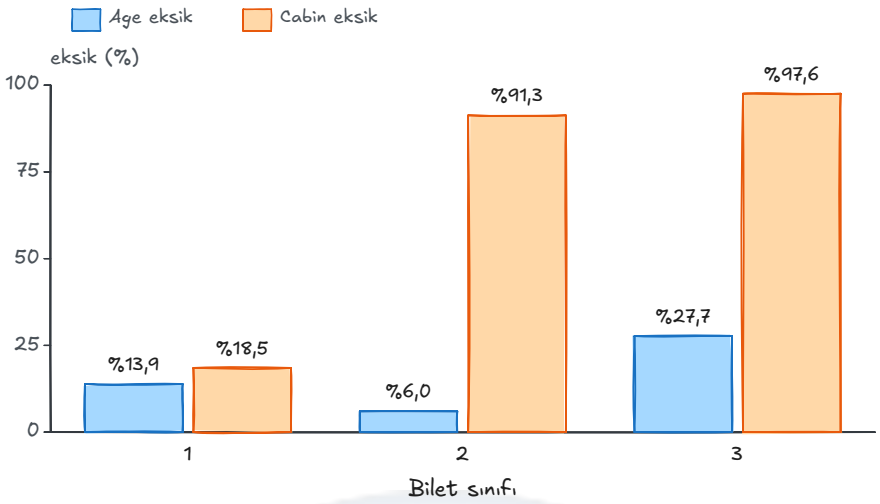
Eksiklik oranları gruplara göre değişiyorsa, yalnızca dolu kayıtlarla yapılan analizler bazı grupları eksik temsil eder. Bunu kontrol edelim:

```
yas_eksigi = df.groupby("Pclass")["Age"].apply(lambda s: s.isna().mean() * 100)
kabin_eksigi = df.groupby("Pclass")["Cabin"].apply(lambda s: s.isna().mean() * 100)
pd.concat([yas_eksigi.rename("Age eksik (%)"),
           kabin_eksigi.rename("Cabin eksik (%)")], axis=1).round(1)
```

çıktı

```
        Age eksik (%)  Cabin eksik (%)
Pclass
1                13.9             18.5
2                 6.0             91.3
3                27.7             97.6
```

`lambda s: ...` kısa ve isimsiz bir işlev tanımlar. `s.isna().mean()` bir gruptaki eksik hücrelerin oranıdır; `True` değerleri 1, `False` değerleri 0 olarak ortalanır.



Şekil 8.5 Bilet sınıfına göre eksiklik oranları. Kabin bilgisi ikinci ve üçüncü sınıfta neredeyse hiç kaydedilmemiş; yaş eksikliği de üçüncü sınıfta belirgin biçimde yüksek.

Yaş eksikliği birinci, ikinci ve üçüncü sınıfta sırasıyla %13,9, %6,0 ve %27,7. Kabin eksikliği ise %18,5, %91,3 ve %97,6 (Şekil 8.5). Bunun iki önemli sonucu var:

- Kabin kodu bilinen yolcular neredeyse tamamen birinci sınıftan geliyor. Bu yolcular üzerinden bütün sınıflar için çıkarım yapmak ciddi bir **seçilim yanlılığı** yaratır.
- Yaşı eksik yolcuları doğrudan silmek, üçüncü sınıfı diğerlerine göre daha fazla dışarıda bırakır ve yolcu bileşimini değiştirir.

Eksikliklerin nedenini bu CSV dosyasından kesin olarak bilemeyiz. Ama örüntü, eksikliğin rastgele dağılmadığını açıkça gösteriyor. Bu bulgu Bölüm 10'daki doldurma kararlarımızın temelini oluşturacak.

8.15 Uç değerleri incelemek, otomatik silmemek

```
q1 = df["Fare"].quantile(0.25)
q3 = df["Fare"].quantile(0.75)
iqr = q3 - q1
ust_esik = q3 + 1.5 * iqr

print("Q1:", round(q1, 4), "| Q3:", round(q3, 4), "| IQR:", round(iqr, 4))
print("Üst inceleme eşiği:", round(ust_esik, 4))
print("Eşik üstündeki kayıt:", df["Fare"].gt(ust_esik).sum())
print("Sıfır ücretli kayıt:", df["Fare"].eq(0).sum())
df.nlargest(5, "Fare")[["PassengerId", "Pclass", "Fare", "Survived"]]
```

çıktı

```
Q1: 7.9104 | Q3: 31.0 | IQR: 23.0896
Üst inceleme eşiği: 65.6344
Eşik üstündeki kayıt: 116
Sıfır ücretli kayıt: 15
     PassengerId  Pclass      Fare  Survived
258          259       1  512.3292         1
679          680       1  512.3292         1
737          738       1  512.3292         1
```

```
27           28       1  263.0000         0
88           89       1  263.0000         1
```

`.quantile(0.25)` ve `.quantile(0.75)` birinci ve üçüncü çeyreği verir. Üst eşik Q3 + 1,5 × IQR, kutu grafiğindeki bıyığın sınırıyla aynı kuraldır. `.gt(esik)` eşikten büyük değerleri, `.eq(0)` sıfırları işaretler. `df.nlargest(5, "Fare")` en yüksek beş ücreti getirir.

Üst inceleme eşiği 65,63. Bu eşiğin üstünde 116 kayıt, tam sıfır değerde 15 kayıt var. En büyük beş ücretin hepsi birinci sınıf yolculara ait. Bu 116 kaydı "hatalı" deyip silmek, gerçek yüksek ücretli yolcuları ve sınıf dağılımını bozar. Sıfır ücret de gerçek, özel ya da hatalı bir kayıt olabilir; kaynak bilgisi olmadan düzeltmeyiz.

> **Dikkat**
>
> Çeyrek kuralı bir **alarmdır**, bir veri temizleme kararı değildir. Eşiğin üstündeki değerler incelenecek gözlemlerdir. Hangi durumda hangi müdahalenin yapılacağını Bölüm 11'de ayrıntılı göreceğiz.

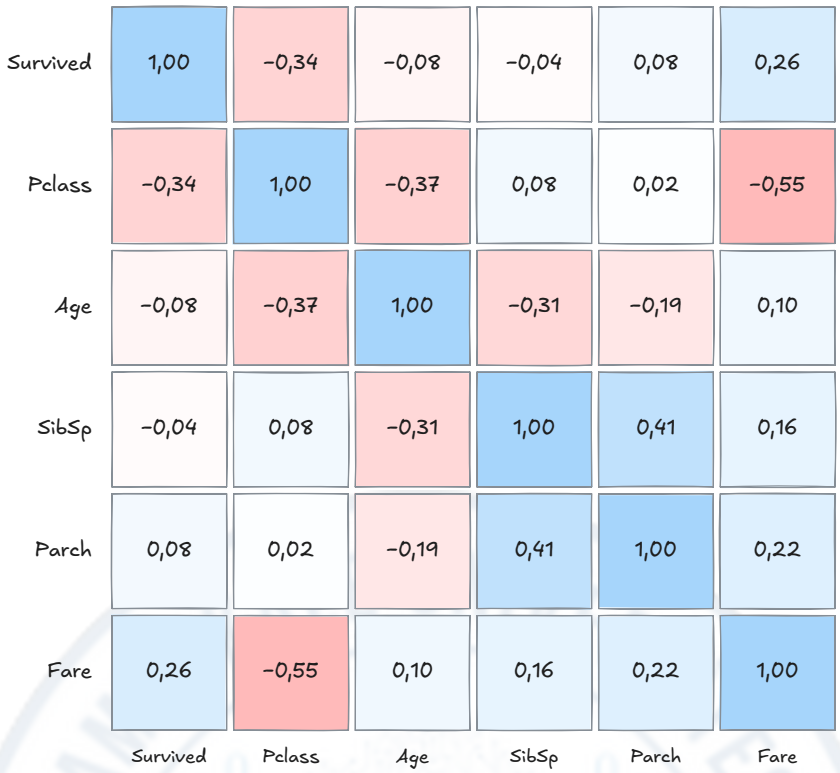
8.16 Korelasyon: kısa bir sayısal özet

```python
sayisal = ["Survived", "Pclass", "Age", "SibSp", "Parch", "Fare"]
df[sayisal].corr().round(2)
```

çıktı

```
          Survived  Pclass   Age  SibSp  Parch  Fare
Survived      1.00   -0.34 -0.08  -0.04   0.08  0.26
Pclass       -0.34    1.00 -0.37   0.08   0.02 -0.55
Age          -0.08   -0.37  1.00  -0.31  -0.19  0.10
SibSp        -0.04    0.08 -0.31   1.00   0.41  0.16
Parch         0.08    0.02 -0.19   0.41   1.00  0.22
Fare          0.26   -0.55  0.10   0.16   0.22  1.00
```

`.corr()` sayısal sütun çiftleri için **Pearson korelasyon katsayısını** hesaplar. Katsayı −1 ile +1 arasında değer alır. +1 kusursuz pozitif, −1 kusursuz negatif doğrusal ilişkiyi, 0 ise doğrusal ilişki olmadığını gösterir. Matris simetriktir; köşegende her değişkenin kendisiyle korelasyonu 1'dir.



Şekil 8.6 Korelasyon matrisi. Mavi tonlar pozitif, kırmızı tonlar negatif ilişkiyi; rengin koyuluğu ilişkinin gücünü gösterir.

`Survived` ile `Pclass` arasında yaklaşık −0,34, `Fare` ile +0,26, `Age` ile −0,08 korelasyon var (Şekil 8.6). Negatif işareti doğru okumak gerekir: `Pclass` kodu büyüdükçe üçüncü sınıfa doğru gidilir, hayatta kalma oranı düşer. `Fare` ile `Pclass` arasındaki −0,55 ise ücret ve sınıfın birbirinden bağımsız iki kanıt gibi yorumlanmaması gerektiğini bir kez daha gösteriyor.

Korelasyonu okurken üç uyarıyı akılda tutalım:

1. `Survived` 0/1, `Pclass` sıralı bir kategori kodudur. Bu kodların korelasyonu kaba bir betimleyici özettir; sürekli ölçümler arasındaki korelasyon gibi yorumlanmaz.
2. Yaşla ilgili küçük korelasyon, yaş gruplarında hiç fark olmadığı anlamına gelmez. Yaş grupları tablosunda gördüğümüz gibi ilişki doğrusal olmayabilir. Pearson katsayısı yalnızca doğrusal ilişkiyi ölçer.
3. Korelasyon nedensellik değildir.

8.17 Keşifsel analizden çıkan sonuçlar

Bu bölümde 891 yolcu üzerinde yaptığımız incelemeyi dört bulguda toplayabiliriz:

1. 891 yolcunun 342'si (%38,4) hayatta kalmış. Sınıf ve cinsiyet grupları arasında belirgin oran farkları var.
2. `Age`, `Cabin` ve `Embarked` sütunlarında eksik değer var. `Cabin` eksikliği çok yüksek ve sınıfa göre eşit dağılmıyor; yaş eksikliği de üçüncü sınıfta yoğunlaşıyor.
3. `Fare` sağa çarpık; medyan ortalamayı tamamlıyor. Yüksek ücretleri otomatik olarak silmedik.

4. İki değişkeni birlikte incelemek, tek değişkenli yorumları zenginleştirdi. Ücret ile sınıf, cinsiyet ile sınıf birlikte düşünülmeli.

Bu analizin sınırlarını da açıkça yazmak gerekir. Bir anlamlılık testi, güven aralığı, nedensel model ya da tahmin modeli kurmadık. Yüzdeler bu örneklemdeki gözlemlerdir. İleride bir model kurduğumuzda `Survived` hedef olarak ayrılacak ve hedef bilgisi girdilere sızdırılmayacak. Eksik değer doldurma, ölçekleme ve benzeri işlemler ise eğitim ve test ayrımından sonra, yalnızca eğitim verisinden öğrenilecek. Kısım III'ün geri kalanı bu işlemleri tek tek ele alıyor.

Örnek 8.1 · Bir bulguyu raporlamak

"Bu veri kümesinde birinci sınıf yolcuların gözlenen hayatta kalma oranı %63,0, üçüncü sınıf yolcularınki %24,2'dir. Bu fark gözlemsel bir ilişkidir; diğer koşullar değerlendirilmeden sınıfın tek başına nedensel etkisi olarak yorumlanmamalıdır."

Bu cümle dört şey yapıyor: Hangi veride ölçüldüğünü söylüyor ("bu veri kümesinde"), sayıyı veriyor, bulgunun türünü adlandırıyor ("gözlemsel bir ilişki") ve sınırını açıkça belirtiyor.

Uygulama 8.1 · Defterde deneyin

1. `Embarked` gruplarındaki oran farklarına sınıf dağılımı katkı yapıyor olabilir mi? `pd.crosstab(df["Embarked"], df["Pclass"])` ile grup sayılarını inceleyin. C limanından binenlerin ne kadarı birinci sınıftır?
2. 61+ yaş grubundaki %22,7 oranına bakarak neden güçlü bir genelleme yapmamalıyız?
3. `Fare` için ortalama ile medyan arasındaki farkı hangi grafik destekliyor? Eşik üstündeki 116 kaydı silersek hangi karşılaştırmalar değişir?

Bölüm özeti

- Keşifsel veri analizi, model kurmadan önce verinin yapısını, sorunlarını ve ilişkilerini tanıma sürecidir. Bulguları ön işleme kararlarının gerekçesini oluşturur.
- Veri sözlüğü her sütunun anlamını ve ölçüm türünü açıklar. Veri tipi değerin nasıl saklandığını, ölçüm türü neyi ifade ettiğini anlatır; `Pclass` tam sayı ama kategoridir.
- `shape`, `dtypes`, `info()`, `duplicated()` ve `isna()` bir tablonun ilk sağlık kontrolünü yapar. Titanic'te 891 satır, 12 sütun vardır; `Cabin` %77,1, `Age` %19,9, `Embarked` %0,2 eksiktir.
- Ortalama uç değerlere duyarlı, medyan dayanıklıdır. `Fare`'in ortalaması (32,20) medyanının (14,45) iki katından fazladır; dağılım sağa çarpıktır.
- Gruplar farklı büyüklükte olduğunda sayılar değil grup içi oranlar karşılaştırılır. 0/1 hedefin ortalaması oranı verir.
- Kutu grafiği medyan, çeyrekler, IQR ve 1,5 × IQR sınırını tek bir şekilde gösterir. Sınır dışındaki noktalar incelenecek gözlemlerdir, otomatik hata değildir.
- İki değişkeni birlikte incelemek karıştırıcı değişkenleri fark etmeyi sağlar. Ücret ile sonuç ilişkisinin bir kısmı sınıftan kaynaklanabilir.
- Eksiklik sınıflara göre eşit dağılmıyor; eksik satırları silmek ya da yalnızca dolu kayıtlara bakmak seçilim yanlılığı yaratabilir.
- Korelasyon yalnızca doğrusal ilişkiyi özetler; küçük bir katsayı ilişki olmadığını, büyük bir katsayı nedensellik olduğunu göstermez.

Kaynaklar

McKinney, W. (2022). *Python for Data Analysis* (3. baskı). O'Reilly. Bölüm 5 ve 10.

Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3. baskı). O'Reilly. Bölüm 2: "Explore and Visualize the Data to Gain Insights".

NIST/SEMATECH. *e-Handbook of Statistical Methods*, Bölüm 1: Exploratory Data Analysis. https://doi.org/10.18434/M32189

pandas geliştirme ekibi. *pandas User Guide: Group by: split-apply-combine*. https://pandas.pydata.org/docs/user_guide/groupby.html

9

BÖLÜM

Veri ön işlemeye giriş: kalite, tekrarlar ve veri ayrımı

Bu bölümde

- Veri ön işlemenin keşifsel analizden farkını ve bir yöntemi seçerken sorulan üç soruyu açıklamak,
- veri sözlüğüne dayanarak şema, geçerli değer, kategori tutarlılığı, eksik değer temsili ve mantıksal tutarlılık kontrollerini yapmak,
- biçim hatası ile geçersiz değeri ayırt etmek,
- tam satır tekrarı, kimlik tekrarı ve seçilmiş alan tekrarını birbirinden ayırmak,
- girdileri ve hedefi ayırıp eğitim ve test kümelerini tabakalı olarak oluşturmak,
- öğrenilen işlemler ile sabit kuralları ayırmak; `fit` ve `transform` adımlarının neden ayrı tutulduğunu sayısal bir örnekle göstermek.

Kaynak: YBS 319 ders sayfası ve uygulama defteri

Bölüm 8'de Titanic verisini tanıdık. Eksik yaşları, çok eksik kabin kodlarını, sağa çarpık ücret dağılımını ve inceleme gerektiren yüksek ücretleri belirledik. Şimdi bu bulgulara göre veriyi bir modele hazırlayacağız.

Veri ön işleme, ham veriyi belirli bir analiz ya da modelleme amacı için kullanılabilir hâle getiren işlemlerin bütünüdür. Keşifsel analiz ile ön işleme farklı sorular sorar:

- **Keşifsel analizin sorusu:** "Yaş hangi yolcularda eksik? Yüksek ücretler hangi sınıflarda yoğunlaşıyor?" Sorunu ve bağlamını anlamamızı sağlar.
- **Ön işlemenin sorusu:** "Yaşı eksik yolcuları silmeli miyiz, doldurmalı mıyız? Ücreti olduğu gibi mi kullanacağız, dönüştürecek miyiz?" Bu sorulara göre bir yöntem seçer ve işlemin veriyi nasıl değiştirdiğine bakarız.

Bu bölüm ve izleyen altı bölüm, ön işlemenin adımlarını tek tek ele alıyor. Bölüm 15'te bütün adımları tek bir tekrarlanabilir hatta birleştireceğiz.

9.1 Ön işlemenin akışı

Ön işlemeyi beş adımlı bir akış olarak düşünebiliriz (Şekil 9.1).



Şekil 9.1 Veri ön işlemenin akışı. Veri ayrımı, öğrenilen işlemlerden önce gelir; böylece hazırlama adımları yalnızca eğitim verisinden öğrenilir.

Akıştaki sıralama tesadüfi değildir. Temel kalite kontrolleri, veri ayrımından önce bütün tablo üzerinde yapılabilir; çünkü bunlar veriden bir parametre öğrenmez. Eksik değer doldurma ve ölçekleme gibi işlemler ise veriden bir şey öğrenir: bir medyan, bir ortalama, bir kategori listesi. Bunların yalnızca eğitim verisinden öğrenilmesi gerekir. Bu yüzden veri ayrımı, hazırlama adımlarından önce gelir.

9.1.1 Bir yöntemi seçerken üç soru

Ön işlemede çok sayıda yöntemle karşılaşacağız. Bir yöntemi seçmeden önce şu üç soruyu sorarız:

1. **Neyi düzeltmek istiyoruz?** Eksik bilgi mi, biçim hatası mı, farklı ölçekler mi, fazla karmaşıklık mı?
2. **Ne kaybedebiliriz?** Satır, yorumlanabilirlik, doğal dağılım ya da alt grupların temsili?
3. **Başarısını nasıl kontrol edeceğiz?** Kalite kontrolleri, işlem öncesi ve sonrası karşılaştırması ya da eğitim içi doğrulama?

> **Dikkat**
>
> Bir yöntem ders kitabında anlatıldı diye kullanılmak zorunda değildir. Titanic için temel bileşen analizi (PCA) ya da yapay örnek üretme (SMOTE) kullanmamız gerekmez. Yüksek ücretli yolcuları da yalnızca ücretleri yüksek olduğu için silmeyiz. Yöntem, verideki somut soruna göre seçilir.

9.2 Veriyi tanıma ve kalite kontrolleri

Bölüm 8'deki veri sözlüğünü ön işleme açısından yeniden okuyalım. Bu kez her sütun için "hazırlarken neye dikkat edeceğiz?" sorusunu soruyoruz.

Tablo 9.1 Titanic sütunlarının hazırlamadaki rolleri.

Sütun ya da grup	Anlamı ve türü	Hazırlamada dikkat
`PassengerId`	Kayıt kimliği	Kayıtları izlemek ve kimlik tekrarını kontrol etmek için kullanılır; girdi yapılmaz
`Survived`	İkili hedef: 0 / 1	Girdi sütunlarından ayrı tutulur
`Pclass`	Sıralı kategori: 1 / 2 / 3	Sınıflar arasındaki mesafenin eşit olduğu varsayılmaz
`Age` , `Fare`	Sayısal ölçümler	Eksiklik, geçerli aralık ve dağılım kontrol edilir
`SibSp` , `Parch`	Negatif olmayan tam sayı sayımlar	Ondalıklı ya da negatif kayıtlar incelenir
`Sex` , `Embarked`	Sırasız (nominal) kategoriler	Beklenen etiketler ve tutarsız yazımlar kontrol edilir
`Name` , `Ticket` , `Cabin`	Metin ya da kod	Ham metin yerine bilgi türetme seçenekleri değerlendirilir

9.2.1 Beş kontrol

Bir veri kümesinin ilk sağlık kontrolünü beş başlıkta yaparız:

Şema
: Beklenen sütunlar var mı? Sayısal olması gereken sütunlarda metin bulunuyor mu?

Geçerli değer
: `Survived` yalnızca 0 ve 1 mi? `Pclass` yalnızca 1, 2 ve 3 mü? Yaş ve ücret negatif değil mi?

Kategori tutarlılığı
: "male", "Male" ve " male " gerçekten aynı etiket mi? Etiketlerin aynı anlamda kullanıldığını doğruladıktan sonra boşlukları temizleyip yazımları birleştirebiliriz.

Eksik değer temsili
: Boş hücre, "?", "unknown" ya da −999 aynı anlama mı geliyor? Örneğin −999 kodunu eksik saymadan önce veri sözlüğünde ne anlama geldiğine bakarız.

Mantıksal tutarlılık

Kimlik benzersiz mi? Sayımlar tam sayı mı? Alan bilgisine göre birbiriyle çelişen kayıtlar var mı?

Geçerli değer kontrolünü birkaç satırlık kodla yapabiliriz:

```
import pandas as pd
df = pd.read_csv("Titanic-Dataset.csv")

print(df["Survived"].isin([0, 1]).all())       # hepsi 0 ya da 1 mi?
print(df["Pclass"].isin([1, 2, 3]).all())      # hepsi 1, 2 ya da 3 mü?
print((df["Age"] < 0).sum(), (df["Fare"] < 0).sum())   # negatif yaş ve ücret
print(sorted(df["Sex"].unique()), sorted(df["Embarked"].dropna().unique()))
```

çıktı

```
True
True
0 0
['female', 'male'] ['C', 'Q', 'S']
```

`.isin([...])` her değerin verilen listede olup olmadığını işaretler; `.all()` bütün işaretlerin `True` olup olmadığını sorar. `.unique()` bir sütundaki farklı değerleri listeler. Titanic dosyası bu kontrollerin hepsinden geçiyor: Hedef ve sınıf kodları geçerli, negatif yaş ya da ücret yok, kategori etiketleri tutarlı.

> **Dikkat**
>
> `Age = 0,42` bir bebeğin yaşı olabilir; ondalıklı yaşları otomatik olarak yuvarlamayın. `Fare = 0` değerini de kaynağa bakmadan eksik ya da hatalı saymayın. Alışılmadık görünen her değer hata değildir.

9.2.2 Biçim hatası ve geçersiz değer

Gerçek veriler bu kadar temiz gelmez. Derste küçük bir örnek üzerinde iki farklı sorunu ayırmıştık:

```
ornek = pd.DataFrame({"Embarked": [" S ", "q", "C"],
                      "Age": ["22", "bilinmiyor", "-5"]})
ornek["Embarked"] = ornek["Embarked"].str.strip().str.upper()
ornek["Age"] = pd.to_numeric(ornek["Age"], errors="coerce")
ornek
```

çıktı

	Embarked	Age
0	S	22.0
1	Q	NaN
2	C	-5.0

`.str.strip()` metnin başındaki ve sonundaki boşlukları kaldırır; `.str.upper()` harfleri büyütür. Böylece `" S "` değeri `"S"` olur. `pd.to_numeric(..., errors="coerce")` metni sayıya çevirir; çevrilemeyen bir değerle karşılaşınca hata vermek yerine onu `NaN` yapar. `"bilinmiyor"` metni

eksik değere dönüştü.

Ama dikkat edin: `"-5"` metni başarıyla −5 sayısına çevrildi. Biçim düzeldi, ama negatif bir yaş hâlâ geçersizdir. Bu örnek iki kavramı ayırır: **Biçim hatası**, değerin yanlış yazılmış ya da yanlış tipte saklanmış olmasıdır. **Geçersiz değer** ise doğru biçimde yazılmış ama mantıksal olarak mümkün olmayan değerdir. Tür dönüşümü, mantıksal geçerlilik kontrolünün yerini tutmaz.

Titanic dosyasında da aynı temizlik kurallarını uyguluyoruz. Bu dosyadaki etiketler zaten tutarlı olduğu için sayılar değişmez; ama aynı kurallar ileride gelecek yeni kayıtları da korur:

```
df["Sex"] = df["Sex"].str.strip().str.lower()
df["Embarked"] = df["Embarked"].str.strip().str.upper()
df[["Sex", "Embarked"]].value_counts(dropna=False).head(3)
```

çıktı

```
Sex     Embarked
male    S           441
female  S           203
male    C            95
Name: count, dtype: int64
```

9.3 Yinelenen kayıtlar

Yinelenen kayıt, aynı gözlemin yanlışlıkla birden fazla kez kaydedilmesidir. Bir tekrarı tanımlamak için önce satırın birimini bilmemiz gerekir. Titanic'te bir satır bir yolcudur. Üç farklı tekrar kontrolü yaparız:

Tablo 9.2 Üç tekrar kontrolü.

Kontrol	Ne sorar?	Nasıl karar veririz?
Tam satır tekrarı	Bütün alanlar aynı mı?	Verinin iki kez yüklenmiş olma ihtimalini araştırırız
Kimlik tekrarı	`PassengerId` tekrar ediyor mu?	Aynı kayıt mı, çelişen kayıt mı kontrol ederiz
Seçilmiş alan tekrarı	İsim, bilet gibi alanlar aynı mı?	Alan bilgisi olmadan otomatik silmeyiz

```
print(df.duplicated().sum())                  # tam satır tekrarı
print(df["PassengerId"].duplicated().sum())   # kimlik tekrarı
print(df["Ticket"].duplicated().sum())        # bilet kodu tekrarı
df = df.drop_duplicates()                     # varsa tam tekrarları ayıklar
```

çıktı

```
0
0
210
```

İlk iki sayı sıfır; bu veride tekrar nedeniyle yolcu silmeyeceğiz. `.drop_duplicates()` satır sayısını değiştirmez. Üçüncü sayıya dikkat edin: 210 yolcunun bilet kodu başka bir yolcuyla aynı. Ama bunlar tekrarlanan kayıtlar değildir. Aile üyeleri ve birlikte seyahat edenler aynı bilet koduyla

yolculuk etmiş olabilir. Ortak bilet kodu, aynı yolcu olduklarını göstermez.

Bir tekrar bulduğumuzda önce kayıtları yan yana inceleriz ve çelişen alanları belirleriz. Hangi kaydın doğru olduğunu kaynak, zaman damgası ya da kayıt kuralıyla araştırırız. Hangi kayıtları neden sildiğimizi ya da birleştirdiğimizi not ederiz.

Tekrar ve sızıntı
Doğrulanmış aynı gözlemler hem eğitim hem test kümesine düşerse, model testte daha önce gördüğü bir kaydı "tahmin eder" ve değerlendirme fazla iyimser çıkar (Bölüm 7).

9.4 Veri ayrımı

9.4.1 Girdi ve hedef

Model kurarken veriyi önce iki parçaya ayırırız. Geleneksel olarak **X** girdi özelliklerini, **y** ise hedefi gösterir. `Survived` sütununu X'ten çıkarırız; aksi hâlde model cevabı girdinin içinde bulur. `PassengerId` de bir kimlik olduğu için girdiye alınmaz.

```
from sklearn.model_selection import train_test_split

X = df.drop(columns=["Survived", "PassengerId"])   # hedef ve kimlik çıkar
y = df["Survived"]                                  # hedef ayrı tutulur

X_egitim, X_test, y_egitim, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42)
print(X_egitim.shape, X_test.shape)
```

çıktı

```
(712, 10) (179, 10)
```

`scikit-learn` (kısaca `sklearn`), Python'daki en yaygın makine öğrenmesi kütüphanesidir. `train_test_split` veriyi rastgele iki parçaya böler ve dört çıktı verir: eğitim girdileri, test girdileri, eğitim hedefleri ve test hedefleri. Parametreleri tek tek açalım:

- `test_size=0.20` verinin yaklaşık %20'sini teste ayırır. 891 yolcudan 712'si eğitime, 179'u teste düştü.
- `stratify=y` hedef sınıf oranlarının iki parçada yaklaşık aynı kalmasını sağlar.
- `random_state=42` bölmenin her çalıştırmada aynı çıkmasını sağlar. 42 sayısının özel bir anlamı yoktur; herhangi bir sabit sayı aynı işi görür.

9.4.2 Tabakalı bölme

Tabakalı bölmenin etkisini görelim:

```
print(y.mean().round(3), y_egitim.mean().round(3), y_test.mean().round(3))
```

çıktı

```
0.384 0.383 0.385
```

Bütün veride, eğitimde ve testte hayatta kalma oranı yaklaşık %38,4. Rastgele bir bölmede şans eseri test kümesine daha çok hayatta kalan düşebilirdi; `stratify` bunu önler.

Ancak tabakalı bölme her sorunu çözmez. Ortak aile ya da bilet nedeniyle bazı yolcular birbirinden bağımsız değildir. Aynı aileden yolcuların bir kısmı eğitimde, bir kısmı testte bulunursa model testte, eğitimde gördüğü bir ailenin üyelerini tahmin etmiş olur. Yeni ailelere genellemeyi ölçmek istiyorsak, aynı gruptaki yolcuları aynı tarafta tutan **grup temelli bölme** ayrıca değerlendirilir.

Bölme ayrıntılarına neden bu kadar önem veriyoruz? Çünkü bundan sonraki her hazırlama adımında şu kuralı uygulayacağız: **Öğrenilen her şey eğitim verisinden öğrenilir; test verisi yalnızca dönüştürülür.**

9.5 Öğrenilen işlemler ve sabit kurallar

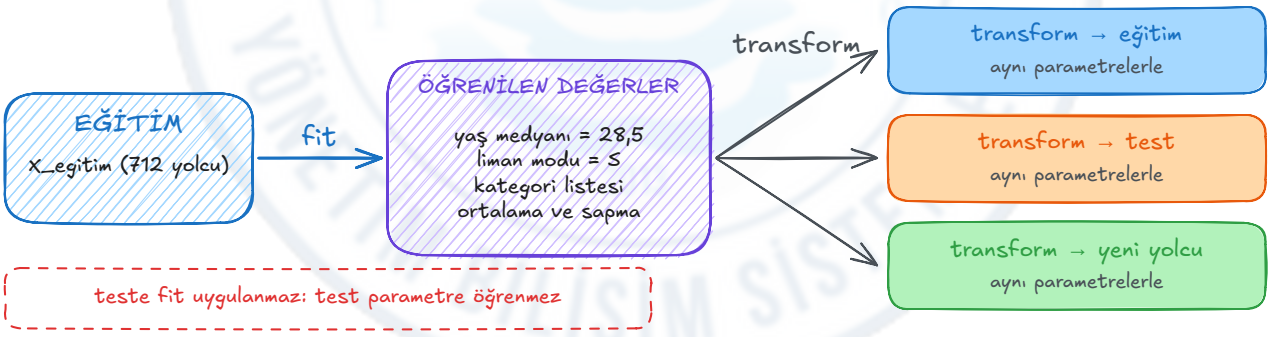
Ön işleme adımlarını iki gruba ayırabiliriz.

Öğrenilen işlemler, veriden bir parametre hesaplar. Yaşı doldurmak için medyan, kategoriyi doldurmak için en sık değer (mod), kodlama için kategori listesi, ölçekleme için ortalama ve standart sapma, aykırı değer taraması için IQR sınırları, PCA bileşenleri ve model tabanlı özellik seçimi bu gruptadır. Bunların hepsi yalnızca eğitim verisinden öğrenilmelidir.

Sabit kurallar ise veri dağılımından hiçbir şey öğrenmez. Veri sözlüğüne dayanan boşluk temizliği ya da `SibSp + Parch + 1` gibi her satıra aynı biçimde uygulanan formüller bu gruptadır. Bunlar eğitim ve teste aynı şekilde, ayrı ayrı uygulanabilir.

9.5.1 fit ve transform

`scikit-learn` bu ayrımı iki yöntemle somutlaştırır (Şekil 9.2):



Şekil 9.2 fit ve transform. Parametreler yalnızca eğitimden öğrenilir; aynı parametreler eğitim, test ve yeni veriye uygulanır.

- `fit`: Parametreleri öğrenir. Örneğin bir doldurucu, eğitim yaşlarının medyanını hesaplar ve saklar.
- `transform`: Öğrenilmiş parametrelerle veriyi dönüştürür. Eksik yaşlara saklanan medyanı yazar.
- `fit_transform`: İkisini aynı veri üzerinde art arda yapar. Yalnızca eğitim verisinde kullanılır.

Test verisine **hiçbir zaman** `fit` uygulamayız; yalnızca `transform` uygularız.

9.5.2 Test değeri eğitim parametresini etkileyebilir mi?

Kuralın neden bu kadar önemli olduğunu küçük bir sayısal örnekle görelim. Eğitim kümesindeki yaşlar [20, 30, 40] olsun. Test kümesinde tek bir yolcu var ve yaşı 80. Medyan yerine ortalama kullanarak sızıntıyı daha görünür kılalım.

```
import numpy as np
egitim_yaslari = np.array([20, 30, 40])
test_yasi = np.array([80])

dogru = egitim_yaslari.mean()
yanlis = np.concatenate([egitim_yaslari, test_yasi]).mean()
print("Yalnızca eğitimden:", dogru)
print("Eğitim + testten:", yanlis)
```

çıktı

```
Yalnızca eğitimden: 30.0
Eğitim + testten: 42.5
```

Doğru yöntemde doldurma değeri 30'dur. Doldurma değerini bütün veriden hesaplarsak 42,5 olur: Test yolcusunun yaşı, eğitim yolcularına yazılacak değeri değiştirmiştir. Model, test kümesi hakkında görmemesi gereken bir bilgiyi dolaylı olarak almış olur. Bölüm 7'de **örtük sızıntı** dediğimiz şey tam olarak budur.

Gerçek kullanımda bunun karşılığı şudur: Model kullanıma alındığında yarın gelecek yolcuların yaşlarını bugünden bilemeyiz. Eğitimde de bilmiyormuş gibi davranmamız gerekir.

> **Tanım 9.1 · Doğru sıra**
>
> Önce temel şema ve kayıt bütünlüğünü kontrol edin. Ardından veriyi ayırın. Doldurma ve diğer öğrenilen işlemler için `fit` yalnızca eğitimde çalışır; test verisi aynı kurallarla `transform` edilir. Ayrıntılı keşifsel analiz ve yöntem seçimi de eğitim bölümünde yürütülmelidir.

9.5.3 Çapraz doğrulamada ön işleme

Bölüm 7'de doğrulama kümesinden söz etmiştik. Pratikte doğrulama çoğu zaman **çapraz doğrulamayla** yapılır: Eğitim verisi birkaç parçaya (kata) bölünür; her seferinde bir kat doğrulama, diğerleri eğitim için kullanılır. Bu durumda aynı kural her katta geçerlidir. Ön işlemeyi bir kez bütün eğitim verisine uygulayıp sonra katlara ayırmak da sızıntı yaratır. Her katta doldurucu ve ölçekleyici, yalnızca o katın eğitim bölümünden öğrenilmelidir. Bölüm 15'te bunu otomatik olarak sağlayan aracı göreceğiz.

Uygulama 9.1 · Sızıntıyı bulun

Bir öğrenci şu sırayla çalışıyor. Hangi adımlar sızıntı yaratır?

1. CSV dosyasını okuyup boşlukları temizledi.
2. Bütün verideki yaş medyanıyla eksik yaşları doldurdu.
3. `SibSp + Parch + 1` ile aile büyüklüğünü hesapladı.
4. Veriyi eğitim ve test olarak ayırdı.
5. Ücreti, eğitim verisinin ortalaması ve standart sapmasıyla ölçekledi; teste de aynı değerleri uyguladı.

Cevap. Yalnızca 2. adım sızıntı yaratır: Medyan, test yolcularının yaşları da katılarak hesaplandı. 1. ve 3. adımlar sabit kurallardır; bölmeden önce yapılmaları sorun değildir. 5. adım doğrudur. Düzeltmek için 2. adımı 4. adımdan sonraya taşımak ve medyanı yalnızca eğitimden hesaplamak gerekir.

Bölüm özeti

- Keşifsel analiz sorunu ve bağlamını anlamaya, ön işleme ise bu sorunlara karşı yöntem seçip uygulamaya odaklanır. Bir yöntem seçilirken neyin düzeltileceği, neyin kaybedilebileceği ve başarının nasıl kontrol edileceği sorulur.
- Kalite kontrolleri beş başlıkta yapılır: şema, geçerli değer, kategori tutarlılığı, eksik değer temsili ve mantıksal tutarlılık.
- Biçim hatası ile geçersiz değer farklıdır: `"-5"` metnini sayıya çevirmek biçimi düzeltir ama negatif yaşı geçerli yapmaz.
- Tam satır tekrarı, kimlik tekrarı ve seçilmiş alan tekrarı farklı sorunlardır. Titanic'te ilk ikisi yoktur; ortak bilet kodu tekrar kaydı değildir.
- Girdiler (X) ve hedef (y) ayrılır; kimlik ve hedef girdiye alınmaz. `stratify=y` sınıf oranlarını korur; ilişkili gözlemler için grup temelli bölme ayrıca düşünülür.
- Öğrenilen işlemler (medyan, mod, kategori listesi, ölçek parametreleri) yalnızca eğitimden öğrenilir. Sabit kurallar (boşluk temizliği, satır bazlı formüller) veri dağılımından bir şey öğrenmez.
- `fit` öğrenir, `transform` uygular. Teste yalnızca `transform` uygulanır. Parametreyi bütün veriden hesaplamak, test bilgisini eğitime sızdırır.
- Çapraz doğrulamada ön işleme her katın eğitim bölümünde yeniden öğrenilmelidir.

Kaynaklar

Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3. baskı). O'Reilly. Bölüm 2: "Create a Test Set" ve "Prepare the Data for Machine Learning Algorithms".

Müller, A. C. ve Guido, S. (2016). *Introduction to Machine Learning with Python*. O'Reilly. Bölüm 3: "Preprocessing and Scaling".

scikit-learn geliştiricileri. *Common pitfalls and recommended practices*. https://scikit-learn.org/stable/common_pitfalls.html

10

BÖLÜM

Eksik veri

Bu bölümde

- Eksik değeri sıfırdan, “hayır”dan ve uygulanamaz alandan ayırmak,
- satır silme, sütun çıkarma ve doldurma seçeneklerini gerekçeleri ve sınırlarıyla karşılaştırmak,
- eksikliğin rastgele olup olmadığının doldurma kararını neden değiştirdiğini açıklamak; MCAR, MAR ve MNAR kavramlarını tanımak,
- sütun eksiklik oranı ile satır kaybının neden farklı olduğunu göstermek,
- ortalama, medyan, mod ve grup medyanıyla doldurmayı `pandas` ve `SimpleImputer` ile uygulamak,
- eksiklik göstergesi üretmenin ve hedefe göre doldurmamanın gerekçesini açıklamak,
- doldurma sonrasında neyin kontrol edileceğini bilmek.

Kaynak: YBS 319 ders sayfası ve uygulama defteri

Eksik değer, bir gözlemin belirli bir alanının kayıtlı olmamasıdır. Sıfır, "hayır" ya da "kabin yok" ile aynı şey değildir. Bölüm 8'de Titanic verisinde `Cabin` sütununun 687, `Age` sütununun 177, `Embarked` sütununun 2 gözlemde eksik olduğunu gördük. Eksikliklerin bilet sınıflarına göre farklılaştığını da gördük. Bu nedenle eksik satırları silmek ya da eksikleri gelişigüzel doldurmak, yolcu bileşimini ve sonuçları değiştirebilir.

Bu bölümde önce eksik değerlerle neler yapabileceğimizi, sonra bu seçenekler arasında nasıl karar vereceğimizi konuşacağız. Bütün hesapları, Bölüm 9'da ayırdığımız 712 kişilik eğitim verisi üzerinde yapıyoruz.

```
X_egitim.isna().sum()
```

çıktı

```
Pclass        0
Name          0
Sex           0
Age         137
SibSp         0
Parch         0
Ticket        0
Fare          0
Cabin       552
Embarked      2
dtype: int64
```

Eğitim verisinde `Age` 137, `Cabin` 552, `Embarked` 2 gözlemde eksik. Bunlar artık eğitim sayılarıdır; bütün veriyle aynı değildir.

10.1 Üç temel seçenek

Eksik değerlerle karşılaştığımızda üç temel seçeneğimiz vardır:

Satır silmek. Herhangi bir kullanılacak alanı eksik olan satırı tamamen çıkarırız. Az sayıda kayıt eksikse ve silmenin etkisi incelendiyse düşünülebilir. Ama örneklem ve alt grupların temsili azalır.

Sütun çıkarmak. Çok eksik ve ek bilgi sağlamayan bir alanı tamamen bırakırız. Ama alanın kısmi olarak gözlenen değerli bilgisi de kaybolur.

Doldurmak. Eksik hücreye makul bir değer yazarız. Bütün gözlemleri korur, ama eksik bilginin kendisini geri getirmez. Doldurduğumuz değer bir tahmindir, ölçülmüş bir değer değildir.

Yaşı eksik satırları silmeyi asıl eğitim verisini değiştirmeden ayrı bir tabloda deneyelim:

```
yas_silinmis = X_egitim.dropna(subset=["Age"])
print(len(X_egitim), "→", len(yas_silinmis))
```

çıktı

```
712 → 575
```

`dropna(subset=["Age"])` yalnızca `Age` sütunu eksik olan satırları siler. 712 yolcudan 575'i kalıyor; 137 yolcu, yani eğitim verisinin beşte biri gidiyor. Peki giden yolcular kimler?

```
eksik_yas = X_egitim["Age"].isna()
pd.DataFrame({"Toplam": X_egitim.groupby("Pclass").size(),
              "Yaşı eksik": eksik_yas.groupby(X_egitim["Pclass"]).sum()})
```

çıktı

```
        Toplam  Yaşı eksik
Pclass
1          171          26
2          150           9
3          391         102
```

Silinecek 137 yolcunun 102'si üçüncü sınıftan. Silme işlemi üçüncü sınıfı orantısız biçimde küçültür. Eksiklik gruplarda farklıysa, silmek grup temsilini de değiştirir. Bu yüzden son hattımızda yolcuları koruyacağız.

10.2 Eksiklik neden oluştu?

Eksikliği görmek kolaydır. Asıl zor soru, değerin **neden** kaydedilmediğidir. Derste bu soruyu şöyle özetlemiştik: Eksiklik rastgele oluştuysa, makul bir değerle doldurmak çoğu zaman güvenli bir başlangıçtır. Ama eksikliğin arkasında bir mekanizma varsa, yani değerin eksik olması o değerin kendisiyle ilişkiliyse, basit bir doldurma sorunu çözmez; hatta gizleyebilir.

İstatistikte bu ayrımı yapmak için üç kavram kullanılır. Bu adları ezberlemekten çok, her birinin arkasındaki soruyu anlamak önemlidir (Şekil 10.1).



Şekil 10.1 Eksikliğin üç mekanizması. Soru, bir değerin eksik olma olasılığının neye bağlı olduğudur.

Tamamen rastgele eksiklik (MCAR)

Missing Completely At Random. Eksik olma olasılığı hiçbir şeye bağlı değildir. Örneğin yaştan, sınıftan ve diğer alanlardan bağımsız olarak rastgele seçilmiş bazı formlar aktarım sırasında kaybolmuştur. Bu durumda tam kayıtlarla çalışmak birçok hesapta yansız sonuç verir; ama örneklem küçülür.

Gözlenen bilgiyle açıklanan eksiklik (MAR)

Missing At Random. Eksiklik, elimizde bulunan başka bir alanla açıklanabilir; o alan hesaba katıldığında eksik değerin kendisine ayrıca bağlı değildir. Örneğin üçüncü sınıfta yaş kaydı daha sık eksiktir; ama aynı sınıfın içinde eksik olma durumu gerçek yaşa bağlı değildir. Bu

durumda doldururken eksikliği açıklayan alandan (burada sınıftan) yararlanmak önemlidir. Adına aldanmayın: MAR, "rastgele birkaç hücre boş" demek değildir.

Gözlenmeyen nedenle eksiklik (MNAR)

Missing Not At Random. Eksiklik, eksik değerin kendisiyle ya da hiç gözlemediğimiz bir etkenle ilişkilidir. Örneğin çok yaşlı kişiler yaşlarını paylaşmamış olsun ve elimizdeki hiçbir alan bunu açıklamasın. Bu durumda standart doldurma yöntemleri yanlılığı gidermez; ek bilgi ve farklı varsayımlarla duyarlılık analizi gerekir.

> **Dikkat**
>
> Bu üç durumu yalnızca tabloya bakarak kesin olarak ayırt edemeyiz. Eksik yaşları görmediğimiz için, eksikliğin gerçek yaşla ilişkili olup olmadığını gözlenen veriden doğrulayamayız. Yapabileceğimiz şey ipuçlarını toplamaktır.

10.2.1 İpuçlarını toplamak

Eksikliğin mekanizmasını anlamak için dört yoldan yararlanırız:

1. **Kayıt sürecini araştırmak.** Soru kimlere sorulmuş? Ölçüm cihazı ne zaman durmuş? Veri aktarımında bir filtre var mı?
2. **Eksikliği bir değişken gibi incelemek.** `Age` eksikliği `Pclass`, `Sex` ya da başka alanlara göre farklı mı? Bölüm 8'de bunu yaptık.
3. **Örüntüye bakmak.** Aynı satırdaki alanlar birlikte mi eksik? Belirli bir tarih, kaynak ya da grup mu eksik?
4. **Sonucu dikkatle yorumlamak.** Eksiklik gözlenen bir alanla ilişkiliyse MCAR varsayımını sorgularız. Ama bu ilişki, tek başına MAR sonucuna varmak için yeterli değildir.

Titanic'te üçüncü sınıfta yaş eksikliğinin daha sık olması önemli bir ipucudur. Bu nedenle doldurmada `Pclass` bilgisinden yararlanabiliriz. Yine de yaş eksikliğinin kesin olarak MAR olduğunu söyleyemeyiz. `Cabin` boşluğu da "yolcunun kabini yoktu" anlamına gelmez; kabin bilgisinin kaydedilmediğini gösterir.

10.2.2 Ölçülmemiş ve uygulanamaz

Bir değerin eksik olmasının bir nedeni daha vardır: Soru o kişi için hiç geçerli olmayabilir. Evi olmayan birine "konut kredisi tutarı" sorulmaz. Bu hücre boş kalır, ama bu bir ölçüm eksikliği değil, **yapısal bir boşluktur**. Veri sözlüğü bunun sıfır mı yoksa ayrı bir durum mu olduğunu belirler. Bu hücreyi diğer kişilerin kredi ortalamasıyla doldurmak, anlamı tamamen bozar.

10.3 Eksiklik oranına göre karar

"Yüzde kaç eksikse ne yapılır?" sorusunun tek bir cevabı yoktur. Önce üç ayrı oranı hesaplarız:

- sütundaki eksik hücre oranı,
- kullanılacak alanlardan en az biri eksik olan satır oranı,
- alt gruplardaki eksik oranları.

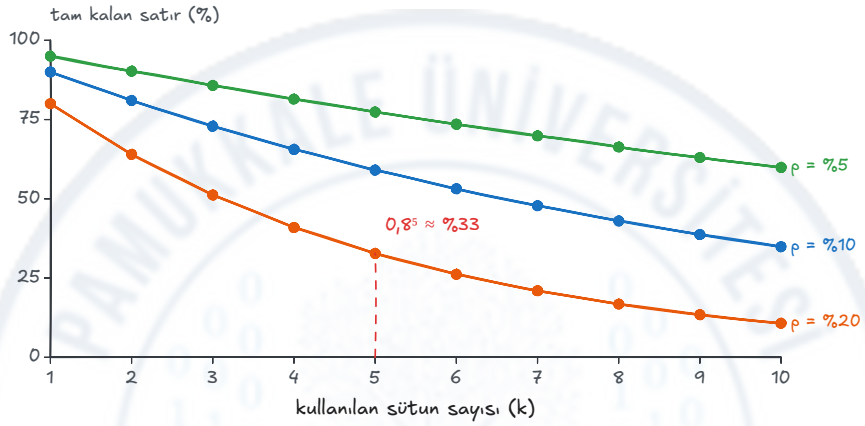
10.3.1 Sütun eksikliği ile satır kaybı aynı değildir

"Her sütunda %5 eksik var" ile "satırların %5'i eksik" aynı durum değildir. Eksikler farklı satırlara dağılmışsa, eksik içeren bütün satırları silmek çok daha büyük bir kayıp yaratır.

Basit bir hesap yapalım. Her sütunun eksik olma olasılığı p olsun ve sütunlar birbirinden bağımsız olsun. k sütun kullandığımızda bir satırın tamamen dolu olma olasılığı:

$$P(\text{satır tam}) = (1-p)^k$$

Her sütunda %20 eksiklik varsa ve 5 sütun kullanıyorsak, satırların yalnızca $0{,}8^5 \approx 0{,}33$'ü, yani üçte biri tam kalır (Şekil 10.2). Her sütun "yalnızca %20" eksik görünürken, satır silme verinin üçte ikisini yok eder.



Şekil 10.2 Kullanılan sütun sayısı arttıkça tam kalan satırların oranı. Her sütunda aynı eksiklik oranı ve sütunlar arasında bağımsızlık varsayılmıştır.

Titanic'te kalan satır sayısını görmek için bu formüle gerek yok; eksikliği doğrudan sayarız. Ama hesap, sütun bazındaki küçük oranların satır düzeyinde nasıl büyüyebileceğini gösteriyor.

10.3.2 Oranlar yalnızca bir başlangıçtır

Tablo 10.1 Eksiklik düzeyine göre sorulacak sorular. Yüzdeler kesin eşik değil, örnektir.

Durum	Ne düşünürüz?	Dikkat
Az eksik, örneğin %1	Kaç gözlem kayboluyor? Belirli bir grupta mı toplanıyor? Kaynaktan geri alınabilir mi?	Oran küçük olsa da hangi grupların kaybolduğuna bakarız
Dikkate değer eksik, örneğin %20	Satırları koruyan doldurma, eksiklik göstergesi, grup temsili ve doğrulama incelenir	Yöntem yalnızca yüzdeye göre seçilmez; değişken türü ve ilişkiler de önemlidir
Çok eksik, örneğin %70	Alan ne kadar değerli? Diğer alanlar doldurmaya yetecek bilgi taşıyor mu?	Çok sayıda hücreyi doldurmak, ölçülmüş bilgi miktarını artırmaz

Aynı %1'lik kayıp küçük bir azınlık grubunun tamamını silebilir. Aynı %70'lik eksiklikte alanın gözlenen kısmı ya da eksikliğin kendisi yararlı bilgi taşıyabilir. Yüzdenin yanında kalan gözlem sayısına, alanın sağlayacağı bilgiye, eksikliğin nedenine ve diğer alanların doldurmaya ne kadar

yardım edebileceğine bakarız.

10.3.3 Hangi seçenek ne zaman?

Satır silme, az sayıda kayıt kayboluyorsa, yeterli örneklem kalıyorsa ve silmenin varsayımları makulse düşünülebilir. Eksiklik tamamen rastgeleyse tam kayıt analizi birçok hesapta yansızdır. Ama eksiklik bir mekanizmaya bağlıysa silme temsili ve sonuçları bozar.

Doldurma, satır kaybı önemliyse, diğer alanlar bilgi taşıyorsa ya da kullanılacak model eksik girdi kabul etmiyorsa düşünülür. Hangi değerin neden üretildiği kayda geçirilir.

Sütun çıkarma, alan büyük ölçüde eksikse, tahmin anında güvenilir biçimde elde edilemiyorsa ya da ek bilgi sağlamıyorsa düşünülür. Ama önce alanın gözlenen kısmının ve eksiklik göstergesinin katkısı değerlendirilir.

> **Dikkat**
>
> Hedef değişkenin kendisi eksikse durum farklıdır. `Survived` etiketi eksik olan bir yolcunun doğru sonucunu kaynaktan öğrenmeye çalışırız. Öğrenemediğimiz kayıtları denetimli öğrenmede etiketli yolcularla birlikte kullanamayız; hedefi "doldurmak" modelin kendi tahminini öğrenmesi demektir.

10.4 Doldurma yöntemleri

Tablo 10.2 Doldurma yöntemleri, gerekçeleri ve yan etkileri.

Yöntem	Ne zaman?	Neyi değiştirebilir?
Ortalama	Gözlenen değerler yaklaşık simetrik dağılıyorsa	Ortalama çevresinde yığılma oluşturur; yayılımı ve ilişkileri değiştirir; uç değerlerden etkilenir
Medyan	Dağılım çarpıksa ya da uç değerler varsa	Bütün eksiklere aynı değer verilir; gerçek çeşitliliği ve belirsizliği yansıtmaz
Mod ya da ayrı "Bilinmiyor" etiketi	Kategorik alanlarda	Mod en sık kategoriyi büyütür; "Bilinmiyor" gerçek bir kategoriyle aynı anlama gelmez
Grup medyanı	Sınıf gibi gözlenen gruplar değişkeni açıklıyorsa	Yeterli gözlem ve eğitimde görülmemiş gruplar için bir yedek değer gerekir
KNN, regresyon, iteratif	Diğer alanların ilişkilerinden yararlanmak istendiğinde	KNN ölçeğe duyarlıdır; tek bir tahmin belirsizliği eksik temsil eder
Çoklu doldurma	İstatistiksel çıkarımda belirsizliği hesaba katmak için	Birden çok analiz sonucu uygun kurallarla birleştirilir

10.4.1 Ortalama ve medyan

```
yas_ortalama = X_egitim["Age"].mean()
yas_medyan = X_egitim["Age"].median()
print(round(yas_ortalama, 2), yas_medyan)

yas_ornegi = X_egitim[["Age"]].copy()
yas_ornegi["DoldurulmusYas"] = yas_ornegi["Age"].fillna(yas_medyan)
yas_ornegi[yas_ornegi["Age"].isna()].head()
```

çıktı

```
29.81 28.5
     Age  DoldurulmusYas
692  NaN            28.5
481  NaN            28.5
527  NaN            28.5
557  NaN            28.5
828  NaN            28.5
```

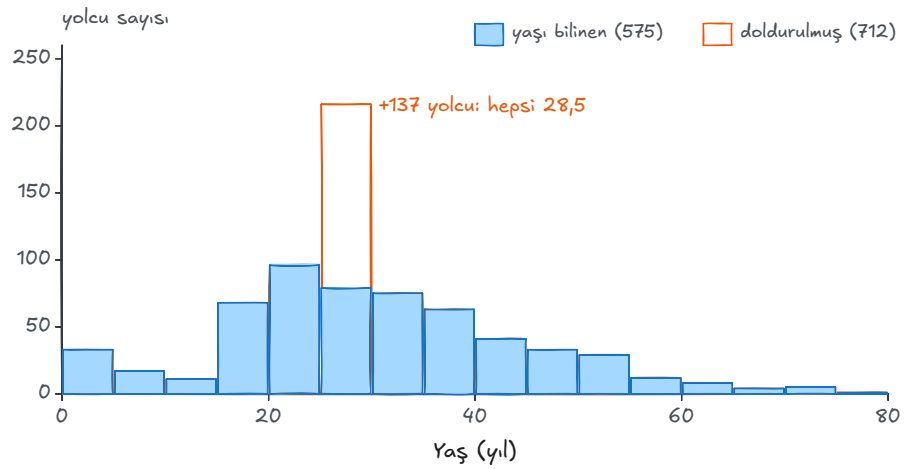
Eğitim yaş ortalaması 29,81, medyanı 28,5. İki değer birbirine yakın, çünkü yaş dağılımı çok çarpık değil. `.fillna(...)` eksik hücrelere verilen değeri yazar. Son satır, yalnızca yaşı eksik olan yolcuları seçer ve doldurma sonrası hâllerini gösterir: Hepsine aynı değer, 28,5 yazıldı.

Bu değerlerin iki özelliğine dikkat edin. Birincisi, yalnızca eğitimden hesaplandılar; test verisi için ayrı bir medyan hesaplanmayacak. İkincisi, doldurulan yaşlar gözlenmiş gerçek yaşlar değil.

10.4.2 Dağılım nasıl değişti?

Ham ve doldurulmuş yaşları aynı eksende karşılaştıralım:

```
yas_ornegi["Age"].dropna().plot.hist(bins=16, alpha=0.5, label="Orijinal Age")
yas_ornegi["DoldurulmusYas"].plot.hist(bins=16, alpha=0.3, label="Doldurulmuş yaş")
plt.xlabel("Yaş (yıl)")
plt.ylabel("Yolcu sayısı")
plt.legend()
plt.show()
```



Şekil 10.3 Medyanla doldurmanın etkisi. Doldurulmuş histogram bütün eğitim yolcularını içerir; 137 eksik yaşın hepsine aynı değer yazıldığı için medyanın bulunduğu aralıkta belirgin bir yığılma oluşur.

Ham histogram yalnızca yaşı bilinen 575 yolcuyu, doldurulmuş histogram ise 712 yolcunun hepsini içerir. Doldurulmuş dağılımda 28,5 yaşın bulunduğu aralık birden yükseliyor (Şekil 10.3). Yayılım daralıyor ve dağılımın şekli değişiyor. Medyanla doldurmanın bedeli budur: Basit ve dayanıklıdır, ama gerçek çeşitliliği yansıtmaz.

10.4.3 Kategoriler için mod

```
liman_modu = X_egitim["Embarked"].mode().iloc[0]
liman_dolu = X_egitim["Embarked"].fillna(liman_modu)
print(liman_modu)
liman_dolu.value_counts()
```

çıktı

```
S
Embarked
S    518
C    139
Q     55
Name: count, dtype: int64
```

Mod en sık görülen değerdir. `.mode()` bir seri döndürür, çünkü birden fazla mod olabilir; `.iloc[0]` sıfırıncı sıradaki ilk modu seçer. Eğitimde en sık liman S; iki eksik liman S olarak doldurulunca S grubu 516'dan 518'e çıktı.

Mod ile doldurma baskın kategoriyi büyütür. `Embarked` yalnızca iki eksik içerdiği için bu etki ihmal edilebilir. Daha fazla kategori eksikliği olan bir durumda ayrı bir "Bilinmiyor" etiketi daha uygun olabilir.

10.4.4 SimpleImputer ile gerçek veriye uygulama

Örnekleri karşılaştırdıktan sonra kararımızı veriyoruz: `Age` ve `Fare` için eğitim medyanı, `Sex` ve `Embarked` için eğitim modu. Bunu `scikit-learn`'ün `SimpleImputer` aracıyla yapıyoruz, çünkü Bölüm 9'daki `fit` ve `transform` ayrımını otomatik olarak sağlar:

```
from sklearn.impute import SimpleImputer

X_egitim_dolu = X_egitim.copy()
X_test_dolu = X_test.copy()
doldurulacak_sayisal = ["Age", "Fare"]

sayisal_doldurucu = SimpleImputer(strategy="median")
sayisal_doldurucu.fit(X_egitim[doldurulacak_sayisal])     # yalnızca eğitimden öğrenir
X_egitim_dolu[doldurulacak_sayisal] =
sayisal_doldurucu.transform(X_egitim[doldurulacak_sayisal])
X_test_dolu[doldurulacak_sayisal] =
sayisal_doldurucu.transform(X_test[doldurulacak_sayisal])
sayisal_doldurucu.statistics_
```

çıktı

```
array([28.5   , 14.4542])
```

`strategy="median"` her sütunun eğitim medyanını kullanır. `.statistics_` öğrenilmiş değerleri gösterir: yaş için 28,5, ücret için 14,4542. `Fare` bu dosyada hiç eksik değil; ama kuralı tanımlamış olduk. Model kullanıma alındığında ücreti eksik gelen yeni bir yolcu olursa ne yapılacağı artık belli.

```
doldurulacak_kategori = ["Sex", "Embarked"]
kategori_doldurucu = SimpleImputer(strategy="most_frequent")
kategori_doldurucu.fit(X_egitim[doldurulacak_kategori])
X_egitim_dolu[doldurulacak_kategori] =
kategori_doldurucu.transform(X_egitim[doldurulacak_kategori])
X_test_dolu[doldurulacak_kategori] =
kategori_doldurucu.transform(X_test[doldurulacak_kategori])
X_egitim_dolu[["Age", "Fare", "Sex", "Embarked"]].isna().sum()
```

çıktı

```
Age         0
Fare        0
Sex         0
Embarked    0
dtype: int64
```

`strategy="most_frequent"` her sütunun en sık eğitim etiketini kullanır. Kullanacağımız dört alanda artık eksik kalmadı. Ham `Cabin` hâlâ eksik içeriyor; onu metin olarak kullanmayacağız. Birazdan nasıl ele alacağımızı göreceğiz.

10.4.5 Grup medyanıyla doldurma

Eksikliğin sınıfa göre değiştiğini biliyoruz ve sınıflar arasında yaş dağılımı da farklı. Bu durumda tek bir genel medyan yerine her sınıfın kendi medyanını kullanmak daha anlamlı olabilir:

```
sinif_medyanlari = X_egitim.groupby("Pclass")["Age"].median()
print(sinif_medyanlari)

grup_yas = X_egitim[["Pclass", "Age"]].copy()
grup_degeri = grup_yas["Pclass"].map(sinif_medyanlari).fillna(yas_medyan)
```

```
grup_yas["Doldurulmus"] = grup_yas["Age"].fillna(grup_degeri)
grup_yas[grup_yas["Age"].isna()].head()
```

çıktı

```
Pclass
1    38.0
2    29.0
3    24.0
Name: Age, dtype: float64
     Pclass  Age  Doldurulmus
692       3  NaN         24.0
481       2  NaN         29.0
527       1  NaN         38.0
557       1  NaN         38.0
828       3  NaN         24.0
```

Birinci sınıfın eğitim yaş medyanı 38, ikinci sınıfınki 29, üçüncü sınıfınki 24. `.map(sinif_medyanlari)` her satırın sınıf kodunu o sınıfın medyanıyla eşler. `.fillna(yas_medyan)` ise eğitimde görülmemiş bir sınıf gelirse genel medyanı yedek olarak kullanır. Eksik yaşlı birinci sınıf yolcusu artık 38, üçüncü sınıf yolcusu 24 alıyor.

Bu yaklaşımın da bir koşulu var: Her grupta yeterli sayıda eğitim gözlemi bulunmalı. Az gözlemli bir grubun medyanı kararsız olur.

10.4.6 Daha gelişmiş yöntemler

KNN ile doldurma, eksik değeri, diğer alanlarda en çok benzeyen eğitim kayıtlarının değerlerinden hesaplar. Örneğin ücreti ve sınıfı benzer beş yolcunun yaşlarından yararlanır. Ancak "benzerlik" bir uzaklık hesabına dayandığı için değişkenlerin ölçeği sonucu doğrudan etkiler; ölçekleme ile doldurmayı birlikte planlamak gerekir (Bölüm 13).

Daha karmaşık bir yöntem her zaman daha iyi değildir. Değişkenler arasında gerçek bir ilişki yoksa ya da eksiklik gözlenmeyen bir nedene bağlıysa, gelişmiş bir algoritma sorunu kendiliğinden çözmez. Yöntemler eğitim içi doğrulamayla karşılaştırılır.

Tek bir doldurulmuş değerin, eksik değer hakkındaki belirsizliği temsil etmediğini de unutmayalım. İstatistiksel çıkarım yapılacaksa (örneğin bir güven aralığı hesaplanacaksa), bu belirsizliği hesaba katmak için **çoklu doldurma** kullanılır: Eksikler birkaç farklı makul biçimde doldurulur, analiz her birinde tekrarlanır ve sonuçlar uygun kurallarla birleştirilir.

10.5 Eksiklik göstergesi

Bir yaşı doldurduğumuzda, o yaşın başlangıçta bilinmediği bilgisi kaybolur. Oysa bu bilgi kendi başına anlamlı olabilir: Yaşın kaydedilmemiş olması, yolcunun sınıfı ya da kayıt süreci hakkında bir şey söylüyor olabilir. Bu yüzden doldurmadan **önce** bir eksiklik göstergesi üretiriz:

```
gostergeler = pd.DataFrame({
    "YasEksik": X_egitim["Age"].isna().astype(int),
    "KabinBilgisiVar": X_egitim["Cabin"].notna().astype(int)})
gostergeler.mean().round(3)
```

çıktı
```
YasEksik           0.192
KabinBilgisiVar    0.225
dtype: float64
```

`.isna()` eksikleri `True`, `.notna()` dolu değerleri `True` olarak işaretler; `.astype(int)` bu değerleri 1 ve 0'a çevirir. Eğitim yolcularının %19,2'sinin yaşı eksik, %22,5'inin kabin bilgisi var.

`KabinBilgisiVar` göstergesi `Cabin` sorununa da bir çözüm getiriyor. Ham kabin kodu %77,5 eksik ve çok çeşitli; doğrudan kullanmak zor. Ama kabin bilgisinin kaydedilip kaydedilmediği, Bölüm 8'de gördüğümüz gibi sınıfla güçlü biçimde ilişkili. Ham kodu çıkarıp yalnızca bu göstergeyi tutabiliriz.

> **Dikkat**
>
> `KabinBilgisiVar = 0` olan bir yolcunun kabini olmadığını bilemeyiz. Gösterge, kabin bilgisinin **kaydedilip kaydedilmediğini** gösterir. Bu gösterge modelde yararlı çıksa bile "kabinsiz yolcular" diye yorumlanmamalıdır.

10.5.1 Hedefe göre doldurmayın

Akla şu fikir gelebilir: Hayatta kalanların yaş medyanı ile kalmayanlarınki farklıysa, eksik yaşları her yolcunun kendi sonuç grubunun medyanıyla dolduralım. Bu ciddi bir hatadır. Yeni bir yolcunun sonucunu, tahmin sırasında bilmeyiz; zaten tahmin etmeye çalıştığımız şey odur. Eksik yaşı `Survived` gruplarına göre doldurmak, hedef bilgisini girdiye sızdırır. Bu bir **hedef sızıntısıdır**. Hedef, doldurmada ya da yeni özellik üretiminde kullanılmaz.

10.6 Doldurma sonrasında neyi kontrol ederiz?

Doldurma işlemi bitti diye iş bitmiş sayılmaz. Şunları kontrol ederiz:

- İşlem öncesi ve sonrası dağılım, medyan ve ortalama, yayılım ve grup oranları nasıl değişti?
- İmkânsız değerler (negatif yaş gibi) ya da kategori listesinde olmayan doldurmalar oluştu mu?
- Eksikliği gösteren alan, doldurmadan önce üretildi mi?

Farklı doldurma yöntemlerini karşılaştırmak için eğitim verisindeki bazı gözlenen değerleri geçici olarak gizleyip her yöntemin onları ne kadar iyi tahmin ettiğine bakabiliriz. Ama bu denemede oluşturduğumuz yapay eksiklik, gerçek eksiklikten farklı olabilir.

Eksikliğin gözlenmeyen bir nedene bağlı olma ihtimali varsa bir **duyarlılık analizi** yaparız: Eksik yaşların medyan çevresinde, daha genç ya da daha yaşlı olduğu farklı makul senaryolar altında sonuçlar değişiyor mu? Böylece varsayımımız değiştiğinde sonuçların ne kadar oynadığını görürüz.

Uygulama 10.1 · Titanic için karar günlüğü

Bu bölümdeki kararlarımızı bir tabloda toplayalım:

Age. %19,2 eksik ve eksiklik sınıfa göre değişiyor. Satır silmek üçüncü sınıfı orantısız küçültür. Karar: eğitim medyanıyla doldur, doldurmadan önce `YasEksik` göstergesini üret. Grup medyanı bir alternatif olarak doğrulamada karşılaştırılabilir.

Embarked. Yalnızca 2 eksik. Karar: eğitim moduyla (S) doldur.

Cabin. %77,5 eksik, ham kod çok çeşitli. Karar: ham kodu çıkar, `KabinBilgisiVar` göstergesini tut ve dikkatle yorumla.

Fare. Eksik yok. Karar: yeni veride eksik gelirse eğitim medyanını kullanacak kuralı tanımla.

Survived. Doldurmada ve özellik üretiminde kullanılmaz.

Bölüm özeti

- Eksik değer sıfır, "hayır" ya da "yok" demek değildir. Uygulanamaz bir alanın boşluğu (yapısal boşluk) da ölçüm eksikliğinden ayrılır.
- Üç temel seçenek satır silmek, sütun çıkarmak ve doldurmaktır. Titanic eğitiminde yaşı eksik satırları silmek 137 yolcuyu, bunların 102'si üçüncü sınıftan olmak üzere, dışarıda bırakır.
- Eksiklik rastgele oluştuysa doldurma makul bir başlangıçtır; eksikliğin arkasında değerin kendisiyle ilişkili bir mekanizma varsa basit doldurma yanlılığı gidermez. Bu durumlar MCAR, MAR ve MNAR olarak adlandırılır ve gözlenen tablodan kesin ayırt edilemez.
- Sütun eksikliği satır kaybıyla aynı değildir: Her sütunda %20 eksik ve 5 bağımsız sütun varsa satırların yalnızca üçte biri tam kalır.
- Medyan çarpık dağılımlara ve uç değerlere ortalamadan daha dayanıklıdır; ama bütün eksiklere aynı değeri yazdığı için yığılma yaratır. Kategoriler için mod ya da ayrı bir "Bilinmiyor" etiketi kullanılır.
- Doldurma değerleri yalnızca eğitimden öğrenilir. `SimpleImputer` bunu `fit` ve `transform` ile sağlar.
- Eksiklik göstergesi doldurmadan önce üretilir. `KabinBilgisiVar` kaydın doluluğunu gösterir, kabin sahipliğini değil.
- Eksik bir girdiyi hedef gruplarına göre doldurmak hedef sızıntısıdır.

Kaynaklar

van Buuren, S. (2018). *Flexible Imputation of Missing Data* (2. baskı). Chapman & Hall/CRC. Bölüm 1.2–1.3.

James, G., Witten, D., Hastie, T., Tibshirani, R. ve Taylor, J. (2023). *An Introduction to Statistical Learning with Applications in Python*. Springer. Bölüm 12.3.

Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3. baskı). O'Reilly. Bölüm 2: "Clean the Data".

scikit-learn geliştiricileri. *Imputation of missing values*. https://scikit-learn.org/stable/modules/impute.html

11

BÖLÜM

Hatalı ve aykırı değerler

Bu bölümde

- Hatalı değer, aykırı değer ve etkili gözlem kavramlarını ayırmak,
- bir aykırılığın üç olası kaynağını (kayıt hatası, geçerli nadir gözlem, farklı süreç) tanımak,
- IQR ve Z-skoru taramalarını hesaplamak; hangisinin hangi dağılımda uygun olduğunu açıklamak,
- iki yöntemin neden farklı yolcuları işaretlediğini göstermek,
- düzeltme, eksik sayma, satır çıkarma, koruma, kırpma ve dönüştürme seçenekleri arasında gerekçeli karar vermek.

Kaynak: YBS 319 ders sayfası ve uygulama defteri

Bölüm 8'de ücret dağılımının uzun bir sağ kuyruğu olduğunu ve 116 ücretin üst inceleme eşiğini aştığını gördük. O bölümde bir uyarı yapmıştık: Çeyrek kuralı bir alarmdır, bir veri temizleme kararı değildir. Bu bölümde alarm çaldıktan sonra ne yapacağımızı konuşuyoruz.

11.1 Hatalı değer ve aykırı değer

İki kavramı baştan ayırmamız gerekiyor.

Hatalı değer, ölçüm ya da kayıt kurallarını ihlal eden değerdir. 250 yaş, negatif bir ücret ya da yanlış para birimiyle girilmiş bir tutar hatalı değerdir.

Aykırı değer ise diğer gözlemlerden belirgin biçimde uzak olan değerdir. Uzak olabilir ama doğru olabilir. Titanic'te 512 birimlik bilet ücreti, birinci sınıf yolcuların gerçek bir özelliğini yansıtıyor olabilir.

Her hatalı değer aykırı değildir: 28 yerine yanlışlıkla 82 yazılmış bir yaş, dağılımın içinde kaybolur. Her aykırı değer de hatalı değildir. Bu bölümün temel mesajı, ikisinin aynı muameleyi görmemesi gerektiğidir.

11.1.1 Aykırılığın üç kaynağı

Bir değer dağılımın geri kalanından uzak duruyorsa, bunun üç farklı açıklaması olabilir:

Tablo 11.1 Aykırılığın kaynakları ve ilk yaklaşım.

Durum	Örnek	İlk yaklaşım
Kayıt ya da ölçüm hatası	250 yaş, yanlış para birimi, ondalık ayırıcının kayması	Kaynaktan doğru değeri araştır; düzeltilemiyorsa hatalı hücreyi eksik olarak ele almayı düşün
Geçerli nadir gözlem	Çok yüksek ücret ödeyen gerçek bir birinci sınıf yolcusu	Bağlamı kontrol et; otomatik silmek yerine koru, dönüştür ya da dayanıklı bir yöntem kullan
Farklı süreç ya da kapsam	Yolcu tablosuna personel kaydının karışması	Analiz evrenini tanımla; kapsam dışı olduğu doğrulanırsa ayırma kuralı uygula

Etkili gözlem
Bir modelin sonucunu tek başına belirgin biçimde değiştiren kayda **etkili gözlem** denir. Her aykırı kayıt etkili olmak zorunda değildir; etkiyi model kurulduktan sonra ayrıca inceleriz.

Aykırılık bağlama göre de değişir. Bütün yolcular içinde yüksek görünen bir ücret, birinci sınıf içinde olağan olabilir. Ayrıca bazen tek tek makul olan değerler bir araya geldiğinde sıra dışı bir birleşim oluşturur: 5 yaşında bir çocuğun yanında 6 kardeş kaydı olağan olabilir, ama 5 yaşında birinin 3 çocuğu olması değildir. Buna **çok değişkenli aykırılık** denir. Tek bir sütuna uygulanan tarama bunları yakalayamaz.

11.2 İki tarama yöntemi

11.2.1 IQR kuralı

Bölüm 8'deki kutu grafiğinde kullandığımız kural:

$$\text{IQR} = Q_3 - Q_1$$

$$\text{Alt sınır} = Q_1 - k \times \text{IQR} \qquad \text{Üst sınır} = Q_3 + k \times \text{IQR}$$

Yaygın seçim $k = 1{,}5$'tir. Bu katsayı, bir değerin hatalı olduğunu tek başına göstermez. Çeyrekler uç değerlerden çok az etkilendiği için IQR kuralı çarpık dağılımlarda da kullanılabilir; normal dağılım varsayımı gerektirmez.

Eşikleri yalnızca eğitim verisinden hesaplıyoruz:

```
q1 = X_egitim["Fare"].quantile(0.25)
q3 = X_egitim["Fare"].quantile(0.75)
iqr = q3 - q1
ust_sinir = q3 + 1.5 * iqr
yuksek_ucretler = X_egitim[X_egitim["Fare"] > ust_sinir]

print(round(ust_sinir, 4), len(yuksek_ucretler))
yuksek_ucretler["Pclass"].value_counts().sort_index()
```

çıktı

```
65.6563 91
Pclass
1    81
2     5
3     5
Name: count, dtype: int64
```

Eğitim verisinde üst sınır 65,66 ve bu sınırın üstünde 91 yolcu var. Bu yolcuların büyük çoğunluğu birinci sınıftan. Bu hücre hiçbir satırı silmez; yalnızca inceleme için işaretler.

11.2.2 Z-skoru

Z-skoru, bir değerin ortalamadan kaç standart sapma uzakta olduğunu gösterir:

$$z = \frac{x - \bar{x}_{\text{eğitim}}}{s_{\text{eğitim}}}$$

z = 0 ortalamayı, z = 2 ortalamanın iki standart sapma üstünü, negatif bir z ise ortalamanın altını gösterir. Yaygın bir başlangıç olarak $|z| > 3$ olan değerleri işaretleriz. Mutlak değer almak iki kuyruğu birlikte kontrol etmemizi sağlar.

```
fare_ortalama = X_egitim["Fare"].mean()
fare_std = X_egitim["Fare"].std(ddof=0)
fare_z = (X_egitim["Fare"] - fare_ortalama) / fare_std
z_aykiri = fare_z.abs() > 3

print(round(fare_ortalama, 2), round(fare_std, 2), z_aykiri.sum())
X_egitim[z_aykiri][["Pclass", "Fare"]].head()
```

çıktı
```
31.82 48.03 15
     Pclass      Fare
527       1  221.7792
557       1  227.5250
716       1  227.5250
742       1  262.3750
118       1  247.5208
```

`ddof=0` standart sapmayı hesaplarken toplamı gözlem sayısına (n) böler. `.abs()` mutlak değeri alır. Z-skoru yalnızca 15 yolcu işaretledi; IQR kuralı ise 91.

11.2.3 Neden farklı sonuç veriyorlar?

```
alt_sinir = q1 - 1.5 * iqr
karsilastirma = X_egitim[["Fare"]].copy()
karsilastirma["IQR"] = (karsilastirma["Fare"] < alt_sinir) | (karsilastirma["Fare"] >
ust_sinir)
karsilastirma["Z_skoru"] = z_aykiri
print(karsilastirma[["IQR", "Z_skoru"]].sum())
karsilastirma[karsilastirma["IQR"] != karsilastirma["Z_skoru"]].head(3)
```

çıktı
```
IQR        91
Z_skoru    15
dtype: int64
        Fare   IQR  Z_skoru
846  69.5500  True    False
412  90.0000  True    False
218  76.2917  True    False
```

`|` işareti "ya da" anlamındadır: İki koşuldan biri doğruysa satır işaretlenir. `!=` "eşit değil" demektir; son satır iki yöntemin farklı karar verdiği yolcuları gösterir. Örneğin 76,29 ya da 90 birimlik bir ücret IQR sınırını aşıyor ama Z-skoru sınırının çok altında kalıyor.

Farkın nedeni, iki yöntemin farklı merkez ve yayılım ölçüleri kullanmasıdır (Şekil 11.1).



Şekil 11.1 Eğitim ücretleri ve iki üst eşik. Uzun sağ kuyruktaki yüksek ücretler ortalamayı ve standart sapmayı büyüttüğü için Z-skoru eşiği çok sağa kayar; çeyreklere dayanan IQR eşiği bu etkiden korunur.

Ücret dağılımı sağa çarpık. Kuyruktaki yüksek değerler ortalamayı (31,82) ve özellikle standart sapmayı (48,03) büyütüyor. Z-skoru eşiği bu iki sayıya dayandığı için 175,90'a kadar yükseliyor. Sonuçta aykırı değerler kendi tespit edilmelerini zorlaştırıyor. Buna **maskeleme** denir. Çeyrekler ise uç değerlerden çok az etkilenir; IQR eşiği 65,66'da kalır.

Ama IQR'nin de bir zayıflığı var: Çok çarpık bir dağılımda kuyruktaki geçerli gözlemleri fazla işaretleyebilir. 91 yolcunun çoğu muhtemelen gerçek birinci sınıf ücretleridir. **Daha çok kayıt işaretleyen yöntem daha iyi değildir.** İşaretlenen değerler her durumda ayrıca incelenmelidir.

11.2.4 Hangi durumda hangisi?

Tablo 11.2 Aykırı tarama yöntemleri.

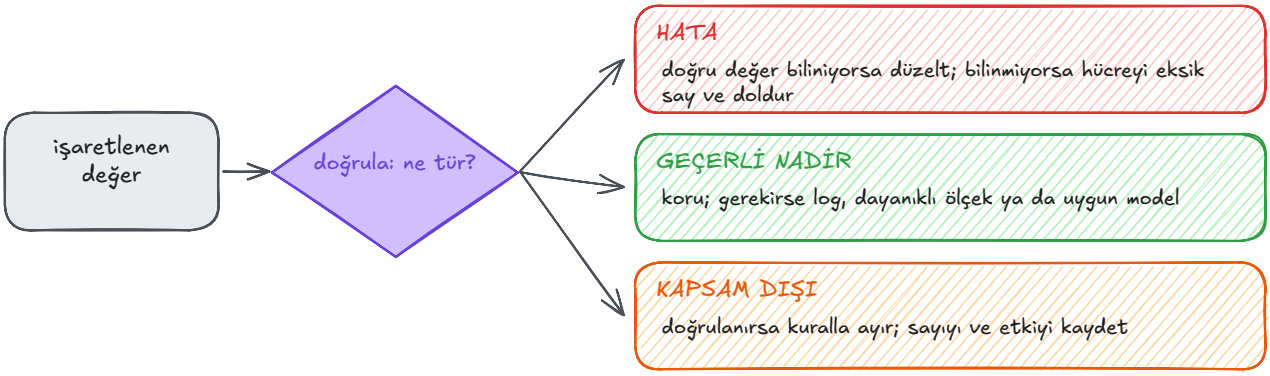
Yöntem	Ne zaman?	Sınırı
Z-skoru	Tek tepeli, yaklaşık simetrik, normale yakın dağılım	Çarpıklık ve büyük uçlar ortalamayı ve sapmayı değiştirerek bazı uçları maskeleyebilir
IQR	Çarpık, uzun kuyruklu ya da belirgin uç değerli dağılım	Normal dağılım gerektirmez; ama çok çarpık dağılımda geçerli kuyruk gözlemlerini fazla işaretleyebilir
Değiştirilmiş Z-skoru (MAD)	Uçlardan daha az etkilenen bir merkez ve yayılım gerektiğinde	Ortalama yerine medyanı, standart sapma yerine medyan mutlak sapmayı kullanır
Grup ve çok değişkenli inceleme	Karışık gruplar, birden fazla tepe, sıra dışı değer birleşimleri	Tek bir genel eşik yeterli olmaz; alan bilgisi gerekir

Titanic için ham ücret sağa çarpık olduğundan IQR makul bir ilk taramadır. Sınıfa göre grafikler (Şekil 8.4) ve veri bağlamı kararı tamamlar. Yaş için de dağılıma bakmadan Z-skoru seçmeyiz.

> **Dikkat**
>
> 1,5 × IQR kuralı ile $|z| > 3$ kuralı eşdeğer değildir. Normal bir dağılımda bile 1,5 × IQR sınırları yaklaşık ±2,7 standart sapmaya karşılık gelir. Bu yüzden aynı dağılımda bile iki yöntemin aynı gözlemleri işaretlemesi beklenmez. Z-skoru her dağılımda hesaplanabilir, ama normal dağılımdaki "±3 sapma" yorumu her dağılıma taşınamaz.

11.3 Alarm çaldıktan sonra: karar

İşaretlenen bir değerle ne yapacağımıza Şekil 11.2'deki sırayla karar veririz.



Şekil 11.2 Aykırı ya da şüpheli bir değer için karar akışı. İlk soru her zaman değerin doğru olup olmadığıdır.

1. **Önce doğrula.** Değer bir hata mı, geçerli bir nadir gözlem mi, yoksa kapsam dışı bir kayıt mı?
2. **Doğru değeri biliyorsak düzelt.** Kaynak, yaşın 25 olduğunu doğruluyorsa 250'yi 25 yaparız. Tahmin ederek düzeltmeyiz.
3. **Yalnızca bir hücre hatalı ve doğru değer bulunamıyorsa** o hücreyi eksik olarak işaretler, Bölüm 10'daki doldurma sürecine bırakırız. Böylece yolcunun diğer yararlı bilgileri korunur.
4. **Kaydın tamamı güvenilmezse**, doğrulanmış bir tekrarsa ya da kapsam dışıysa satırı çıkarmayı düşünürüz. Kuralı, çıkarılan kayıt sayısını ve alt gruplara etkisini kaydederiz.
5. **Değer gerçekse**, sırf eşik dışında diye medyanla değiştirmeyiz. Korumayı, log dönüşümünü, dayanıklı ölçeklemeyi ya da uçlara duyarlı olmayan bir modeli karşılaştırırız.

> **Örnek 11.1**
>
> `Age = 250` değerinin hatalı olduğunu biliyoruz ama doğru yaşa ulaşamıyoruz. Yalnızca yaş hücresini eksik olarak işaretleyip eğitim medyanıyla doldurabiliriz; yolcunun sınıfı, ücreti ve diğer bilgileri korunur. Buna karşılık `Fare = 512,33` gerçek bir ücretse, aynı işlemi sırf yüksek olduğu için yapmak gerçek sınıf farkını siler.

11.3.1 Kırpma, winsorization ve trimming

Uç değerlerin etkisini azaltmanın üç yolu sıklıkla birbirine karıştırılır:

Kırpma (sınırlandırma)
: Sınırın ötesindeki değeri sınıra taşır. Gözlem kalır, değeri değişir.

Winsorization
: Kırpmanın, sınırların seçilmiş alt ve üst yüzdeliklere (örneğin %1 ve %99) göre belirlendiği biçimidir.

Trimming
: Uç gözlemi analizden tamamen çıkarır.

```
fare_sinirli = X_egitim["Fare"].clip(upper=ust_sinir)
print(X_egitim["Fare"].max(), "→", round(fare_sinirli.max(), 4))
print((fare_sinirli == fare_sinirli.max()).sum())
```

çıktı
```
512.3292 → 65.6563
91
```

`.clip(upper=...)` verilen sınırı aşan değerleri sınır değerine indirir; satır çıkarmaz. 512,33 birimlik ücret 65,66'ya indi. Ama ikinci satıra bakın: Artık 91 farklı yolcu tam olarak aynı ücrete sahip. 80 birim ödeyen yolcu ile 512 birim ödeyen yolcu arasındaki fark tamamen silindi. Her üç yöntem de gerçek farklılıkları ve modelin göreceği dağılımı değiştirir. Kırpma ya da yüzdelik sınırları da yalnızca eğitimden öğrenilir. Son hattımızda ücretleri sınırlandırmayacağız.

11.3.2 Seçenekleri karşılaştırmak

Tablo 11.3 Uç değerler için seçenekler.

Seçenek	Gerekçe	Risk
Korumak	Kayıt gerçek ve amaç için anlamlı	Bazı yöntemlerin sonucunu güçlü etkileyebilir
Kaynağa göre düzeltmek	Doğru değer güvenilir kaynaktan belirlendi	Tahminle yapılan düzeltme yeni hata yaratır
Sınırlandırmak	Aşırı etkinin kontrollü biçimde azaltılması amaçlanıyor	Gerçek farkları bastırır
Dönüştürmek ya da dayanıklı yöntem kullanmak	Çarpıklık ya da uçlara duyarlılık önemli	Dönüşüm hatayı düzeltmez; yorum değişir
Silmek	Doğrulanmış hata ya da kapsam dışı kayıt	Örneklem ve sınıf bileşimi değişir

Titanic için kararımız: Yüksek ücretin hata olduğunu gösteren hiçbir bilgi yok. Yolcuları koruyacak, ücrete Bölüm 13'te log dönüşümü uygulayacağız. Diğer seçenekler eğitim verisi içinde karşılaştırılabilir.

Dikkat

Bir yöntemi değiştirerek sonuçların ne kadar değiştiğine bakmak bir **duyarlılık kontrolüdür** ve iyi bir uygulamadır. Ama test verisindeki "zor" yolcuları silip başarı puanını yükseltmek bir çözüm değildir. Model kullanıma alındığında yüksek ücretli yolcularla da karşılaşacak; değerlendirme verisi bu yolcuları temsil etmelidir. Test yolcularına eğitimde belirlediğimiz dönüşüm uygulanır, ama hiçbiri silinmez.

Bölüm özeti

- Hatalı değer kayıt kurallarını ihlal eder; aykırı değer dağılımın geri kalanından uzaktır ama doğru olabilir. Etkili gözlem modelin sonucunu belirgin biçimde değiştiren kayıttır.
- Aykırılığın üç kaynağı vardır: kayıt hatası, geçerli nadir gözlem ve farklı süreç. Her biri farklı bir müdahale gerektirir.
- IQR kuralı çeyreklere dayanır ve çarpık dağılımlarda kullanılabilir; Z-skoru ortalama ve standart sapmaya dayanır ve normale yakın dağılımlarda uygundur. İki kural eşdeğer değildir.
- Titanic eğitim ücretlerinde IQR 91, Z-skoru 15 yolcu işaretler. Yüksek ücretler ortalamayı ve sapmayı büyüterek Z-skoru eşiğini 175,90'a iter; buna maskeleme denir.
- Daha çok kayıt işaretleyen yöntem daha iyi değildir; işaret incelemenin başlangıcıdır.
- Karar sırası: doğrula, biliniyorsa düzelt, yalnızca hücre hatalıysa eksik say, kayıt güvenilmezse çıkar, değer gerçekse koru ya da dönüştür.
- Kırpma değeri sınıra taşır, trimming gözlemi çıkarır; ikisi de gerçek farkları siler. Eşikler eğitimden öğrenilir; test yolcuları silinmez.

Kaynaklar

NIST/SEMATECH. *e-Handbook of Statistical Methods*, Bölüm 1.3.5.17: Detection of Outliers. https://www.itl.nist.gov/div898/handbook/eda/section3/eda35h.htm

Iglewicz, B. ve Hoaglin, D. C. (1993). *How to Detect and Handle Outliers*. ASQC Quality Press.

Müller, A. C. ve Guido, S. (2016). *Introduction to Machine Learning with Python*. O'Reilly. Bölüm 3: "Preprocessing and Scaling".

12

BÖLÜM

Özellik mühendisliği ve kategorik kodlama

Bu bölümde

- Özellik mühendisliğinin amacını ve türetilen bir özelliğin nasıl belgeleneceğini açıklamak,
- `AileBuyuklugu`, `TekYolcu`, eksiklik göstergeleri, unvan ve güverte gibi özellikleri Python ile üretmek,
- aralıklara ayırmanın ve nadir kategorileri birleştirmenin yararını ve bedelini tartışmak,
- sırasız (nominal) ve sıralı (ordinal) kategorileri ayırmak; one-hot, ordinal ve ikili kodlamayı uygulamak,
- eğitimde görülmemiş bir kategoriyle karşılaşıldığında ne olacağını açıklamak,
- yüksek kategori sayısının ve hedef kodlamanın risklerini tanımak.

Kaynak: YBS 319 ders sayfası ve uygulama defteri

Ham veri sütunları çoğu zaman bir modelin doğrudan kullanabileceği biçimde gelmez. Bir yolcunun ismi bir metindir; bilet sınıfı bir sayı gibi görünür ama bir kategoridir; kardeş ve ebeveyn sayıları ayrı sütunlardadır. Bu bölümde iki işi ele alıyoruz: mevcut alanlardan probleme uygun yeni temsiller üretmek ve kategorileri modelin anlayacağı sayılara dönüştürmek.

12.1 Özellik mühendisliği

Özellik mühendisliği, mevcut alanlardan probleme uygun yeni temsiller üretmektir. Örneğin `SibSp` ve `Parch` alanlarını birleştirerek aile büyüklüğünü gösterebiliriz. Bir özellik türettiğimizde iki şeyi mutlaka yazarız: nasıl hesaplandığını ve neyi ifade ettiğini.

Tablo 12.1 Titanic için türetilebilecek özellikler.

Yeni özellik	Kaynak ya da kural	Nasıl yorumlarız?
`AileBuyuklugu`	`SibSp + Parch + 1`	Bu iki alanda belirtilen yakınlar ile yolcunun kendisi
`TekYolcu`	`AileBuyuklugu == 1`	Bu alanlarda yolcuya eşlik eden kayıtlı bir yakın yok
`YasEksik`	`Age` boş mu?	Yaşın başlangıçta bilinmediği (Bölüm 10)
`KabinBilgisiVar`	`Cabin` dolu mu?	Kabin bilgisinin kaydedilip kaydedilmediği
`Unvan`	`Name` içindeki unvan	Mr, Mrs, Miss gibi; nadir unvanlar ve farklı yazımlar incelenir
`Guverte`	`Cabin` kodunun ilk harfi	Eksik kodlar ve birden fazla kabini olanlar ayrıca ele alınır
`YasGrubu`	Önceden belirlenen yaş aralıkları	Yorum kolaylaşır, aralık içindeki ayrıntı kaybolur

12.1.1 Aynı kuralları her yerde uygulamak

Özellik üretme kurallarını bir **işlev** içinde topluyoruz. Böylece aynı kuralları eğitime, teste ve ileride gelecek yeni yolculara hiç değiştirmeden uygulayabiliriz:

```
def ozellik_uret(tablo):
    sonuc = tablo.copy()                          # girdiyi değiştirmeden kopyalar
    sonuc["Sex"] = sonuc["Sex"].astype("string").str.strip().str.lower()
    sonuc["Embarked"] = sonuc["Embarked"].astype("string").str.strip().str.upper()
    sonuc["AileBuyuklugu"] = sonuc["SibSp"] + sonuc["Parch"] + 1
    sonuc["TekYolcu"] = (sonuc["AileBuyuklugu"] == 1).astype(int)
    sonuc["YasEksik"] = sonuc["Age"].isna().astype(int)
    sonuc["KabinBilgisiVar"] = sonuc["Cabin"].notna().astype(int)
    return sonuc

X_egitim_ozellik = ozellik_uret(X_egitim)
X_test_ozellik = ozellik_uret(X_test)
X_egitim_ozellik[["SibSp", "Parch", "AileBuyuklugu", "TekYolcu", "YasEksik"]].head()
```

çıktı

	SibSp	Parch	AileBuyuklugu	TekYolcu	YasEksik
692	0	0	1	1	1
481	0	0	1	1	1
527	0	0	1	1	1
855	0	1	2	0	0
801	1	1	3	0	0

`def` bir işlev tanımlar; parantez içindeki `tablo`, işleve verilecek girdinin adıdır. Girintili satırlar işlevin gövdesidir. `return` sonucu geri döndürür. İşlevi bir kez tanımladıktan sonra `ozellik_uret(X_egitim)` ve `ozellik_uret(X_test)` diye iki kez çağırdık.

`astype("string")` sütunu, eksik değerleri destekleyen metin tipine çevirir. Bu sayede sütunun tamamı eksik olsa bile `.str` işlemleri hata vermez. `astype(int)` ise `True` ve `False` değerlerini 1 ve 0'a dönüştürür.

Bu işlevdeki kuralların hepsi **sabit kurallardır** (Bölüm 9): Hiçbiri veri dağılımından bir parametre öğrenmez. Her satır yalnızca kendi değerleriyle hesaplanır. Bu yüzden bölmeden önce ya da sonra uygulanmaları sızıntı yaratmaz.

Dikkat

`TekYolcu` ve `AileBuyuklugu` aynı kaynaktan türetildi; biri ötekinin özel bir durumudur. Bu yüzden son tabloda `SibSp` ve `Parch` sütunlarını ayrıca tutmuyoruz; aynı bilgiyi birkaç kez saymamak için aile temsilini sade bırakıyoruz. Ayrıca `AileBuyuklugu` gerçek ailenin tamamını ölçmez; yalnızca bu iki alanda kayıtlı yakınları kapsar.

12.1.2 Metinden bilgi çıkarmak: unvan ve güverte

Ham `Name` sütununu doğrudan bir modele vermek anlamsızdır: 891 yolcunun 891 farklı ismi vardır. Ama isimlerin içinde bir yapı bulunur. "Braund, Mr. Owen Harris" isminde virgül ile nokta arasında bir **unvan** durur. Kabin kodlarının ilk harfi de geminin **güvertesini** gösterir.

```
metin_ornegi = X_egitim[["Name", "Cabin"]].copy()
isim = metin_ornegi["Name"].astype("string")
metin_ornegi["Unvan"] = isim.str.extract(r",\s*([^\.]+)\.", expand=False)
kabin = metin_ornegi["Cabin"].astype("string")
metin_ornegi["Guverte"] = kabin.str.extract(r"^\s*([A-Za-z])", expand=False)
print(metin_ornegi["Unvan"].value_counts().head(6))
metin_ornegi[["Unvan", "Guverte"]].head()
```

çıktı

```
Unvan
Mr        412
Miss      141
Mrs       107
Master     31
Dr          6
Rev         5
Name: count, dtype: Int64
    Unvan Guverte
692    Mr    <NA>
481    Mr    <NA>
```

```
527    Mr       C
855   Mrs    <NA>
801   Mrs    <NA>
```

`.str.extract(...)` bir **düzenli ifade** (*regular expression*, regex) desenine uyan metin parçasını çıkarır. Birinci desen `,\s*([^\.]+)\.` şunu söyler: bir virgül, ardından boşluklar, sonra nokta olmayan karakterler (yakalanacak kısım) ve bir nokta. İkinci desen `^\s*([A-Za-z])` ise metnin başındaki boşluklardan sonraki ilk harfi yakalar. `expand=False` sonucun tek sütunluk bir seri olarak dönmesini sağlar. Kabin kodu eksik olan yolcuların güvertesi de eksik kalır.

Unvan dağılımına bakın: Mr, Miss, Mrs ve Master yaygın; geri kalan unvanlar ise birkaç kişiye ait. Bu **nadir kategoriler** sorun yaratır: Az gözlemli bir kategori hakkında model güvenilir bir şey öğrenemez. Çözüm genellikle nadir kategorileri "Diğer" gibi tek bir grupta birleştirmektir. Ama hangi kategorinin nadir sayılacağına karar veren eşik ve kategori listesi, **eğitim verisinden** belirlenir; çünkü bu bir öğrenilen işlemdir.

Bu iki özelliği son hattımıza eklemiyoruz. Katkılarını, ileride modelleme aşamasında eğitim içi doğrulamayla ayrı ayrı değerlendirmek daha doğru olur.

12.1.3 Aralıklara ayırmak

Bölüm 8'de yaşı aralıklara bölmüştük. Aralıklara ayırmak (*binning*) yorumu kolaylaştırır ve doğrusal olmayan ilişkileri yakalamaya yardım edebilir. Ama bir bedeli vardır: 19 ile 35 yaş arasındaki bütün farkları tek bir etikette toplar.

Sınırlar alan bilgisine dayanan sabit değerlerse (örneğin "18 yaşından küçükler çocuk sayılır") kural baştan tanımlanabilir. Sınırlar verinin yüzdeliklerine göre seçiliyorsa (örneğin "yaşları dört eşit gruba böl"), eşikler eğitimden öğrenilir.

12.2 Kategorik kodlama

Birçok makine öğrenmesi algoritması yalnızca sayısal girdi kabul eder. Bu yüzden `Sex`, `Embarked` gibi metin kategorilerini sayılara dönüştürmemiz gerekir. Dönüştürürken dikkat etmemiz gereken tek bir kural var: **Kategoriler arasında gerçekte bulunmayan bir sıra ya da uzaklık yaratmamak.**

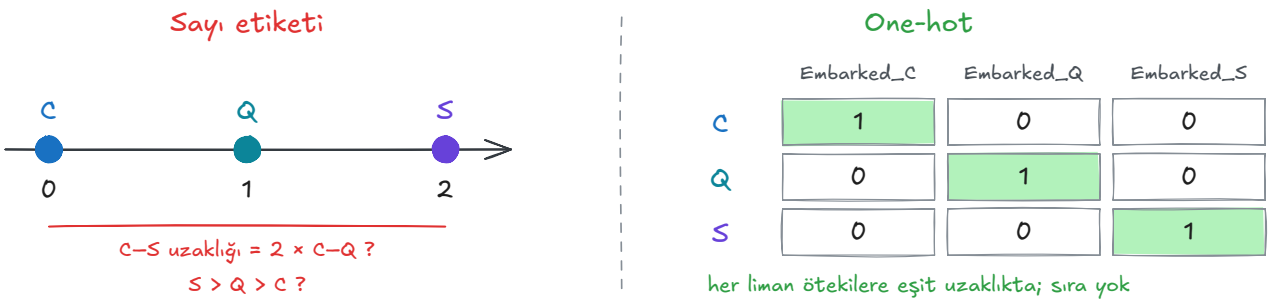
Hesaplanabilir ama kullanılabilir mi?
Bir özelliğin hesaplanabilmesi, tahmin anında kullanılabileceğini göstermez. "Aynı biletle seyahat eden toplam yolcu sayısı" eğitim verisinde hesaplanır; ama yeni bir yolcu geldiğinde biletteki diğer yolcuları biliyor muyuz? Bölüm 6'daki karar anı sorusu burada da geçerlidir.

12.2.1 Nominal ve ordinal kategoriler

Nominal (sırasız) kategorilerde doğal bir sıra yoktur. Kadın ile erkek, C, Q ve S limanları arasında bir büyüklük ilişkisi kurulamaz. **Ordinal** (sıralı) kategorilerde ise anlamlı bir sıra vardır: "düşük, orta, yüksek" ya da "1., 2., 3. sınıf".

12.2.2 Neden 0, 1, 2 vermiyoruz?

Limanlara C = 0, Q = 1, S = 2 diye sayı vermek en kolay yol gibi görünür. Ama bu durumda model, S limanının Q'dan "bir birim daha büyük" olduğunu, S ile C arasındaki uzaklığın Q ile C arasındakinin iki katı olduğunu varsayar (Şekil 12.1). Bu ilişkiler gerçekte yoktur ve özellikle uzaklığa ya da doğrusal ilişkiye dayanan modeller bu yapay yapıyı öğrenir.



Şekil 12.1 Aynı üç limanın iki gösterimi. Sayı etiketi yapay bir sıra ve uzaklık yaratır; one-hot gösteriminde her liman ötekilere eşit uzaklıktadır.

12.2.3 One-hot kodlama

One-hot kodlama her kategori için ayrı bir gösterge sütunu oluşturur. Limanlar C, Q, S ise bir S yolcusu [0, 0, 1] ile, bir C yolcusu [1, 0, 0] ile temsil edilir. Bu kodlama hiçbir sıra varsaymaz. Bedeli şudur: Çok sayıda farklı kategori, sütun sayısını hızla büyütür.

Önce `pandas` ile yapalım:

```
kategori_ornegi = X_egitim_islem[["Sex", "Embarked", "Pclass"]]
dummy_kodlari = pd.get_dummies(kategori_ornegi,
                               columns=["Sex", "Embarked", "Pclass"], dtype=int)
dummy_kodlari.head()
```

çıktı

```
     Sex_female  Sex_male  Embarked_C  Embarked_Q  Embarked_S  Pclass_1  Pclass_2  \
692           0         1           0           0           1         0         0
481           0         1           0           0           1         0         1
527           0         1           0           0           1         1         0
855           1         0           0           0           1         0         0
801           1         0           0           0           1         0         1

     Pclass_3
692         1
481         0
527         0
855         1
801         0
```

`pd.get_dummies` her kategori için 0/1 sütunu oluşturur. Sex için 2, Embarked için 3, Pclass için 3 olmak üzere toplam 8 gösterge sütunu elde ettik. Kadın bir yolcuda `Sex_female` 1, `Sex_male` 0 olur. Her yolcuda kendi limanının ve sınıfının sütunu da 1'dir. `columns` parametresini yazmasaydık pandas yalnızca metin sütunlarını kodlardı; `Pclass` sayısal saklandığı için onu açıkça listeledik.

`Pclass`'ı neden one-hot kodladık? Sınıflar sıralı olsa da birinci sınıftan ikinciye geçişin, ikinciden üçüncüye geçişle aynı "miktarda" fark yarattığını varsaymak istemiyoruz. Bölüm 8'deki oranlar da bu farkların eşit olmadığını gösteriyordu (%63, %47, %24). Sıralı kodu korumak da bir seçenektir; bu bir modelleme tercihidir.

12.2.4 get_dummies neden yetmez?

`pd.get_dummies` kullanışlıdır ama bir zayıflığı vardır: Eğitimi ve testi ayrı ayrı kodladığımızda, bir kategori yalnızca bir tarafta bulunuyorsa iki tablonun sütunları farklı çıkar. Test verisinde hiç Q limanı yoksa test tablosunda `Embarked_Q` sütunu oluşmaz ve model sütunları yanlış eşleştirir.

Çözüm, kategori listesini eğitimden **öğrenip** aynısını teste uygulamaktır. Bu tam olarak bir `fit` ve `transform` işidir. `scikit-learn`'ün `OneHotEncoder` aracı bunu yapar:

```
from sklearn.preprocessing import OneHotEncoder

kategorik_sutunlar = ["Sex", "Embarked", "Pclass"]
kodlayici = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
kodlayici.fit(X_egitim_islem[kategorik_sutunlar])       # kategorileri eğitimden öğrenir
kod_sutunlari = kodlayici.get_feature_names_out()
print(kod_sutunlari)
```

çıktı

```
['Sex_female' 'Sex_male' 'Embarked_C' 'Embarked_Q' 'Embarked_S' 'Pclass_1'
 'Pclass_2' 'Pclass_3']
```

`sparse_output=False` sonucu sıradan bir dizi olarak verir. `handle_unknown="ignore"` ayarını birazdan açıklayacağız. `get_feature_names_out()` öğrenilen sütun adlarını verir.

```
egitim_kodlari = kodlayici.transform(X_egitim_islem[kategorik_sutunlar])
test_kodlari = kodlayici.transform(X_test_islem[kategorik_sutunlar])
X_egitim_kategori = pd.DataFrame(egitim_kodlari, columns=kod_sutunlari,
                                 index=X_egitim.index)
X_test_kategori = pd.DataFrame(test_kodlari, columns=kod_sutunlari,
                               index=X_test.index)
X_test_kategori.head(3)
```

çıktı

```
     Sex_female  Sex_male  Embarked_C  Embarked_Q  Embarked_S  Pclass_1  Pclass_2  \
565         0.0       1.0         0.0         0.0         1.0       0.0       0.0
160         0.0       1.0         0.0         0.0         1.0       0.0       0.0
553         0.0       1.0         1.0         0.0         0.0       0.0       0.0

     Pclass_3
565       1.0
160       1.0
553       1.0
```

Eğitim ve test aynı kategori listesiyle kodlandı; sütun sırası ve anlamı aynı. `index=X_egitim.index` satır etiketlerini korur, böylece kodlanmış tablolar doğru yolcularla eşleşir.

12.2.5 Görülmemiş ve eksik kategori

Kodlayıcı kategori listesini eğitimde öğrenir. Peki kullanım sırasında hiç görmediği bir liman etiketi, örneğin "X", gelirse ne olur? Üç seçenek vardır:

- Hata vermek (varsayılan davranış). Sistem durur; biri sorunu incelemek zorunda kalır.
- `handle_unknown="ignore"` ile o alanın bütün gösterge sütunlarını 0 yapmak. Yolcu hiçbir limana atanmaz.

- "Bilinmeyen" için ayrı bir kategori ya da sütun tanımlamak.

Biz ikinci seçeneği kullandık. Bunu küçük bir örnekle görelim:

```
yeni = pd.DataFrame({"Sex": ["female"], "Embarked": ["X"], "Pclass": [2]})
pd.DataFrame(kodlayici.transform(yeni), columns=kod_sutunlari).astype(int)
```

çıktı

```
   Sex_female  Sex_male  Embarked_C  Embarked_Q  Embarked_S  Pclass_1  Pclass_2  \
0           1         0           0           0           0         0         1

   Pclass_3
0         0
```

Bilinmeyen "X" limanı, üç liman sütununun hepsinde 0 üretti. Cinsiyet ve sınıf normal kodlandı. Dikkat edin: **Bilinmeyen kategori ile eksik kategori farklı şeylerdir.** Eksik bir liman, kodlamadan önce eğitim moduyla (S) doldurulur ve `Embarked_S` 1 olur. Bilinmeyen "X" ise doldurulmaz; tüm göstergeler 0 kalır.

12.2.6 Referans kategori

`drop="first"` (ya da `get_dummies` için `drop_first=True`) ayarı her alanın ilk kategorisinin sütununu çıkarır; bizim örneğimizde 8 yerine 5 sütun kalır. Çıkarılan kategori **referans kategori** olur: Diğer tüm göstergeleri 0 olan bir yolcu, o kategoridedir.

Bu ayar neden var? Bir alanın bütün gösterge sütunları toplandığında her satırda 1 eder. Doğrusal bir modelde sabit terim de her satırda 1'dir. Bu durumda sütunlar arasında tam bir doğrusal bağımlılık oluşur ve düzenlileştirilmemiş doğrusal modeller katsayıları tek bir şekilde belirleyemez. Bölüm 18'de çoklu doğrusal regresyonda bu konuya döneceğiz. Her algoritma için zorunlu değildir; biz bu aşamada bütün kategorilerin sütunlarını koruyoruz.

12.2.7 Ordinal ve ikili kodlama

Gerçekten sıralı bir kategoride sırayı korumak isteyebiliriz. **Ordinal kodlama** kategorileri tanımlanan sırada 0, 1, 2 diye kodlar:

```
from sklearn.preprocessing import OrdinalEncoder

duzeyler = pd.DataFrame({"Duzey": ["Orta", "Düşük", "Yüksek", "Orta"]})
sirali = OrdinalEncoder(categories=[["Düşük", "Orta", "Yüksek"]])
sirali.fit_transform(duzeyler)
```

çıktı

```
array([[1.],
       [0.],
       [2.],
       [1.]])
```

`categories` parametresinde sırayı açıkça yazıyoruz: Düşük → 0, Orta → 1, Yüksek → 2. Sırayı yazmazsak kodlayıcı kategorileri harf sırasına dizer. Bu örnekte harf sırası tesadüfen doğru çıkar; ama "Kötü, İyi, Mükemmel" gibi bir listede İ harfi Türkçeye özgü olduğu için en sona düşer ve

"İyi" en yüksek düzey gibi kodlanır. Ordinal kodun da bir sınırı var: Sıra, eşit mesafe olduğunu garanti etmez. Bazı modeller 0 ile 1 arasındaki farkı 1 ile 2 arasındaki farkla eşit sayar.

İki kategorili bir alan ise açık bir eşlemeyle tek bir 0/1 sütununda temsil edilebilir: `female → 1, male → 0` gibi. Bu durumda one-hot'un iki sütunu gereksizdir.

> **Dikkat**
>
> `scikit-learn`'deki `LabelEncoder` aracı, **hedef** değişkenin sınıf etiketlerini sayıya çevirmek için tasarlanmıştır. Girdi sütunlarında kategori türüne göre `OneHotEncoder` ya da `OrdinalEncoder` kullanılır. Titanic'te `Survived` zaten 0/1 olduğu için hedefe ayrıca bir kodlama uygulamıyoruz.

12.2.8 Yüksek kategori sayısı

Ham `Name`, `Ticket` ya da `Cabin` sütunlarını one-hot kodlarsak yüzlerce sütun oluşur ve her sütunda yalnızca bir ya da birkaç yolcu 1 olur. Bunun yerine şu yollar düşünülür:

- metinden anlamlı bilgi çıkarmak (unvan, güverte),
- nadir kategorileri birleştirmek,
- kategorileri sabit sayıda sütuna eşleyen *hashing* yöntemleri,
- **hedef kodlama** (*target encoding*): Her kategoriyi o kategorideki hedef ortalamasıyla değiştirmek.

Hedef kodlama özellikle dikkat ister. Bir kategorinin hedef ortalaması bütün veriden hesaplanırsa, test yolcularının sonuçları doğrudan girdiye taşınır; bu ciddi bir sızıntıdır. Doğru kullanım, ortalamaların her eğitim katında ayrı ayrı ve doğrulama satırlarını dışarıda bırakarak hesaplanmasını gerektirir. Titanic tablomuzda bu yöntemi kullanmıyoruz.

Bölüm özeti

- Özellik mühendisliği mevcut alanlardan probleme uygun yeni temsiller üretmektir. Her özelliğin hesaplanma kuralı ve anlamı belgelenir.
- Özellik kuralları bir işlevde toplanır ve eğitim, test ve yeni veriye aynen uygulanır. `AileBuyuklugu`, `TekYolcu`, `YasEksik`, `KabinBilgisiVar` sabit kurallardır; veri dağılımından bir şey öğrenmez.
- Metinden düzenli ifadelerle unvan ve güverte çıkarılabilir. Nadir kategorileri birleştirme eşiği ve kategori listesi eğitimden öğrenilir.
- Aralıklara ayırmak yorumu kolaylaştırır ama aralık içindeki ayrıntıyı siler. Yüzdeliklere dayanan sınırlar eğitimden öğrenilir.
- Nominal kategorilere 0, 1, 2 vermek yapay bir sıra ve uzaklık yaratır. One-hot kodlama her kategori için ayrı bir gösterge sütunu oluşturur.
- `pd.get_dummies` eğitim ve testte farklı sütunlar üretebilir; `OneHotEncoder` kategori listesini eğitimden öğrenip teste aynen uygular.
- `handle_unknown="ignore"` görülmemiş bir kategoride bütün göstergeleri 0 yapar. Bilinmeyen kategori ile eksik kategori farklıdır.
- `drop="first"` referans kategori oluşturur; düzenlileştirilmemiş doğrusal modellerde tam doğrusal bağımlılığı önler.
- Ordinal kodlamada sıra açıkça verilir; sıra eşit mesafe anlamına gelmez. `LabelEncoder` hedef içindir.
- Hedef kodlama bütün veriden hesaplanırsa ciddi sızıntı yaratır.

Kaynaklar

Zheng, A. ve Casari, A. (2018). *Feature Engineering for Machine Learning*. O'Reilly. Bölüm 5.

Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3. baskı). O'Reilly. Bölüm 2: "Handling Text and Categorical Attributes".

scikit-learn geliştiricileri. *Encoding categorical features*. https://scikit-learn.org/stable/modules/preprocessing.html#encoding-categorical-features

13

BÖLÜM

Dağılım dönüşümleri ve ölçeklendirme

Bu bölümde

- Sağa çarpık bir dağılımda log dönüşümünün ne yaptığını sayılarla açıklamak; log, log1p ve karekökün tanım aralıklarını bilmek,
- bir dönüşümün ne zaman gerekli, ne zaman gereksiz ya da sakıncalı olduğuna karar vermek,
- ölçek farkının uzaklık hesaplarını nasıl etkilediğini gerçek yolcularla göstermek,
- StandardScaler, MinMaxScaler ve RobustScaler'ı formülleriyle uygulamak ve sonuçlarını yorumlamak,
- dağılım dönüşümü ile ölçeklemenin farklı işler yaptığını ve neden art arda kullanılabildiğini açıklamak,
- hangi modellerin ölçeğe duyarlı olduğunu bilmek.

Kaynak: YBS 319 ders sayfası ve uygulama defteri

Bu bölümde sayısal sütunlarla ilgili iki farklı sorunu ele alıyoruz. Birincisi **dağılımın şekli**: Ücret gibi sağa çarpık bir değişkende birkaç büyük değer ham farkları büyütür. İkincisi **ölçek**: Yaş 0 ile 80 arasında, ücret 0 ile 512 arasında değişir; iki değişkenin birimleri birbirinden çok farklıdır. Bu iki sorunun çözümleri de farklıdır ve sık sık birbirine karıştırılır. Bölümün sonunda neden art arda kullanılabildiklerini göreceğiz.

13.1 Dağılım dönüşümleri

Sağa çarpık bir dağılımda gözlemlerin çoğu küçük değerlerde toplanır; az sayıda gözlem ise çok büyük değerlere uzanır. Titanic'te `Fare` böyledir. Bir dönüşüm büyük değerleri sıkıştırabilir ve bazı ilişkileri modelin daha rahat temsil etmesini sağlayabilir.

Tablo 13.1 Sık kullanılan dağılım dönüşümleri.

Dönüşüm	Tanımı	Geçerlilik ve dikkat
Logaritma	$\ln(x)$	$x > 0$ olmalı; sıfırda tanımsız
log1p	$\ln(1+x)$	$x > -1$ olmalı; negatif olmayan veride sıfırı korur
Karekök	$\sqrt{x}$	$x \geq 0$ olmalı; logdan daha hafif sıkıştırır
Box-Cox, Yeo-Johnson	Veriden öğrenilen kuvvet dönüşümü	Box-Cox pozitif değer ister; Yeo-Johnson sıfır ve negatif değerleri de ele alır

13.1.1 Logu sayılarla okumak

Bu bölümde "log" derken doğal logaritmayı (ln) kastediyoruz. Ücretin 0, 10, 100 ve 1000 olduğunu düşünelim:

```
import numpy as np
degerler = np.array([0, 10, 100, 1000])
np.log1p(degerler).round(4)
```

çıktı

```
array([0.    , 2.3979, 4.6151, 6.9088])
```

Tablo 13.2 log1p dönüşümünün sayısal karşılıkları.

Ham değer x	$\ln(1+x)$	Yorum
0	0,0000	Sıfır ücret tanımlı kalır
10	2,3979	Değer log birimine taşınır
100	4,6151	Büyük ham fark daha küçük bir log farkına dönüşür
1000	6,9088	Sağ kuyruktaki çok büyük değer sıkıştırılır

Ham ölçekte 10 ile 1000 arasında 990 birimlik bir uçurum var. Log ölçekte bu fark yaklaşık 4,5 birime iniyor. Log, büyük değerler arasındaki farkı küçük değerler arasındaki farktan çok daha güçlü sıkıştırır.

Logun bir başka özelliği de **oransal farkları** görünür kılmasıdır. Ham ölçekte 10'dan 20'ye artış 10 birim, 100'den 200'e artış 100 birimdir. Ama ikisi de iki katına çıkmadır. Doğrudan logda bu iki artış tamamen eşit uzaklıktadır:

$$\ln(20) - \ln(10) = \ln(200) - \ln(100) = \ln(2)$$

log1p'de +1 nedeniyle bu eşitlik tam değildir; değerler büyüdükçe yaklaşır. Titanic'te 15 yolcunun ücreti sıfır olduğu için doğrudan log yerine log1p seçiyoruz.

> **Dikkat**
>
> Gerçek sayılarda $\ln(x)$ için $x > 0$, $\ln(1 + x)$ için $x > -1$ gerekir. −5'e log1p uygulamak geçerli bir gerçek sayı üretmez; −1'de sonuç eksi sonsuzdur. Ücret negatif olmamalıdır. Negatif bir ücretle karşılaşırsak önce kaydı araştırırız (Bölüm 11). Log uygulamak geçersiz bir değeri geçerli bir ölçüme dönüştürmez.

13.1.2 Önce hangi sorunu çözdüğümüzü bilelim

Bir dönüşümü alışkanlıkla değil, belirli bir sorunu çözmek için uygularız:

Tablo 13.3 Sorun ve uygun yaklaşım.

Sorun ya da amaç	Uygun yaklaşım	Titanic bağlantısı
Farklı birimler uzaklığı ya da cezayı etkiliyor	Ölçekleme	Yaş, ücret ve aile sayımı farklı büyüklüklerde
Sağ kuyruktaki birkaç büyük değer ham farkları büyütüyor	Uygunsa log1p, karekök ya da kuvvet dönüşümü	Yüksek ücretleri silmek yerine log1p seçiyoruz
Oransal bir ilişkiyi temsil etmek istiyoruz	Log	10 → 20 ile 100 → 200 aynı oranda artış
Doğrulanmış yanlış kayıt var	Kaynak düzeltmesi ya da hücreyi eksik sayma	Negatif ücret log ile düzelmez
Amaç yalnızca değişkeni "normal dağıtmak"	Önce modelin gerçekte ne varsaydığını incele	Ağaç modelleri ve KNN girdilerin normal dağılmasını gerektirmez

Son satır önemli bir yanılgıyı düzeltiyor. Birçok tahmin algoritması, girdi sütunlarının normal dağılmasını gerektirmez. Doğrusal regresyon gibi çıkarım modellerinde normallik varsayımı da çoğu zaman girdilerle değil, modelin **hatalarıyla** ilgilidir (Bölüm 17). Varsayım doğru yerde kontrol edilmelidir.

Dönüşüm ne zaman gereksiz ya da sakıncalıdır?

- 0/1 göstergelere ve nominal kategorilere dağılım dönüşümü uygulamak çoğunlukla anlamsızdır.
- Ham birimlerdeki fark analizin esas sorusuysa log, sorunun anlamını değiştirir.

- Yaklaşık simetrik ve makul aralıktaki bir alana alışkanlıkla log uygulamak gereksizdir. Yaşa, sırf sayısal diye log uygulamıyoruz.

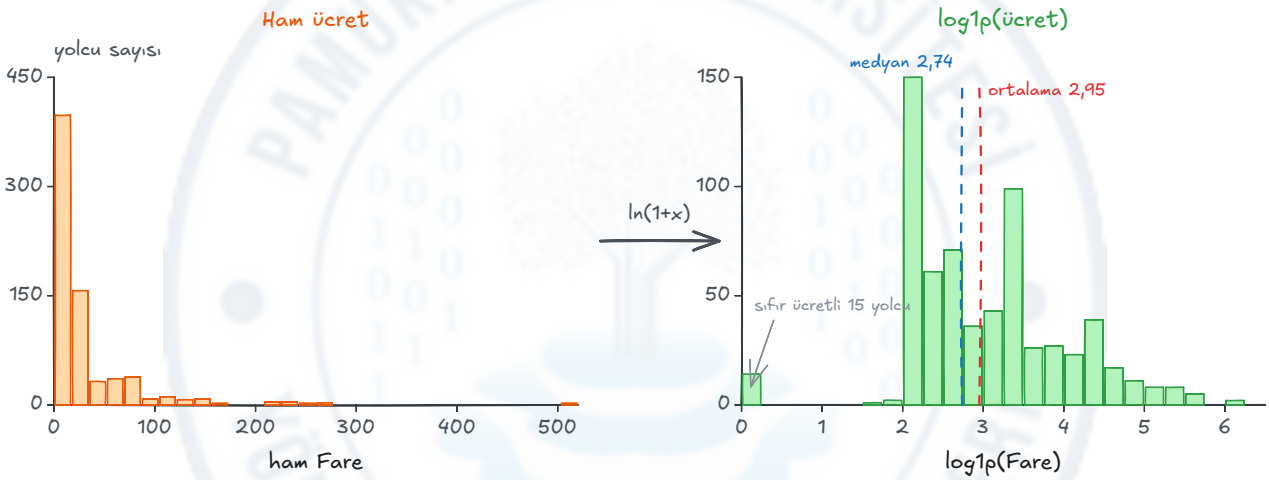
13.1.3 Ücret dönüşümünü uygulamak

log1p dağılımdan öğrenilecek bir parametre içermez; sabit bir formüldür. Bu yüzden eğitime ve teste aynen uygulanabilir:

```
fare_ham = X_egitim_islem["Fare"].copy()            # karşılaştırma için sakla
X_egitim_islem["Fare"] = np.log1p(X_egitim_islem["Fare"])
X_test_islem["Fare"] = np.log1p(X_test_islem["Fare"])
print(fare_ham.agg(["min", "median", "mean", "max"]).round(2).to_dict())
print(X_egitim_islem["Fare"].agg(["min", "median", "mean", "max"]).round(2).to_dict())
```

çıktı

```
{'min': 0.0, 'median': 14.45, 'mean': 31.82, 'max': 512.33}
{'min': 0.0, 'median': 2.74, 'mean': 2.95, 'max': 6.24}
```



Şekil 13.1 Eğitim ücretlerinin ham ve log1p gösterimi. Log, uzun sağ kuyruğu sıkıştırır; ortalama ile medyan birbirine yaklaşır. Ama dağılım tam simetrik olmaz.

Ham ücrette ortalama (31,82) medyanın (14,45) iki katından fazlaydı. Log ölçekte ortalama 2,95, medyan 2,74; birbirlerine çok yaklaştılar (Şekil 13.1). Sağ kuyruk sıkıştı. Ama sıfır ücretli 15 yolcu hâlâ en solda ayrı bir küme oluşturuyor; log dönüşümü normal dağılım garantisi vermez.

Logaritma sonrası değerler artık ham ücret biriminde değildir. Bunu grafik eksenlerinde ve sütun adlarında belirtir, veri sözlüğünü güncelleriz. Gerektiğinde ham birime geri dönmek için log1p'nin tersi olan `np.expm1` kullanılır: $\mathrm{expm1}(2{,}3979) \approx 10$.

13.1.4 Box-Cox ve Yeo-Johnson

Log ya da karekök yetersiz kalırsa, veriye en uygun kuvveti kendisi bulan dönüşümler değerlendirilebilir. Bu dönüşümlerin kuvvet parametresi veriden öğrenildiği için yalnızca eğitimde `fit` edilir. Box-Cox pozitif veri ister; Yeo-Johnson sıfır ve negatif değerleri de ele alabilir. Ama negatif bir değerin alan açısından geçersiz olması ayrı bir kalite sorunudur.

13.2 Ölçeklendirme

13.2.1 Ölçek farkı uzaklığı nasıl değiştirir?

Bazı yöntemler gözlemler arasındaki **uzaklığı** hesaplar. Ölçeklenmemiş verilerde büyük sayısal aralığı olan bir özellik, yalnızca birimi nedeniyle baskın hâle gelebilir. Titanic dosyasındaki ilk iki yolcuyu yalnızca yaş ve ücret üzerinden düşünelim:

Tablo 13.4 İki yolcunun yaş ve ücreti.

Yolcu	Yaş	Ham ücret
1	22	7,2500
2	38	71,2833

Öklid uzaklığında farkların kareleri toplanır:

$$d(A,B) = \sqrt{(38-22)^2 + (71{,}2833-7{,}25)^2}$$

```
yas_katki = (38 - 22) ** 2
ucret_katki = (71.2833 - 7.25) ** 2
print(yas_katki, round(ucret_katki, 2))
print("Ücretin payı: %", round(100 * ucret_katki / (yas_katki + ucret_katki), 1))
```

çıktı

```
256 4100.26
Ücretin payı: % 94.1
```

Yaşın katkısı 256, ücretin katkısı yaklaşık 4100,26. Toplam kare uzaklığın %94,1'i ücret farkından geliyor. İki yolcunun ne kadar "benzer" olduğu sorusunu neredeyse tamamen ücret yanıtlıyor.

> **Dikkat**
>
> Bu oran, ücretin hayatta kalmayı açıklamakta daha önemli olduğunu **göstermez**. Yalnızca iki değişkenin birimlerinin ve bu iki yolcu arasındaki farkların hesaba yansıma biçimidir. Yaşın birimini yıldan aya çevirseydik (16 yıl = 192 ay), yaşın katkısı 36.864'e çıkar ve bu kez yaş baskın olurdu. Aynı veri, birim değişikliğiyle farklı bir "benzerlik" verir.

Ölçekleme bu birim etkisini azaltır. Ama iki şeyi yapmaz: Değişkenleri eşit derecede önemli yapmaz ve gereksiz bilgiyi temizlemez. Birbirine çok benzeyen iki alanı birlikte eklemek, aynı bilgiyi uzaklık hesabında iki kez saydırır.

13.2.2 Hangi modeller ölçeğe duyarlıdır?

Tablo 13.5 Ölçeğin model türlerine etkisi.

Yöntem	Ölçeğe duyarlılığın nedeni
KNN, K-Means	Komşuluk ve küme kararları uzaklık hesabına dayanır
RBF çekirdekli SVM	Çekirdek bir uzaklık fonksiyonudur
Düzenlileştirilmiş doğrusal ve lojistik modeller	Aynı ceza, farklı birimlerdeki katsayıları farklı etkiler
PCA	Varyansı büyük alan, bileşenlerde baskın olur (Bölüm 14)
Sinir ağları	Girdi büyüklükleri optimizasyonun ilerleyişini etkiler
Karar ağacı, Random Forest	Bölmeler değerlerin sırasına dayanır; ölçekleme genellikle gerekmez

Bu liste bir model ile bir ölçekleyici arasında zorunlu bir eşleşme kurmaz (“KNN için yalnızca MinMax” gibi). Seçim veriye ve eğitim içi doğrulamaya göre yapılır.

13.2.3 Üç ölçekleyici

Tablo 13.6 Üç yaygın ölçekleyici.

Yöntem	Formül	Sonuç ve sınırı
StandardScaler	$\frac{x - \mu}{\sigma}$	Eğitimde ortalama 0, standart sapma 1; uç değerlerden etkilenir
MinMaxScaler	$\frac{x - x_{\min}}{x_{\max} - x_{\min}}$	Eğitim aralığı [0, 1]; uç değerlere çok duyarlıdır
RobustScaler	$\frac{x - \text{medyan}}{\text{IQR}}$	Medyan ve çeyrekleri kullanır; uçları silmez, aralığı sınırlamaz

Formüllerdeki bütün parametreler (μ, σ, en küçük ve en büyük değer, medyan, IQR) **eğitim verisinden** öğrenilir. Bir ölçekleyici, test verisinin ortalamasını ya da en büyük değerini hiçbir zaman hesaplamaz.

Üçünün aynı veride nasıl davrandığını küçük bir örnekle görelim. Eğitim ücretleri [10, 20, 30, 40, 100] olsun:

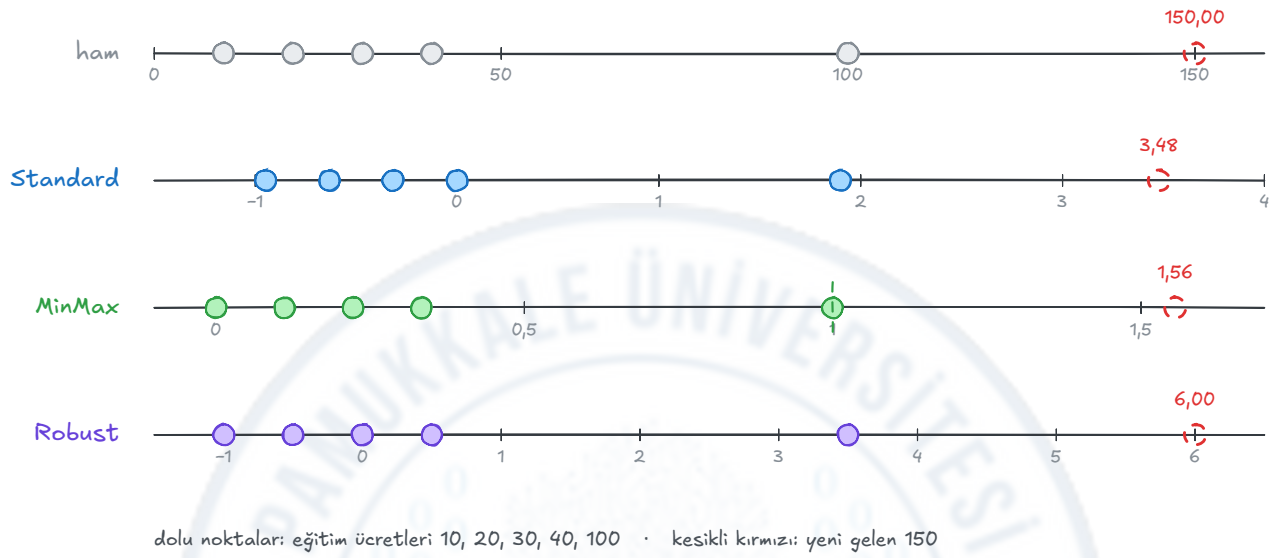
```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler

ornek = pd.DataFrame({"Ucret": [10, 20, 30, 40, 100]})
yeni = pd.DataFrame({"Ucret": [150]})
for ad, olcekleyici in [("Standard", StandardScaler()),
                        ("MinMax", MinMaxScaler()),
                        ("Robust", RobustScaler())]:
    olcekleyici.fit(ornek)
```

```
    print(f"{ad:9s}", olcekleyici.transform(ornek).ravel().round(2),
          "| yeni 150 →", olcekleyici.transform(yeni).ravel().round(2))
```

çıktı

```
Standard  [-0.95 -0.63 -0.32  0.    1.9 ] | yeni 150 → [3.48]
MinMax    [0.   0.11 0.22 0.33 1.  ] | yeni 150 → [1.56]
Robust    [-1.  -0.5  0.   0.5  3.5] | yeni 150 → [6.]
```



Şekil 13.2 Aynı beş ücretin üç ölçekleyicideki karşılığı. Sıra ve göreli konumlar her üçünde de korunur; değişen merkez ve birimdir. Eğitim dışından gelen 150, MinMax ölçeğinde 1'i aşar.

Şekil 13.2'de üç önemli gözlem var:

- **Sıra korunur.** Üç ölçekleyici de pozitif doğrusal dönüşümdür; hangi ücretin hangisinden büyük olduğunu değiştirmezler. Dağılımın çarpık şekli de korunur: 100, diğer dört değerden hâlâ çok uzaktadır.
- **Uç değer etkisi farklıdır.** MinMax'te 100 değeri en büyük olduğu için tam 1'e oturur ve diğer dört değeri 0 ile 0,33 arasına sıkıştırır. RobustScaler medyana (30) ve IQR'ye (20) dayandığı için diğer değerleri geniş tutar, ama 100'ü 3,5'e taşır.
- **Yeni veri eğitim aralığını aşabilir.** 150, MinMax ölçeğinde 1,56 olur. Bu beklenen bir sonuçtur. Testte en küçük ve en büyük değer yeniden hesaplanmaz; aksi hâlde aynı sayı her veri kümesinde farklı bir anlam taşır.

13.2.4 Titanic'te StandardScaler

Yaş, aile büyüklüğü ve loglanmış ücret için StandardScaler'ı uyguluyoruz:

```
sayisal_sutunlar = ["Age", "AileBuyuklugu", "Fare"]     # Fare burada loglanmış
standard = StandardScaler()
standard.fit(X_egitim_islem[sayisal_sutunlar])          # yalnızca eğitimden öğrenir
X_egitim_sayisal = pd.DataFrame(standard.transform(X_egitim_islem[sayisal_sutunlar]),
                                columns=sayisal_sutunlar, index=X_egitim.index)
X_test_sayisal = pd.DataFrame(standard.transform(X_test_islem[sayisal_sutunlar]),
                              columns=sayisal_sutunlar, index=X_test.index)
print("Öğrenilen ortalamalar:", standard.mean_.round(3))
print("Eğitim ortalaması:", X_egitim_sayisal.mean().round(3).to_dict())
```

```
print("Eğitim sapması:  ", X_egitim_sayisal.std(ddof=0).round(3).to_dict())
print("Test ortalaması: ", X_test_sayisal.mean().round(3).to_dict())
```

çıktı

```
Öğrenilen ortalamalar: [29.556  1.883  2.951]
Eğitim ortalaması: {'Age': 0.0, 'AileBuyuklugu': 0.0, 'Fare': 0.0}
Eğitim sapması:   {'Age': 1.0, 'AileBuyuklugu': 1.0, 'Fare': 1.0}
Test ortalaması:  {'Age': -0.036, 'AileBuyuklugu': 0.066, 'Fare': 0.058}
```

Eğitimde üç alanın ortalaması 0, standart sapması 1 oldu. Testin ortalaması ise tam 0 değil. Bu doğaldır: Test yolcuları eğitim yolcularıyla aynı değildir. Testi yeniden `fit` etmek bu farkı gizler ve veri sızıntısı yaratır.

13.2.5 Standardize değer ne anlama gelir?

$$z = \frac{x - \mu_{\text{eğitim}}}{\sigma_{\text{eğitim}}}$$

Eğitim yaş ortalamasının 30, standart sapmasının 10 olduğunu varsayalım. 20 yaş −1, 30 yaş 0, 40 yaş 1, 50 yaş 2 olur. Standardize değer, "eğitim ortalamasından kaç standart sapma uzakta" sorusunun cevabıdır.

Bu formül Bölüm 11'deki Z-skoruyla aynıdır. Fark, kullanım amacındadır: Orada $|z| > 3$ olan değerleri işaretlemek için kullandık; burada bütün değerleri bu birime taşıyoruz ve hiçbir satırı silmiyoruz.

> **Dikkat**
>
> StandardScaler veriyi normal dağılıma **dönüştürmez**; yalnızca merkezi ve ölçeği değiştirir. Çarpık bir dağılım standardize edildikten sonra da çarpık kalır. Çıktı −3 ile +3 arasına sınırlı da değildir. Normal dağılımdaki "±1, ±2, ±3 sapma aralıkları gözlemlerin yaklaşık %68, %95 ve %99,7'sini kapsar" bilgisi normal dağılıma aittir; standardize edilmiş her sütun için geçerli değildir.

13.2.6 MinMaxScaler alternatifi

```
minmax = MinMaxScaler(feature_range=(0, 1))
minmax.fit(X_egitim_islem[sayisal_sutunlar])
X_egitim_minmax_sayisal = pd.DataFrame(minmax.transform(X_egitim_islem[sayisal_sutunlar]),
                                       columns=sayisal_sutunlar, index=X_egitim.index)
X_egitim_minmax_sayisal.describe().loc[["min", "mean", "max"]].round(3)
```

çıktı

```
        Age  AileBuyuklugu   Fare
min   0.000          0.000  0.000
mean  0.366          0.088  0.473
max   1.000          1.000  1.000
```

Eğitim alanlarının her biri artık 0 ile 1 arasında. Bu, StandardScaler'ın üzerine uygulanmış bir işlem değil; aynı hazırlanmış alanlardan üretilmiş **alternatif** bir gösterimdir. MinMax ile StandardScaler'ı aynı alana art arda uygulamak genellikle ek bir yarar sağlamaz. Birini seçeriz;

hangisinin daha iyi çalıştığına modelleme aşamasında eğitim içi doğrulamayla karar veririz, test sonucuna bakarak değil.

13.2.7 Aynı ücretin dört gösterimi

```
karsilastirma = pd.DataFrame(index=X_egitim.index)
karsilastirma["Ham Fare"] = fare_ham
karsilastirma["Log Fare"] = X_egitim_islem["Fare"]
karsilastirma["Standard Fare"] = X_egitim_sayisal["Fare"]
karsilastirma["MinMax Fare"] = X_egitim_minmax_sayisal["Fare"]
karsilastirma.head().round(3)
```

çıktı

	Ham Fare	Log Fare	Standard Fare	MinMax Fare
692	56.496	4.052	1.125	0.649
481	0.000	0.000	-3.014	0.000
527	221.779	5.406	2.508	0.866
855	9.350	2.337	-0.627	0.374
801	26.250	3.305	0.362	0.530

Bu dört sütun dört farklı yolcu değil, aynı eğitim satırlarının dört farklı gösterimidir. Negatif bir "Standard Fare", log ücretin eğitim ortalamasının altında olduğunu söyler; ücretin negatif olduğunu değil. Sıfır bir "MinMax Fare" ise o yolcunun eğitimdeki en düşük ücrete sahip olduğunu gösterir; ücretsiz seyahat ettiğini değil. Yorum yaparken hangi gösterimde olduğumuzu unutmamalıyız.

13.3 Dönüşüm ve ölçeklemeyi birlikte kullanmak

Log ile StandardScaler'ı ücrete art arda uyguladık. Bunun gerekçesi, ikisinin farklı işler yapmasıdır:

Tablo 13.7 Dönüşüm ve ölçekleme neyi değiştirir?

Özellik	MinMax	Standard	log1p
Dağılımın çarpıklığı	Korunur	Korunur	Değişebilir
Çıktı aralığı	Eğitimde [0, 1]; yeni veri aşabilir	Sabit aralık yok	Sabit aralık yok
Merkez ve yayılım	Eğitim en küçük ve en büyüğüne göre	Eğitimde ortalama ≈ 0, sapma ≈ 1	Sonrasında ölçekleme düşünülebilir
Sıra	Korunur	Korunur	Korunur

Log, dağılımın **şeklini**; ölçekleyiciler **merkezi ve ölçeği** değiştirir. Biri ötekinin yerini tutmaz. Yalnızca MinMax uygulamak ücretin sağa çarpıklığını gidermez; yalnızca log uygulamak da ücreti diğer değişkenlerle ortak bir ölçeğe getirmez.

Ücret için kullandığımız birleşik dönüşüm:

$$u = \ln(1 + x), \qquad z = \frac{u - \mu_{\log,\text{eğitim}}}{\sigma_{\log,\text{eğitim}}}$$

Burada μ ve σ, **ham ücretin değil, loglanmış eğitim ücretinin** ortalaması ve standart sapmasıdır. Ham birime geri dönmek için sıra tersine çevrilir:

$$u = z \cdot \sigma_{\log,\text{eğitim}} + \mu_{\log,\text{eğitim}}, \qquad x = e^{u} - 1$$

Kodda önce ölçekleyicinin `inverse_transform` işlemi, ardından `np.expm1` kullanılır. Bu ters dönüşüm, doldurulmuş bir değeri geri getirir; eksik olan gerçek değeri keşfetmez.

13.3.1 İşlemlerin sırası

Basit bir sayısal hatta şu sıra kullanılır:

geçerlilik kontrolü → eksik doldurma → seçilen alanda log → ölçekleme

Önce eksikleri doldururuz, çünkü log ve ölçekleyiciler eksik değerle çalışmaz. Sonra log uygularız, çünkü ölçekleyicinin parametreleri logdan sonraki dağılıma göre öğrenilmelidir. One-hot sütunları ve 0/1 göstergeleri, sırf sayısal oldukları için ölçeklemek zorunlu değildir; biz onları olduğu gibi koruyoruz.

Bu sıra her durumda geçerli değildir. KNN ile doldurma gibi ölçeğe bağlı bir yöntem seçersek, uzaklık hesabından önce ölçekleri nasıl ele alacağımızı yeniden tasarlamamız gerekir.

> **"Normalizasyon" hangisi?**
> "Normalizasyon" sözcüğü bazen MinMax ölçekleme, bazen de her satır vektörünün uzunluğunu 1'e getirme işlemi için kullanılır. `scikit-learn`'deki `Normalizer` ikincisini yapar ve sütunları değil satırları ölçekler. Bir kaynakta bu sözcüğü gördüğünüzde hangi işlemin kastedildiğine bakın.

Bölüm özeti

- Dağılım dönüşümleri şekli değiştirir. log1p büyük değerleri sıkıştırır, sıfırı korur ve oransal farkları görünür kılar. Tanım aralıklarına dikkat edilir; log geçersiz bir değeri düzeltmez.
- Dönüşüm belirli bir sorun için uygulanır. Birçok model girdilerin normal dağılmasını gerektirmez; göstergelere, kategorilere ve yaklaşık simetrik alanlara dönüşüm gereksizdir.
- Titanic eğitim ücretinde log1p, ortalama ile medyanı birbirine yaklaştırır ama normal dağılım üretmez. Ters dönüşüm `np.expm1` ile yapılır.
- Ölçeklenmemiş verilerde birimi büyük olan değişken uzaklığı domine eder: İlk iki yolcunun kare uzaklığının %94,1'i ücretten gelir. Bu bir önem ölçüsü değildir.
- KNN, SVM, düzenlileştirilmiş doğrusal modeller, PCA ve sinir ağları ölçeğe duyarlıdır; ağaç modelleri genellikle değildir.
- StandardScaler ortalama 0 ve sapma 1, MinMaxScaler [0, 1] aralığı, RobustScaler medyan ve IQR kullanır. Parametreler yalnızca eğitimden öğrenilir; test verisi eğitim aralığını aşabilir.
- Ölçekleyiciler sırayı ve çarpıklığı korur; StandardScaler normal dağılım üretmez.
- Log şekli, ölçekleyici merkezi ve birimi değiştirir; bu yüzden art arda kullanılabilirler. Sıra: doldurma → log → ölçekleme.

Kaynaklar

Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3. baskı). O'Reilly. Bölüm 2: "Feature Scaling and Transformation".

Müller, A. C. ve Guido, S. (2016). *Introduction to Machine Learning with Python*. O'Reilly. Bölüm 3: "Preprocessing and Scaling" ve Bölüm 4: "Univariate Nonlinear Transformations".

scikit-learn geliştiricileri. *Preprocessing data*. https://scikit-learn.org/stable/modules/preprocessing.html

14

BÖLÜM

Öznitelik seçimi, boyut indirgeme ve sınıf dengesizliği

Bu bölümde

- Öznitelik seçimi ile boyut indirgemenin farkını açıklamak,
- alan bilgisi, filtre, sarmalayıcı ve model tabanlı seçim yaklaşımlarını tanımak; düşük korelasyonun neden tek başına eleme gerekçesi olmadığını göstermek,
- temel bileşen analizinin (PCA) ne yaptığını ve sınırlarını açıklamak,
- sınıf dengesizliğinde doğruluğun neden yanıltıcı olabildiğini göstermek; kesinlik, duyarlılık ve F1 ölçütlerini hesaplamak,
- sınıf ağırlığı, azaltarak örnekleme, çoğaltarak örnekleme ve SMOTE seçeneklerini ve risklerini karşılaştırmak.

Kaynak: YBS 319 ders sayfası ve uygulama defteri

Bu bölümde ön işleme ile modellemenin sınırında duran üç konuyu ele alıyoruz: hangi sütunların modele girmesi gerektiği, çok sayıda sütunun daha az sayıda yeni sütunla özetlenip özetlenemeyeceği ve hedef sınıflar eşit büyüklükte olmadığında ne yapılacağı. Üç konuda da aynı uyarı geçerli: Titanic için bu yöntemlerin hiçbiri zorunlu değil. Bir yöntem gerekçesi olduğunda eklenir.

14.1 Öznitelik seçimi

Çok sayıda ya da gereksiz özellik işlem maliyetini artırabilir, aşırı uyumu kolaylaştırabilir (Bölüm 7) ve modelin yorumunu zorlaştırabilir. Ama daha az sütun her zaman daha iyi sonuç vermez. Önemli bir bilgiyi atarsak model temel bir farkı göremez.

İki kavramı ayıralım. **Öznitelik seçimi** mevcut sütunlardan bir alt küme tutar; seçilen sütunlar aynen kalır. **Boyut indirgeme** ise sütunları yeni bir temsile dönüştürür; ortaya çıkan sütunlar eski sütunların birleşimidir.

14.1.1 Dört yaklaşım

Alan bilgisi

Problemi bilen birinin kararı. `PassengerId` bir kimlik olduğu için girdiden çıkarıldı. Ham `Name` yerine uygun bir unvan bilgisi değerlendirilebilir. `AileBuyuklugu` kullanıldığı için `SibSp` ve `Parch` ayrıca eklenmedi. Bu yaklaşım çoğu zaman en etkilisidir.

Filtre

Modelden bağımsız bir ölçüye göre eleme: sabit ya da çok düşük varyanslı sütunlar, birbirine çok benzeyen sütunlar ya da hedefle ilişki ölçüleri. İlişki ölçüsü değişken türüne uygun seçilmelidir.

Sarmalayıcı

Farklı özellik alt kümelerini bir modelle deneyip eğitim içi doğrulamayla karşılaştırmak. Daha doğrudan ama hesaplama açısından pahalıdır.

Model tabanlı

Bir modelin kendi içinde ürettiği önem bilgisini kullanmak: düzenlileştirmenin sıfıra çektiği katsayılar ya da ağaç modellerinin önem skorları. Bu önemler kullanılan yönteme ve değişkenler arasındaki ilişkiye bağlıdır.

14.1.2 Düşük korelasyon eleme gerekçesi değildir

Bölüm 8'de `Survived` ile `Age` arasındaki Pearson korelasyonunun yalnızca −0,08 olduğunu gördük. Bu sayıya bakıp yaşı modelden çıkarmak doğru olmaz. Aynı bölümde yaş gruplarına baktığımızda 0–12 yaş grubunun belirgin biçimde farklı olduğunu görmüştük. İlişki doğrusal değilse Pearson katsayısı onu yakalayamaz. Etkileşimler (yaşın etkisinin cinsiyete göre değişmesi gibi) ve eksiklik de sonucu etkiler.

Doğrusal olmayan ilişkileri de yakalayabilen bir ölçü **karşılıklı bilgidir** (*mutual information*). Bir değişkeni bilmenin hedef hakkındaki belirsizliği ne kadar azalttığını ölçer:

```
from sklearn.feature_selection import SelectKBest, mutual_info_classif
```

```
secim_girdi = X_egitim_ozellik[["Age", "Fare", "AileBuyuklugu"]].copy()
secim_girdi["Age"] = secim_girdi["Age"].fillna(yas_medyan)

def bilgi_skoru(girdi, hedef):
    return mutual_info_classif(girdi, hedef,
                               discrete_features=[False, False, True],
                               random_state=42)

secici = SelectKBest(score_func=bilgi_skoru, k=2)
secici.fit(secim_girdi, y_egitim)              # yalnızca eğitimde öğrenir
pd.Series(secici.scores_, index=secim_girdi.columns).round(4)
```

çıktı

```
Age              0.0399
Fare             0.1685
AileBuyuklugu    0.0536
dtype: float64
```

`SelectKBest` her sütuna bir skor verir ve en yüksek skorlu k sütunu seçer. `discrete_features` hangi sütunların ayrık (sayım) olduğunu belirtir: yaş ve ücret sürekli, aile büyüklüğü ayrık. En yüksek skoru ücret aldı; yaş ile aile büyüklüğü daha düşük ama sıfırdan farklı skorlar aldı. Bu skorlar bir **seçim kararı değildir**; yalnızca tek tek değişkenlere bakan bir ölçüdür ve değişkenlerin birlikte taşıdığı bilgiyi görmez.

> **Dikkat**
>
> Seçim adımı da bir öğrenilen işlemdir: Hangi sütunların seçileceği hedefe bakılarak belirlenir. Bu yüzden yalnızca eğitim verisinde, çapraz doğrulamada ise her eğitim katında ayrı ayrı öğrenilmelidir. Test sonuçlarına bakarak özellik seçmek, test kümesini dolaylı olarak eğitime katar.

14.2 Boyut indirgeme: temel bileşen analizi

Temel bileşen analizi (*principal component analysis*, PCA), verideki değişkenliği en çok açıklayan doğrusal yönleri bulur. Ortaya çıkan **bileşenler**, mevcut özelliklerin ağırlıklı toplamlarıdır.

Sezgisel olarak şöyle düşünebiliriz. İki değişkenli bir nokta bulutu, bir yönde uzanıyor olsun (Şekil 14.1). Birinci bileşen, noktaların en çok yayıldığı yöndür. İkinci bileşen ona dik olan yöndür. Noktaların çoğu birinci yön boyunca dağılıyorsa, her noktayı yalnızca birinci bileşen üzerindeki konumuyla temsil edebiliriz. İki sütun bire iner; kaybettiğimiz şey, noktaların ana eksene olan dik uzaklıklarıdır.



Şekil 14.1 PCA'nın iki boyuttaki sezgisi. Birinci bileşen değişkenliğin en büyük olduğu yöndür. Noktaları bu eksene izdüşürmek iki sütunu bire indirir ve dik yöndeki ayrıntıyı atar.

Titanic'in sayısal alanlarına uygulayalım:

```
from sklearn.decomposition import PCA

pca_girdi = X_egitim_hazir[["sayisal__Age", "ucret__Fare", "sayisal__AileBuyuklugu"]]
pca = PCA(n_components=2)
pca.fit(pca_girdi)                      # yönleri eğitimden öğrenir
bilesenler = pca.transform(pca_girdi)
print(bilesenler.shape)
print(pca.explained_variance_ratio_.round(3))
```

çıktı

```
(712, 2)
[0.464 0.364]
```

Üç standardize alanı iki bileşene indirdik. `explained_variance_ratio_` her bileşenin toplam değişkenliğin ne kadarını taşıdığını gösterir: Birinci bileşen yaklaşık %46'sını, ikincisi %36'sını; ikisi birlikte %83'ünü koruyor. Üçüncü boyutu attığımızda değişkenliğin yaklaşık %17'sini kaybettik.

PCA'yı kullanırken dört şeyi aklımızda tutmalıyız:

1. **PCA hedefi kullanmaz.** En çok değişkenlik taşıyan bileşen, hedefi tahmin etmek için en yararlı bileşen olmayabilir. Korunan varyans oranı bir tahmin doğruluğu değildir.
2. **Ölçek belirleyicidir.** PCA veriyi merkezler ama ölçeklemez. Birimi büyük olan alan varyansta baskın olur (Bölüm 13). Bizim girdimiz zaten standardize olduğu için bu sorun yok.
3. **Yorum zorlaşır.** "Bileşen 1" artık yaş ya da ücret değildir; ikisinin ağırlıklı bir karışımıdır.
4. **Bileşenler eğitimden öğrenilir** ve test verisine aynen uygulanır.

Titanic az sayıda özgün sütuna sahip olduğu için PCA temel hattımızın zorunlu bir parçası değil. Sırf PCA anlatıldı diye veriyi indirgemek gerekmez. PCA, yüzlerce ya da binlerce sütunlu verilerde, örneğin görüntülerde ya da sensör ölçümlerinde, çok daha anlamlıdır.

14.3 Sınıf dengesizliği

Titanic'te 549 yolcu için hedef 0, 342 yolcu için 1'dir: yaklaşık %61,6 ve %38,4. Eğitim verisinde de oranlar aynı:

```
y_egitim.value_counts()
```

çıktı

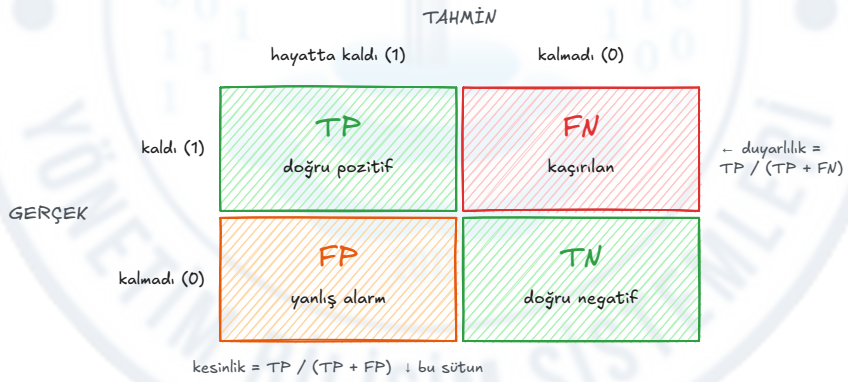
```
Survived
0    439
1    273
Name: count, dtype: int64
```

Sınıfların %50–%50 olması gerekmez. Asıl soru, hangi hatanın daha önemli olduğu ve modelin hangi sınıfta zorlandığıdır.

14.3.1 Doğruluk yetmeyebilir

Bölüm 5'teki sahtecilik örneğini hatırlayın: Her işleme "normal" diyen bir model %99 doğruluk elde ediyordu. Titanic'te de aynı durumu kurabiliriz. Herkese "hayatta kalmadı" diyen bir sınıflandırıcı yaklaşık %61,6 doğruluk elde eder; ama hayatta kalanların **hiçbirini** yakalayamaz. Bu yüzden doğruluğun yanında başka ölçütlere de bakarız.

Bu ölçütleri tanımlamak için önce tahminleri dört gruba ayırırız. Pozitif sınıf `Survived = 1` olsun (Şekil 14.2):



Şekil 14.2 Karışıklık matrisi. Satırlar gerçek sınıfı, sütunlar tahmini gösterir. Köşegendeki hücreler doğru, diğerleri yanlış tahminlerdir.

Doğru pozitif (TP)
: Hayatta kalan ve model de "hayatta kaldı" dedi.

Yanlış negatif (FN)
: Hayatta kalan ama model "kalmadı" dedi; kaçırılan.

Yanlış pozitif (FP)
: Hayatta kalmayan ama model "kaldı" dedi; yanlış alarm.

Doğru negatif (TN)
: Hayatta kalmayan ve model de "kalmadı" dedi.

Bu dört sayıdan üç ölçüt hesaplanır:

$$\text{Kesinlik (precision)} = \frac{TP}{TP + FP} \qquad \text{Duyarlılık (recall)} = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2 \times \text{Kesinlik} \times \text{Duyarlılık}}{\text{Kesinlik} + \text{Duyarlılık}}$$

Kesinlik, "hayatta kaldı" dediklerimizin ne kadarının gerçekten hayatta kaldığını; **duyarlılık**, gerçekten hayatta kalanların ne kadarını yakalayabildiğimizi ölçer. **F1**, ikisinin harmonik ortalamasıdır ve ikisinden biri çok düşükse düşük çıkar.

Örnek 14.1

Herkese 0 diyen sınıflandırıcıyı eğitim verisinde değerlendirelim. 712 yolcunun 439'u 0, 273'ü 1. Model hiç "1" demediği için TP = 0, FP = 0, FN = 273, TN = 439.

Doğruluk = 439 / 712 ≈ %61,7. Duyarlılık = 0 / 273 = 0. Kesinlik = 0 / 0, tanımsız. F1 = 0.

Yüksek görünen doğruluk, modelin hayatta kalanlar hakkında hiçbir şey öğrenmediğini gizliyor. Bu ölçütleri dersin ilerleyen konularında sınıflandırma modelleriyle birlikte ayrıntılı kullanacağız.

14.3.2 Dengesizlikle başa çıkma seçenekleri

Tablo 14.1 Sınıf dengesizliği için seçenekler.

Seçenek	Ne değişir?	Dikkat
Sınıf ağırlıkları	Hataların eğitimdeki ağırlığı	Yeni gözlem üretmez; modelin desteklemesi gerekir
Azaltarak örnekleme	Çoğunluk sınıfından kayıt çıkarılır	Bilgi kaybı olur
Çoğaltarak örnekleme	Azınlık örnekleri tekrarlanır	Aynı örnekler tekrarlandığı için model onları ezberleyebilir
SMOTE	Komşular arasında yapay örnekler üretilir	Kategorik ya da karma özelliklerde anlamsız değerler üretebilir

Sınıf ağırlıkları en hafif müdahaledir. Model eğitilirken azınlık sınıfındaki bir hata, çoğunluk sınıfındaki bir hatadan daha pahalı sayılır:

```
from sklearn.utils.class_weight import compute_class_weight

agirliklar = compute_class_weight(class_weight="balanced",
                                  classes=np.array([0, 1]),
                                  y=y_egitim.to_numpy())
agirliklar.round(3)
```

çıktı

```
array([0.811, 1.304])
```

`class_weight="balanced"` her sınıfa, sınıf büyüklüğüyle ters orantılı bir ağırlık verir: $n/(2 \times n_{\text{sınıf}})$. Sınıf 0 için 712 / (2 × 439) ≈ 0,811, sınıf 1 için 712 / (2 × 273) ≈ 1,304. Hayatta kalan bir yolcuyu yanlış sınıflandırmak, kalmayan birini yanlış sınıflandırmaktan yaklaşık 1,6 kat daha pahalı sayılır.

SMOTE (*Synthetic Minority Over-sampling Technique*) ise azınlık sınıfından bir örnek ile ona yakın başka bir azınlık örneği arasındaki doğru parçası üzerinde yeni, yapay bir nokta üretir (Şekil 14.3):

```
nokta_1 = np.array([20.0, 10.0])          # azınlık sınıfından bir gözlem
nokta_2 = np.array([30.0, 30.0])          # aynı sınıftan bir komşu
nokta_1 + 0.5 * (nokta_2 - nokta_1)       # aradaki yapay nokta
```

çıktı

```
array([25., 20.])
```



Şekil 14.3 SMOTE'un mantığı ve bir tuzağı. Sayısal özelliklerde iki komşunun arasında makul bir nokta üretilir; ama one-hot sütunlarında ara değer, hiçbir gerçek kategoriye karşılık gelmez.

[20, 10] ile [30, 30] arasında, tam ortada [25, 20] noktası üretildi. Sayısal özelliklerde bu makul olabilir. Ama aynı işlemi one-hot sütunlarına uygularsak sorun çıkar: `Embarked_C = 1` olan bir yolcu ile `Embarked_S = 1` olan bir yolcunun ortasında `Embarked_C = 0,5`, `Embarked_S = 0,5` olan bir "yolcu" oluşur. Böyle bir liman yoktur. Kategorik özellikler varsa bu sorunu dikkate alan SMOTENC gibi yöntemler değerlendirilir.

Dikkat

Bütün örnekleme işlemleri **yalnızca eğitim verisinde**, çapraz doğrulamada ise yalnızca eğitim katlarında yapılır. Test verisinin doğal sınıf dağılımı korunur. Testi dengelemek, modelin gerçek kullanımda karşılaşacağı dağılımı bozar ve sonuçları anlamsız kılar. Örneklemeyi bölmeden önce yapmak ise yapay örneklerin hem eğitime hem teste düşmesine, yani sızıntıya yol açar.

Titanic'teki %62–%38 oranı, yeniden örneklemeyi tek başına gerektirmez. Bu çalışmada dengeleme yapmıyoruz. İleride bir model kurduğumuzda kesinlik, duyarlılık ve hata maliyetini birlikte değerlendireceğiz. Gerekirse sınıf ağırlıkları ya da karar eşiğini ayarlamak (Bölüm 6) ilk denenecek seçeneklerdir.

Bölüm özeti

- Öznitelik seçimi mevcut sütunlardan bir alt küme tutar; boyut indirgeme sütunları yeni bir temsile dönüştürür.
- Seçim yaklaşımları alan bilgisi, filtre, sarmalayıcı ve model tabanlıdır. Seçim bir öğrenilen işlemdir; yalnızca eğitimde ya da eğitim katlarında yapılır, test sonucuna bakılarak yapılmaz.
- Düşük Pearson korelasyonu tek başına eleme gerekçesi değildir; ilişki doğrusal olmayabilir. Karşılıklı bilgi doğrusal olmayan ilişkileri de yakalayabilir.
- PCA değişkenliği en çok açıklayan doğrusal yönleri bulur. Hedefi kullanmaz, ölçeğe duyarlıdır ve yorumu zorlaştırır. Titanic gibi az sütunlu bir veride zorunlu değildir.
- Sınıf dengesizliğinde doğruluk yanıltabilir: Herkese 0 diyen bir model Titanic'te %61,6 doğruluk alır ama hiç hayatta kalan yakalamaz.
- Kesinlik "dediklerimizin ne kadarı doğru", duyarlılık "gerçek pozitiflerin ne kadarını yakaladık" sorusunu yanıtlar; F1 ikisini birleştirir.
- Sınıf ağırlıkları yeni gözlem üretmez; azaltma bilgi kaybettirir; çoğaltma ezberlemeye yol açabilir; SMOTE kategorik sütunlarda anlamsız ara değerler üretebilir.
- Örnekleme yalnızca eğitimde yapılır; test dağılımı korunur.

Kaynaklar

Guyon, I. ve Elisseeff, A. (2003). An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3, 1157–1182.

Chawla, N. V., Bowyer, K. W., Hall, L. O. ve Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.

James, G., Witten, D., Hastie, T., Tibshirani, R. ve Taylor, J. (2023). *An Introduction to Statistical Learning with Applications in Python*. Springer. Bölüm 12.2.

imbalanced-learn geliştiricileri. *Common pitfalls and recommended practices*. https://imbalanced-learn.org/stable/common_pitfalls.html

15

BÖLÜM

Tekrarlanabilir bir ön işleme hattı

Bu bölümde

- Elle yürütülen ön işlemenin risklerini ve tekrarlanabilir bir hattın sağladığı güvenceyi açıklamak,
- `Pipeline` ve `ColumnTransformer` ile farklı sütun gruplarına farklı işlem zincirleri uygulamak,
- adım adım hazırlanan tablo ile hattın ürettiği tablonun aynı olduğunu doğrulamak,
- yeni bir kaydı, eksik ve görülmemiş değerleriyle birlikte aynı hatla hazırlamak,
- öğrenilmiş bir hattı kaydetmek ve yeniden yüklemek,
- çapraz doğrulamada ön işlemenin neden her katta yeniden öğrenilmesi gerektiğini göstermek,
- ön işlemenin sonunda yapılacak kontrolleri ve karar günlüğünü hazırlamak.

Kaynak: YBS 319 ders sayfası ve uygulama defteri

Bölüm 9'dan bu yana Titanic verisini adım adım hazırladık: Eksikleri eğitim medyanı ve moduyla doldurduk, özellikler türettik, kategorileri kodladık, ücrete log uyguladık, sayısal alanları ölçekledik. Her adımda aynı kuralı uyguladık: Öğrenilen her şey eğitimden öğrenildi, test yalnızca dönüştürüldü.

Bu işler elle yürütüldüğünde kolayca karışabilir. Sütun sırası değişebilir, bir doldurma değeri yanlış tablodan hesaplanabilir, test verisine yanlışlıkla `fit` uygulanabilir, ücret iki kez loglanabilir. Kullanıma alındıktan sonra gelen yeni bir yolcunun, eğitimdekinden farklı bir kuralla hazırlanması da mümkündür. Bu bölümde bütün adımları tek bir tekrarlanabilir **hatta** topluyoruz.

15.1 fit, transform ve fit_transform

Bölüm 9'da tanıdığımız üç işlemi bir kez daha hatırlayalım, çünkü hattın bütün mantığı bunlara dayanır:

Tablo 15.1 Üç işlem ve kullanıldıkları yer.

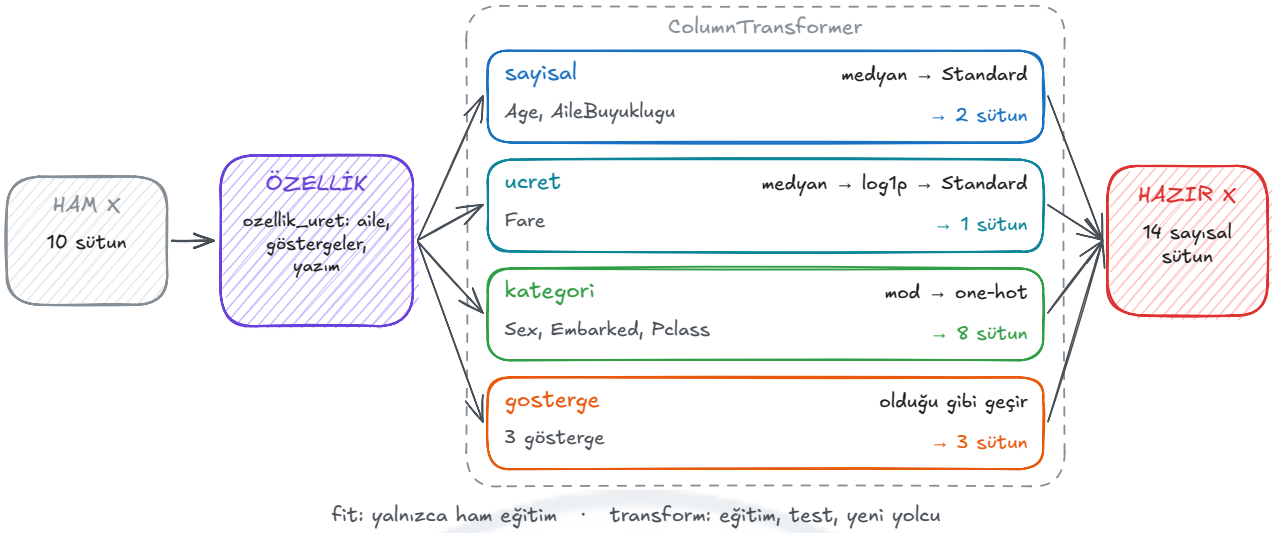
İşlem	Anlamı	Nerede?
`fit`	Parametreleri öğren: medyan, ölçek, kategori listesi	Yalnızca eğitim ya da ilgili eğitim katı
`transform`	Öğrenilmiş parametrelerle veriyi dönüştür	Eğitim, doğrulama, test ve yeni veri
`fit_transform`	Öğren ve aynı eğitim verisini dönüştür	Yalnızca eğitimde; testte kullanılmaz

15.2 Pipeline ve ColumnTransformer

`scikit-learn` iki araç sunar:

- **`Pipeline`**, birden çok işlemi bir zincirde toplar. Zincir `fit` edildiğinde her adım sırayla öğrenir ve bir sonraki adıma dönüştürülmüş veriyi verir.
- **`ColumnTransformer`**, farklı sütun gruplarına farklı zincirler uygular ve sonuçları yan yana birleştirir.

Kurmak istediğimiz yapı Şekil 15.1'de görülüyor.



Şekil 15.1 Titanic ön işleme hattı. Özellik üretme adımından sonra ColumnTransformer dört dala ayrılır; her dal kendi sütunlarına kendi zincirini uygular ve sonuçlar 14 sütunluk tek bir tabloda birleşir.

Tablo 15.2 Hattın dalları ve işlem sırası.

Dal	Sütunlar	İşlem sırası
sayisal	`Age`, `AileBuyuklugu`	Eğitim medyanı → StandardScaler
ucret	`Fare`	Eğitim medyanı → log1p → StandardScaler
kategori	`Sex`, `Embarked`, `Pclass`	Eğitim modu → One-hot
gosterge	`TekYolcu`, `YasEksik`, `KabinBilgisiVar`	Değiştirmeden geçir

15.2.1 Sayısal dallar

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import FunctionTransformer

sayisal_hat = Pipeline([
    ("doldur", SimpleImputer(strategy="median")),
    ("olcekle", StandardScaler()),
])
ucret_hat = Pipeline([
    ("doldur", SimpleImputer(strategy="median")),
    ("log", FunctionTransformer(np.log1p, feature_names_out="one-to-one")),
    ("olcekle", StandardScaler()),
])
```

Bir `Pipeline`, `(ad, işlem)` çiftlerinden oluşan bir listeyle tanımlanır. Adlar bizim seçtiğimiz etiketlerdir; sonradan bir adıma ulaşmak için kullanılır. `FunctionTransformer`, `np.log1p` gibi sıradan bir işlevi hattın bir adımına dönüştürür. `feature_names_out="one-to-one"` ise çıktı sütununun adının girdiyle aynı kalmasını sağlar.

Ücret için ayrı bir dal kurmamızın nedeni, log dönüşümünün yalnızca `Fare`'e uygulanmasını garanti etmektir. Yaşa ve aile büyüklüğüne log uygulanmaz.

15.2.2 Kategorik dal ve birleştirme

```
from sklearn.compose import ColumnTransformer

kategorik_hat = Pipeline([
    ("doldur", SimpleImputer(strategy="most_frequent", missing_values=pd.NA)),
    ("kodla", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
])
gosterge_sutunlar = ["TekYolcu", "YasEksik", "KabinBilgisiVar"]

hazirlama = ColumnTransformer([
    ("sayisal", sayisal_hat, ["Age", "AileBuyuklugu"]),
    ("ucret", ucret_hat, ["Fare"]),
    ("kategori", kategorik_hat, ["Sex", "Embarked", "Pclass"]),
    ("gosterge", "passthrough", gosterge_sutunlar),
], remainder="drop")
```

`ColumnTransformer` üçlülerden oluşan bir liste alır: dalın adı, uygulanacak işlem ve sütun listesi. `"passthrough"` göstergeleri değiştirmeden geçirir. `remainder="drop"` listede adı geçmeyen bütün ham alanları (`Name`, `Ticket`, `Cabin`, `SibSp`, `Parch`) çıktıdan çıkarır.

`missing_values=pd.NA` ayrıntısına dikkat edin. `ozellik_uret` işlevi `Sex` ve `Embarked` sütunlarını eksik değerleri destekleyen metin tipine çeviriyor; bu tipte eksik değer `pd.NA` olarak temsil edilir. Doldurucuya hangi değeri "eksik" sayacağını söylememiz gerekiyor. `Pclass` bu dosyada hiç eksik değil, ama yeni bir kayıtta eksik gelirse ne yapılacağı da tanımlanmış oldu.

15.2.3 Ham veriden hazır girdiye tek hat

Son olarak, özellik üretme işlevini ve `ColumnTransformer`'ı tek bir `Pipeline`'da birleştiriyoruz:

```
on_isleme = Pipeline([
    ("ozellik", FunctionTransformer(ozellik_uret, validate=False)),
    ("hazirla", hazirlama),
])
on_isleme.fit(X_egitim)                          # her şeyi yalnızca ham eğitimden öğrenir
X_egitim_dizi = on_isleme.transform(X_egitim)
X_test_dizi = on_isleme.transform(X_test)

son_sutunlar = hazirlama.get_feature_names_out()
X_egitim_hazir = pd.DataFrame(X_egitim_dizi, columns=son_sutunlar, index=X_egitim.index)
X_test_hazir = pd.DataFrame(X_test_dizi, columns=son_sutunlar, index=X_test.index)
print(list(son_sutunlar))
```

çıktı

```
['sayisal__Age', 'sayisal__AileBuyuklugu', 'ucret__Fare', 'kategori__Sex_female',
'kategori__Sex_male', 'kategori__Embarked_C', 'kategori__Embarked_Q',
'kategori__Embarked_S', 'kategori__Pclass_1', 'kategori__Pclass_2', 'kategori__Pclass_3',
'gosterge__TekYolcu', 'gosterge__YasEksik', 'gosterge__KabinBilgisiVar']
```

`validate=False`, özellik işlevine tabloyu diziye çevirmeden, DataFrame olarak vermesini söyler; işlevimiz sütun adlarıyla çalıştığı için bu gerekli.

Hattın tamamına yalnızca bir kez `fit` çağırdık. Bu çağrı, zincirdeki her adımın kendi parametrelerini sırayla öğrenmesini sağladı: önce özellikler üretildi, sonra her dalın doldurucusu, log adımı, ölçekleyicisi ve kodlayıcısı ham eğitim verisinden öğrendi. Sütun adlarındaki önekler (`sayisal__`, `ucret__`, `kategori__`, `gosterge__`) her sütunun hangi daldan geldiğini gösteriyor.

15.2.4 Hat, adım adım yaptığımızla aynı mı?

Önceki bölümlerde aynı işlemleri adım adım yapmış ve `X_egitim_adim_adim` adlı tabloyu üretmiştik. Hattın aynı sonucu verdiğini doğrulayalım:

```
print(np.allclose(X_egitim_hazir, X_egitim_adim_adim))
print(np.allclose(X_test_hazir, X_test_adim_adim))
```

çıktı

```
True
True
```

`np.allclose`, iki tablodaki sayıların, bilgisayarın küçük yuvarlama farkları dışında, birbirine eşit olup olmadığını kontrol eder. İki sonuç da `True`: Hat, derste adım adım uyguladığımız işlemlerin aynısını yapıyor.

> **Dikkat**
>
> Hatta **ham** veriyi veriyoruz. Önceden doldurulmuş ya da loglanmış bir tabloyu hatta verirsek doldurma iki kez yapılmaz ama log iki kez uygulanır: $\ln(1 + \ln(1 + x))$. Hiçbir hata mesajı almayız; yalnızca yanlış bir veriyle çalışmış oluruz.

15.3 Son kontroller

Ön işlemenin sonunda yalnızca “eksik değer kaldı mı?” diye bakmak yetmez. Yanlış kodlama, sonsuz değerler, hedef sızıntısı ya da kaymış satırlar olabilir.

```
print(X_egitim_hazir.shape, X_test_hazir.shape)
print(X_egitim_hazir.isna().sum().sum(), X_test_hazir.isna().sum().sum())
print(np.isfinite(X_egitim_hazir.to_numpy()).all())
print(X_egitim_hazir.index.equals(y_egitim.index),
X_test_hazir.index.equals(y_test.index))
print(list(X_egitim_hazir.columns) == list(X_test_hazir.columns))
```

çıktı

```
(712, 14) (179, 14)
0 0
True
True True
True
```

Beklenen sonuçlar geldi: 712 eğitim ve 179 test yolcusu, aynı 14 sütun; hiç eksik hücre yok; bütün değerler sonlu; girdiler ve hedefler aynı yolcularla hizalı; eğitim ve testin sütunları aynı sırada.

`index.equals` kontrolü özellikle önemlidir. Satır sırası bir yerde karışırsa, bir yolcunun girdileri başka bir yolcunun hedefiyle eşleşir. Model hiçbir hata vermeden anlamsız bir ilişki öğrenir.

Bu kontrolleri bir listeye dönüştürebiliriz:

- [x] Hedef girdi sütunlarından ayrıldı; X ve y satırları aynı yolculara karşılık geliyor.
- [x] Satır sayısındaki değişikliklerin gerekçesi ve alt gruplara etkisi kaydedildi.
- [x] Gerekli eksikler ele alındı; sayısal çıktılarda NaN ya da sonsuz değer kalmadı.
- [x] Eğitim ve test aynı sütun adlarına, sırasına ve dönüşüm anlamına sahip.
- [x] Doldurma, kodlama, ölçekleme ve seçim yalnızca eğitimden öğrenildi.
- [x] Ham veri korundu; özellik formülleri ve kategori kuralları belgelendi.
- [x] Tohum, bölme düzeni, paket sürümleri ve öğrenilmiş hat kaydedilecek.
- [x] Yeni kayıtta eksik bilgi ve görülmemiş kategori davranışı kontrol edildi.

15.4 Yeni bir yolcu

Hazır tablonun 891 satır içermesi tek başına bir başarı göstergesi değildir. Bütün satırları korumuş ama yanlış kurallarla doldurmuş ya da yanlış hizalamış olabiliriz.

Modelin asıl işi, yeni yolcular için tahmin yapmak olacak. Yeni bir yolcunun hazırlanmasını, iki zor durumu birden içeren bir senaryoyla sınayalım: Yaşı eksik, biniş limanı ise eğitimde hiç görülmemiş bir “X”.

```
yeni_yolcu = X_test.head(1).copy()
yeni_yolcu["Age"] = np.nan              # eksik yaş
yeni_yolcu["Embarked"] = "X"            # eğitimde olmayan liman
yeni_dizi = on_isleme.transform(yeni_yolcu)        # fit yok, yalnızca transform
yeni_hazir = pd.DataFrame(yeni_dizi, columns=son_sutunlar)
yeni_hazir.T.rename(columns={0: "değer"}).round(3)
```

çıktı

```
                          değer
sayisal__Age             -0.081
sayisal__AileBuyuklugu    0.703
ucret__Fare               0.280
kategori__Sex_female      0.000
kategori__Sex_male        1.000
kategori__Embarked_C      0.000
kategori__Embarked_Q      0.000
kategori__Embarked_S      0.000
kategori__Pclass_1        0.000
kategori__Pclass_2        0.000
kategori__Pclass_3        1.000
gosterge__TekYolcu        0.000
gosterge__YasEksik        1.000
gosterge__KabinBilgisiVar 0.000
```

`.T` tabloyu çevirir (satırları sütun, sütunları satır yapar); tek satırlık geniş bir tabloyu okumayı kolaylaştırır.

Hat iki durumu da beklediğimiz gibi ele aldı. Yaş eğitim medyanıyla (28,5) dolduruldu ve standardize edildi; `gosterge__YasEksik` 1 oldu, böylece yaşın başlangıçta bilinmediği bilgisi korundu. Bilinmeyen "X" limanı, üç liman sütununun hepsinde 0 üretti. Bölüm 12'de belirttiğimiz gibi bu, eksik bir limandan farklıdır: Eksik liman modla (S) doldurulup `Embarked_S` 1 olurdu.

15.5 Hattı kaydetmek ve yeniden kullanmak

Bir sonraki derste modelleme yaparken hazırlanmış verilere ve öğrenilmiş hatta ihtiyacımız olacak. Hazırlanmış tabloları CSV olarak, öğrenilmiş hattı ise `joblib` ile kaydederiz:

```
from pathlib import Path
import joblib

cikti_klasoru = Path("ciktilar")
cikti_klasoru.mkdir(exist_ok=True)
X_egitim_hazir.to_csv(cikti_klasoru / "X_egitim.csv", index_label="Satir")
X_test_hazir.to_csv(cikti_klasoru / "X_test.csv", index_label="Satir")
y_egitim.to_csv(cikti_klasoru / "y_egitim.csv", index_label="Satir")
y_test.to_csv(cikti_klasoru / "y_test.csv", index_label="Satir")
X_egitim.to_csv(cikti_klasoru / "X_egitim_ham.csv", index_label="Satir")
X_test.to_csv(cikti_klasoru / "X_test_ham.csv", index_label="Satir")
joblib.dump(on_isleme, cikti_klasoru / "titanic_on_isleme.joblib")
```

`mkdir(exist_ok=True)` klasörü oluşturur; klasör zaten varsa hata vermez. `/` işareti `Path` nesnelerinde klasör ile dosya adını birleştirir. `index_label="Satir"` satır etiketlerini bir sütun olarak kaydeder; böylece dosyalar yeniden okunduğunda girdiler ile hedefler doğru eşleştirilebilir. Bu sütun bir model özelliği değildir; yalnızca yolcuları izlemek içindir.

Ham eğitim ve test dosyalarını da ayrıca kaydettik. Bunun nedenini bir sonraki başlıkta göreceğiz.

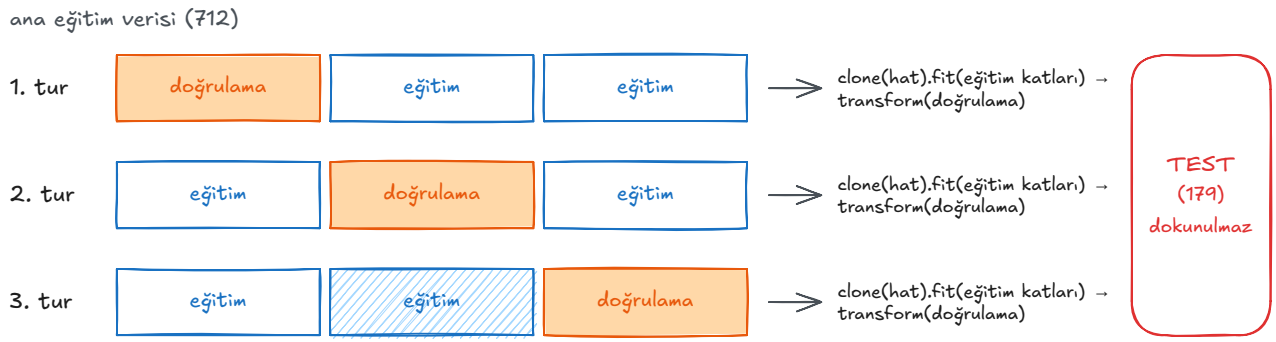
Kaydedilen hattı yeniden yükleyip aynı sonucu verdiğini doğrulayabiliriz:

```
yuklenen = joblib.load(cikti_klasoru / "titanic_on_isleme.joblib")
print(np.allclose(yuklenen.transform(yeni_yolcu), yeni_dizi))
```

> **Dikkat**
>
> `joblib` dosyaları içlerinde çalıştırılabilir kod taşıyabilir. Yalnızca kendi kaydettiğiniz ya da güvendiğiniz bir kaynaktan gelen dosyaları yükleyin. Bir hattı başka bir bilgisayarda kullanırken paket sürümlerinin aynı olmasına da dikkat edin.

15.6 Çapraz doğrulamada hat

Bölüm 9'da bir uyarı yapmıştık: Çapraz doğrulamada ön işlemeyi bir kez bütün eğitim verisine uygulayıp sonra katlara ayırmak sızıntı yaratır. Her katta doldurucu ve ölçekleyici, yalnızca o katın eğitim bölümünden öğrenilmelidir. Hat bunu kolaylaştırır (Şekil 15.2).



Şekil 15.2 Üç katlı çapraz doğrulamada hattın kullanımı. Her turda hattın temiz bir kopyası yalnızca o turun eğitim katlarında fit edilir; doğrulama katı yalnızca dönüştürülür.

İlk katı elle hazırlayarak mantığı görelim:

```
from sklearn.model_selection import StratifiedKFold
from sklearn.base import clone

katlar = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
kat_egitim_yeri, kat_dogrulama_yeri = next(katlar.split(X_egitim, y_egitim))
kat_egitim = X_egitim.iloc[kat_egitim_yeri]
kat_dogrulama = X_egitim.iloc[kat_dogrulama_yeri]

kat_hat = clone(on_isleme)          # öğrenilmiş değer taşımayan temiz kopya
kat_hat.fit(kat_egitim)             # yalnızca kat eğitiminde öğrenir
kat_dogrulama_hazir = kat_hat.transform(kat_dogrulama)
print(len(kat_egitim), len(kat_dogrulama), kat_dogrulama_hazir.shape)
print(kat_hat.named_steps["hazirla"].named_transformers_["sayisal"]
      .named_steps["doldur"].statistics_)
```

çıktı

```
474 238 (238, 14)
[29.  1.]
```

`StratifiedKFold` eğitim verisini sınıf oranlarını koruyarak üç kata böler. `next(...)` ilk turun satır konumlarını alır. `clone` hattın yalnızca **tanımını** kopyalar; ana eğitimde öğrenilmiş değerleri taşımaz. Bu temiz kopya yalnızca 474 kişilik kat eğitiminde `fit` edildi ve 238 kişilik doğrulama katına uygulandı.

Son satıra bakın: Bu katta öğrenilen yaş doldurma medyanı 29,0 (ikinci sayı aile büyüklüğünün medyanı); ana eğitimdeki 28,5'ten farklı. Çünkü doğrulama katındaki yolcuların yaşları hesaba girmedi. Asıl test verisi ise hiç kullanılmadı.

Bir sonraki derste modeli bu hattın sonuna ekleyeceğiz. Böylece çapraz doğrulama her katta hem hazırlığı hem modeli yeniden öğrenecek ve bu işi bizim yerimize otomatik olarak yapacak. Hazırlanmış CSV tablolarının yanında ham eğitim dosyasını da kaydetmemizin nedeni buydu.

15.7 Karar günlüğü

Kısım III boyunca verdiğimiz kararları bir günlükte toplamak, çalışmamızı hem bize hem başkalarına açıklanabilir kılar:

Tablo 15.3 Titanic ön işleme karar günlüğü.

Hat sızıntıyı önlemeye yardım eder, ama yanlış girdi seçimini kendiliğinden düzeltmez. `Survived` sütununun X'e eklenmesi ya da tahmin anında bilinmeyen bir sütunun kullanılması, hattın içinde de hatadır.

Alan ya da işlem	Seçilen işlem	Gerekçe ve kontrol
`Age`	Eğitim medyanı + `YasEksik` göstergesi	Satırları koru; eksiklik sınıfa göre değişiyor; dağılımı ve eğitim içi sonuçları incele
`Embarked`	Eğitim modu → one-hot	Yalnızca iki eksik; yeni etiketlerin nasıl kodlandığı kontrol edildi
`Cabin`	Ham kodu çıkar, `KabinBilgisiVar` tut	Çok eksik; gösterge kaydın doluluğunu yansıtır
`Fare` uçları	Koru, log1p + StandardScaler	Yüksek ücretin hata olduğunu gösteren bilgi yok
`Pclass`	One-hot	Sınıflar arası farkın eşit olduğu varsayılmıyor
`SibSp`, `Parch`	`AileBuyuklugu` ve `TekYolcu`	Aile temsilini sade tutmak
`PassengerId`, `Name`, `Ticket`	Model girdisinden çıkar	Kimlik ve yüksek çeşitlilikte metin; izleme için saklanır
PCA, SMOTE	Kullanılmıyor	Gerekçe yok; gerekirse eğitim içi doğrulamayla değerlendirilir

Uygulama 15.1 · Tartışma soruları

1. Yaşı eksik yolcuları silmek üçüncü sınıfı nasıl etkilerdi?
2. Kabin eksikliği göstergesi modelde yararlı çıksa bile neden dikkatle yorumlanmalı?
3. Log dönüşümünü ve ölçeklemeyi hangi durumlarda birlikte kullanırız?
4. Yeni yolcunun limanı eğitimde görülmediyse hattımız ne yapıyor? Bu davranışı değiştirmek isteseydik ne yapardık?
5. İşlenmiş tablonun 891 satır olması neden tek başına başarı göstergesi değildir?
6. Hazır tabloyu kaydetmek ile ham eğitim verisini hatla birlikte çapraz doğrulamaya vermek arasındaki fark nedir?

Bölüm özeti

- Elle yürütülen ön işlemede sütun sırası, doldurma değerleri ve kategori listeleri kolayca karışır. Hat, bütün adımları tek ve tekrarlanabilir bir yapıda toplar.
- `Pipeline` işlemleri sıralı bir zincirde, `ColumnTransformer` farklı sütun gruplarına farklı zincirleri uygular. Titanic hattı dört daldan oluşur ve 14 sütun üretir.
- Hatta ham veri verilir; tek bir `fit` çağrısı bütün adımların yalnızca eğitimden öğrenmesini sağlar. Önceden işlenmiş veriyi hatta vermek işlemleri iki kez uygular.
- Hattın sonucu adım adım hazırlanan tabloyla `np.allclose` ile karşılaştırılarak doğrulanır.
- Son kontroller boyut, eksik ve sonsuz değer, girdi ile hedefin hizası ve sütun tutarlılığını kapsar. 891 satır olması tek başına başarı değildir.
- Yeni bir kayıt yalnızca `transform` ile hazırlanır. Eksik yaş eğitim medyanıyla doldurulur ve gösterge 1 olur; görülmemiş liman bütün liman sütunlarında 0 üretir.
- Öğrenilmiş hat `joblib` ile kaydedilir; yalnızca güvenilir kaynaktan yüklenir.
- Çapraz doğrulamada hattın temiz bir kopyası (`clone`) her katın eğitim bölümünde `fit` edilir. Bu yüzden ham eğitim verisi de saklanır.
- Kararlar bir günlükte gerekçeleriyle belgelenir.

Kaynaklar

Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3. baskı). O'Reilly. Bölüm 2: "Transformation Pipelines".

scikit-learn geliştiricileri. *Pipelines and composite estimators*. https://scikit-learn.org/stable/modules/compose.html

scikit-learn geliştiricileri. *Cross-validation: evaluating estimator performance*. https://scikit-learn.org/stable/modules/cross_validation.html

KISIM IV

Doğrusal regresyon

16 Basit doğrusal regresyon

17 Regresyon modelini değerlendirmek

18 Çoklu doğrusal regresyon

16

BÖLÜM

Basit doğrusal regresyon

Bu bölümde

- Regresyonun koşullu ortalamayı modellediğini açıklamak; tahmin ve çıkarım amaçlarını ayırmak,
- “doğrusal” sözcüğünün katsayılarla ilgili olduğunu göstermek,
- popülasyon modeli ile tahmin edilen doğruyu, hata terimi ile artığı ayırmak,
- eğimi ve kesişimi birimleriyle, “ortalamada”, “karşılaştırma” ve “veri aralığında” kısıtlarıyla yorumlamak,
- en küçük kareler ölçütünü gerekçelendirmek ve OLS formüllerini türetmek,
- OLS doğrusunun cebirsel özelliklerini ve eğim ile korelasyon arasındaki ilişkiyi açıklamak,
- basit regresyonu Python’da elle ve `scikit-learn` ile hesaplamak.

Kaynak: YBS 319 ders sayfası

Kısım III'te veriyi bir modele hazırladık. Şimdi ilk modelimizi kuruyoruz. Bölüm 6'da denetimli öğrenmeyi hedefin türüne göre ikiye ayırmıştık: Hedef kategorikse sınıflandırma, sayısalsa regresyon. Bu kısımda regresyonun en temel ve en yorumlanabilir biçimini, **doğrusal regresyonu** ele alıyoruz.

Derste de söylediğimiz gibi regresyonu vizede ayrıntılı sormayacağız; ama regresyonun ne olduğunu ve bir regresyon doğrusunun neyi özetlediğini bilmeniz gerekiyor. Bu üç bölümü, dönemin ikinci yarısına sağlam bir temel olarak da okuyabilirsiniz.

16.1 Regresyon nedir?

Denetimli öğrenmede her gözlem için bir dizi girdi değişkeni (X) ve bir hedef (Y) gözleriz. Hedef sayısal ve sürekli olduğunda (fiyat, satış miktarı, teslim süresi, enerji tüketimi) problem bir **regresyon** problemidir. Regresyon iki farklı amaca hizmet edebilir:

Tahmin

> "Bu hafta ilana çıkan 125 m², 8 yıllık dairenin fiyatı ne olur?" Amaç, yeni gözlemler için olabildiğince isabetli bir sayı üretmektir. Modelin iç yapısı ikinci plandadır; başarı, modelin görmediği verideki hatayla ölçülür (Bölüm 7).

Çıkarım

> "Aynı yaştaki daireler arasında alan 1 m² arttığında fiyat ortalama ne kadar değişir? Bu fark rastlantıyla açıklanabilir mi?" Amaç, ilişkinin yönünü, büyüklüğünü ve belirsizliğini anlamaktır.

Pek çok makine öğrenmesi yöntemi yalnızca birinci amaca hizmet eder. Doğrusal regresyonun ayırt edici yanı, iki amaca birlikte cevap verebilmesidir: Hem tahmin üretir hem de her değişkenin katkısını açık bir sayıyla yorumlamamıza izin verir.

16.1.1 Regresyon koşullu ortalamayı modeller

Aynı alana sahip iki dairenin fiyatı aynı değildir. Konum, kat, cephe, tadilat durumu ve pazarlık payı fiyatı değiştirir. Bu nedenle regresyon tek tek fiyatları değil, belirli bir X değerine sahip gözlemlerin **ortalama** fiyatını modeller. Buna **koşullu beklenen değer** denir:

$$E[\,Y \mid X = x\,]$$

Bu ifade "X değeri x olan gözlemlerde Y'nin ortalaması" diye okunur. 120 m² dairelerin ortalama fiyatı tek bir sayıdır; tek tek 120 m² dairelerin fiyatları bu ortalamanın çevresine dağılır.

Tanım 16.1

Regresyon doğrusu, her x değerindeki koşullu ortalamaların üzerinden geçen doğrudur. Bundan sonraki her kavram bu fikre dayanır: Hata terimi bu ortalamadan sapmadır; R^2 bu sapmaların ne kadarının açıklandığını ölçer; güven aralığı ortalamanın, kestirim aralığı ise tek bir gözlemin belirsizliğini anlatır.

16.1.2 “Doğrusal” neyi kısıtlar?

Doğrusal regresyonda doğrusallık **katsayılarda** aranır, değişkenlerde değil. Bilinmeyen katsayıların her biri bir sayıyla çarpılıp toplanarak yazılabiliyorsa model doğrusaldır:

Adı nereden geliyor?
Francis Galton 1886’da ebeveyn ve çocuk boylarını karşılaştırdı. Çok uzun ebeveynlerin çocukları da uzundu, ama ortalamada ebeveynlerinden daha az uzundu. Galton buna “ortalamaya dönüş” (*regression*) dedi. En küçük kareler yöntemi ise daha eskidir: Legendre 1805’te, Gauss 1809’da yayımladı.

Tablo 16.1 Hangi modeller katsayılarda doğrusaldır?

Model	Katsayılarda doğrusal mı?	Açıklama
$Y = \beta_0 + \beta_1 X + \varepsilon$	Evet	Düz bir doğru
$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \varepsilon$	Evet	Bir parabol çizer, ama X^2 yeni bir değişken gibi davranır
$Y = \beta_0 + \beta_1 \log X + \varepsilon$	Evet	Değişken dönüştürülmüştür; katsayılar doğrusal girer
$Y = \beta_0\, e^{\beta_1 X} + \varepsilon$	Hayır	β_1 üstte yer alır; başka yöntemler gerekir

“Doğrusal model yalnızca düz çizgi çizebilir” düşüncesi bu yüzden eksiktir. Bölüm 18’de aynı yöntemin eğri yüzeylere nasıl genişletildiğini göreceğiz.

16.1.3 Terimler

Aynı kavramlar istatistik ve makine öğrenmesi kaynaklarında farklı adlarla geçer:

Tablo 16.2 Regresyon terimleri.

Sembol	Türkçe	İngilizce
Y	Bağımlı değişken, yanıt, hedef	response, dependent variable, target
$X_1, \ldots, X_p$	Bağımsız değişken, açıklayıcı değişken, öznitelik	predictor, regressor, feature
β_0	Kesişim, sabit terim	intercept, bias
$\beta_1, \ldots, \beta_p$	Eğim, regresyon katsayıları	slope, coefficients, weights
ε	Hata terimi	error term
$e_i = y_i - \hat{y}_i$	Artık	residual
$\hat{\beta}$	Tahmin edilen katsayı (“beta şapka”)	estimated coefficient
OLS	Sıradan en küçük kareler	ordinary least squares

16.1.4 Örnek veri

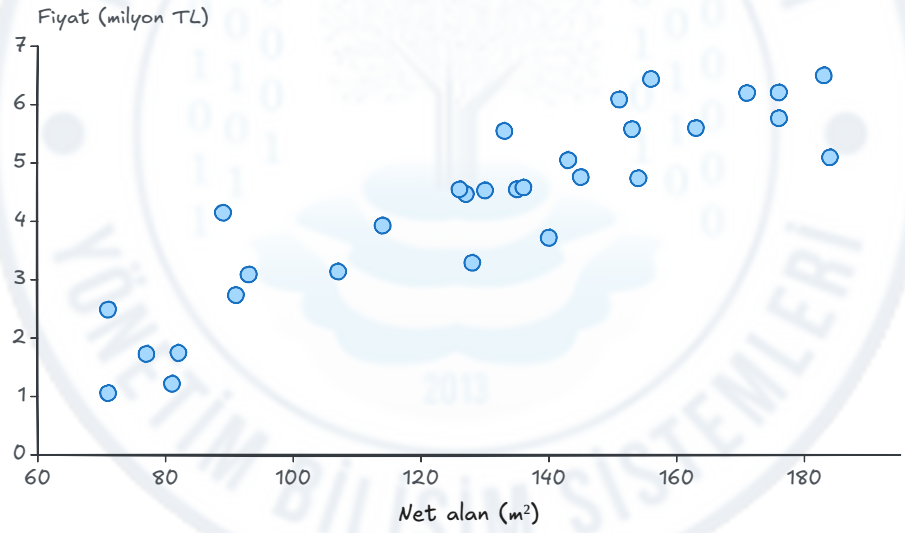
Bu kısım boyunca, derste kullandığımız küçük bir veri kümesiyle çalışacağız: Denizli'de satışa çıkmış 30 dairenin net alanı (m^2), bina yaşı (yıl) ve satış fiyatı (milyon TL). Veri ders için üretilmiş kurgusaldır; ama değerler gerçekçi aralıklarda tutulmuştur. Alan 71 ile 184 m^2, bina yaşı 0 ile 37 yıl, fiyat 1,06 ile 6,50 milyon TL arasındadır. Bu bölümde yalnızca alanı, Bölüm 18'de alan ile yaşı birlikte kullanacağız.

```python
import numpy as np
import pandas as pd

daire = pd.read_csv("daireler.csv")
print(daire.shape)
daire.describe().loc[["mean", "std", "min", "max"]].round(3)
```

çıktı

```
(30, 3)
         alan     yas  fiyat
mean  129.533  19.067  4.286
std    34.922   8.874  1.575
min    71.000   0.000  1.060
max   184.000  37.000  6.500
```



Şekil 16.1 30 dairenin alan ve fiyatı. Büyük daireler genel olarak daha pahalı; ama aynı alandaki daireler arasında da belirgin fiyat farkları var.

Şekil 16.1'deki saçılım grafiği bize iki şey söylüyor: Alan ile fiyat arasında güçlü ve yaklaşık doğrusal bir ilişki var; ama noktalar bir doğrunun üzerinde değil, onun çevresinde dağılıyor.

16.2 Basit doğrusal regresyon modeli

Basit doğrusal regresyon, hedefi tek bir açıklayıcı değişkenle modeller. Popülasyon düzeyinde model şöyle yazılır:

$$Y = \beta_0 + \beta_1 X + \varepsilon$$

Örneğimizde Y fiyat (milyon TL), X net alan (m^2).

β_0: kesişim
: X = 0 olduğunda Y'nin beklenen değeri. Doğrunun dikey ekseni kestiği yükseklik.

β_1: eğim
: X bir birim arttığında Y'nin beklenen değerindeki değişim. Birimi "Y'nin birimi / X'in birimi"dir.

ε: hata terimi
: Y'nin X ile açıklanamayan bölümü: modelde olmayan değişkenler (konum, kat), ölçüm hataları ve öngörülemeyen rastlantısallık.

Hata teriminin ortalamasının, X hangi değeri alırsa alsın sıfır olduğu varsayılır: $E[\varepsilon \mid X] = 0$. Bu varsayım altında modelin iki tarafının koşullu beklenen değerini alırsak koşullu ortalamaya ulaşırız: $E[Y \mid X] = \beta_0 + \beta_1 X$. Yani doğru, ortalamaların doğrusudur; tek tek gözlemler bu doğrunun çevresine ε kadar saçılır.

16.2.1 Popülasyon doğrusu ve tahmin edilen doğru

β_0 ve β_1 bilinmeyen sabitlerdir. Satılık dairelerin tamamını, yani **popülasyonu** tanımlarlar. Elimizde ise yalnızca n gözlemlik bir **örneklem** vardır. Örneklemden hesapladığımız değerlere **tahmin** denir ve şapka işaretiyle gösterilir:

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$$

Bu denklemde ε yoktur, çünkü $\hat{y}$ tek bir dairenin fiyatı değil, o alandaki dairelerin ortalama fiyatının tahminidir. Farklı 30 daireyle çalışsaydık farklı $\hat{\beta}$ değerleri bulurduk. Bu nedenle $\hat{\beta}$'lar sabit sayılar değil, örneklemden örnekleme değişen rastlantı değişkenleridir. Bu değişkenliği Bölüm 17'de ölçeceğiz.

> **Tanım 16.2 · Hata terimi ve artık**
>
> **Hata**, $\varepsilon_i = y_i - (\beta_0 + \beta_1 x_i)$, gözlemin gerçek popülasyon doğrusuna dikey uzaklığıdır. Gerçek β'ları bilmediğimiz için hiçbir zaman hesaplanamaz. **Artık**, $e_i = y_i - \hat{y}_i$, gözlemin tahmin edilen doğruya dikey uzaklığıdır. Veriden hesaplanır ve hataların gözlenebilir vekilidir.

İkisi birbirine yakındır ama aynı şey değildir. Modelin varsayımlarını kontrol ederken gözlenemeyen hatalar yerine artıkları inceleyeceğiz.

16.3 Katsayıları yorumlamak

Örnek verimizde, nasıl hesaplandığını birazdan göreceğimiz yöntem şu doğruyu verir:

$$\hat{y} = -0{,}984 + 0{,}0407 \times \text{alan} \qquad (R^2 = 0{,}814,\ n = 30)$$

16.3.1 Eğim: 0,0407

Eğimin birimi "milyon TL / m²"dir. 0,0407 milyon TL, 40,7 bin TL eder. Doğru yorum şöyledir: **Net alanı 1 m² daha büyük olan daireler, ortalamada 40,7 bin TL daha pahalıdır.** 10 m²'lik fark için beklenen fiyat farkı 407 bin TL'dir. Bu cümledeki her ifade bir kısıt taşır:

"Ortalamada"
: Eğim tek bir dairenin değil, koşullu ortalamaların değişimidir. 1 m² daha büyük olup daha ucuz olan daireler de veride vardır.

"Daha büyük olan daireler"
: Bu bir **karşılaştırmadır**, bir müdahale değil. Veri gözlemseldir. Örneğimizde büyük daireler aynı zamanda daha yeni binalarda yer alıyor: Alan ile yaş arasındaki korelasyon −0,60. Bu yüzden 40,7 bin TL'nin bir bölümü aslında bina yaşının farkını taşır. "Daireme 1 m² eklersem değeri 40,7 bin TL artar" çıkarımı bu modelden yapılamaz. Bölüm 18'de iki etkiyi ayıracağız.

"Veri aralığında"
: Modelde 71 ile 184 m² arasındaki daireler var. 350 m²'lik bir villa için tahmin, doğrusal ilişkinin veri dışında da sürdüğünü varsaymak demektir. Buna **dışdeğerleme** (*extrapolation*) denir ve güvenilirliği veriden denetlenemez.

> **Dikkat**
>
> Eğimin büyüklüğü birimlere bağlıdır. Fiyatı bin TL cinsinden ölçseydik eğim 40,7 olurdu; ilişki hiç değişmezdi. Bu yüzden "katsayı büyük, demek ki değişken önemli" yargısı tek başına anlamsızdır. Önem, katsayının belirsizliğine ve değişkenin gerçekçi değişim aralığına göre değerlendirilir.

16.3.2 Kesişim: −0,984

Kesişim, x = 0 için tahmin edilen değerdir. Burada "0 m²'lik bir dairenin beklenen fiyatı eksi 984 bin TL" demek olur. Böyle bir daire yoktur; negatif fiyat da imkânsızdır. Bu değer yanlış değildir; sadece yorumlanabilir bir nicelik değildir. x = 0 veri aralığının çok dışında kalır (en küçük daire 71 m²) ve kesişim bu noktaya yapılmış bir dışdeğerlemedir. Kesişimin buradaki görevi, doğrunun veri bulutunun içinden geçmesi için dikey konumunu ayarlamaktır.

Kesişim, x = 0 hem anlamlı hem de veri aralığına yakın olduğunda yorumlanır. Reklam harcaması ile satış arasındaki bir modelde kesişim "reklam yapılmayan haftalarda beklenen satış", deneyim yılı ile maaş arasındaki bir modelde "yeni mezunun beklenen başlangıç maaşı" anlamına gelir.

16.3.3 Kesişim anlamsızsa neden atmıyoruz?

Kesişimi atmak, $\beta_0 = 0$ kısıtını koymak, yani doğruyu orijinden geçmeye zorlamaktır. Gerçek ilişki orijinden geçmiyorsa doğru, (0, 0) noktasına bağlı kalarak veriye uymaya çalışır ve eğim yanlı tahmin edilir. Örnek verimizde kesişimsiz model eğimi 0,0336 (33,6 bin TL/m²) olarak verir; bu, kesişimli modeldeki 40,7 bin TL'den belirgin biçimde düşüktür. Kesişim yalnızca, kuramsal olarak x = 0 iken y = 0 olmak zorunda olduğu ve bunun veriyle de uyumlu olduğu özel durumlarda atılır.

16.3.4 Merkezleme

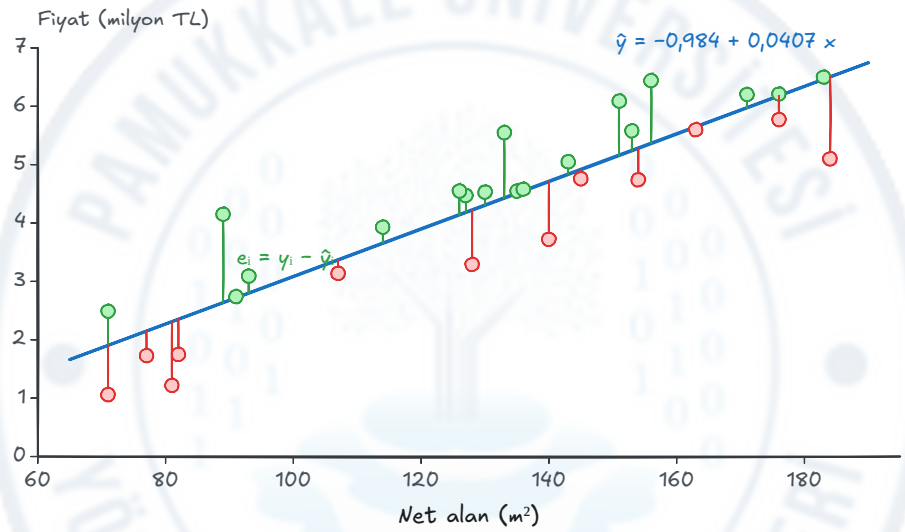
Kesişimi anlamlı kılmanın bir yolu, açıklayıcı değişkeni ortalamasından sapma olarak yazmaktır. Ortalama alan $\bar{x} = 129{,}5\ \text{m}^2$ olduğuna göre:

$$\hat{y} = 4{,}286 + 0{,}0407 \times (\text{alan} - 129{,}5)$$

Eğim değişmez. Kesişim artık "ortalama büyüklükteki bir dairenin beklenen fiyatı" anlamına gelir ve fiyatların ortalamasına (4,286) eşittir. Bu, Bölüm 13'teki StandardScaler'ın ilk adımıyla aynı işlemdir.

16.4 "En iyi doğru" ne demek?

Bir veri bulutundan sonsuz sayıda doğru geçirilebilir ve her doğru her gözlem için bir artık üretir (Şekil 16.2). "En iyi" doğruyu seçebilmek için 30 artığı tek bir ölçüte indirmemiz gerekir.



Şekil 16.2 Artıklar. Her gözlemin doğruya dikey uzaklığı onun artığıdır. Doğrunun üstündeki gözlemlerin artığı pozitif, altındakilerin negatiftir.

Akla gelen üç aday var:

Tablo 16.3 Bir doğrunun uyumunu ölçmek için üç aday.

Ölçüt	Formül	Değerlendirme
Artıkların toplamı	$\sum e_i$	Kullanılamaz: Pozitif ve negatif artıklar birbirini götürür; $(\bar{x}, \bar{y})$ 'den geçen her doğru için toplam sıfırdır
Mutlak artıkların toplamı	$\sum \lvert e_i \rvert$	Aykırı değerlere dayanıklıdır; ama kapalı bir çözümü yoktur ve çözüm tek olmayabilir
Kare artıkların toplamı	$\sum e_i^2$	Her yerde türevlenebilir; tek ve kapalı bir çözümü vardır; koşullu ortalamayı modeller

Üçüncü ölçütü en küçük yapan yönteme **en küçük kareler** denir. Ölçütün kendisine **artık kareler toplamı** (SSE) denir:

$$\mathrm{SSE}(\hat{\beta}_0, \hat{\beta}_1) = \sum_{i=1}^{n} \left(y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i \right)^2$$

16.4.1 Neden kareler?

Matematiksel kolaylık işin bir yanıdır. Kare artıkların seçilmesinin iki kuramsal gerekçesi daha vardır. **Gauss-Markov teoremine** göre, Bölüm 17'de sayacağımız varsayımlar altında OLS tahmin edicisi, doğrusal ve yansız tahmin ediciler arasında en küçük varyansa sahiptir. Hatalar normal dağılıyorsa da veriyi en olası kılan katsayılar tam olarak OLS katsayılarıdır.

Bedeli de vardır: 1 birimlik bir artık toplama 1, 3 birimlik bir artık 9, 10 birimlik bir artık 100 katkı yapar. Kare kayıp büyük hataları orantısından fazla cezalandırır. Tek bir uç gözlem doğruyu kendine doğru çekebilir (Bölüm 17).

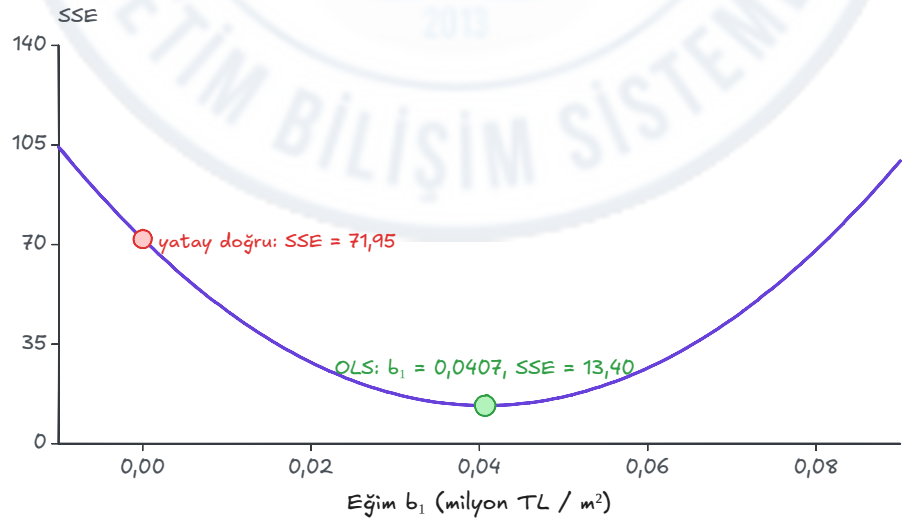
16.4.2 Eğim değiştikçe SSE nasıl değişir?

Doğruyu $(\bar{x}, \bar{y})$ noktası etrafında döndürdüğümüzü düşünelim. Her eğim için farklı bir SSE elde ederiz. SSE yalnızca eğime bağlı bir fonksiyona dönüşür:

Makine öğrenmesindeki adı
Makine öğrenmesi kaynaklarında aynı ölçüt genellikle **ortalama kare hata** olarak yazılır: MSE = SSE / *n*. Sabit bir sayıya bölmek en küçük noktanın yerini değiştirmez. Géron bu fonksiyonu "maliyet fonksiyonu" olarak adlandırır.

$$\mathrm{SSE}(b_1) = S_{yy} - 2b_1 S_{xy} + b_1^2 S_{xx}$$

Burada $S_{xx} = \sum (x_i - \bar{x})^2$, $S_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y})$, $S_{yy} = \sum (y_i - \bar{y})^2$'dir. Bu ifade b_1'in ikinci dereceden bir fonksiyonudur: yukarı açık bir parabol ve tek bir en düşük noktası vardır (Şekil 16.3).



Şekil 16.3 SSE'nin eğime göre değişimi. Parabolün en düşük noktası OLS eğimidir. Eğim sıfırken doğru yataydır: Artıkların toplamı yine sıfırdır ama kareler toplamı çok büyüktür.

Bu tarama yalnızca SSE'nin biçimini göstermek içindir. OLS eğimleri tek tek denemez. Parabolün en düşük noktası, türev alınarak tek bir formülle bulunur.

16.5 OLS çözümünün türetilmesi

Amacımız iki bilinmeyenli bir fonksiyonu en küçük yapmaktır:

$$\min_{b_0,\,b_1} S(b_0, b_1) = \sum \left(y_i - b_0 - b_1 x_i\right)^2$$

S, iki katsayıya göre ikinci dereceden ve dışbükey bir fonksiyondur; grafiği bir kâsedir. Kâsenin en düşük noktasında iki kısmi türevin de sıfır olması gerekir.

Adım 1. Kısmi türevleri sıfıra eşitle.

$$\frac{\partial S}{\partial b_0} = -2\sum \left(y_i - b_0 - b_1 x_i\right) = 0 \qquad \frac{\partial S}{\partial b_1} = -2\sum x_i\left(y_i - b_0 - b_1 x_i\right) = 0$$

Parantez içindeki ifade artıktır. Dolayısıyla bu iki koşul, sırasıyla $\sum e_i = 0$ ve $\sum x_i e_i = 0$ demektir.

Adım 2. Normal denklemler. -2'leri atıp toplamları dağıtınca iki bilinmeyenli iki doğrusal denklem elde edilir. Bunlara **normal denklemler** denir:

$$\sum y_i = n\,b_0 + b_1 \sum x_i \qquad \sum x_i y_i = b_0 \sum x_i + b_1 \sum x_i^2$$

Adım 3. Kesişimi çöz. İlk denklemi n'ye bölersek $\bar{y} = b_0 + b_1 \bar{x}$ elde ederiz:

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

Bu eşitlik aynı zamanda OLS doğrusunun her zaman **ağırlık merkezinden**, yani $(\bar{x}, \bar{y})$ noktasından geçtiğini söyler.

Adım 4. Eğimi çöz. b_0 ifadesini ikinci denkleme yerleştirip düzenlersek:

$$\hat{\beta}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} = \frac{S_{xy}}{S_{xx}} = \frac{\mathrm{Kov}(x, y)}{\mathrm{Var}(x)} = r \cdot \frac{s_y}{s_x}$$

Formülün tek koşulu $S_{xx} > 0$'dır: x değerleri en az iki farklı değer almalıdır. Bütün daireler 120 m² olsaydı, alanın fiyata etkisi hakkında veride hiçbir bilgi olmazdı ve eğim tanımsız kalırdı.

16.5.1 Küçük bir örnekle elle hesap

Formülü önce beş gözlemle elle uygulayalım. x bir öğrencinin haftalık çalışma saati, y 10 üzerinden quiz notu olsun.

Tablo 16.4 Beş gözlemle OLS hesabı.

i	x_i	y_i	$x_i - \bar{x}$	$y_i - \bar{y}$	çarpım	$(x_i - \bar{x})^2$	$\hat{y}_i$	e_i
1	1	2	−2	−2	4	4	2,8	−0,8
2	2	4	−1	0	0	1	3,4	0,6
3	3	5	0	1	0	0	4,0	1,0
4	4	4	1	0	0	1	4,6	−0,6
5	5	5	2	1	2	4	5,2	−0,2
Top.	15	20	0	0	6	10	20	0

$\bar{x} = 3$, $\bar{y} = 4$. Eğim $\hat{\beta}_1 = 6/10 = 0{,}6$, kesişim $\hat{\beta}_0 = 4 - 0{,}6 \times 3 = 2{,}2$. Doğru: $\hat{y} = 2{,}2 + 0{,}6x$. Haftada bir saat fazla çalışan öğrencilerin notu ortalamada 0,6 puan yüksektir. Son sütundaki artıkların toplamının sıfır olduğuna dikkat edin.

Aynı hesabı daire verisinde Python'la yapalım:

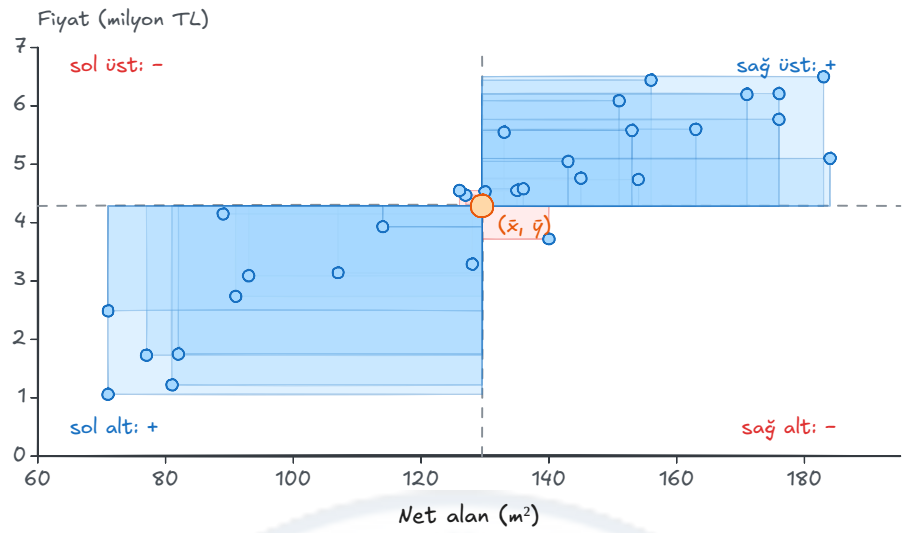
```
x = daire["alan"].to_numpy()
y = daire["fiyat"].to_numpy()
Sxy = np.sum((x - x.mean()) * (y - y.mean()))
Sxx = np.sum((x - x.mean()) ** 2)
b1 = Sxy / Sxx
b0 = y.mean() - b1 * x.mean()
print(round(Sxy, 2), round(Sxx, 2))
print("eğim:", round(b1, 4), "kesişim:", round(b0, 4))
```

çıktı

```
1438.99 35367.47
eğim: 0.0407 kesişim: -0.9843
```

16.5.2 Eğimin payı ne anlatıyor?

Payı oluşturan her terim, bir gözlemin ağırlık merkezine göre yatay ve dikey sapmalarının çarpımıdır. Geometrik olarak bu, $(\bar{x}, \bar{y})$ noktası ile gözlem arasına çizilen dikdörtgenin alanıdır (Şekil 16.4). Gözlem sağ üst ya da sol alt bölgedeyse çarpım pozitif, sol üst ya da sağ alt bölgedeyse negatiftir. Pozitif alanlar baskınsa eğim pozitif çıkar. Payda ise yalnızca x'in kendi yayılımını ölçer ve sonucu "m² başına" birime çevirir.



Şekil 16.4 Eğimin payı. Ağırlık merkezinden çizilen her dikdörtgen bir sapma çarpımıdır. Mavi dikdörtgenler pozitif, kırmızılar negatif katkı yapar.

16.5.3 scikit-learn ile

Aynı sonucu `scikit-learn`'ün `LinearRegression` aracıyla da alabiliriz. Kısım III'teki `fit` mantığı burada da geçerlidir: Model katsayıları `fit` sırasında öğrenir.

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(daire[["alan"]], daire["fiyat"])     # X iki boyutlu, y tek boyutlu
print(model.intercept_.round(4), model.coef_.round(4))
model.predict(pd.DataFrame({"alan": [120]})).round(3)
```

çıktı

```
-0.9843 [0.0407]
array([3.898])
```

`daire[["alan"]]` iki köşeli parantezle bir tablo (iki boyutlu) seçer; `scikit-learn` girdilerin bu biçimde olmasını bekler. `.predict(...)` yeni bir girdi için tahmin üretir: 120 m² bir daire için 3,898 milyon TL.

OLS, deneme yanılma yapmadan, veriyi bir kez dolaşarak kesin çözümü verir. İstatistik yazılımları ve kütüphaneler sayısal kararlılık için hesabı farklı matris yöntemleriyle yapar; ama çözdükleri problem aynıdır: SSE'nin en küçük noktası.

16.6 OLS doğrusunun özellikleri

Modelde kesişim bulunduğu sürece aşağıdaki eşitlikler **her veri kümesinde**, hiçbir varsayıma gerek kalmadan geçerlidir. Hepsi iki normal denklemin doğrudan sonucudur.

Tablo 16.5 OLS doğrusunun cebirsel özellikleri.

Özellik	Formül	Anlamı
Artıkların toplamı sıfırdır	$\sum e_i = 0$	Doğru ortalamada ne yukarıda ne aşağıdadır
Artıklar x ile ilişkisizdir	$\sum x_i e_i = 0$	x'te kullanılmamış doğrusal bilgi kalmamıştır
Doğru ağırlık merkezinden geçer	$\bar{y} = \hat{\beta}_0 + \hat{\beta}_1 \bar{x}$	Ortalama alandaki dairenin tahmini, ortalama fiyattır
Tahminlerin ortalaması $\bar{y}$'dir	$\frac{1}{n}\sum \hat{y}_i = \bar{y}$	Model toplamda fiyat düzeyini doğru yakalar
Tahminler ile artıklar ilişkisizdir	$\sum \hat{y}_i e_i = 0$	Bölüm 17'deki kareler toplamı ayrışımını mümkün kılar

```
e = y - (b0 + b1 * x)
print(np.isclose(e.sum(), 0), np.isclose((x * e).sum(), 0))   # ikisi de sıfır mı?
```

çıktı

```
True True
```

Dikkat

"Artıkların ortalaması sıfır çıktı, demek ki hata teriminin ortalamasının sıfır olduğu varsayımı sağlanıyor" çıkarımı yanlıştır. Kesişimli bir OLS modelinde artıkların ortalaması **zorunlu olarak** sıfırdır; bu bir sınav değil, yöntemin cebirsel sonucudur.

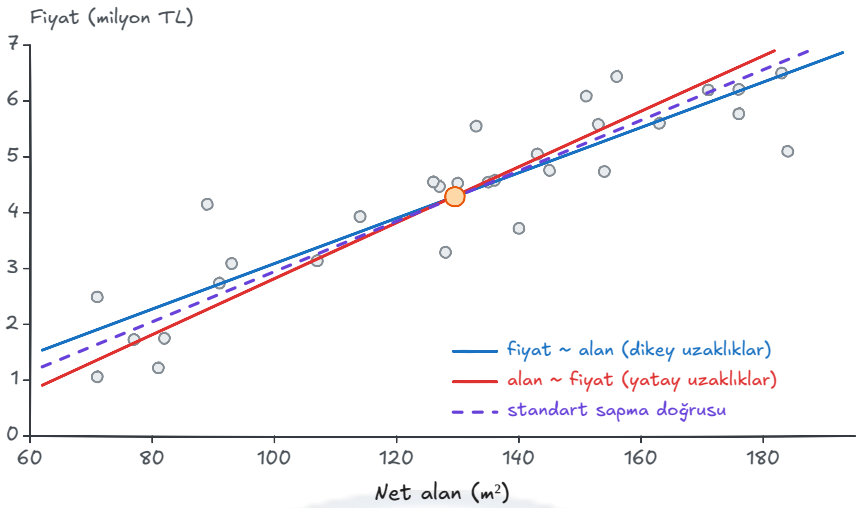
16.6.1 Eğim ve korelasyon

$\hat{\beta}_1 = r \cdot s_y / s_x$ eşitliği, eğim ile Pearson korelasyonu arasındaki ilişkiyi verir. Örneğimizde r = 0,902, s_y = 1,575 milyon TL ve s_x = 34,92 m²; çarpım 0,0407'dir. İki değişkeni z-skorlarına çevirirsek (Bölüm 13) eğim tam olarak r olur. Korelasyon ilişkinin gücünü ve yönünü birimsiz ölçer; eğim aynı ilişkiyi y'nin ve x'in birimleriyle ifade eder.

Bu eşitlik Galton'un gözlemini de açıklar. Standartlaştırılmış değişkenlerde tahmin $\hat{z}_y = r \cdot z_x$ biçimini alır. $|r| < 1$ olduğu sürece, ortalamadan 2 standart sapma büyük bir dairenin fiyatının ortalamadan yalnızca $2r = 1{,}80$ standart sapma yüksek olması beklenir. Tahminler her zaman x'e göre ortalamaya daha yakındır. Bu biyolojik bir mekanizma değil, korelasyonun mükemmel olmamasının istatistiksel sonucudur.

16.6.2 Yön önemlidir

OLS yalnızca **dikey** uzaklıkları küçültür, çünkü hatanın y'de olduğunu varsayar. Fiyatı alana göre değil de alanı fiyata göre tahmin etmek isteseydik yatay uzaklıkları küçültürdük ve farklı bir doğru bulurduk (Şekil 16.5). İki doğru yalnızca $|r| = 1$ iken çakışır.



Şekil 16.5 Üç farklı doğru. Fiyatın alana regresyonu dikey, alanın fiyata regresyonu yatay uzaklıkları küçültür. Standart sapma doğrusu ikisinin arasında kalır. Üçü de ağırlık merkezinden geçer.

Bölüm özeti

- Regresyon, sayısal bir hedefin koşullu ortalamasını, $E[Y \mid X]$, modeller. Tahmin ve çıkarım olmak üzere iki amaca hizmet eder.
- "Doğrusal" sözcüğü katsayılarla ilgilidir; X^2 ya da $\log X$ içeren modeller de doğrusal regresyonla çözülür.
- Popülasyon modeli $Y = \beta_0 + \beta_1 X + \varepsilon$'dir. Örneklemden tahmin edilen doğru $\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$ 'tir. Hata gerçek doğruya, artık tahmin edilen doğruya uzaklıktır.
- Daire verisinde $\hat{y} = -0{,}984 + 0{,}0407 \times \text{alan}$. Eğim: 1 m² büyük daireler ortalamada 40,7 bin TL pahalıdır. Bu bir karşılaştırmadır, bir müdahale değil; veri aralığında geçerlidir.
- Kesişim yalnızca x = 0 anlamlı ve veriye yakınsa yorumlanır; anlamsız olsa da modelden atılmaz. Merkezleme kesişimi anlamlı kılar.
- En küçük kareler, artık kareler toplamını (SSE) en küçük yapar. Büyük hataları orantısından fazla cezalandırır.
- OLS çözümü: $\hat{\beta}_1 = S_{xy}/S_{xx}$, $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$. Doğru her zaman ağırlık merkezinden geçer.
- Kesişimli OLS'de artıkların toplamı ve x ile çarpımlarının toplamı zorunlu olarak sıfırdır; bu bir varsayım sınaması değildir.
- $\hat{\beta}_1 = r \cdot s_y/s_x$. OLS dikey uzaklıkları küçültür; x'in y'ye regresyonu farklı bir doğru verir.

Kaynaklar

James, G., Witten, D., Hastie, T., Tibshirani, R. ve Taylor, J. (2023). *An Introduction to Statistical Learning with Applications in Python*. Springer. Bölüm 3.1.

Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3. baskı). O'Reilly. Bölüm 4: "Linear Regression".

Galton, F. (1886). Regression towards mediocrity in hereditary stature. *Journal of the Anthropological Institute*, 15, 246–263.

17

BÖLÜM

Regresyon modelini değerlendirmek

Bu bölümde

- Toplam değişkenliği açıklanan ve artık parçalarına ayırmak; R^2 ve artık standart hatasını hesaplayıp yorumlamak,
- MSE, RMSE, MAE ve R^2 ölçütlerini karşılaştırmak; eğitim ve test verisinde ölçmenin farkını açıklamak,
- R^2'nin neyi söylemediğini Anscombe dörtlüsüyle göstermek,
- doğrusal regresyonun beş varsayımını, ihlallerinin sonuçlarını ve artık grafiklerinde nasıl görüneceklerini tanımak,
- aykırı değer, yüksek kaldıraç ve etkili gözlem kavramlarını ayırmak,
- katsayıların standart hatasını, t testini ve güven aralığını yorumlamak,
- ortalama yanıt için güven aralığını tek bir gözlem için kestirim aralığından ayırmak.

Kaynak: YBS 319 ders sayfası

Bölüm 16'da daire verisine bir doğru uydurduk: $\hat{y} = -0{,}984 + 0{,}0407 \times \text{alan}$. Bu bölümde üç soru soruyoruz. Doğru verideki değişkenliğin ne kadarını açıklıyor? Doğrunun güvenilirliği hangi varsayımlara dayanıyor? Katsayıların ve tahminlerin belirsizliği ne kadar?

17.1 Model uyumu

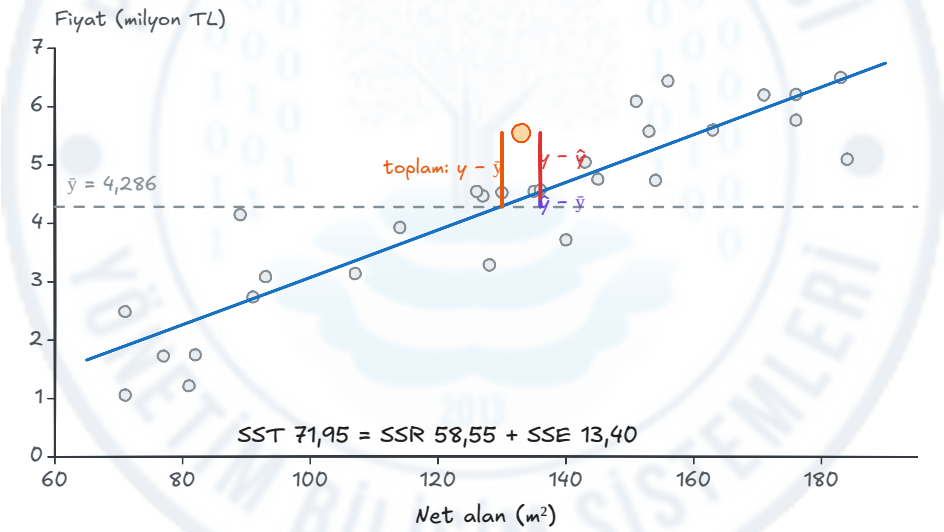
17.1.1 Kareler toplamının ayrışımı

Hiç açıklayıcı değişken kullanmasaydık her daire için yapabileceğimiz en iyi tahmin ortalama fiyat $\bar{y}$ olurdu. Regresyonun başarısını bu en basit modele göre ölçeriz. Her gözlemin ortalamadan sapması iki parçaya ayrılır:

$$\underbrace{(y_i - \bar{y})}_{\text{toplam sapma}} = \underbrace{(\hat{y}_i - \bar{y})}_{\text{modelin açıkladığı}} + \underbrace{(y_i - \hat{y}_i)}_{\text{artık}}$$

İki tarafın karesini alıp topladığımızda, Bölüm 16'daki $\sum \hat{y}_i e_i = 0$ ve $\sum e_i = 0$ özellikleri sayesinde çapraz terimler sıfırlanır ve kareler toplamı da aynı biçimde ayrışır (Şekil 17.1):

$$\underbrace{\sum (y_i - \bar{y})^2}_{\text{SST}} = \underbrace{\sum (\hat{y}_i - \bar{y})^2}_{\text{SSR}} + \underbrace{\sum (y_i - \hat{y}_i)^2}_{\text{SSE}}$$



Şekil 17.1 Bir gözlemin toplam sapmasının ayrışımı. Ortalamadan doğruya kadar olan parça modelin açıkladığı, doğrudan gözleme kadar olan parça artıktır.

```
b1 = np.sum((x - x.mean()) * (y - y.mean())) / np.sum((x - x.mean()) ** 2)
b0 = y.mean() - b1 * x.mean()
y_sapka = b0 + b1 * x

SST = np.sum((y - y.mean()) ** 2)
SSR = np.sum((y_sapka - y.mean()) ** 2)
SSE = np.sum((y - y_sapka) ** 2)
print(round(SST, 2), "=", round(SSR, 2), "+", round(SSE, 2))
```

çıktı

```
71.95 = 58.55 + 13.4
```

Örneğimizde 71,95 = 58,55 + 13,40.

17.1.2 Belirlilik katsayısı R^2

Kısaltmalara dikkat
Bu kitapta SST toplam, SSR açıklanan (*regression*), SSE artık (*error*) kareler toplamıdır. ISLR toplamı TSS, artığı RSS olarak adlandırır. Bazı kitaplarda SSR "residual" anlamında kullanılır. Bir kaynağı okurken kısaltmanın tanımına bakın.

$$R^2 = \frac{\text{SSR}}{\text{SST}} = 1 - \frac{\text{SSE}}{\text{SST}} = 1 - \frac{13{,}40}{71{,}95} = 0{,}814$$

Yorum: Fiyatların ortalama etrafındaki değişkenliğinin %81,4'ü alanla doğrusal bir ilişkiyle açıklanmaktadır. Kesişimli bir OLS modelinde, eğitim verisinde R^2 0 ile 1 arasındadır. Basit regresyonda R^2, korelasyonun karesine eşittir: $0{,}902^2 = 0{,}814$.

17.1.3 Artık standart hatası

$$\hat{\sigma} = \text{RSE} = \sqrt{\frac{\text{SSE}}{n-2}} = \sqrt{\frac{13{,}40}{28}} = 0{,}692 \text{ milyon TL}$$

RSE, hata teriminin standart sapması σ'nın tahminidir ve y'nin biriminde ifade edilir. Gerçek regresyon doğrusunu bilsek bile, bir dairenin fiyatı tipik olarak bu doğrudan yaklaşık 0,69 milyon TL sapar. Ortalama fiyat 4,29 milyon TL olduğuna göre bu yaklaşık %16'lık bir tipik sapmadır. R^2 "ne kadarı açıklandı?" sorusuna, RSE "geriye ne kadar belirsizlik kaldı?" sorusuna cevap verir.

Paydada neden $n - 2$ var? İki parametre ($\hat{\beta}_0$, $\hat{\beta}_1$) veriden tahmin edildi. Artıklar iki doğrusal kısıtı ($\sum e_i = 0$ ve $\sum x_i e_i = 0$) sağlamak zorunda olduğundan, 30 artıktan yalnızca 28'i serbestçe değişebilir. Buna **serbestlik derecesi** denir. Genel durumda p açıklayıcı değişkenle payda $n - p - 1$ olur.

17.1.4 Makine öğrenmesinde kullanılan hata ölçüleri

```python
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

mse = mean_squared_error(y, y_sapka)
print("MSE :", round(mse, 3))
print("RMSE:", round(np.sqrt(mse), 3))
print("MAE :", round(mean_absolute_error(y, y_sapka), 3))
print("R²  :", round(r2_score(y, y_sapka), 3))
```

çıktı

```
MSE : 0.447
RMSE: 0.668
MAE : 0.511
R²  : 0.814
```

Tablo 17.1 Regresyon hata ölçüleri (eğitim verisinde).

Ölçü	Formül	Değer	Not
MSE	$\frac{1}{n}\sum e_i^2$	0,447	Birimi milyon TL^2; doğrudan yorumlanması zordur
RMSE	$\sqrt{\text{MSE}}$	0,668	Hedefin biriminde; RSE'den farkı paydada n kullanılmasıdır
MAE	$\frac{1}{n}\sum\lvert e_i\rvert$	0,511	Büyük hatalara daha az duyarlıdır
R^2	$1 - \text{SSE}/\text{SST}$	0,814	Birimsiz

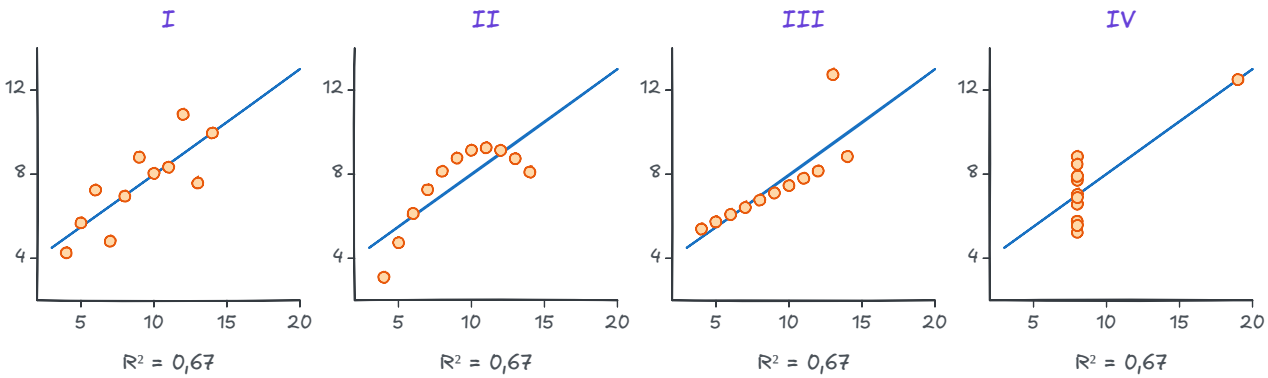
RMSE her zaman MAE'den büyük ya da ona eşittir. İkisi arasındaki fark büyükse, birkaç büyük hata var demektir.

> **Dikkat**
>
> Bu tablodaki değerler modelin eğitildiği veride hesaplandı. Tahmin amaçlı bir modelde katsayılar yalnızca eğitim verisinden tahmin edilir; MSE, RMSE, MAE ve R^2 ise modelin hiç görmediği test verisinde raporlanır (Bölüm 7). Test verisinde R^2 negatif bile çıkabilir: Bu, modelin test gözlemlerini eğitim ortalamasından daha kötü tahmin ettiği anlamına gelir.

17.1.5 R^2 neyi söylemez?

- **Yüksek R^2 modelin doğru olduğunu göstermez.** Eğri bir ilişkiye düz bir doğru uydurmak da yüksek R^2 verebilir.
- **Düşük R^2 ilişkinin önemsiz olduğunu göstermez.** İnsan davranışı gibi gürültülü alanlarda R^2 = 0,10 olan bir model, güçlü ve kesin tahmin edilmiş bir eğim içerebilir.
- **R^2 nedensellik ölçmez.**
- **Değişken eklemek R^2'yi asla düşürmez.** İşe yaramaz bir değişken bile R^2'yi biraz artırır (Bölüm 18).

Bunun klasik örneği Francis Anscombe'un 1973'te oluşturduğu dört veri kümesidir (Şekil 17.2). Dördünün de ortalamaları, varyansları, korelasyonları, regresyon doğruları ve R^2 değerleri neredeyse aynıdır. Grafikleri ise tamamen farklı hikâyeler anlatır.



Şekil 17.2 Anscombe dörtlüsü. Dört veri kümesinin de regresyon doğrusu yaklaşık y = 3 + 0,5x ve $R^2 \approx 0{,}67$'dir.

I. kümede doğrusal bir ilişki ve rastgele bir saçılım var; doğru uygundur. II. kümede açıkça eğri bir ilişki var; düz doğru yanlış modeldir. III. kümede tek bir aykırı değer doğrunun eğimini değiştiriyor. IV. kümede eğimi tek bir gözlem belirliyor; o nokta olmasa x'te hiç değişkenlik yok. Ders şu: **Çıktıdaki sayılara bakmadan önce veriyi çizin.**

17.2 Varsayımlar ve artık tanısı

OLS doğrusu, x'te değişkenlik olan her veri için hesaplanabilir. Varsayımlar doğrunun var olup olmayacağını değil, **ne kadar güvenilir olduğunu** belirler: Katsayılar gerçek değerlerin çevresinde mi dağılır? Standart hatalar ve testler doğru mu hesaplanır?

Tablo 17.2 Doğrusal regresyonun varsayımları.

Varsayım	Anlamı	İhlal edilirse	Nasıl incelenir?
1. Doğrusallık	$E[Y \mid X]$ gerçekten $\beta_0 + \beta_1 X$ biçimindedir	Doğru ilişkiyi yanlış özetler; bazı bölgelerde sistematik hata	Artık-tahmin grafiğinde eğri örüntü
2. Dışsallık	$E[\varepsilon \mid X] = 0$	Eğim yanlıdır; örneklem büyüse de düzelmez	Veriden tam sınanamaz; alan bilgisi
3. Bağımsızlık	Hatalar birbirinden bağımsızdır	Standart hatalar küçük, testler fazla iyimser	Zaman ya da grup sırasına göre artıklar
4. Sabit varyans	Hata varyansı her x için aynıdır	Katsayılar yansız ama standart hatalar yanlış	Huni biçimli artık grafiği
5. Normallik	$\varepsilon \sim N(0, \sigma^2)$	Küçük örneklemde testler yaklaşık kalır	Artıkların histogramı, Q-Q grafiği

Bu beş maddeye bir teknik koşul eklenir: x'te değişkenlik olmalıdır ($S_{xx} > 0$). Çoklu regresyonda bunun karşılığı, açıklayıcı değişkenler arasında tam doğrusal bağımlılık bulunmamasıdır (Bölüm 18).

Tanım 17.1 · Hangi sonuç hangi varsayıma dayanır?

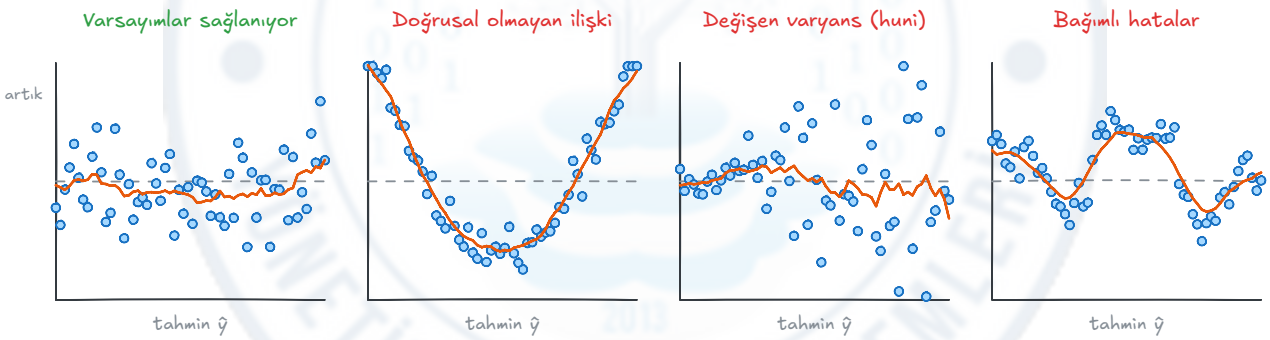
Katsayıları **hesaplamak** için yalnızca x'te değişkenlik yeterlidir. **Yansızlık** için doğrusallık, dışsallık ve rastgele örnekleme gerekir. Gauss-Markov anlamında **en iyi** olmak için bunlara ek olarak sabit varyans ve bağımsızlık gerekir. Kesin **t ve F dağılımları** için hepsine ek olarak normallik gerekir. Tahmin amaçlı kullanımda varsayımların bir kısmı gevşetilebilir; çıkarım amaçlı kullanımda hepsi önem kazanır.

Dikkat

Normallik varsayımı X'in ya da Y'nin dağılımıyla değil, **hata teriminin** dağılımıyla ilgilidir. Fiyatlar sağa çarpık olabilir; önemli olan, belirli bir alandaki fiyatların doğru çevresindeki dağılımıdır. Bu yüzden normallik ham değişkenlere değil artıklara bakılarak incelenir. Bölüm 13'te "ham girdileri normal dağıtmak gerekmez" dememizin bir nedeni de buydu.

17.2.1 Artık-tahmin grafiğini okumak

Varsayımları incelemenin en etkili aracı, artıkları tahmin edilen değerlere karşı çizen grafiktir. Varsayımlar sağlandığında bu grafik sıfır çizgisi çevresinde, belirli bir biçimi olmayan yatay bir bant gibi görünür. Her ihlalin kendine özgü bir görüntüsü vardır (Şekil 17.3).



Şekil 17.3 Artık-tahmin grafiklerinde dört tipik görüntü. Turuncu çizgi artıkların yerel ortalamasıdır.

- **Varsayımlar sağlanıyor:** yatay, biçimsiz bir bant.
- **Doğrusal olmayan ilişki:** Artıklar bir eğri çizer; uçlarda pozitif, ortada negatif. Çözüm olarak log dönüşümü ya da X^2 terimi düşünülür.
- **Değişen varyans:** Artıkların yayılımı tahmin büyüdükçe artar; huni biçimi. Çözüm olarak log dönüşümü ya da değişen varyansa dayanıklı standart hatalar kullanılır.
- **Bağımlı hatalar:** Zaman sırasıyla gözlenen verilerde ardışık artıklar birbirine benzer; dalgalı bir örüntü. Aynı binadaki daireler ya da ardışık aylar tipik örneklerdir.

17.2.2 Aykırı değer, kaldıraç ve etki

Kare kayıp büyük artıkları ağır cezalandırdığı için tek tek gözlemler doğruyu belirgin biçimde etkileyebilir. Bölüm 11'de aykırı değer ile etkili gözlemi ayırmıştık. Regresyonda bu ayrım üç kavramla yapılır (Şekil 17.4):

Aykırı değer
: y yönünde doğrudan uzak, yani artığı büyük olan gözlem. Doğrunun ortasında yer alıyorsa eğimi pek değiştirmez, ama RSE'yi ve standart hataları büyütür.

Yüksek kaldıraç
: x yönünde ortalamadan uzak gözlem. Kaldıraç $h_i = \frac{1}{n} + \frac{(x_i - \bar{x})^2}{S_{xx}}$ ile ölçülür. Ortalaması $(p+1)/n$ 'dir; bu değerin iki katını aşan gözlemler dikkatle incelenir.

Etkili gözlem
: Veriden çıkarıldığında katsayıları belirgin biçimde değiştiren gözlem. Etki, büyük artık ile yüksek kaldıracın birleşimidir ve **Cook uzaklığı** ile ölçülür.



Şekil 17.4 Tek bir gözlem doğruyu ne kadar değiştirebilir? Kesikli gri doğru, turuncu nokta olmadan elde edilen doğrudur.

Bir aykırı gözlemi silmek için "aykırı olması" yeterli bir gerekçe değildir; Bölüm 11'deki karar akışı burada da geçerlidir. Önce kayıt hatası olup olmadığı araştırılır. Gerçek bir gözlemse, sonuçlar o gözlemle ve onsuz raporlanabilir ya da aykırı değerlere dayanıklı yöntemler kullanılabilir.

17.3 Katsayılar için çıkarım

17.3.1 Standart hata

$\hat{\beta}$'lar örneklemden örnekleme değişir. Bu değişkenliğin standart sapmasına **standart hata** denir. Varsayımlar altında formülleri şöyledir:

$$\mathrm{SE}(\hat{\beta}_1) = \frac{\hat{\sigma}}{\sqrt{S_{xx}}} \qquad \mathrm{SE}(\hat{\beta}_0) = \hat{\sigma}\sqrt{\frac{1}{n} + \frac{\bar{x}^2}{S_{xx}}}$$

Eğim formülü üç şeyi açıkça söyler. Eğim daha kesin tahmin edilir: (1) gürültü (σ) küçükse, (2) gözlem sayısı büyükse, çünkü S_{xx} gözlem sayısıyla büyür, (3) x değerleri geniş bir aralığa yayılmışsa. Bütün daireler 125 ile 135 m² arasında olsaydı, alanın etkisini çok belirsiz tahmin ederdik.

17.3.2 Hipotez testi ve güven aralığı

En sık sorulan soru, X ile Y arasında doğrusal bir ilişki olup olmadığıdır:

$$H_0 : \beta_1 = 0 \qquad H_1 : \beta_1 \neq 0 \qquad t = \frac{\hat{\beta}_1 - 0}{\mathrm{SE}(\hat{\beta}_1)} \sim t_{n-2}$$

H_0 doğruysa t istatistiği $n - 2$ serbestlik dereceli t dağılımına uyar. Bu çıktıyı `statsmodels` kütüphanesiyle alabiliriz:

```
import statsmodels.formula.api as smf

model = smf.ols("fiyat ~ alan", data=daire).fit()
model.summary2().tables[1].round(4)
```

çıktı

```
            Coef.  Std.Err.        t   P>|t|  [0.025  0.975]
Intercept -0.9843    0.4929  -1.9969  0.0556 -1.9940  0.0254
alan       0.0407    0.0037  11.0611  0.0000  0.0332  0.0482
```

`statsmodels` istatistiksel çıkarım için tasarlanmış bir kütüphanedir; kurulu değilse `%pip install statsmodels` ile kurulur. `"fiyat ~ alan"` ifadesi "fiyatı alana göre modelle" anlamına gelir; kesişim otomatik olarak eklenir.

Tablo 17.3 Basit regresyon çıktısı.

	Tahmin	Std. hata	*t*	*p*	%95 güven aralığı
Kesişim	−0,984	0,493	−2,00	0,056	[−1,994; 0,025]
Alan (m²)	0,0407	0,0037	11,06	< 0,001	[0,0332; 0,0482]

Eğim: t = 11,06 ve p < 0,001. Alan ile fiyat arasında hiçbir ilişki olmasaydı, 30 dairelik bir örneklemde bu kadar büyük bir t değeri görmek son derece düşük olasılıklı olurdu. %95 güven aralığı, m² başına yaklaşık 33 ile 48 bin TL'dir.

Kesişim: p = 0,056. Bu test "$\beta_0 = 0$ mı?" sorusunu sınar. Kesişim zaten yorumlanabilir bir nicelik olmadığından bu p değeri burada bir anlam taşımaz ve kesişim modelden atılmaz (Bölüm 16).

> **Dikkat**
>
> p değeri, H_0 doğruyken en az gözlenen kadar uç bir t değeri elde etme olasılığıdır. "H_0'ın doğru olma olasılığı" değildir ve etkinin büyüklüğünü ölçmez; çok büyük örneklemlerde önemsiz büyüklükteki etkiler de çok küçük p değerleri verir. %95 güven aralığı ise bu yöntemle çok sayıda örneklemden kurulacak aralıkların %95'inin gerçek β_1'i kapsayacağı anlamına gelir.

17.4 Ortalama tahmini mi, tek bir dairenin tahmini mi?

120 m² için doğru 3,898 milyon TL verir. Bu sayı iki farklı soruya cevap olarak kullanılabilir ve iki sorunun belirsizliği çok farklıdır:

Güven aralığı: ortalama yanıt

"120 m² dairelerin **ortalama** fiyatı nedir?" Belirsizlik yalnızca doğrunun konumundan gelir.

$$\mathrm{SE} = \hat{\sigma}\sqrt{\frac{1}{n} + \frac{(x_0 - \bar{x})^2}{S_{xx}}}$$

Kestirim aralığı: tek bir gözlem

"İlana çıkacak **belirli bir** 120 m² dairenin fiyatı ne olur?" Doğrunun belirsizliğine dairenin kendi sapması eklenir.

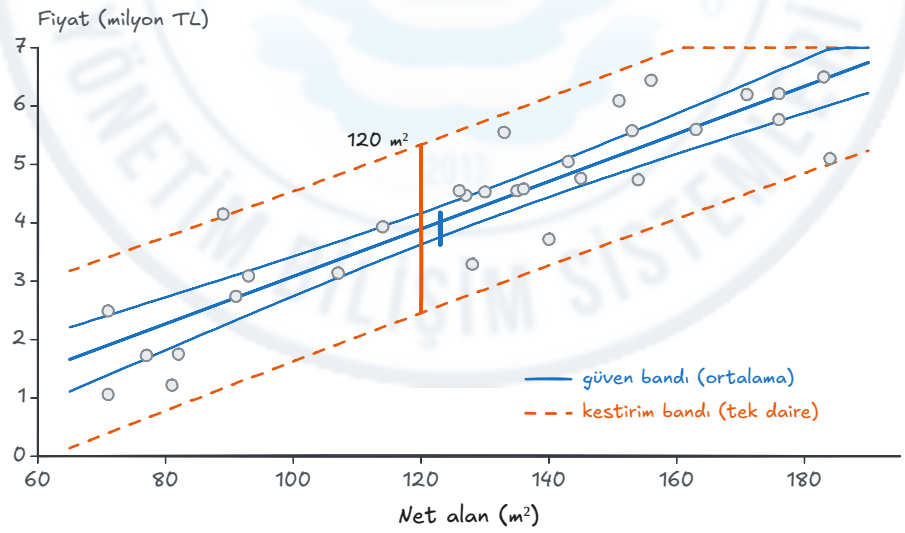
$$\mathrm{SE} = \hat{\sigma}\sqrt{1 + \frac{1}{n} + \frac{(x_0 - \bar{x})^2}{S_{xx}}}$$

```
yeni = pd.DataFrame({"alan": [120]})
aralik = model.get_prediction(yeni).summary_frame(alpha=0.05)
aralik[["mean", "mean_ci_lower", "mean_ci_upper", "obs_ci_lower",
"obs_ci_upper"]].round(2)
```

çıktı

	mean	mean_ci_lower	mean_ci_upper	obs_ci_lower	obs_ci_upper
0	3.9	3.63	4.17	2.46	5.34

Ortalama için %95 güven aralığı [3,63; 4,17] milyon TL; tek bir daire için %95 kestirim aralığı [2,46; 5,34] milyon TL. İki formüldeki tek fark, kestirim aralığındaki "1 +" terimidir. Örneklem büyüdükçe güven aralığı sıfıra doğru daralır; kestirim aralığı ise hiçbir zaman σ'dan dar olamaz. Ne kadar veri toplarsak toplayalım, tek bir dairenin kendine özgü sapmasını ortadan kaldıramayız.



Şekil 17.5 Güven ve kestirim bantları. İki bant da ortalama alan civarında en dar, uçlara doğru daha geniştir.

Şekil 17.5'te iki bandın da $\bar{x} = 129{,}5$ m² civarında en dar olduğuna dikkat edin. Bu, formüldeki $(x_0 - \bar{x})^2$ teriminden gelir: Veri bulutunun ortasında doğrunun konumu daha kesin bilinir. Veri aralığının dışına çıkıldığında bantlar formül gereği yalnızca biraz genişler; ama bu genişleme

dışdeğerlemenin gerçek riskini yansıtmaz, çünkü formül doğrusal ilişkinin veri dışında da sürdüğünü varsayar.

Bölüm özeti

- Toplam değişkenlik açıklanan ve artık parçalarına ayrılır: SST = SSR + SSE. Daire verisinde 71,95 = 58,55 + 13,40.
- $R^2 = 1 - SSE/SST = 0{,}814$: Fiyat değişkenliğinin %81,4'ü alanla açıklanır. Basit regresyonda $R^2 = r^2$.
- RSE = $\sqrt{SSE/(n-2)}$ = 0,692 milyon TL; hata teriminin tipik büyüklüğüdür. Payda serbestlik derecesidir.
- MSE, RMSE ve MAE makine öğrenmesinde kullanılan hata ölçüleridir. Tahmin amaçlı modellerde test verisinde raporlanır; test R^2'si negatif olabilir.
- Yüksek R^2 doğru model, düşük R^2 önemsiz ilişki demek değildir; R^2 nedensellik ölçmez ve değişken eklendikçe düşmez. Anscombe dörtlüsü veriyi çizmeyi öğretir.
- Beş varsayım: doğrusallık, dışsallık, bağımsızlık, sabit varyans, normallik. Normallik hatalarla ilgilidir ve artıklarla incelenir.
- Artık-tahmin grafiğinde eğri örüntü doğrusal olmayan ilişkiyi, huni değişen varyansı, dalga bağımlı hataları gösterir.
- Aykırı değer y'de, yüksek kaldıraç x'te uzaktır; etkili gözlem ikisinin birleşimidir ve katsayıları değiştirir.
- Eğimin standart hatası gürültü azaldıkça, örneklem büyüdükçe ve x'in yayılımı arttıkça küçülür. Alanın eğimi için t = 11,06, %95 güven aralığı [0,0332; 0,0482].
- Güven aralığı ortalama yanıt, kestirim aralığı tek bir gözlem içindir. Kestirim aralığı σ 'dan dar olamaz.

Kaynaklar

James, G., Witten, D., Hastie, T., Tibshirani, R. ve Taylor, J. (2023). *An Introduction to Statistical Learning with Applications in Python*. Springer. Bölüm 3.1.2–3.1.3 ve 3.3.3.

Anscombe, F. J. (1973). Graphs in statistical analysis. *The American Statistician*, 27(1), 17–21.

Wooldridge, J. M. (2020). *Introductory Econometrics: A Modern Approach* (7. baskı). Cengage. Bölüm 2–4.

18

BÖLÜM

Çoklu doğrusal regresyon

Bu bölümde

- Çoklu doğrusal regresyonu bir düzlem olarak yorumlamak; katsayıları “diğerleri sabitken” ifadesiyle doğru okumak,
- modeli matris gösterimiyle yazmak ve \hat{\beta} = (X^\top X)^{-1} X^\top y çözümünü Python’da uygulamak,
- bir çözümün ne zaman var olduğunu açıklamak; tam çoklu bağlantıyı tanımak,
- dışlanmış değişken yanlılığını sayısal olarak göstermek ve yönünü belirlemek,
- “bir değişkeni kontrol etmenin” ne demek olduğunu Frisch-Waugh-Lovell teoremiyle açıklamak,
- çoklu doğrusal bağlantıyı VIF ile ölçmek,
- düzeltilmiş R^2 ve F testini kullanmak; doğrusal modeli kategorik değişkenler, etkileşimler ve eğrisel terimlerle genişletmek.

Kaynak: YBS 319 ders sayfası

Fiyatı yalnızca alanla açıklamak, aynı alandaki yeni ve eski binaları aynı kefeye koymak demektir. Bölüm 16'da alanın eğimini yorumlarken bir uyarı yapmıştık: Büyük daireler aynı zamanda daha yeni binalarda olduğu için 40,7 bin TL'nin bir kısmı aslında bina yaşının etkisini taşıyor olabilir. Bu bölümde iki değişkeni aynı modele koyarak bu etkileri ayırıyoruz.

18.1 Doğrudan düzleme

Çoklu doğrusal regresyon, birden fazla açıklayıcı değişkeni aynı modelde kullanır:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \varepsilon$$

Örneğimizde X_1 net alan (m²), X_2 bina yaşı (yıl). Genel durumda p değişkenle $Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p + \varepsilon$ yazılır.

Neden her değişken için ayrı bir basit regresyon kurmuyoruz? Birincisi, ayrı modellerden tek bir tahmin üretmenin tutarlı bir yolu yoktur. İkincisi ve daha önemlisi, açıklayıcı değişkenler birbiriyle ilişkiliyse her basit regresyon, diğer değişkenin etkisinin bir kısmını kendi eğimine katar. Örneğimizde alan ile yaş arasındaki korelasyon −0,60'tır.

18.1.1 Regresyon düzlemi

İki açıklayıcı değişkenle $E[Y \mid X_1, X_2] = \beta_0 + \beta_1 X_1 + \beta_2 X_2$ üç boyutlu uzayda bir **düzlemdir.** Taban düzlemi alan ve yaş eksenlerinden oluşur, dikey eksen fiyattır. Her daire bu uzayda bir noktadır. OLS, noktaların düzleme fiyat yönündeki uzaklıklarının karelerinin toplamını en küçük yapan düzlemi seçer.

```
model2 = smf.ols("fiyat ~ alan + yas", data=daire).fit()
print(model2.params.round(4))
print("R²:", round(model2.rsquared, 3), "| düzeltilmiş R²:", round(model2.rsquared_adj,
3))
```

çıktı

```
Intercept    1.5902
alan         0.0305
yas         -0.0661
dtype: float64
R²: 0.902 | düzeltilmiş R²: 0.895
```

Örnek verimizde OLS düzlemi:

$$\hat{y} = 1{,}590 + 0{,}0305 \times \text{alan} - 0{,}0661 \times \text{yaş} \qquad (R^2 = 0{,}902,\ \hat{\sigma} = 0{,}511)$$

Aynı modeli `scikit-learn` ile de kurabiliriz; tek fark girdiye iki sütun vermektir:

```
from sklearn.linear_model import LinearRegression

model = LinearRegression().fit(daire[["alan", "yas"]], daire["fiyat"])
print(model.intercept_.round(4), model.coef_.round(4))
```

çıktı

```
1.5902 [ 0.0305 -0.0661]
```

18.1.2 “Diğerleri sabitken”

$\hat{\beta}_1 = 0{,}0305$ **(alan)**

Aynı yaştaki daireler karşılaştırıldığında, 1 m² daha büyük olan daire ortalamada 30,5 bin TL daha pahalıdır. Geometrik olarak düzlemin alan ekseni yönündeki eğimidir.

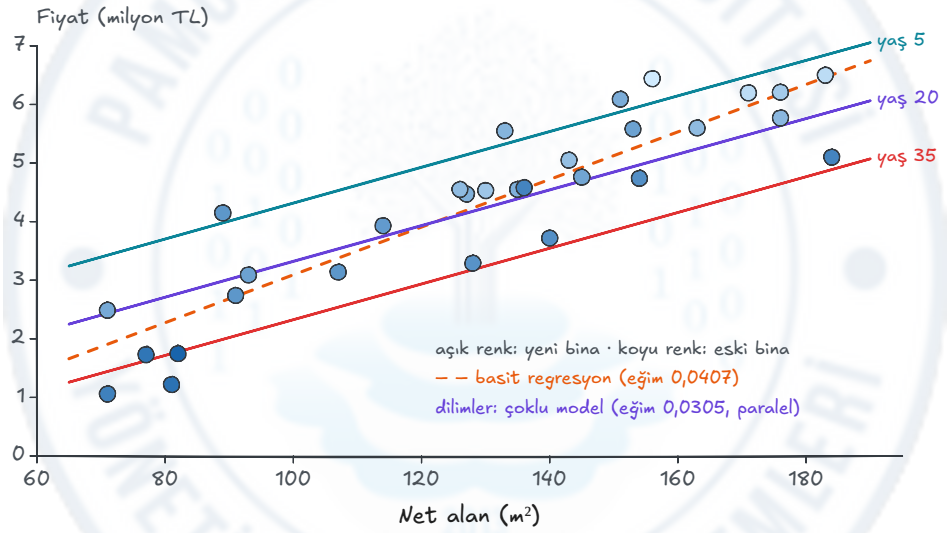
$\hat{\beta}_2 = -0{,}0661$ **(yaş)**

Aynı alandaki daireler karşılaştırıldığında, binası 1 yıl daha eski olan daire ortalamada 66,1 bin TL daha ucuzdur. 10 yıllık fark yaklaşık 661 bin TL eder.

$\hat{\beta}_0 = 1{,}590$ **(kesişim)**

Alan 0 m² ve yaş 0 iken beklenen fiyat. İki değişken birden sıfırdayken anlamsızdır. Düzlem her zaman ağırlık merkezinden, yani (129,5 m², 19,1 yıl, 4,286 milyon TL) noktasından geçer.

Düzlemi sabit yaşlarda dilimlediğimizi düşünelim (Şekil 18.1). Her dilim, o yaştaki daireler için alan ile fiyat arasındaki ilişkiyi gösteren bir doğrudur. Yaş değiştikçe doğru yukarı aşağı kayar ama eğimi hep 0,0305’tir: Dilimler paraleldir. Bu, modelin **toplamsallık** varsayımıdır: Alanın etkisi yaşa bağlı değildir.



Şekil 18.1 Regresyon düzleminin sabit yaş dilimleri. Noktalar bina yaşına göre renklendirilmiştir. Üç paralel doğru 5, 20 ve 35 yıllık binalar için alan–fiyat ilişkisidir; kesikli doğru ise yaşı hesaba katmayan basit regresyondur.

> **Dikkat**
>
> “Yaş sabitken” ifadesi, veride aynı yaştaki dairelerin karşılaştırıldığı anlamına gelir; bir deney yapılmış değildir. Modelde olmayan değişkenler (konum, kat, site içinde olma) hâlâ sabit tutulmamaktadır. Bu değişkenler alan ya da yaşla ilişkiliyse katsayılar onların etkisini de taşır.

Tahmin örneği: 120 m², 10 yıllık bir daire için $1{,}590 + 0{,}0305 \times 120 - 0{,}0661 \times 10 = 4{,}594$ milyon TL. Basit modelin aynı daire için tahmini 3,898 milyon TL idi. Basit model yaşı bilmediği için bu görece yeni daireyi, veri kümesindeki ortalama yaştaki (yaklaşık 19 yıl) dairelerle aynı kefeye koyuyordu.

18.2 Matris gösterimi

Değişken sayısı arttıkça her katsayı için ayrı formül yazmak imkânsızlaşır. Matris gösterimiyle model, kaç değişken olursa olsun tek satırda yazılır:

$$\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\varepsilon}$$

Örneğimizde **y**, 30 × 1 boyutlu fiyat vektörü; X, 30 × 3 boyutlu **tasarım matrisi**; **β**, 3 × 1 boyutlu katsayı vektörüdür. Tasarım matrisinin ilk sütunu tamamen 1'lerden oluşur. Her satırda β_0 bu 1 ile çarpılır; böylece kesişim, değeri her gözlemde 1 olan sabit bir değişkenin katsayısı gibi davranır. Makine öğrenmesi kaynaklarında kesişimin "bias" olarak adlandırılması ve girdiye eklenmiş bir 1 sütunu olarak gösterilmesi bu yüzdendir.

18.2.1 Normal denklemler ve çözüm

Kare artıkların toplamı matrislerle $\text{SSE}(\boldsymbol{\beta}) = (\mathbf{y} - X\boldsymbol{\beta})^\top(\mathbf{y} - X\boldsymbol{\beta})$ olarak yazılır. Türevi alıp sıfıra eşitlersek:

$$X^\top X\hat{\boldsymbol{\beta}} = X^\top \mathbf{y} \quad \Longrightarrow \quad \hat{\boldsymbol{\beta}} = (X^\top X)^{-1} X^\top \mathbf{y}$$

Tek değişkenli durumda bu ifade Bölüm 16'daki formüllere indirgenir; oradaki iki normal denklem, buradaki matris denkleminin satırlarıdır.

```
np.set_printoptions(suppress=True)       # bilimsel gösterimi kapatır
X = np.column_stack([np.ones(len(daire)), daire["alan"], daire["yas"]])
y = daire["fiyat"].to_numpy()

XtX = X.T @ X
Xty = X.T @ y
beta = np.linalg.inv(XtX) @ Xty
print(XtX.round(0))
print(Xty.round(2))
print(beta.round(4))
```

çıktı

```
[[    30.   3886.    572.]
 [  3886. 538734.  68665.]
 [   572.  68665.  13190.]]
[  128.58 18094.39  2134.84]
[ 1.5902  0.0305 -0.0661]
```

`np.column_stack` sütunları yan yana birleştirir; ilk sütun 1'lerden oluşur. `@` işareti matris çarpımıdır; `.T` matrisin devriğini (*transpose*) alır; `np.linalg.inv` tersini hesaplar. $X^\top X$'in her elemanı tanıdık bir toplamdır: sol üst köşede gözlem sayısı (30), ilk satırda alanların ve yaşların toplamı, köşegende karelerin toplamı. Sonuç `statsmodels` ile aynı: [1,5902; 0,0305; −0,0661].

18.2.2 Çözüm ne zaman vardır?

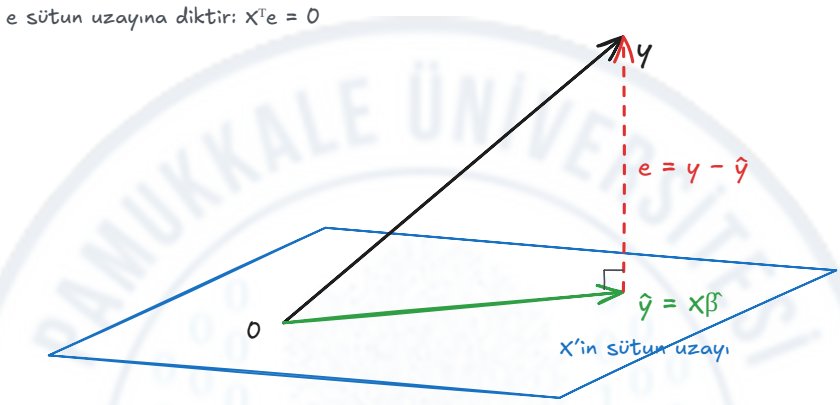
$X^\top X$'in tersinin alınabilmesi için X'in sütunlarının doğrusal olarak bağımsız olması gerekir. Bu iki şey demektir:

Yeterli gözlem: $n \geq p + 1$. Üç katsayıyı iki daireyle tahmin edemezsiniz: İki noktadan sonsuz sayıda düzlem geçer.

Tam çoklu bağlantı yok: Hiçbir sütun diğerlerinin tam doğrusal birleşimi olmamalıdır. Örnekler: alanın hem m² hem de ft² olarak modele girmesi; "oda sayısı", "banyo sayısı" ve bunların toplamı olan "toplam oda sayısı"nın birlikte kullanılması; kesişim varken bir kategorik değişkenin **bütün** kategorileri için gösterge sütunu eklenmesi. Sonuncusuna **kukla değişken tuzağı** denir. Bölüm 12'deki `drop="first"` seçeneğinin nedeni buydu.

18.2.3 Geometrik yorum: dik izdüşüm

30 fiyatı 30 boyutlu uzayda tek bir vektör olarak düşünün. X'in üç sütunu bu uzayda üç boyutlu bir alt uzay gerer. $X\beta$ biçiminde yazılabilen her tahmin vektörü bu alt uzaydadır. OLS, alt uzayda $\mathbf{y}$'ye en yakın noktayı seçer; en yakın nokta, $\mathbf{y}$'nin alt uzaya **dik izdüşümüdür** (Şekil 18.2).



Şekil 18.2 OLS bir dik izdüşümdür. Tahmin vektörü ŷ, y'nin X'in sütun uzayına izdüşümüdür; artık vektörü e bu uzaya diktir. SST = SSR + SSE ayrışımı bu dik üçgende Pisagor teoremidir.

Artık vektörü $\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}$ sütun uzayına diktir. Matris diliyle $X^\top \mathbf{e} = \mathbf{0}$: Artıklar birler sütununa dik ($\sum e_i = 0$) ve her açıklayıcı değişkene dik. Bölüm 16'daki özellikler bu dikliğin satır satır yazılmış hâlidir.

18.2.4 Katsayıların varyansı

$$\mathrm{Var}(\hat{\boldsymbol{\beta}}) = \sigma^2 (X^\top X)^{-1}$$

Standart hatalar, σ^2 yerine $\hat{\sigma}^2 = \mathrm{SSE}/(n-p-1)$ konarak bu matrisin köşegeninden okunur. $(X^\top X)^{-1}$'in köşegen elemanları büyüdükçe standart hatalar da büyür. Açıklayıcı değişkenler birbirine çok benzediğinde bu elemanlar şişer; çoklu doğrusal bağlantının matematiksel kaynağı budur.

18.3 Dışlanmış değişken yanlılığı

Alanın katsayısı basit modelde 0,0407, çoklu modelde 0,0305. İkisi de doğru hesaplanmıştır; ama farklı sorulara cevap verirler:

Basit model: 40,7 bin TL / m²

> "1 m² daha büyük bir daire, ortalamada ne kadar pahalı?" Büyük dairelerin genellikle daha yeni binalarda olduğu gerçeği bu karşılaştırmaya dahildir.

Çoklu model: 30,5 bin TL / m²

"Aynı yaştaki iki daireden 1 m² büyük olan, ortalamada ne kadar pahalı?" Yaş farkının fiyata yansıyan kısmı ayrılmıştır.

İki katsayı arasındaki fark tam olarak hesaplanabilir. Yaşı alana göre regresyona soktuğumuzda eğim $\hat{\delta} = -0{,}1535$ çıkar: 10 m² daha büyük daireler ortalamada 1,5 yıl daha yeni binalardadır. Basit modelin eğimi şu özdeşliği sağlar:

$$\tilde{\beta}_1 = \hat{\beta}_1 + \hat{\beta}_2 \cdot \hat{\delta} \qquad 0{,}0407 = 0{,}0305 + (-0{,}0661) \times (-0{,}1535)$$

```
basit = smf.ols("fiyat ~ alan", data=daire).fit().params["alan"]
delta = smf.ols("yas ~ alan", data=daire).fit().params["alan"]
b1, b2 = model2.params["alan"], model2.params["yas"]
print(round(basit, 4), "=", round(b1, 4), "+", round(b2 * delta, 4))
```

çıktı

```
0.0407 = 0.0305 + 0.0101
```

Basit modeldeki 40,7 bin TL'nin yaklaşık 10,1 bin TL'si aslında alanın değil, büyük dairelerin daha yeni binalarda olmasının payıdır. Popülasyon düzeyinde bu farka **dışlanmış değişken yanlılığı** denir.

Yanlılık iki koşul birlikte sağlandığında ortaya çıkar: Dışlanan değişken Y'yi etkilemeli **ve** dahil edilen değişkenle ilişkili olmalıdır. Biri sıfırsa basit ve çoklu eğimler aynı çıkar. Yanlılığın yönü iki işaretin çarpımıyla belirlenir:

Tablo 18.1 Dışlanmış değişken yanlılığının yönü.

	Dışlanan değişken dahil edilenle pozitif ilişkili	Negatif ilişkili
Dışlanan değişkenin Y'ye etkisi pozitif	Yukarı yanlılık	Aşağı yanlılık
Dışlanan değişkenin Y'ye etkisi negatif	Aşağı yanlılık	Yukarı yanlılık

Bizim durumumuz sağ alt hücredir: Yaş fiyatı düşürür ve alanla negatif ilişkilidir. Basit modelin alan eğimi bu yüzden yukarı yanlıdır.

18.3.1 "Kontrol etmek" ne demek?

Çoklu modeldeki alan katsayısını iki basit regresyonla da elde edebiliriz. Bu sonuca **Frisch-Waugh-Lovell teoremi** denir:

1. Alanı yaşa göre regresyona sokun ve artıkları alın. Bu artıklar, alanın yaşla açıklanamayan kısmıdır: "Bu yaştaki bir daireye göre beklenenden ne kadar büyük?"
2. Fiyatı bu artıklara göre regresyona sokun. Bulunan eğim tam olarak 0,0305'tir.

```
daire["alan_arinmis"] = smf.ols("alan ~ yas", data=daire).fit().resid
smf.ols("fiyat ~ alan_arinmis", data=daire).fit().params.round(4)
```

çıktı
```
Intercept       4.2860
alan_arinmis    0.0305
dtype: float64
```

Teorem, çoklu regresyon katsayısının ne anlama geldiğini kesin olarak söyler: $\hat{\beta}_1$, alanın diğer değişkenlerle örtüşmeyen kısmındaki değişimin fiyatla ilişkisidir. **"Yaşı kontrol etmek", alan değişkeninden yaşla ortak olan bilgiyi çıkarmak demektir.**

> **Dikkat**
>
> Dışlanmış değişken yanlılığı "ne kadar çok değişken, o kadar iyi" izlenimi verebilir. Ama açıklayıcı değişkenin **sonucu** olan değişkenler modele girerse, etkinin bir kısmını kendi üzerine alarak katsayının anlamını değiştirebilir. Alanın fiyata etkisini incelerken "oda sayısı"nı eklemek buna örnektir: Büyük daireler daha çok odaya sahiptir. Hangi değişkenlerin modele gireceği, sorulan soruya göre alan bilgisiyle belirlenir.

18.4 Çoklu doğrusal bağlantı

Tam çoklu bağlantıda OLS tek bir çözüm vermez. Pratikte daha sık karşılaşılan durum, değişkenlerin yüksek ama tam olmayan biçimde ilişkili olmasıdır. Çözüm vardır, ama katsayılar örneklemden örnekleme çok oynar ve standart hataları büyür.

18.4.1 Varyans şişirme faktörü

$$\mathrm{VIF}_j = \frac{1}{1 - R_j^2}$$

Burada R_j^2, j. açıklayıcı değişkenin diğer açıklayıcı değişkenlere göre regresyonundan elde edilen R^2'dir. VIF, katsayının varyansının diğer değişkenlerle örtüşme nedeniyle kaç kat büyüdüğünü gösterir; standart hata $\sqrt{\mathrm{VIF}}$ kat büyür.

```python
r = daire["alan"].corr(daire["yas"])
vif = 1 / (1 - r ** 2)
print(round(r, 3), round(vif, 2), round(np.sqrt(vif), 2))
```

çıktı
```
-0.604 1.57 1.25
```

İki değişkenli modelde $R_j^2 = r^2$'dir. Örneğimizde r = −0,604 ve VIF = 1,57; standart hatalar yaklaşık 1,25 kat büyümüştür. Bu kabul edilebilir bir düzeydir. Uygulamada VIF 5'i aştığında dikkat, 10'u aştığında ciddi sorun olarak değerlendirilir. Bu eşikler kuramsal değil, deneyime dayalı kurallardır.

18.4.2 Geometrik sezgi: menteşe

Alan ve yaş çok güçlü ilişkili olsaydı, noktalar tabanda dar bir şerit boyunca dizilirdi. Bu şeridin üzerinden geçen düzlem, şeridin çevresinde bir menteşe gibi dönebilir: Şerit boyunca tahminler neredeyse aynı kalırken düzlemin eğimi çok farklı olabilir. Veri, düzlemin şeride dik yöndeki eğimi

hakkında çok az bilgi taşır. Sonuç olarak tek tek katsayılar çok belirsiz olur, ama toplamları kesin tahmin edilir. Veri "ikisinin birlikte etkisini" bilir, ama bu etkiyi ikisi arasında nasıl paylaştıracağını bilemez.

Ne yapılabilir? Amaç tahminse çoklu bağlantı çoğu zaman sorun değildir: Yeni veride değişkenler arasındaki ilişki aynı kaldığı sürece tahminler isabetli olur. Amaç yorumsa daha fazla veri toplamak, ilişkili değişkenlerden birini çıkarmak, değişkenleri anlamlı bir indekste birleştirmek, PCA (Bölüm 14) ya da ileride göreceğimiz düzenlileştirme yöntemleri düşünülür.

18.5 Çoklu modelde uyum

18.5.1 R^2 neden her değişkenle artar?

Yeni bir değişken eklendiğinde OLS, o değişkenin katsayısını sıfır yaparak eski modeli her zaman yeniden elde edebilir. Dolayısıyla SSE asla artmaz, R^2 asla azalmaz. Tamamen rastgele bir değişken bile tesadüfi küçük ilişkiler üzerinden SSE'yi biraz düşürür. Bu yüzden değişken sayısı farklı modelleri R^2 ile karşılaştırmak yanıltıcıdır.

$$\bar{R}^2 = 1 - (1 - R^2)\frac{n-1}{n-p-1}$$

Düzeltilmiş R^2, her ek değişken için serbestlik derecesi kaybını cezalandırır. Yeni değişken SSE'yi yeterince düşürmezse düzeltilmiş R^2 azalır. Örneğimizde basit model R^2 = 0,814, $\bar{R}^2$ = 0,807; çoklu model R^2 = 0,902, $\bar{R}^2$ = 0,895. Yaşın eklenmesi cezaya rağmen açıklama gücünü belirgin biçimde artırmıştır.

Modele fiyatla hiçbir ilişkisi olmayan rastgele değişkenler ekleyelim. Tek bir denemenin sonucu şansa bağlı olabileceği için her durumu 200 kez tekrarlayıp ortalamaya bakıyoruz:

```
rng = np.random.default_rng(0)
y = daire["fiyat"].to_numpy()
n = len(y)

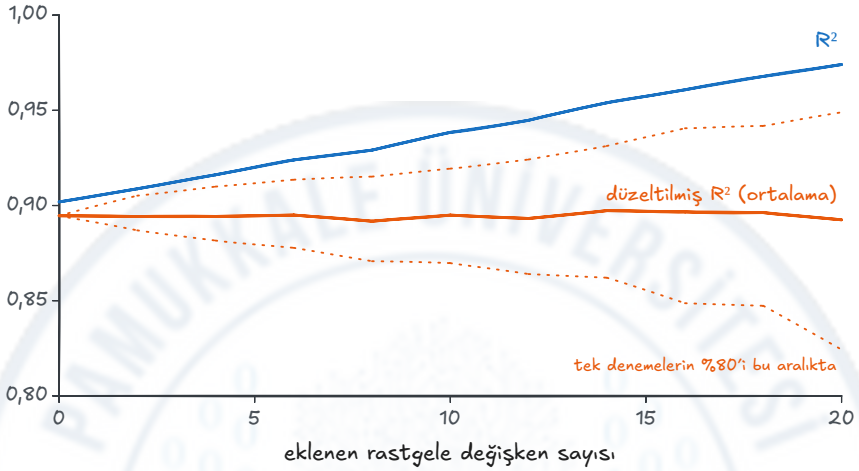
def r2_hesapla(X):
    X = np.column_stack([np.ones(n), X])
    b = np.linalg.lstsq(X, y, rcond=None)[0]          # OLS çözümü
    e = y - X @ b
    r2 = 1 - (e @ e) / np.sum((y - y.mean()) ** 2)
    p = X.shape[1] - 1
    return r2, 1 - (1 - r2) * (n - 1) / (n - p - 1)

sonuc = []
for k in [0, 5, 10, 15, 20]:
    tekrar = [r2_hesapla(np.column_stack([daire["alan"], daire["yas"]] +
                                         [rng.normal(size=n) for _ in range(k)]))
              for _ in range(200)]
    r2, r2d = np.array(tekrar).T
    sonuc.append((k, r2.mean().round(3), r2d.mean().round(3),
                  r2d.min().round(3), r2d.max().round(3)))
pd.DataFrame(sonuc, columns=["rastgele", "R² ort.", "düz. R² ort.",
                             "düz. R² en küçük", "düz. R² en büyük"])
```

çıktı

	rastgele	R^2 ort.	düz. R^2 ort.	düz. R^2 en küçük	düz. R^2 en büyük
0	0	0.902	0.895	0.895	0.895
1	5	0.919	0.894	0.873	0.932
2	10	0.938	0.894	0.847	0.962
3	15	0.956	0.895	0.804	0.981
4	20	0.974	0.893	0.722	0.982

`np.linalg.lstsq` en küçük kareler çözümünü, ters matris almadan, sayısal olarak daha kararlı biçimde hesaplar.



Şekil 18.3 Modele anlamsız değişkenler eklemek (200 tekrarın ortalaması). R^2 her eklemede sistematik olarak yükselir; düzeltilmiş R^2 ortalamada yerinde sayar ama tek tek denemelerde giderek daha çok oynar.

Tablo ve Şekil 18.3 iki şey gösteriyor. Birincisi, R^2 hiçbir anlamı olmayan değişkenlerle 0,902'den 0,97'nin üzerine çıkıyor: 30 gözlemle 20 rastgele değişken eklediğimizde model artık ilişkiyi değil, eğitim verisindeki gürültüyü ezberliyor. Bu, Bölüm 3 ve 7'de konuştuğumuz **aşırı uyumun** doğrusal regresyondaki görünümüdür. İkincisi, düzeltilmiş R^2 ortalamada bu artışı yakalamıyor; ama tek bir denemede şans eseri yükselebiliyor. Yani düzeltilmiş R^2 de tek başına güvenilir bir seçim ölçütü değildir. Böyle bir modelin test verisindeki hatası büyük olur. Makine öğrenmesinde model seçimi bu yüzden doğrulama verisinde ya da çapraz doğrulamayla ölçülen hataya dayanır.

18.5.2 F testi

$$H_0 : \beta_1 = \beta_2 = \cdots = \beta_p = 0 \qquad F = \frac{(\mathrm{SST} - \mathrm{SSE})/p}{\mathrm{SSE}/(n-p-1)} = \frac{(71{,}95 - 7{,}06)/2}{7{,}06/27} = 124{,}1$$

F testi modelin bir bütün olarak işe yarayıp yaramadığını sınar. Tek tek t testleri varken neden buna ihtiyaç var? Çok sayıda değişken olduğunda, hiçbirinin gerçek bir etkisi olmasa bile, %5 düzeyinde yaklaşık her 20 testten biri tesadüfen anlamlı çıkar. F testi bütün katsayıları birlikte sınar ve bu sorunu önler. Tersi de olabilir: Çoklu bağlantı altında tek tek t testlerinin hiçbiri anlamlı değilken F testi çok anlamlı çıkabilir.

```
model2.summary2().tables[1].round(4)
```

çıktı

```
              Coef.  Std.Err.       t  P>|t|  [0.025  0.975]
Intercept   1.5902    0.6373  2.4952  0.019  0.2826  2.8978
alan        0.0305    0.0034  8.9526  0.000  0.0235  0.0375
yas        -0.0661    0.0134 -4.9239  0.000 -0.0936 -0.0386
```

Tablo 18.2 Çoklu regresyon çıktısı (n = 30, R^2 = 0,902, $\bar{R}^2$ = 0,895, F(2, 27) = 124,1).

	Tahmin	Std. hata	*t*	*p*	%95 güven aralığı
Kesişim	1,590	0,637	2,50	0,019	[0,283; 2,898]
Alan (m^2)	0,0305	0,0034	8,95	< 0,001	[0,0235; 0,0375]
Yaş (yıl)	−0,0661	0,0134	−4,92	< 0,001	[−0,0936; −0,0386]

18.6 Doğrusal modelin genişletilmesi

"Doğrusal" modelin esnekliği, açıklayıcı değişkenlerin nasıl tanımlandığından gelir. Aşağıdaki genişletmelerin hepsi OLS ile, aynı $(X^\top X)^{-1}X^\top \mathbf{y}$ formülüyle çözülür; değişen yalnızca X'in sütunlarıdır.

Kategorik değişkenler

"Asansör var mı?" gibi bir değişken 0/1 gösterge olarak girer: $+\,\beta_3 \cdot \text{Asansör}$. β_3, aynı alan ve yaştaki asansörlü ve asansörsüz daireler arasındaki ortalama fiyat farkıdır; düzlemi paralel olarak yukarı kaydırır. k kategorili bir değişken için k − 1 gösterge kullanılır; dışarıda kalan kategori karşılaştırma tabanıdır (Bölüm 12).

Etkileşim

$+\,\beta_3 \cdot \text{Alan} \cdot \text{Yaş}$ terimi, alanın etkisinin yaşa bağlı olmasına izin verir: Alanın eğimi artık $\beta_1 + \beta_3 \cdot \text{Yaş}$ olur. Şekil 18.1'deki dilimler paralel olmaktan çıkar ve düzlem bükülmüş bir yüzeye dönüşür. Etkileşim terimi varken ana etkiler de modelde tutulur.

Eğrisel ilişkiler

$+\,\beta_3 \cdot \text{Alan}^2$ ya da $\log(\text{fiyat})$ gibi dönüşümler eğri ilişkileri modeller. Log dönüşümünde katsayılar yüzde değişim olarak yorumlanır: $\log(\text{fiyat})$ modelinde alan katsayısı 0,008 ise 1 m^2 yaklaşık %0,8 fiyat farkı demektir. Değişen varyansı azaltmak için de sık kullanılır (Bölüm 13 ve 17).

Düzenlileştirme

Değişken sayısı çok olduğunda ya da çoklu bağlantı ciddi olduğunda SSE'ye katsayıların büyüklüğünü cezalandıran bir terim eklenir: Ridge, Lasso ve Elastic Net. Bu yöntemler dersin ilerleyen konularındandır.

18.6.1 Güçlü ve zayıf yönler

Doğrusal regresyonun **güçlü yönleri**: Katsayılar doğrudan yorumlanabilir ve her değişkenin katkısı kendi birimiyle ifade edilir. Çözüm kapalı biçimde, hızlı ve tekrarlanabilir olarak bulunur; ayarlanacak bir hiperparametre yoktur. Standart hatalar, güven aralıkları ve testlerle belirsizlik ölçülebilir. Küçük veride bile kararlı çalışır ve daha karmaşık modeller için güçlü bir karşılaştırma tabanıdır (Bölüm 7'deki başlangıç düzeyi fikri).

Zayıf yönleri: Doğrusallık ve toplamsallık varsayılır; etkileşimleri ve eğrileri analistin elle tanımlaması gerekir. Kare kayıp nedeniyle aykırı ve etkili gözlemlere duyarlıdır. Çoklu bağlantı altında katsayılar kararsızlaşır. Değişken sayısı gözlem sayısına yaklaştığında aşırı uyar; $p > n$ olduğunda tek bir çözüm yoktur.

18.7 Bir regresyon çıktısını okurken

Kısım IV'ün kavramlarını bir kontrol listesinde toplayalım:

1. **Önce grafiğe bakın.** Saçılım grafiği ve artık-tahmin grafiği, sayıların anlatamadığını gösterir (Anscombe).
2. **Katsayıları birimleriyle yorumlayın.** "Diğerleri sabitken, ortalamada" ifadesini unutmayın; kesişimin yorumlanabilir olup olmadığını x = 0'ın anlamına ve veri aralığına göre değerlendirin.
3. **Belirsizliğe bakın.** Standart hatalar ve güven aralıkları katsayının ne kadar kesin olduğunu söyler. p değeri tek başına etki büyüklüğü değildir.
4. **Uyumu doğru ölçütle değerlendirin.** R^2, RSE; amaç tahminse test verisinde RMSE ve MAE.
5. **Varsayımları artıklar üzerinden kontrol edin.** Etkili gözlemleri ve VIF değerlerini inceleyin.
6. **Nedensellik iddiasından kaçının.** Gözlemsel veride katsayılar ilişkiyi ölçer; dışlanmış değişkenler bu ilişkiyi yanlı kılabilir.

Tablo 18.3 Formül özeti.

Nicelik	Basit regresyon	Çoklu regresyon (p değişken)
Model	$Y = \beta_0 + \beta_1 X + \varepsilon$	$\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\varepsilon}$
Katsayılar	$\hat\beta_1 = S_{xy}/S_{xx} = r \cdot s_y/s_x$	$\hat{\boldsymbol{\beta}} = (X^\top X)^{-1} X^\top \mathbf{y}$
Kesişim	$\hat\beta_0 = \bar y - \hat\beta_1 \bar x$	$\hat\beta_0 = \bar y - \sum_j \hat\beta_j \bar x_j$
RSE	$\sqrt{\mathrm{SSE}/(n-2)}$	$\sqrt{\mathrm{SSE}/(n-p-1)}$
Standart hata	$\hat\sigma/\sqrt{S_{xx}}$	$\hat\sigma\sqrt{[(X^\top X)^{-1}]_{jj}}$
R^2	$1 - \mathrm{SSE}/\mathrm{SST} = r^2$	$1 - \mathrm{SSE}/\mathrm{SST}$; $\bar R^2 = 1 - (1 - R^2)\frac{n-1}{n-p-1}$
Test	$t = \hat\beta_1/\mathrm{SE}$, sd = n − 2	$t = \hat\beta_j/\mathrm{SE}$, sd = n − p − 1; genel F testi

Bölüm özeti

- Çoklu regresyon iki değişkenle bir düzlem tanımlar. Daire verisinde $\hat{y} = 1{,}590 + 0{,}0305 \times$ alan $- 0{,}0661 \times$ yaş, $R^2 = 0{,}902$.
- Katsayılar "diğerleri sabitken, ortalamada" yorumlanır: Aynı yaştaki dairelerde 1 m² 30,5 bin TL, aynı alandaki dairelerde 1 yıl −66,1 bin TL. Bu istatistiksel bir karşılaştırmadır, deney değildir. Sabit yaş dilimleri paraleldir (toplamsallık).
- Matris çözümü $\hat{\boldsymbol{\beta}} = (X^\top X)^{-1} X^\top \mathbf{y}$'dir. Birler sütunu kesişimi üretir. Çözüm için yeterli gözlem ve tam çoklu bağlantının olmaması gerekir; kukla değişken tuzağı bunun bir örneğidir.
- OLS, y'nin X'in sütun uzayına dik izdüşümüdür; artıklar bütün sütunlara diktir.
- Basit modelin alan eğimi (0,0407) ile çoklu modelinki (0,0305) arasındaki fark dışlanmış değişken yanlılığıdır: $0{,}0407 = 0{,}0305 + (-0{,}0661)(-0{,}1535)$. Yanlılık, dışlanan değişken hem Y'yi etkiliyor hem de dahil edilenle ilişkiliyse oluşur.
- Frisch-Waugh-Lovell: Bir değişkeni kontrol etmek, diğer değişkenden onunla ortak olan bilgiyi çıkarmaktır.
- VIF = $1/(1 - R_j^2)$ çoklu bağlantının varyansı kaç kat şişirdiğini gösterir; örnekte 1,57. Çoklu bağlantı tahmini değil, yorumu zorlaştırır.
- R^2 değişken eklendikçe düşmez; düzeltilmiş R^2 serbestlik derecesi kaybını cezalandırır ama tek bir örneklemde şans eseri yükselebilir. Anlamsız değişkenlerle R^2'nin 1'e yaklaşması aşırı uyumdur; model seçimi doğrulama hatasına dayanır. F testi modeli bir bütün olarak sınar.
- Kategorik değişkenler, etkileşimler ve eğrisel terimler aynı OLS formülüyle çözülür; değişen yalnızca X'in sütunlarıdır.

Kaynaklar

James, G., Witten, D., Hastie, T., Tibshirani, R. ve Taylor, J. (2023). *An Introduction to Statistical Learning with Applications in Python*. Springer. Bölüm 3.2–3.3.

Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3. baskı). O'Reilly. Bölüm 4: "The Normal Equation".

Müller, A. C. ve Guido, S. (2016). *Introduction to Machine Learning with Python*. O'Reilly. Bölüm 2: "Linear Models for Regression".

Wooldridge, J. M. (2020). *Introductory Econometrics: A Modern Approach* (7. baskı). Cengage. Bölüm 3.